

Versioning & Identity of Structured Material — A Per-Paper Research Reading

For the Softland code ingestor and object-container kernel · 2026-06-11

What this is

We started at a narrow question — *what is the identity of a code file, and how should the code ingestor store it* — and it opened, correctly, into a much older and larger one: **what is the identity and versioning of any structured material, at multiple granularities at once**. A function, a grouping of functions, a knowledge graph that must version at graph / node / relation-type / node-data level simultaneously — these are one problem, and people with names like McCarthy, Kay, and Hickey are only the visible surface of a literature that has worked it since the 1980s.

This document is the deep reading. Twenty-four sources — research papers and production systems, famous and obscure — were each read in full, **section by section**, then mapped back to our parts. The structure of every entry is the same:

- **The paper, section by section** — what each section actually argues, builds, proves, or measures, in the authors' own words. This is the substance; we do not skip to conclusions.
- **How it maps to our parts** — a zoom-out: which Softland concern (the object-container kernel, the code-identity question, versioning-as-its-own-layer, groupings, four-level graph versioning, the trail-to-code join, preserved disagreement, the Rama substrate) this paper supports, names, warns against, or hands a mechanism.
- **Takeaway** — the single most important thing it contributes.

At the end of each of the five angles there is a **"What we can gather"** synthesis, and the document closes with an overall synthesis and the concrete amendments it implies for the slice-1 code-ingestor contract.

Calibration

The per-section content was produced by 24 parallel deep-reader agents (Opus), one per source, each instructed to read the full text — chasing an open-access copy when the publisher gated it — and to flag inference versus what it read. Twenty-three were read at full text; one (Sciore 1994) via an open secondary source; one (TerminusDB) partially. This is a *reading*, not a peer-reviewed survey: quotes are reproduced as the readers extracted them, and where a mapping is an analogy rather than a claim from the source, the reader was told to say so. Treat it as a high-quality literature map with citations to follow, not as settled fact.

The five angles

1. **Program-unit identity** — what *is* a code unit across time (Unison, Molhado, MolhadoRef, Monticello).
 2. **Version-model theory** — what *is* versioning, formally (Conradi-Westfechtel, DSST persistence, Pijul, the categorical and homotopical patch theories, Darcs).
 3. **Versioned databases** — the two-layer pattern in production (Irmin, Dolt/Prolly, the version-controlled-database category, XTDB bitemporality).
 4. **Graph & RDF versioning** — the multi-level case, stress-tested (TerminusDB, OSTRICH, the BEAR benchmark, the two-track RDF-archiving framework, Wikidata history).
 5. **Composite & assembly versioning** — the mature, solved part (Katz, ORION, Sciore, PLM/PDM effectivity).
-

Angle 1 — Program-Unit Identity: What *Is* a Code Unit Across Time?

1. The Big Idea (Unison documentation)

Unison Computing (Paul Chiusano, Rúnar Bjarnason, et al.) · 2024 · [unison-lang.org](https://www.unison-lang.org) official documentation

In one line. Each Unison definition is identified by the hash of its normalized syntax tree; names are mutable metadata pointing at immutable, content-addressed code.

Source access: full. Read the full public doc page at <https://www.unison-lang.org/docs/the-big-idea/> via WebFetch (two passes: full-text extraction, then per-heading verbatim quotes). This is a documentation page, not a peer-reviewed paper, so I used its real section headings rather than paper-style sections. Not gated.

THE PAPER, SECTION BY SECTION

§ The big idea (core claim). States the single founding axiom: every Unison definition is identified by a hash of its syntax tree, not by a file, path, or name. The hash is a 512-bit SHA3 over a NORMALIZED form of the AST: all named arguments are replaced by positionally-numbered variable references (so alpha-equivalent definitions hash identically), and every dependency reference is replaced by the hash of that dependency. This makes the hash a Merkle-style content address that recursively captures the full transitive meaning of the definition while being insensitive to naming. Names are explicitly NOT part of the hash; they are separately stored metadata that map onto hashes.

“Each Unison definition is identified by a hash of its syntax tree.”

“all named arguments are replaced by positionally-numbered variable references, and all dependencies...are replaced by their hashes”

“names are just separately stored metadata that don't affect the function's hash”

§ Benefits (overview). Frames the rest of the document as consequences of the one starting assumption. The claim is that content-addressing is not a niche optimization but a root cause whose downstream effects are broad: it simplifies distributed programming, eliminates builds and dependency conflicts, enables typed durable storage, supports structured refactorings, and unlocks richer codebase tooling. The rhetorical move is that all later sections are derivations from the single identity axiom rather than independent features.

“This starting assumption provides some surprising benefits: it simplifies distributed programming, eliminates builds and dependency conflicts, supports typed durable storage, structured refactorings, enables better tools for working with code, and lots more.”

§ Simplifying distributed programming. Argues that because a definition's identity is a content hash, computations are location-independent and self-describing. Code (as bytecode referencing hashes) can be shipped from one node to another, and any dependencies the receiving node is missing can be identified by hash and deployed on the fly, because the hash unambiguously names exactly the needed definition. There is no risk of the wrong version being present, since the hash IS the version. This turns distributed deployment into hash-resolution plus on-demand sync rather than environment matching.

“In Unison, since definitions are identified by a content hash, arbitrary computations can just be moved from one location to another, with missing dependencies deployed on the fly.”

§ No builds. Establishes immutability as the consequence of content-addressing: since a definition's hash is computed from its content, the definition behind a given hash can never change. What CAN change is the name-to-hash mapping (the basis of refactoring and code evolution), but never the content under a hash. Because of this,

parsing and typechecking results can be cached permanently against the hash and never invalidated, so there is effectively no build step. The codebase ships with a perfect, permanent compilation cache.

“Because Unison definitions are identified by their hash, they never change. We may change which names are associated with which hashes (and this is used for Unison's approach to refactoring and code evolution), but the definition associated with a hash never changes.”

§ No dependency conflicts. Diagnoses dependency hell as fundamentally a naming-collision problem: conflicts arise only because two different definitions compete for the same name. Since Unison references dependencies by hash rather than name, two libraries depending on different versions of the same logical definition simply reference two different hashes that coexist without conflict. The version/name competition that produces diamond-dependency failures in conventional ecosystems is structurally dissolved; a name clash becomes a resolvable metadata question, not a hard runtime conflict.

“Dependency conflicts are, fundamentally, due to different definitions “competing” for the same names. But why do we care if two different definitions use the same name?”

“definitions are referenced by hash...so dependency conflicts...are just not a thing”

§ Typed, durable storage. Because definitions never change, any value at all (including closures and values that contain functions) can be serialized and later deserialized without the programmer hand-writing an encoder/decoder or pinning a serialization format. The runtime can persist a value by reference to the immutable hashes of all code it transitively depends on, and rehydrate it later with type information intact. This makes durable, typed storage a property of the identity model rather than a separate library, and it stays sound across time because the referenced definitions are immutable.

“In Unison, any value at all (including functions or values that contain functions) can be persisted and unpersisted at a later time, without the programmer needing to manually specify a serialization format or write an encoder and decoder.”

“Definitions never change.”

§ Richer codebase tools. Reframes the codebase itself as a structured database rather than a folder of text files. Because every definition is hashed, typed, and has fully known dependencies, the codebase can maintain a perfect compilation cache, a complete dependency graph, type-indexed search (find by type signature), and other indices that conventional text-file codebases cannot reliably build. Tooling becomes queryable over a content-addressed store with strong invariants, instead of re-parsing and guessing from filesystem layout.

“The Unison codebase is a proper database which knows the type of everything it stores, has a perfect compilation cache, perfect knowledge of dependencies, indices for type-based search, and more.”

§ Structured refactoring. Explains how evolution works given immutability: you never edit code in place, you change which names map to which hashes within structured refactoring sessions. Adding or modifying a definition produces a NEW hash; the refactoring session tracks the set of names being re-pointed and surfaces exactly what must be updated, keeping the codebase in a never-broken state throughout. This contrasts sharply with the conventional loop of mutating text files and then chasing a cascade of misleading compile errors and failing tests, which Unison characterizes as both error-prone and misleading.

“In Unison, changing what names are mapped to which hashes is done with structured refactoring sessions, rather than the typical approach of just mutating text files in place and then fixing a long list of misleading compile errors and failed tests.”

“the codebase is never in a broken state”

HOW IT MAPS TO OUR PARTS

- **Object-container kernel: stable CONTAINER = identity thread; REVISIONS = immutable values inside; 'inferred structure never accepted as truth at ingest.'** — GIVES A NAME + MECHANISM. Unison's hash-of-AST IS an immutable revision value; names are the mutable pointer (the container/identity thread). The split maps near-exactly onto our container-vs-revision split: hash = revision (content), name-binding = container head (current-revision). But Unison inverts our 'inferred structure never truth' rule: the hash is DERIVED

(normalized AST) yet treated AS the truth. Reconcile by noting Unison's hash is a DECLARED canonical form computed deterministically at authoring, not an inference reconstructed by diffing — so it does not violate our rule, it sharpens what counts as 'declared.'

- **The code-identity question: file vs path vs name/FQN vs normalized AST vs git hash; atomic unit = file or function?** — DIRECTLY ANSWERS IT. Unison's whole thesis is the strongest possible vote: identity = hash of the normalized AST of a single DEFINITION (function), with names/paths/files demoted to metadata. The atomic unit is the function, not the file; the file is not even a storage primitive (the codebase is a database). This is the maximal-purity end of our design axis — useful as the reference pole even if we adopt a weaker version.
- **Versioning as its own layer: arrival-time history (Rama, partial persistence) vs committed/minted versions (git-style, full persistence). Two times.** — SUPPORTS + WARNS. Supports the separation: Unison literally separates the immutable content layer (hashes never change) from the mutable naming/refactoring layer (which names map to which hashes) — exactly our 'content revision vs committed pointer' two-layer idea. Warns: Unison collapses 'when-it-was-true' into content-addressing and has NO arrival-time axis at all; it gets reproducibility but loses cheap time-travel over edit history. Our Rama arrival-log gives the axis Unison lacks; we should keep both.
- **Groupings as versioned objects: a curated local world / grouping has its own version history, members pinned/floating/effectivity-scoped.** — OFFERS A MECHANISM. Unison's 'namespace' is a versioned map from names to hashes — i.e. a grouping whose identity is independent of its members, and whose members are PINNED by hash. This is concretely our 'pinned member binding.' What Unison lacks is floating/effectivity-scoped binding (a name always resolves to one hash within a namespace version), so it models the pinned case well and shows the cost: refactoring becomes a structured session because pins must be re-pointed deliberately.
- **Four-level graph versioning: graph / node / relation-type / node-data, composing.** — PARTIAL MECHANISM + GAP. Unison cleanly versions node-DATA (the definition body -> hash) and graph-level (namespace version) and composes them (a namespace hash is built from member hashes, dependencies replaced by their hashes -> a Merkle DAG, structural composition for free). But it has no notion of relation-TYPE versioning (edges are just 'depends-on' by hash). Lesson: Merkle composition gives us node-data + graph levels almost free, but typed-edge versioning is our own problem to solve.
- **Trail-to-code join: resolve a trail's path+timestamp to code-at-that-moment; DECLARED vs INFERRED change.** — WARNS / SHOWS TENSION. Unison resolves identity by hash, not path+timestamp — the opposite of our trail join key. Agent trails carry PATHS and edit ops (mutable, name-space coordinates); Unison says those coordinates are unstable metadata. Implication: if we hash-address code units, a trail's recorded path may no longer resolve, so we must keep the name/path->hash mapping AS OF the trail timestamp (Unison's namespace-version is exactly the table to snapshot). Reinforces preferring DECLARED change (the agent names the unit) over path-diff inference.
- **Preserved disagreement / plurality: conflicting views representable, synthesis late-bound.** — OFFERS A MECHANISM. Because two definitions can share a name without conflict (the name is just metadata, the hashes coexist), Unison structurally HOLDS competing definitions side by side — 'dependency conflicts are different definitions competing for the same name.' Content-addressing lets plural versions coexist unmerged; a name conflict is surfaced for late-bound resolution in a refactoring session rather than silently overwritten. This is a working model of preserved disagreement at the code layer.
- **Substrate choices: Rama event-sourced, immutable values, durable typed storage.** — SUPPORTS / CONVERGENT. Unison's 'typed durable storage' (any value persists/unpersists with no hand-written codec because definitions never change) is the same bet as Rama's immutable event log: identity by content, no serialization drift, perfect compilation/dependency cache. Confirms that an immutable-value substrate buys reproducibility and a 'codebase as database' — validating our Rama-as-truth posture for the code layer.

Takeaway. Unison is the canonical, fully-realized answer to our code-identity question: identity = hash of the normalized AST of a single definition (function), with names, paths, and files demoted to mutable metadata pointing

at immutable content. For Softland this is the reference pole on the identity axis and a direct architectural gift — it splits cleanly into our container-vs-revision model (hash = immutable revision, name-binding = container head), gives us Merkle composition for node-data and graph-level versioning nearly for free, and proves an immutable-value substrate buys reproducibility, a codebase-as-database, and structural plurality (competing definitions coexist under one name). Its key WARNING: Unison has no arrival-time axis and addresses code by hash rather than path+timestamp, so adopting it would break our trail-to-code path join unless we snapshot the name/path-to-hash table as-of each trail moment — which is exactly the second (Rama arrival-time / namespace-version) layer Unison lacks and we must keep.

2. On Product Versioning for Hypertexts

Tien N. Nguyen, Cheng Thao, Ethan V. Munson · 2005 · SCM (Software Configuration Management Workshop) — ACM / Springer LNCS

In one line. Molhado is the first hypertext-versioning system to apply the product-versioning model, using a node/slot/attribute data model (Fluid IR) and a single global version space so that documents, fine-grained logical units (AST/XML nodes), first-class links, and hypermedia structure are all versioned uniformly and resolved at a 'current version.'

Source access: full. Read the full PDF directly from the author/UCSC mirror (users.soe.ucsc.edu/~ejw/scm12/papers/Nguyen-Chapter9.pdf). WebFetch could not parse the binary, so the PDF was downloaded and text-extracted locally with `pdftotext -layout` (962 lines, all sections + references recovered). This is the SCM 2005 paper, a sibling of the Molhado Hypertext Versioning System work (Nguyen/Munson/Boyland/Thao, Hypertext 2004, ref [3]); the deeper node/slot/attribute and per-attribute fine-grained versioning machinery the prompt hints at lives in [3] and Nguyen's PhD thesis [32], which this paper summarizes and applies to hypertext.

THE PAPER, SECTION BY SECTION

§ Abstract. States that versioned hypermedia helps manage evolving document collections but raises structural and cognitive issues. Claims Molhado as 'the first hypermedia infrastructure that applies the product versioning model to versioned hypermedia.' Molhado supports fine-grained configuration management of hypermedia structure AND version control for individual hyperlinks and document nodes, and the paper argues this addresses issues raised by prior versioned-hypermedia research (notably Osterbye) while introducing new issues.

“Molhado, the first hypermedia infrastructure that applies the product versioning model to versioned hypermedia”
“provides version control for individual hyperlinks and document nodes”

§ 1. Introduction. Defines hypertext versioning as storing/retrieving/navigating prior states of a hypertext and letting collaborators develop new states. Motivates with three domains: software engineering (capturing evolution of relationships between requirements/design/implementation artifacts, not just the artifacts themselves), legal systems (the version of a law in effect at the time of an infraction matters), and the Web (rolling a site back to a specific time). Names the structural issues (immutability of versions, version control for links, versioning for hypermedia structure) and cognitive issues (version creation, element version selection) from Osterbye. Introduces Molhado as using 'a single versioning mechanism for all software components and for the hypertext structures' that can return to a consistent prior state 'not only of a hypertext network but also of a single node or link,' supporting both embedded (HTML) and first-class (XLink) hyperlinks.

“Molhado can track changes at a very fine-grained level, allowing users to return to a consistent previous state not only of a hypertext network but also of a single node or link”
“Molhado uses a single versioning mechanism for all software components and for the hypertext structures (including anchors and links) that connect them”

§ 2. Related work. Lays out three version-control approaches and positions product versioning against the other two. (a) COMPOSITION model: atomic components (usually files) each have a version history; composites are defined by Revision Selection Rules (RSRs). Systems: Xanadu, KMS, DIF, Hyperform; HyperWeb and HyperCASE

built on RCS where 'the smallest versionable information unit is a file.' RSRs make structure hard to analyze without executing the rules, prompting 'current context' notions (Neptune, CoVer, VerSE) with maintenance overhead. (b) TOTAL versioning: every item — composite and atomic — has its own version space; PCTE recursively copies the whole composition hierarchy; suffers the 'version proliferation problem' where a leaf change spawns versions of all ancestors/linked nodes. (c) PRODUCT versioning: 'establishes a single view of a software product... by arranging versions of all items in a uniform, global version space.' Popular in SCM (change-based COV, Aide-de-camp, PIE; state-based Voodoo; Wagner's infrastructure). Molhado is positioned as the only versioned-hypermedia system applying product versioning.

"product versioning establishes a single view of a software product... by arranging versions of all items in a uniform, global version space"

"the version proliferation problem, where each small change in a leaf of the hierarchy creates versions of multiple objects higher in the compositional hierarchy"

"in which the smallest versionable information unit is a file"

§ 3. Issues with versioning for hypertexts (Osterbye). Catalogs the structural and cognitive problems Molhado must answer. 3.1 IMMUTABILITY: a frozen version's contents must be immutable, but it is unclear whether new annotation/comment links to/from it should force a new version, whereas semantically substantial links should. 3.2 VERSION CONTROL FOR LINKS: not versioning individual links blocks adoption in software engineering; three sub-problems — (i) navigation ambiguity when a link has versions pointing to different destinations (v1 $A \rightarrow B$, v2 $A \rightarrow C$), (ii) modifying a target $B \rightarrow B'$ forces a new link version because links and resources have separate histories, (iii) whether a link is a first-class object or a relation of anchor values. 3.3 STRUCTURE: must allow returning to a consistent prior state of the WHOLE network, not just a node or link. 3.4 ELEMENT SELECTION (cognitive): determining which element in a versioned group a link points to (specific element, whole group, or current); query-language selection imposes cognitive overhead and discourages linking; debates where to store selection rules (on structure vs on containment arcs). 3.5 VERSION CREATION (cognitive): forcing users to name every node or explicitly create/maintain versions distracts from the real work.

"The fact that existing hypermedia systems for software development do not version individual links is a significant factor preventing their wider use"

"links and resources have their own version histories"

"If a selection query must be specified each time a link is created, the users are discouraged from making links"

§ 4.1 Data model (Fluid IR). Describes Molhado's underlying data model: the Fluid Internal Representation, whose core concepts are node, slot, attribute, and sequence. 'A node is the basic unit of identity' representing any abstraction; 'a slot is a location that can store a value in any data type, possibly a reference to a node'; 'an attribute is a mapping from nodes to slots,' optionally with per-node slots and a default slot. The model is an attribute table: rows = nodes, columns = attributes, cells = slots. Slots have three kinds: CONSTANT (immutable, set once), SIMPLE (reassignable), and VERSIONED (different values in different versions = slot revisions). A sequence is a same-type slot container with a unique id. Adding versioning gives 'the attribute table a third dimension: the version.' This is the paper's most load-bearing identity machinery.

"A node is the basic unit of identity and is used to represent any abstraction"

"An attribute is a mapping from nodes to slots"

"The third kind of slot is the versioned slot, which may have different values in different versions (slot revisions)"

"the attribute table gets a third dimension: the version"

§ 4.2 Product versioning. Molhado versions the whole project, not individual components: 'All system objects including (atomic and composite) components and hypertext structures are versioned in a uniform, global version space.' Properties can be declared versioned or un-versioned. 'A version is global across the whole project and is a point in a tree-structured discrete time abstraction' — the table's third dimension is tree-structured, moving discretely. Only captured versions (v1.0, v2.0...) are retrievable later; the 'current version' designates the current project state, and setting it rewinds the whole project. Edits to versioned objects create a TEMPORARY version

branching off current, recorded only if explicitly captured. To record one object's history, the whole project is captured — efficient because 'the versioning system only records changes and works at a very small granularity.'

"A version is global across the whole project and is a point in a tree-structured discrete time abstraction, rather than being a particular state of an object"

"Changes made to versioned objects of the project at the current version create a temporary version, branching off the current version"

"Capturing the whole project is quite efficient because the versioning system only records changes and works at a very small granularity"

§ 4.3 Structure-oriented representation. To version at fine granularity, each document is decomposed into 'structural units or logical units' (as in SGML/XML structured-document research), represented as a document tree or graph where each node encodes one logical unit. XML is used for non-program artifacts (syntax defined by DTD/XSchema); 'for a program, an abstract syntax tree (AST) perfectly represents its logical structure.' Trees/graphs are built from nodes/slots/attributes: a 'children' attribute maps a node to a sequence of its children (directed graph), a 'parent' attribute adds tree structure. Molhado has 'a fine-grained versioning scheme for tree and directed graph data structures' recording fine-grained changes to structure AND to associated slot contents. XML example: each node carries an 'operator' (intermediate or text) defining its syntactic type and children; text nodes get a 'content' slot for CDATA; element nodes get a slot per XML attribute. The tree is versioned by the tree-based scheme.

"each document is considered to be logically structured into fine units, called structural units or logical units"

"For a program, an abstract syntax tree (AST) perfectly represents its logical structure"

"fine-grained changes to the structure of a tree or a graph and to the contents of associated slots are recorded"

§ 4.4 Hypermedia entities. Defines the hypermedia layer over the document trees/graphs. An 'href' attribute on each node holds a URL to a document node, enabling embedded HTML-style links to/from any node. The first-class model has four concepts: LINKBASE (a container for hypertext networks and/or other linkbases — like a directory to a file), HYPERTEXT NETWORK (contains links and anchors, belongs to one linkbase), LINK ('a first-class entity and is an association among a set of anchors'), and ANCHOR (denotes a region of interest, refers to a document or document node, may belong to multiple links/networks). Crucially, separating anchors from document nodes 'allows for the separation between hypertext networks and document contents.' Links and anchors can carry arbitrary attribute-value pairs.

"A link is a first-class entity and is an association among a set of anchors"

"This separation between anchors and document nodes allows for the separation between hypertext networks and document contents"

§ 4.5 Version control for hypermedia structure. Provides structure versioning at coarse and fine grain. Coarse: a composite component contains atomic and/or other composite components. Fine: an atomic component may have internal structure containing logical units. A linkbase is a composite component whose internals are a tree (versioned by the tree scheme); a hypertext network is an atomic component whose internals are a directed graph where each link or anchor is a node, a directed edge connects an anchor node to a link node it belongs to, and a 'ref' attribute on each anchor node references a component or logical unit (not a network/linkbase). Figure 6 walks two versions v1/v2: nodes added/deleted (node 5 deleted with doc3 removal, nodes 9/10 created), the 'ref' cell updated, and the 'content' slot of a section changing from 'old' to 'new' — all recorded via directed-graph versioning. Returning to a prior version restores documents, linkbases, and networks together.

"A hypertext network is implemented as a type of atomic component, whose internal structure is a directed graph. Each link or anchor is represented by a node in that graph"

"The content slot of n(section1) has changed from old to new to reflect the change in the content"

"returning to previous state of entire project is a basic versioning operation"

§ 4.6 Version control for links. Shows the directed-graph scheme inherently versions individual links (each is a graph node whose connection structure and slots are captured over time), and resolves all three of Osterbye's link

problems. Navigation ambiguity (Fig 1) disappears because 'the destination of the link will be automatically chosen at the current version' — the global version disambiguates. The target-modification cascade (Fig 2) disappears because all entities live in one global version space; 'link traversal occurs only among nodes at the current version.' The first-class-vs-relation question: a link IS a first-class node; when a versioned attribute, the anchor connection, or an underlying document node changes, 'a new global version point is created in the tree-structured discrete time line, rather than a new version of resources or a new version of links.' Net: per-link and per-node version control without the problems of composition/total models.

"the destination of the link will be automatically chosen at the current version"

"link traversal occurs only among nodes at the current version"

"a new global version point is created in the tree-structured discrete time line, rather than a new version of resources or a new version of links"

§ 4.7 Revisiting other issues. Resolves the remaining Osterbye issues. IMMUTABILITY: users designate 'substantial' links (modeled as versioned slots via 'href') whose change forces a new version, vs annotation/comment links (simple slots) that change without a new version — leveraging Molhado's ability to declare aspects versioned/un-versioned and its single global space. First-class links+anchors pointing into documents do create a version (treated as annotations on top of content), but the underlying components are unchanged between the two versions. ELEMENT SELECTION: because 'an individual entity do not have its own version numbers' and the project's current version is selected implicitly (mouse) or explicitly (commands), there are NO selection rules/queries on links, anchors, or networks — easing GUI construction (used in the Software Concordance). VERSION CREATION: entities share one version space, so users commit the whole project rather than checking nodes/links in and out individually; intermediate session versions support undo. Caveat: 'a single change to a versioned entity might potentially create a new version of entire project.'

"users are allowed to specify the set of substantial links (i.e. the ones that if modified would create a new version)"

"an individual entity do not have its own version numbers"

"a single change to a versioned entity might potentially create a new version of entire project"

§ 5. The versioned hypermedia tool. Describes the implementation as library functions in two groups: (1) linkbase/network services (create, delete, re-structure, relocate, open, activate, import/export to XLink at any version) and (2) link/anchor services (create, delete, rename, view attributes, view history; add/remove/rename anchors, show the structural unit an anchor refers to). Integrated into the Software Concordance (SC) environment with CM handled by Molhado's CM infrastructure. User operations: open a project, pick a current version from a history window, browse/edit the directory and documents; any edit at the current version creates a temporary branch version held until a 'capture' or 'commit' (capture attaches a unique name, date, authors, descriptions and acts as a retrievable checkpoint). Supports branching by switching current version and 'multi-version editing' (working on several versions in one session). Commit saves chosen temporary versions storing only differences. Structured editors (Java, XML, HTML, SVG, UML) are link-aware; users can view a single document node's history (Fig 10: 'loadDelta' not created until v9.1, earlier versions disabled) and a hyperlink's history (Fig 11), and edit networks with causal vs non-causal links (Fig 12). Defining a new network does not alter document content (structure separated from content).

"This switching feature allows the user to work on many versions at the same time during one session. This capability is called multi-version editing"

"The capture command changes a temporary version into a captured one"

"Only the differences are stored. Saving complete version snapshots can improve version access time"

§ 6. Issues for future research. Identifies three problems unique to applying product versioning that composition/total systems did not face. 6.1 HYPERLINKS ACROSS VERSIONS: links exist only within a single version, so there is 'no way to say make a link to document D at version vk'; two fixes proposed — computational links whose anchors are queries against a link database ('get document D for version vk'), or adding a version identifier to all access interfaces (rejected as inconvenient in common cases). 6.2 HYPERLINKS ACROSS PROJECTS: a link lives in one project's version space; the computational-link/query approach (open another

project, load a version, then a document) should suffice. 6.3 SCOPE OF THE PRODUCT VERSIONING SPACE: the whole-project version space fits software/web/legal collections, but some hypertext collections are ill-suited to one version space, and linking Molhado-managed material with material under other versioning models is a challenge. They explicitly reject treating this 'incompatibility' as grounds to abandon product versioning, calling instead for 'a generalized access and maintenance model that covers all approaches.'

"links only exist within a single version. So, there is no way to say make a link to document D at version vk"

"The anchors of a computational link are essentially queries against a database of links"

"versioning needs a generalized access and maintenance model that covers all approaches"

§ 7. Conclusions. Restates Molhado as the first system to apply product versioning to hypertexts, claims it well-addresses Osterbye's structural and cognitive issues, and notes the model eases GUI development and reduces cognitive overhead in version creation and element selection. Software components and hypertext structures are 'uniformly versioned in a fine-grained manner,' letting users return to a consistent prior state of a network, a single node, or a single hyperlink. Acknowledges the new issues (cross-version, cross-project linking) and commits to future experimental and usability studies.

"It is the first system that applies the product versioning model to hypertexts"

"Users are allowed to return to a consistent previous state not only of a hypertext network but also of a single node and of a hyperlink"

HOW IT MAPS TO OUR PARTS

- **Object-container kernel: stable CONTAINER = identity thread; REVISIONS = immutable values; current-revision = state** — Molhado's Fluid IR gives this its canonical name and mechanism. A 'node is the basic unit of identity' (the container), a 'slot' holds a value across versions ('slot revisions'), and 'the current version is the version designating the current state.' This is almost exactly our container/revision/current-revision triad, validated 20 years prior. Molhado's 'versioned slot, which may have different values in different versions' is our immutable-revisions-inside-a-stable-identity. The node persists; its slot values are the time-indexed revisions.
- **Versioning as its own layer: arrival-time history (partial persistence) vs explicit committed/minted versions (full persistence); two times** — Molhado makes our two-times distinction concrete and warns about a tradeoff. Editing creates a 'temporary version, branching off the current version' that is discarded unless explicitly captured ('The capture command changes a temporary version into a captured one'). This is exactly arrival-time (temporary/working) vs committed (captured, named, dated, authored). It SUPPORTS having both, and shows the commit gate ('a user has to commit changes to entire project') is the minting boundary. Warning: in their model 'individual entities do not have their own version numbers' — version is global to the whole product.
- **Groupings as versioned objects: a curated local world has its own version history, partly independent of members** — This is a sharp WARNING. Molhado's product-versioning model deliberately COLLAPSES group-vs-member independent histories: 'a version is global across the whole project... rather than being a particular state of an object.' It buys coherence (a single global timeline, no version-selection queries) at the cost of independence — 'a single change to a versioned entity might potentially create a new version of entire project.' Our goal of grouping histories PARTLY INDEPENDENT of members is precisely what they gave up. The composition/total models they reject preserved member independence but suffered 'version proliferation.' We sit between: we want both, so we must avoid both failure modes.
- **Four-level graph versioning: graph, node, relation-TYPE, node-DATA, composing** — Molhado offers a working MECHANISM for fine-grained graph versioning. Hypertext networks are 'attributed directed graphs'; their 'fine-grained versioning scheme for tree and directed graph data structures' records 'fine-grained changes to the structure... and to the contents of associated slots.' This separates structure-level change (edges/nodes added/deleted — Fig 6: node 5 deleted, node 9 created) from data-level change (a slot's content 'changed from old to new'). That is two of our four levels (graph/node structure vs node-DATA) composing in one scheme via the attribute table (rows=nodes, columns=attributes, cells=slots, 3rd dim=version).

- **The trail-to-code join: resolve a path+timestamp to code-at-that-moment; declared vs inferred change** — Molhado's 'return to a consistent previous state' is exactly point-in-time resolution: 'When users set the current version to a recorded version, the state of entire software project will be set back to that version.' Their key invariant — 'link traversal occurs only among nodes at the current version' — is our resolve-at-timestamp guarantee: a reference resolves to the version coherent with the selected time, never a dangling cross-version pointer. Supports our join. But note their identity is global-version-indexed, not path-indexed; we resolve by path+timestamp, which they'd model as a node + current-version.
- **Code-identity question: is the atomic unit the file or the function (sub-file logical unit)?** — Molhado argues strongly for SUB-FILE identity and warns against file-granularity. It explicitly critiques RCS-based systems where 'the smallest versionable information unit is a file.' Instead it takes a 'structure-oriented approach where each document is... logically structured into fine units, called structural units or logical units'; 'For a program, an abstract syntax tree (AST) perfectly represents its logical structure.' Each AST/XML node is a Molhado node = identity unit. This supports our instinct that the function/logical-unit (AST node), not the file, is the atomic identity-bearing thing — and that the AST is a legitimate identity basis.
- **Preserved disagreement / plurality; substrate choices (Rama event-sourced, multi-version)** — Partial support, partial warning. Molhado supports MULTI-VERSION coexistence: 'while working on one version, the user can always switch to work on... any other version... multi-version editing,' and branching by 'just switching the current version.' This is plurality-as-branches. But it is single-writer / single-coherent-state per version — there is no notion of two conflicting views co-present WITHIN a version. Our preserved-disagreement requirement (synthesis late-bound, conflicts not overwritten) goes beyond what product versioning gives; their model forces one consistent state per global version point.

Takeaway. The single most important contribution is the node/slot/attribute identity machinery (Fluid IR) paired with the product-versioning stance: a node is a stable identity, a versioned slot holds time-indexed immutable revisions, and a global 'current version' resolves every reference coherently — which is almost exactly our object-container kernel and our resolve-at-timestamp join, demonstrated working over fine-grained AST/XML logical units rather than files. Equally important is the warning it carries: Molhado buys coherence and zero-cognitive-overhead selection by making versioning GLOBAL to the whole project, deliberately surrendering the partly-independent group-vs-member histories we want and forcing one consistent state per version (no preserved disagreement). So it validates container/revision/current-version and sub-file AST identity, while marking the exact tradeoff frontier — global coherence vs independent grouping histories vs version proliferation — that our two-times, four-level, plurality-preserving design must navigate between rather than collapse.

3. Refactoring-aware Configuration Management for Object-Oriented Programs (MolhadoRef)

Danny Dig, Kashif Manzoor, Ralph Johnson (UIUC), Tien N. Nguyen (Iowa State) · 2006 · OOPSLA 2006 (ACM) / UIUC tech report UIUCDCS-R-2006-2770; the ResearchGate 'MolhadoRef tool demo' record (221321476) is the companion tool paper

In one line. An operation-based, ID-based SCM that records refactorings and edits as first-class operations and merges them by inverting, textually merging, then replaying — preserving program-element identity and history across renames and moves where file-based SCM (CVS) loses it.

Source access: full. ResearchGate URL (221321476) returned HTTP 403. Read the FULL author PDF instead:

<http://dig.cs.illinois.edu/papers/MolhadoRef.pdf> (the 'Refactoring-aware Configuration Management for Object-Oriented Programs' paper, same authors/content). Extracted via pdftotext locally; all sections, the merge algorithm pseudocode, and Table 1 numbers were read in full.

THE PAPER, SECTION BY SECTION

§ Abstract. States the core problem: text-based SCM systems handle refactorings badly because refactorings produce global, cross-cutting changes that trigger merge conflicts. A refactoring-aware SCM reduces conflicts, preserves history, and clarifies evolution. MolhadoRef's mechanism: record change operations (refactorings AND

edits) used to produce a version, and replay them when merging. Because refactorings have well-defined semantics, they are treated intelligently. The empirical claim is comparative: a case study shows MolhadoRef auto-resolves more conflicts than CVS while producing fewer merge errors.

“MolhadoRef records change operations (refactorings and edits) used to produce one version, and replays them when merging versions.”

“A case-study shows that MolhadoRef solves automatically more merge conflicts than CVS while resulting in fewer merge errors.”

§ 1. Introduction. Motivates why refactoring breaks file-based SCM: refactorings cut across module boundaries, so SCM flags a conflict whenever two programmers touch the same line even for unrelated renames. Cites the authors' prior study of 5 mature Java components — Struts had 136 API changes over 14 months, and >80% of API changes were caused by refactorings — to argue the problem is real and common. Identifies a sharp failure mode unique to OO: a textually 'successful' merge can still be wrong (e.g., one user renames a virtual method while another adds a method in a subclass → accidental overriding, a silent run-time bug). Lists three advantages of MolhadoRef: (1) better merging (resolves more conflicts, no compile errors, semantics preserved), (2) better history preservation (tracks elements even when renamed/moved across files), (3) better evolution understanding (shows high-level refactoring ops instead of thousands of scattered low-level changes). Names the technical crux: edits and refactorings are interleaved, so to merge you must INVERT refactorings to commute them past edits, and refactorings have inter-dependencies. Built on Eclipse (front-end) + Molhado (ID-based SCM backend). Claims the first algorithm to merge refactorings and edits.

“Struts had 136 API changes over a period of 14 months. In each system, more than 80% of the API changes were caused by refactorings.”

“even a successful merge might result in an incorrect program... textual merging could result in accidental method overriding, thus leading to unexpected runtime behavior.”

“it converts refactorings from being the weakest link in an SCM system to being the strongest”

“it presents the first algorithm to effectively merge refactorings and edit operations”

§ 2. Motivating Example. A concrete LAN-simulation program (a known refactoring teaching example) with 5 classes (Packet, LANNode superclass, PrintServer/NetworkTester/Workstation subclasses). Two users branch from V0: Alice (V1) moves getPacketInfo from PrintServer to Packet (τ_1), adds methods (τ_2, τ_3, τ_4), renames originate → generatePacket (τ_5). Bob (V2) renames getPacketInfo → getPacketInformation (τ_6), renames polymorphic send → sendPacket (τ_7), renames class WorkStation → Workstation (τ_8). The example concretely shows three failure classes of textual merge: (a) an UNNECESSARY conflict — Alice moved a method's declaration while Bob renamed it, same lines touched, but semantically mergeable; (b) a COMPILE error — testLosePacket calls send whose declaration was renamed to sendPacket; (c) a silent RUN-TIME error — Alice's new NetworkTester.sendPacket plus Bob's send → sendPacket rename causes accidental overriding, so packets are silently lost. Also shows LOST HISTORY: once getPacketInfo moves PrintServer → Packet, a file-based repo treats it as a brand-new method and the history is gone.

“a textual merge results in accidental method overriding... the merged program contains bugs that require many hours to find.”

“once Alice moved method getPacketInfo... from class PrintServer to Packet, the history of the getPacketInfo is effectively lost since a file-based SCM repository maintains the method as if it is a newly defined method”

§ 3. Background and Terminology. Defines the operation-based paradigm: an SCM records the operations transforming one version into another and replays them; a version IS the sequence of operations used to create it. Distinguishes three operation classes by semantic richness: API refactorings (richest — rename package/class/method, move class/method, change method signature), API edits (add/delete package/class/method-decl/field-decl), and code edits (everything else, weakest semantics, merged textually). Formalizes operations as program transformations τ : Program → Program, with preconditions; an inappropriate application yields $\perp$ (invalid program). Composition $\tau_i;\tau_j$ means apply τ_i then τ_j . Gives three formal definitions: (1) two operations COMMUTE if any order yields the same valid program; (2) they CONFLICT if any order yields $\perp$

(compile conflicts; MolhadoRef also catches run-time conflicts involving method overriding); (3) τ_j **DEPENDS** on τ_i ($\tau_i < \tau_j$) if they don't commute, a strict partial order. Crucial insight: dependences exist only because refactoring engines are NAME-based — e.g., $\tau_5 < \tau_8$ because once the class is renamed (τ_8) the FQN the method-rename (τ_5) needs no longer exists. The fix would be ID-based engines, but rewriting them is infeasible, so MolhadoRef **EMULATES** ID-based behavior via two tactics: reorder refactorings, and modify subsequent refactorings' descriptors during replay.

“An operation-based system treats a version as the sequence of operations used to create it.”

“refactorings are the operations with the most well defined semantics, so the ones that can benefit the most”

“If the refactoring engine used the IDs of the program elements, scenarios in which the names of program entities change would never pose a problem... the next best solution is to emulate ID-based engines.”

§ 4. Merging Algorithm. The technical core, a 5-step pipeline over inputs {V0 base, V1, V2, refLog1, refLog2}. Step #1: detect API/code edits via 3-way differencing (3-way, not 2-way, because 2-way can't distinguish additions from deletions). Step #2 (loop): search for compile and run-time conflicts among edits+refactorings using a predicate matrix; build a refactoring dependence DAG; detect circular dependences; user resolves conflicts (e.g., picks losePacket instead of the colliding sendPacket) and breaks cycles by deleting an operation; repeat to a fixed point (guaranteed to converge because each manual resolution removes operations). Step #3: **INVERT** every refactoring in V1 and V2 by applying an inverse refactoring (not the engine's location-based undo) — this rewrites edits that referenced refactored entities back to the V0 names, yielding V1-Ref and V2-Ref. Step #4: standard 3-way textual merge of the now refactoring-free versions → V_merged-Ref; same-line conflicts caused by refactorings have vanished. Step #5: topologically sort the refactoring DAG and **REPLAY** all refactorings onto V_merged-Ref, modifying descriptors so a name-based engine emulates an ID-based one. Sub-sections detail: conflict detection via the predicate matrix (e.g., two RenameMethod ops conflict if same source or same destination, with extra cases for polymorphic overriding); inverting refactorings with preconditions and heuristics (saving call-site parameter values like the '7' default-arg case so replay doesn't lose them; tagging name-collision cases for late inversion; falling back to textual treatment when no inversion heuristic exists, degrading gracefully to plain text merge); and replay ordering (topological sort so method refactorings precede their class renames; descriptor rewriting when one version renames and the other moves the same entity).

“Version V_1_minusRef= invertRefactorings(V_1, refs)... 3-wayTextualMerge... orderedRefs= topologicalSort(refsDAG)... replayRefactorings(V_merged_minusRef, orderedRefs)”

“The only way to undo a refactoring is to first undo all the operations that come after it. MolhadoRef must be able to undo a refactoring without undoing later operations.”

“When no heuristic for inverting a refactoring is found, the algorithm treats the refactoring as a textual edit... in the worst case it is as good as classic textual merging.”

“MolhadoRef uses a topological sort algorithm to reorder the nodes in the refactorings DAG.”

§ 5. Implementation. MolhadoRef is an Eclipse plugin: Eclipse Java editor as front-end, Molhado (an OO SCM infrastructure / 'a database that keeps track of history') as backend. At check-in it parses Java (all 1.4 syntax) down to method/field declarations, creates a Molhado counterpart for each program element with a **UNIQUE IDENTIFIER**, and stores method/field bodies as attributes of declarations. Refactorings update Molhado properties (names, args) but element identity stays intact across refactoring. Discusses the operation-based vs state-based tradeoff: pure operation-based is accurate but slow (many operations to record/replay); state-based deltas at commit are efficient but lose change semantics. MolhadoRef uses a **HYBRID** — Eclipse compare engine for body/element deltas plus captured Eclipse refactoring operations for semantics. Built on Eclipse's processor-participant refactoring architecture; their participants rewrite the FQNs in subsequent refactoring descriptors (the descriptor-rewriting emulation of ID-based engines). Notably argues these qualities could be **RETROFITTED** onto CVS: store identifier-to-name maps in metadata files in the repo, load at checkout, update during refactoring, write back at check-in. Current scope is a subset of Fowler's catalog; the key extensibility requirement is the ability to **INVERT** a refactoring.

“For each entity, Molhado gives a unique identifier... the identity of program entities remains intact even after refactoring operations.”

“MolhadoRef uses a mixture of both paradigms to maximize efficiency and accuracy.”

“Identifier-to-name maps can be saved in metadata files and stored in the repository... The key property we require of refactorings is the ability to invert refactorings.”

§ 6. Case Study. Evaluated on MolhadoRef’s own three weeks of parallel development by two programmers (who stopped pair-programming and merged real edits+refactorings). The authors note such datasets are rare because developers avoid the problem (notes asking teammates to check in before refactoring are common), so they dogfooded. Many CVS same-line conflicts traced to two refactorings: renaming a central class
LightRefactoring → Operation and moving it to another package. Table 1 quantifies: on the MolhadoRef codebase, CVS produced 36 conflicts, 41 compile errors, 7 run-time errors; MolhadoRef produced 1 conflict, 0 compile errors, 0 run-time errors. MolhadoRef auto-merged all 36 CVS same-line conflicts and asked for help only once (both devs added `getID()` to the same class). Time: 105 minutes for the two developers to reach a correct merge with CVS vs under 1 minute with MolhadoRef. Two further wins: granularity reduction — over 12 weeks, 67 refactorings corresponded to 1267 changed lines, a 1:19 compression for program understanding; and history — over the 3 weeks CVS LOST the history of two core files containing 73 API methods, which MolhadoRef preserved.

“MolhadoRef was able to automatically merge all 36 same-line conflicts reported by CVS... MolhadoRef did not introduce any compile-time or run-time errors while CVS had 48 such errors”

“it took 105 minutes for the two developers to produce the final, correct version using CVS, while it takes less than one minute for MolhadoRef.”

“there were 67 refactorings which correspond to 1267 changed lines... a reduction of 1 : 19.”

“CVS lost the history of two core files containing 73 API methods.”

§ 7. Related Work. Positions MolhadoRef across SCM lineages. Traditional file/directory systems (CVS); Subversion adds metadata/rename/copy versioning but still file-focused. Fine-grained SCM systems (COOP/Orm, Coven, POEM, Westfechtel, Ohst) version sub-file artifacts but none handle refactorings. Software merging taxonomy (per Mens’s survey): text-based (RCS/CVS, lines as indivisible units, can’t combine two edits to one line); syntactic (parse-tree/AST/graph-based, ignores comment/whitespace conflicts but misses semantic invalidity); semantic (dependency graphs, program slicing, denotational semantics, e.g., Yang-Horwitz-Reps). Operation-based merging lineage (Lippe & van Oosterom gave a theoretical framework but no refactoring application; Edwards; GINA’s redo mechanism, which can’t handle long histories/fine granularity). Closest prior work: Ekman & Asklund’s refactoring-aware versioning in Eclipse — but they keep element IDs only in volatile memory (short-lived history) and present no merge algorithm, whereas MolhadoRef persists IDs across the whole project lifecycle and contributes the merge algorithm. Claims novelty as the combination of operation-based + fine-grained + ID-based SCM for refactorings, and the first algorithm to merge refactorings and edits.

“none of them handle refactorings as MolhadoRef does.”

“Ekman and Asklund... keeps the program elements and their IDs in volatile memory, thus allowing for a short-lived history... In our approach, program elements and their IDs are stored throughout the lifecycle of the project”

“the novelty of this work is the combination of operation-based, fine-grained, ID-based SCM to handle refactorings and high-level edit operations.”

§ 8. Conclusions and Future Work. Restates that file-level SCM fails under refactoring three ways (more conflicts, lost history, harder evolution understanding) and that MolhadoRef, being aware of program entities and the refactoring operations that change them, records/detects and replays changes to merge edits and refactorings effectively without compile/run-time errors. Persistent IDs track history; explicit refactoring representation reduces the cognitive load of understanding evolution. Future work: empirical productivity study of teams using it; the larger research goal is upgrading component-based applications to new component versions by replaying the component’s refactorings on both component and application sides — framed as a special case of the same merge algorithm. Closes with the social claim that refactoring-tolerant SCM will make developers bolder about refactoring.

“Storing the IDs of program entities across versions tracks the history better, while explicit representation of refactorings reduces the load of understanding the program evolution.”

“This research is part of our larger goal to upgrade component-based applications... by replaying the component's refactorings”

“software developers will have the freedom to make their designs easier to understand and reuse.”

HOW IT MAPS TO OUR PARTS

- **Object-container kernel: stable CONTAINER = identity thread; REVISIONS = immutable values; 'inferred structure is never accepted as truth at ingest'** — Directly NAMES and validates the core mechanism. Molhado gives every program element (class, method, field) a unique persistent ID at check-in; refactorings update properties (name, args) but 'the identity of program entities remains intact.' This is exactly our container/revision split: ID = container, source-at-a-version = revision. It also SUPPORTS our hard rule via a sharp DECLARED-vs-INFERRED contrast: MolhadoRef records refactorings as declared operations from Eclipse's engine; only as a fallback does it INFER edits via 3-way diff. The paper's whole thesis is that declared semantics beat reconstructed diffs.
- **The code-identity question: is the unit the file or the function? Is identity file/path/name/FQN/AST/hash?** — Gives a strong, evidenced ANSWER and a failure mode. Identity is NOT file, path, name, or FQN. File-based identity loses history when a method moves class; name/FQN-based identity breaks under rename (their $\tau_5 < \tau_8$ dependence: renaming a class invalidates a pending method-rename because the FQN no longer exists). Their answer: identity is an opaque persistent ID, atomic unit is the program element (method/field/class), not the file. 'If the refactoring engine used the IDs of the program elements, scenarios in which the names change would never pose a problem.'
- **Versioning as its own layer: arrival-time history vs committed/minted versions; declared change vs inferred change** — Offers a concrete MECHANISM for the two-times distinction and a build-vs-buy lesson. MolhadoRef mixes paradigms: operation-based (record each declared change when it happens = our committed/minted, semantic time) vs state-based deltas computed at commit (= arrival-time diffing). It explicitly weighs them: operation-based is 'accurate' but slow; state-based is 'efficient' but 'cannot recover the semantics.' Their hybrid (capture refactorings as operations, diff bodies) is a template for our 'declared change vs inferred change' join.
- **The trail-to-code join (first consumer): resolve trail path+timestamp to code-at-that-moment; DECLARED vs INFERRED change** — This is the paper's strongest gift. An agent reasoning trail that DECLARES 'I renamed method X to Y, moved Z to class W' is exactly MolhadoRef's refactoring log — first-class operations with well-defined semantics. The paper proves that resolving such declared ops gives correct history tracking across moves/renames where path+diff alone fails (CVS lost history of 2 core files / 73 API methods). Our trail should carry declared edit-operations, and we resolve them as operations, falling back to diff-inference (their RefactoringCrawler path) only when undeclared.
- **Groupings as versioned objects: a curated grouping/local-world with its own history, members pinned/floating** — PARTIAL support + a gap to fill. Molhado is described as 'an object-oriented SCM infrastructure' that versions structure, not just files (parses to method/field declaration, bodies as attributes). It versions a tree of elements with IDs — closer to our grouping-with-members than a flat file repo. But the paper does NOT give groupings independent version history or pinned/floating member binding; it tracks individual element identity. So it warrants the element-identity foundation but leaves the grouping-version layer to us.
- **Preserved disagreement / plurality: conflicting views representable, synthesis late-bound** — WARNS against eager auto-resolution and offers a structured arbitration model. MolhadoRef does NOT silently overwrite: it builds a dependency DAG of operations, detects conflicts (compile + run-time, incl. accidental method overriding) via a predicate matrix, and escalates to user-assisted resolution — late-bound human synthesis exactly where automation can't safely decide. Their commute/conflict/dependence formalism (Defs 1-3) is a usable vocabulary for our preserved-disagreement arbitration: two ops commute (auto-merge), conflict (surface), or depend (order them).

- **Four-level graph versioning: graph / node / relation-type / node-data must compose** — PARTIAL precedent. MolhadoRef versions node identity (element IDs), node-data (method bodies as attributes), and a kind of relation-type change (override relationships, which it specifically guards — accidental override is a run-time conflict). Replay + topological sort is a working example of composing changes across these levels in one merge. It does not formalize a 4-level model, but demonstrates that node-identity, node-data, and relation (override/containment) changes must be merged together with ordering, validating that the levels must COMPOSE.
- **Substrate choices: Rama (event-sourced) as append-only operation log** — SUPPORTS the event-sourced substrate bet at the granularity of code. MolhadoRef IS an operation-based (event-sourced) SCM: 'an SCM tool records the operations that were performed... and replays them.' A version = the sequence of operations used to create it. This is precisely Rama's append-only log + replay/projection model applied to code edits. The paper's cost note (pure operation-based 'can be slow') is a real warning for a Rama-backed code kernel: don't replay everything; checkpoint state (their hybrid).

Takeaway. The single most important contribution: code-unit identity should be a persistent, opaque ID attached to the program element (method/field/class), NOT its file, path, name, or FQN — and the durable way to track that identity across renames/moves is to record changes as DECLARED, first-class operations (refactorings) with well-defined semantics, replaying them rather than reconstructing intent from text diffs. This is direct, dogfooded validation (1 conflict and 0 merge errors vs CVS's 36 conflicts + 48 errors; 73 API methods' history preserved where CVS lost it) of our container/revision kernel and our DECLARED-vs-INFERRED rule. The transferable mechanism for our trail-to-code join is their invert → textual-merge → replay pipeline plus the commute/conflict/dependence formalism (Defs 1-3) for deciding what auto-merges, what must be ordered, and what is escalated to late-bound human synthesis — with the explicit caveat that pure operation replay is slow, so a Rama-backed kernel must checkpoint state (their operation/state hybrid) rather than replay everything.

4. Versioning Your Code with Monticello (Chapter 7, Deep into Pharo / Pharo by Example)

Co-written with Oscar Nierstrasz (Software Composition Group, University of Bern); part of the Pharo book series by Nierstrasz, Ducasse, et al. · 2013 · Book chapter, Deep into Pharo / Pharo by Example (Square Bracket Associates); hosted at scg.unibe.ch

In one line. A detailed how-to and conceptual exposition of Monticello, Pharo's package-granular versioning system, where classes/methods (not text files) are the units of change, packages are intensional containers defined by category/protocol rules, versions are immutable UUID-stamped snapshots with DAG ancestry, and umbrella packages with pinned dependencies act as configurations.

Source access: full. Read the FULL chapter. The WebFetch of the PDF returned only raw binary, so I downloaded the PDF locally and ran `pdftotext -layout` on it, yielding the complete 1012-line chapter (scg.unibe.ch/archive/papers/Nier13bMonticello.pdf). This is a book CHAPTER, not a conference paper — no Abstract/Related Work; I walk its real section headings (7.1 Basic usage through 7.7 Chapter summary).

THE PAPER, SECTION BY SECTION

§ Preamble / Chapter intro (why a code-granular versioning system). The chapter opens by motivating versioning in general (store multiple versions, manage concurrent access, track changes, enable collaboration 'as soon as the size of your software increases beyond a few classes'). Its core argument is that general-purpose text VCS (CVS, Subversion, Git) are the WRONG granularity for Smalltalk: 'CVS-like tools only version plain text files and not individual packages, classes or methods. We would therefore lack the ability to track changes at the appropriate level of granularity.' Because Monticello knows it stores classes and methods rather than lines of text, 'they can do a better job of supporting the development process.' It positions Monticello as the CVS-equivalent and SmalltalkHub/Squeaksource as the SourceForge-equivalent online repositories — establishing the program-unit-as-unit-of-change thesis the whole research angle rests on.

"CVS-like tools only version plain text files and not individual packages, classes or methods."

"If the versioning tools know that you store classes and methods instead of plain text, they can do a better job of supporting

the development process.”

“SmalltalkHub is the equivalent of SourceForge, and Monticello the equivalent of CVS.”

§ 7.1 Basic usage — Creating a package (the container as an intensional object). Defines the central container abstraction. A package is not a folder or file list but a live object: 'a package is an object — an instance of PackageInfo — that knows how to identify the classes and methods that belong to it.' Membership is INTENSIONAL, defined by naming rules rather than enumeration: a package Perfect contains (1) all classes in category Perfect or categories starting with Perfect-, (2) all methods in protocols named *perfect* / Perfect or starting with *perfect-* (*these cross-cutting members are 'class extensions'*), and (3) all methods of those classes *EXCEPT* ones in -prefixed protocols (which belong to other packages). Monticello auto-maps Smalltalk categories to packages. This is the key identity mechanism: the container's contents are computed from a declared rule, so the same identity thread can gain/lose members without the container itself changing identity.

“a package is an object — an instance of PackageInfo — that knows how to identify the classes and methods that belong to it.”

“All methods belonging to any class (in any category) that are defined in a protocol named *perfect* or Perfect... Such methods are known as extensions.”

“Monticello will map categories to packages for us.”

§ 7.1 Basic usage — Committing changes & versions (immutable snapshots, non-global numbering).

Introduces the immutable-revision concept and the repository. A repository is 'a package container' (local or remote); package-cache is a default local directory mirror of everything saved/loaded. The crucial definition: 'A version is an immutable snapshot of a package that has been written to a repository.' Each version gets a number that is LOCAL to a repository, NOT globally unique: 'this number is not globally unique — in another repository you might have the same file identifier for a different snapshot.' Saving auto-assigns the next number but it is editable, and critically 'version branches do not interfere with the numbering scheme (as with CVS or Subversion)' — numbering is a flat per-repo counter, identity/ancestry is tracked separately. Files are named like Perfect-XX.1.mcz where XX is author initials/name.

“A version is an immutable snapshot of a package that has been written to a repository.”

“this number is not globally unique — in another repository you might have the same file identifier for a different snapshot.”

“version branches do not interfere with the numbering scheme (as with CVS or Subversion).”

§ 7.1 Basic usage — Class extensions & clean/dirty state. Develops the cross-cutting-membership idea concretely: methods *isPerfect* and *divisors* are added to the system class *Integer* but placed in a **perfect* protocol, so 'they do belong to the Perfect package since they are in a protocol whose name starts with * and matches the package name. Such methods are known as class extensions.' These extension methods 'will be available only to someone who loads the Perfect package' — membership, not file location, governs what travels with a version. The clean/dirty mechanism defines the relationship between the immutable saved revision and the mutable working state: 'Modifying the code in a package... makes that package dirty. This means that the version of the package in the image is different from the version that has been saved or loaded.' A dirty package shows an asterisk; saving cleans it. This is precisely a container with an immutable current-revision plus a divergence flag.

“they do belong to the Perfect package since they are in a protocol whose name starts with * and matches the package name.”

“This means that the version of the package in the image is different from the version that has been saved or loaded.”

“Saving a dirty package cleans it.”

§ 7.1 Basic usage — Repository inspector, load/unload/update. Describes how installed-vs-repository state is surfaced through typeface conventions in the inspector, which encode the ancestry relationship between the image's current revision and stored revisions: underlined = installed; bold = 'not an ancestor of the installed version' (newer OR a different branch); normal = older. At the package level: underlined name = installed; bold underlined = installed but a newer version exists; normal = not installed. The exercise loads version 1 (reverting), then version 2 (updating), demonstrating that you can move the current-revision pointer freely across immutable snapshots. This

operationalizes 'current-revision = state': loading is just repointing the image at a chosen immutable snapshot, and the UI computes the ancestry relation to tell you whether that snapshot is ahead, behind, or on a sibling branch.

“a bold version name means that this version is not an ancestor of the installed version. This may mean that it is a newer version, or that it belongs to a different branch.”

“an underlined version name means that this version is installed in the image.”

§ 7.1 Basic usage — Branching. Defines branching as divergent lineage over a shared ancestor: 'A branch is a line of development versions that exists independently of another line, yet still shares a common ancestor version if you look far enough back in time.' You branch by basing a new save on an EARLIER version than the latest. The worked example loads v1, makes a different isPerfect implementation, and saving produces v3 as a sibling of v2 (both children of v1). The system detects divergence and warns on conflicting loads ('warning that you have unsaved changes'). This establishes the DAG (not linear) version model and grounds preserved-plurality: two independent valid implementations of the same methods coexist as separate branches, neither overwriting the other.

“A branch is a line of development versions that exists independently of another line, yet still shares a common ancestor version if you look far enough back in time.”

“versions 2 and 3 constitute independent branches.”

§ 7.1 Basic usage — Merging (fine-grained, per-element conflict resolution). Describes merge as re-integration of branches with method/class-level granularity — the heart of the program-unit-as-unit-of-change claim. 'The merge tool... allows for fine-grained package version merging'; differences are shown per element (a new method, two changed methods), with source-level diff coloring (new=red, removed=struck blue, unchanged=black). Conflict is defined precisely: 'A method or a class is in conflict if its definition has been altered.' Resolution is per-element and human-decided via typeface/arrow conventions: right-arrow = repository replaces current, left-arrow = repository version rejected, red = conflicting, requiring 'Keep current version or Use incoming version.' The merged version records BOTH source versions as ancestors. This shows late-bound, granular synthesis where disagreement is preserved up to the merge decision and resolved per unit, not per file.

“The merge tool is a tool that allows for fine-grained package version merging.”

“A method or a class is in conflict if its definition has been altered.”

“The merged version will have as its ancestor the two versions it was merged from.”

§ 7.2 Exploring Monticello repositories — Browse, Changes & History. Covers three inspection tools (folding in the 7.3 History item, which is conceptually identical). The Browse button opens a 'snapshot browser' whose advantage over the normal browser 'is its ability to display class extensions' — it shows the package's full intensional contents including cross-cutting extension methods on foreign classes, which the ordinary class browser hides; best practice is to 'always browse the code of your package before publishing it, to ensure that it really contains what you think it does' (an admission that intensional membership can surprise you). The Changes button 'computes the difference between the code in the image and the most recent version of the package in the repository,' opening a patch browser of changed/new methods — the inferred-diff tool (vs the declared snapshot). The History item opens a version-history viewer listing every version of a package with its commit comments, and lets you per-version explore changes relative to the working copy or 'spawn a new history browser relative to the selected version' — each container carries its own ordered, annotated revision log.

“The advantage of the snapshot browser over the browser is its ability to display class extensions.”

“The Changes button computes the difference between the code in the image and the most recent version of the package in the repository.”

“It opens a version history viewer that displays the comments committed along with each version of the selected package.”

“always browse the code of your package before publishing it, to ensure that it really contains what you think it does.”

§ 7.3 Advanced topics — Dependencies & umbrella/configuration packages. The most important section for our groupings-as-versioned-objects question. Monticello declares required packages so they load in dependency order, recursively, leaves-first. Critically, version pinning auto-tracks: 'Whenever new versions of required packages

are checked in, then new versions of the packages that depend on them will automatically depend on the new versions.' Hard limit: 'Dependencies cannot be expressed across repositories.' The umbrella pattern (Pier-All) is named explicitly: 'an empty package that acts as a kind of umbrella' that requires all real packages, so installing it installs everything and 'the only package that needs to be saved is Pier-All; all dependent dirty packages are saved automatically.' A worked refactor splits Perfect into NewPerfect-Tests + NewPerfect-Extensions under umbrella NewPerfect-All. The failure mode is stressed: committing a member directly (NewPerfect-Tests) instead of through the umbrella 'effectively break[s] the dependency... Don't do it!' — plus a naming caveat against suffix-overlapping package names. The chapter recommends graduating to a Metacello configuration (the +Config button, Chapter 9) for 'more flexible dependencies.'

"Package Pier-All is an empty package that acts as a kind of umbrella. It requires Pier-Blog, Pier-Caching and all the other Pier packages."

"Whenever new versions of required packages are checked in, then new versions of the packages that depend on them will automatically depend on the new versions."

"Dependencies cannot be expressed across repositories. All requiring and required packages must live in the same repository."

"If instead you save NewPerfect-Tests, this will not cause NewPerfect-All to be saved. This is bad because you effectively break the dependency."

§ 7.3 Advanced topics — Class initialization. Explains a load-time hook: 'any class that defines an initialize method on the class side will be sent the initialize message' when the package loads — only to classes that define it themselves (not inherited), and 'not invoked when you merely reload a package.' Its flagship use is working around a limitation of class extensions: extensions can only ADD METHODS, not instance variables, so a class-side initialize is used to add an instance variable at load time (e.g. adding lastRedRun to TestCase). A side-effect noted: extending/altering a class you don't own dirties ITS package ('After installing TestCaseExtension, the package SUnit will become dirty'). This reveals a subtlety relevant to our membership model — an extension lives in one package but mutating a foreign class can mark the foreign owner's package dirty, i.e. cross-container side effects on shared nodes.

"any class that defines an initialize method on the class side will be sent the initialize message."

"Class extensions are strictly limited to adding new methods to a class."

"if you change a class that is not in your package, the other package will become dirty."

§ 7.4 Getting a change set from two versions (snapshot vs patch). Draws the explicit distinction between a full snapshot and a derived patch — directly the declared-vs-inferred axis. 'A Monticello version is the snapshot of one or more packages. A version contains the complete set of class and method definitions.' Against this, 'A patch is the set of all necessary side effect in the system to go from one version A to another version B.' Pharo change sets are the patch mechanism (global side effects on the system). The technique to capture a diff is procedural and inference-based: create an empty change set, then load v2 while v1 is loaded, and the change set records the delta, which can be filed out to a DiffPerfect.X.cs file. This confirms Monticello's stance: store complete declared snapshots as truth; derive patches by replaying loads and recording side effects.

"A version contains the complete set of class and method definitions that constitute the underlying packages."

"A patch is the set of all necessary side effect in the system to go from one version A to another version B."

§ 7.5 Kinds of repositories. Enumerates repository backends, each a different physical store for the same immutable-snapshot abstraction, with access modes (read-only/write-only/read-write) and optional per-user rights. HTTP (most popular, used by SmalltalkHub, web-browsable/WebDAV-able, runnable on plain Apache); FTP (similar over FTP); GOODS (stores versions in a distributed OO database with transactions/journaling/replication, 'suitable for large repositories used by many clients'); Directory (local filesystem dir, the package-cache kind, only option for disconnected development, versions later copyable to shared repos via SmalltalkHub's Import Versions); Directory-with-Subdirectories (hierarchical variant); SMTP (write-only, emails versions to an address/mailling-list). It also shows programmatically adding repositories via MCDirectoryRepository / MCRepositoryGroup. Takeaway for us:

version/snapshot identity is independent of the storage substrate — the same .mcz can live across many backend types.

“Repositories can be read-only, write-only or read-write. Access rights may be defined globally or can be tied to a particular user.”

“A directory repository stores versions in a directory in the local file system... it's the only option for disconnected development.”

§ 7.6 The .mcz file format (what an immutable version physically is). Reveals the on-disk structure of a version — concretely grounding the identity model. An .mcz is just a ZIP ('Monticello zip') containing four conceptual things: (1) package — name info; (2) VersionInfo — metadata including 'the author initials, date and time the snapshot was taken, and the ancestry of the snapshot'; (3) Snapshot — 'a record of the state of the package at a particular time' under a snapshot/ dir (currently a single source.st fileout); (4) Dependencies — a dependencies/ dir with one member per required package. The pivotal statement on identity/ancestry: 'A version doesn't contain a full history of the source code. It's a snapshot of the code at a single point in time, with a UUID identifying that snapshot, and a record of the UUIDs of all the previous snapshots it's descended from.' Metadata is S-expression key/value (nestable dicts); an example version file shows name/message/date/time/author/ancestors. This is exactly a content-snapshot + UUID identity + UUID ancestry-DAG — our immutable-revision-inside-a-container made physical.

“A version doesn't contain a full history of the source code. It's a snapshot of the code at a single point in time, with a UUID identifying that snapshot, and a record of the UUIDs of all the previous snapshots it's descended from.”

“VersionInfo... contains the author initials, date and time the snapshot was taken, and the ancestry of the snapshot.”

“an mcz file is simply a zipped file containing the source code and other meta-data.”

§ 7.7 Chapter summary. Recaps the model: packages map to Smalltalk categories + method protocols (class extensions via *foo protocols anywhere in the system); editing marks a package dirty/saveable; many repository kinds exist (HTTP via SmalltalkHub most common); saved packages cache to package-cache; the repository inspector browses and selectively loads/unloads versions; branches are created by basing a new version on an earlier-than-latest one and the inspector tracks ancestry to identify separate branches; branches merge with fine-grained per-conflict control and the merged version records both parents; dependencies are tracked so saving a depending package mints a new version pinned to latest required versions; class-side initialize runs at load (notably to add instance variables for extensions); and versions are stored as .mcz zips holding a complete source snapshot plus metadata/dependencies. The summary reaffirms the program-unit (class/method/package) — not the text file — as the unit of change throughout.

“You can create a new branch of a package by basing a new version on another version which is earlier than the latest version.”

“The merged version will have as its ancestor the two versions it was merged from.”

“The mcz file contains a snapshot of the complete source code of that version of your package, as well as files containing other important metadata.”

HOW IT MAPS TO OUR PARTS

- **Object-container kernel (stable container = identity thread; revisions = immutable values; current-revision = state)** — GIVES IT A NAME + supporting precedent. Monticello's PackageInfo IS exactly our stable container: 'a package is an object — an instance of PackageInfo — that knows how to identify the classes and methods that belong to it.' A version is 'an immutable snapshot of a package' — our immutable revision. The image's loaded copy (clean/dirty) is the current-revision/state. The container is intensional (a membership rule over categories/protocols), not a frozen list — directly models a container whose identity persists while its contents change. CAVEAT for our hard rule 'inferred structure is never accepted as truth at ingest': Monticello's membership is a *declared* rule (category prefix + *protocol pattern), so what belongs is computed deterministically from declared metadata, not inferred — aligns with our rule only if the rule itself is treated as declared truth.

- **The code-identity question (is the unit a file or a function? identity = path/name/AST/hash?)** — OFFERS A DIRECT ANSWER and a warning. Monticello's whole thesis is that the file is the WRONG unit: 'CVS-like tools only version plain text files and not individual packages, classes or methods. We would therefore lack the ability to track changes at the appropriate level of granularity.' Identity here is the NAME/FQN: package = name + membership rule; method identity = Class>>selector in a named protocol; class identity = its name. NOT file, NOT path, NOT AST, NOT content-hash. Supports our instinct that the function/method (not the file) is the atomic unit, but warns that name-based identity breaks under rename (Monticello sees a renamed method as delete+add — pure INFERRED-by-diff, no declared identity continuity).
- **Versioning as its own layer; two times (arrival-time vs committed/minted-time)** — SUPPORTS the two-times split and shows a pure version-2 of it. Monticello has NO arrival-time substrate — every version is an explicit, human-MINTED commit ('the next available number is automatically assigned'). VersionInfo records 'date and time the snapshot was taken, and the ancestry' — committed-time + DAG ancestry. This is full-persistence (branch from any node) with zero partial-persistence. Validates our committed/minted layer design but shows the gap our Rama substrate fills: Monticello cannot answer 'what was the code when this trail-event arrived' because it has no arrival log — only discrete human commits. Our two-times model is strictly more expressive.
- **Groupings as versioned objects with their own history, member binding pinned/floating** — OFFERS A CONCRETE MECHANISM (umbrella package) AND a failure mode. Pier-All / NewPerfect-All is 'an empty package that acts as a kind of umbrella' — a grouping with its OWN version history, partly independent of members. Member binding is explicitly version-PINNED: 'a new version of NewPerfect-All which then requires the new version of NewPerfect-Tests... will also depend on the existing, unmodified version of NewPerfect-Extensions.' That is per-member effectivity (some pinned-updated, some pinned-frozen) — exactly our pinned-vs-floating distinction. FAILURE MODE is loud: 'If instead you save NewPerfect-Tests, this will not cause NewPerfect-All to be saved... you effectively break the dependency. Don't do it!' — a grouping whose member pins silently desync when you commit a member directly. Warns our grouping layer needs commit-through-the-container discipline.
- **Four-level graph versioning (graph / node / relation-type / node-data must compose)** — PARTIAL precedent + shows the missing levels. Monticello composes exactly TWO of our four levels: node-DATA (method/class source defs inside a snapshot) and GROUPING/graph (package version aggregating member versions, umbrella aggregating packages). It has NO relation-TYPE versioning — the only 'relation' is the untyped 'requires' dependency, and dependencies 'cannot be expressed across repositories,' a hard composition boundary. So Monticello demonstrates that data-level + container-level versioning DO compose (umbrella depends on pinned member versions), but its lack of typed-relation versioning is precisely the gap our four-level model must fill — a worked example of the two easy levels and a blank where the relation-type level should be.
- **Trail-to-code join: declared change vs inferred change** — GIVES US THE EXACT VOCABULARY. Monticello cleanly separates the two: the SNAPSHOT (declared full state, 'the complete set of class and method definitions') vs the PATCH/change-set (inferred diff, 'A patch is the set of all necessary side effect in the system to go from one version A to another'). Section 7.4 shows the diff is RECONSTRUCTED by loading v1, recording, loading v2 — pure inference. The merge tool's red/blue/struck typefaces ARE diff-inference made visible. For our trail-join: store the DECLARED snapshot as truth and compute the INFERRED patch on demand — never the reverse. Directly supports our rule that the agent's declared edit-op is primary and diff-reconstruction is secondary.
- **Preserved disagreement / plurality; late-bound synthesis** — OFFERS A MECHANISM (branch + fine-grained merge) for late-bound synthesis. Branches are first-class independent lines ('a line of development versions that exists independently of another line, yet still shares a common ancestor'). Conflict is preserved, not auto-resolved: 'A method or a class is in conflict if its definition has been altered'; resolution is per-element, deferred to a human ('Keep current version or Use incoming version'), and the merged version records BOTH parents as ancestors. Exactly our preserved-disagreement-then-late-synthesis pattern at method granularity.

The limit: Monticello forces resolution AT merge time (you must pick); it cannot hold an unresolved conflict as a durable representable object the way our plurality layer wants.

- **Substrate choices (Rama event-sourced log vs discrete commits)** — CONTRASTS / clarifies our substrate bet. Monticello is a pure commit-graph substrate: discrete human-minted snapshots, DAG ancestry by UUID, no event log between commits, no time-travel except to committed nodes. 'A version doesn't contain a full history of the source code. It's a snapshot of the code at a single point in time, with a UUID... and a record of the UUIDs of all the previous snapshots it's descended from.' This is the full-persistence half of our design WITHOUT the arrival-time half. Confirms that a UUID-keyed immutable-snapshot + ancestry model is sound and battle-tested, and that our addition (Rama arrival log underneath the commit graph) is the genuinely new contribution, not a reinvention.

Takeaway. Monticello is a working, decades-proven existence proof of our object-container kernel at code granularity: a package is a stable INTENSIONAL container (a PackageInfo membership rule over names/protocols, not a file list) whose identity persists while its immutable UUID-stamped snapshot revisions accumulate a DAG ancestry — and it validates two of our hardest design moves, (a) name/protocol-based unit identity that lets cross-cutting 'class extensions' travel with the right container regardless of file location, and (b) the umbrella/configuration package as a separately-versioned grouping whose members are pinned per-version. But it equally maps our frontier: it has ONLY committed/minted time (no arrival-time log — the gap our Rama substrate fills), only untyped same-repo 'requires' relations (no relation-TYPE versioning level), and name-based identity that silently degrades to delete+add under rename — and it broadcasts the exact failure mode to avoid (commit a member directly instead of through its umbrella and you silently desync the pins: 'Don't do it!')."

What we can gather — Angle 1

The four systems triangulate a single answer with three variants. **The atomic unit of code identity is the program element — the function, method, or definition — not the file**, and the identity must be a *stable handle decoupled from the mutable name and path*. Every one of them rejects file-granularity explicitly (Monticello: "we would lack the ability to track changes at the appropriate level"; Molhado critiques RCS's "smallest unit is a file"; Dolt later re-bases git from files to rows for the same reason). They differ only in **how the stable handle is minted**, and that difference is the whole design axis:

- **Unison derives it from content** — the hash of the normalized AST. Rename becomes a metadata edit by definition; the diamond-dependency problem evaporates; plural definitions coexist under one name. Maximal purity. The cost: it has *no* arrival-time axis and addresses code by hash, not path+timestamp — which is the exact key our trail-to-code join needs.
- **Molhado/MolhadoRef mint an opaque persistent ID** and demote the name to a *versioned attribute*. This is the closest system to our kernel, and it carries the strongest evidence in the whole angle: persistent IDs preserved the history of 73 renamed/moved API methods that CVS lost (1 merge conflict vs CVS's 36 + 48 errors). And it proves the principle our kernel already holds — **declared operations beat inferred diffs**: refactorings are recorded as first-class operations by the engine that performed them, never reconstructed from text.
- **Monticello uses an intensional container** (a membership rule over names/protocols) with name-based identity, battle-tested for two decades — and it shows the failure mode of name-based identity in the open: a rename silently degrades to delete+add, because nothing carries the identity across the name change.

So the convergence is sharp. The container/revision split is validated four independent times. The function is the unit; **the file is an assembly that pins function versions** — which is already the bridge to Angle 5, before we even get there. And the decisive practical guidance: adopt **opaque-ID identity (Molhado), not content-hash identity (Unison)** — because we need the path+timestamp trail resolution and the arrival-time history that content-addressing actively destroys — while keeping a content hash as a *derived* attribute for dedup and for the exactness

flag. The one piece Unison has that we would otherwise lack is the **namespace-version: the name/path → identity table, snapshotted as-of a moment** — which is exactly what we must keep so a trail's recorded path still resolves later.

Angle 2 — Version-Model Theory: What *Is* Versioning, Formally?

5. Version Models for Software Configuration Management

Reidar Conradi (NTNU Trondheim) and Bernhard Westfechtel (Aachen University of Technology) · 1998 · ACM Computing Surveys, Vol. 30, No. 2

In one line. The canonical taxonomy of software versioning: it separates product space from version space, extensional from intensional, state-based from change-based, and binds them together through AND/OR graphs and configuration rules.

Source access: full. ACM (dl.acm.org) returned HTTP 403. Read the FULL original PDF (51 pages, Acrobat Distiller 1998) from an open university mirror: <https://www.kiv.zcu.cz/~brada/files/aswi/.../conradi98version-models.pdf> — extracted locally with pdftotext -layout and read line-by-line. All quotes below are verbatim from that authoritative PDF (pages 232-282), not from the lossy WebFetch summary.

THE PAPER, SECTION BY SECTION

§ Abstract + 1. Introduction. After 20+ years of SCM, building consistent configurations of versioned products remains hard; the survey classifies the VERSION MODELS underlying commercial systems and research prototypes, defines the four fundamental concepts (revisions, variants, configurations, changes), and foregrounds INTENSIONAL versioning (construction from configuration rules) as the hard center. The Introduction installs the paper's organizing dichotomy: a product lives in a PRODUCT SPACE (objects + relationships, ignoring versions) and a VERSION SPACE (how versions are organized); a VERSIONED OBJECT BASE combines them. A version model is defined by how the version space is structured, which objects are versioned externally vs internally, how different objects' version spaces relate, and how old/new versions are reconstructed/constructed. Notes the field's curse — every system reinvents terminology over near-identical concepts — so the contribution is a UNIFIED terminology plus a canonical graph formalism (nodes/edges/attributes). Surveys representational families: file-based, language-based (module interconnection languages), and database-founded (EER, OO, deductive).

“we focus on intensional versioning, that is, construction of versions based on configuration rules”

“Software objects and their relationships constitute the product space, their versions are organized in the version space. A versioned object base combines product and version space.”

“each system comes with its own terminology. On the other hand, the underlying concepts are often very similar.”

“A version model defines the objects to be versioned, version identification and organization, as well as operations for retrieving existing versions and constructing new versions.”

§ 2. Product Space (Objects, Relationships, Representations). Describes product structure WITHOUT versioning. The product space is a product graph; nodes = software objects, edges = relationships. Every object carries an OBJECT IDENTIFIER (OID): an external OID is a user-assigned name; a system-generated unique OID may be used internally — the identity primitive. Objects are coarse-grained but internally structured (a module = declarations + statements; fine-grained content held as 'long attributes'). Two relationship kinds: COMPOSITION (subsystem → module → leaf; composite objects = configurations, leaves = atomic objects) and DEPENDENCY (orthogonal: import/include, lifecycle, build), each with a dependent that must be kept consistent with its master. Objects split into SOURCE (human-authored) vs DERIVED (tool-produced). Representation choice (text files vs syntax trees/graphs) changes what 'diff' even means — differing text lines vs differing syntactic units. Distinguishes domain-independent vs domain-specific product models.

“each software object carries an object identifier (OID) that serves to identify it uniquely within a certain context. An external OID is a name assigned by the user, whereas a system-generated, unique OID may be used internally.”

“Objects that are decomposed are called composite objects or configurations. Objects residing at the leaves of the

composition hierarchy are denoted atomic objects.”

“a diff command... returns differing text lines in the case of text files and differing syntactic units in the case of syntax trees”

§ 3.1 Versions, Versioned Items, and Deltas. The core identity machinery. A version v is a state of an evolving item i , formally $v=(ps,vs)$: a point in product space AND a point in version space. The ITEM is anything put under version control at ANY granularity. A VERSIONED ITEM is ‘a container for a set V of versions’ — exactly Softland’s object-container split. Versioning requires a SAMENESS CRITERION (decide two versions belong to the same item), provided by a unique identifier (OID). Within an item each version has a VERSION IDENTIFIER (VID): auto-number, symbolic name, or an expression (the intensional scheme). All versions share INVARIANTS; they differ in specific properties, and the difference is a DELTA — either SYMMETRIC (set-difference of properties) or DIRECTED / CHANGE (a sequence of operations $op1...opm$ transforming $v1$ into $v2$, explicitly likened to database transaction logs). Warns deltas are not always small; commonality erodes as items evolve, with the escape being MULTILEVEL / RECURSIVE versioning (a version may itself have versions — Adele interfaces realized by body versions; DAMOKLES generalizes to fully recursive).

“ v is characterized by a pair $v = (ps, vs)$, where ps and vs denote a state in the product space and a point in the version space, respectively.”

“A versioned item is a container for a set V of versions.”

“Versioning requires a sameness criterion; that is, there must be some way to decide whether two versions belong to the same item.”

“a directed delta, also called a change, is a sequence of (elementary) change operations $op1...opm$ which, when applied to one version $v1$, yields another version $v2$ (note the correspondence to transaction logs in databases).”

§ 3.2 Extensional and Intensional Versioning. The pivotal distinction for Softland’s two-times problem. EXTENSIONAL versioning enumerates the set $V = \{v1,...,vn\}$: every version was explicitly checked in once, is retrievable, and is typically made IMMUTABLE (at check-in or via explicit freeze), guaranteeing safe reconstruction. INTENSIONAL versioning defines the set by a predicate $V = \{v \mid c(v)\}$: versions are implicit, constructed ON DEMAND, identified by an expression/query (a tuple of attribute values mapped by q : $A1 \times ... \times An \rightarrow V$). Illustrative contrast: SCCS/RCS store-and-reconstruct text files (extensional) vs C conditional compilation building any source from preprocessor variable values (intensional). The paper insists these are NOT mutually exclusive and should be combined in one system.

“Extensional versioning means that V is defined by enumerating its members: $V = \{v1,...,vn\}$.”

“To ensure safe retrieval of previously constructed versions, versions can be made immutable.”

“Instead of enumerating its members, the version set is defined by a predicate: $V = \{v \mid c(v)\}$.”

“extensional and intensional versioning are by no means mutually exclusive, but can (and should) be combined into a single SCM system.”

§ 3.3 Intents of Evolution: Revisions, Variants, Cooperation. Defines the three orthogonal INTENTS behind making a version. A REVISION is intended to SUPERSEDE its predecessor, evolving along the TIME dimension (bug fixes, enhancements, library adaptation); old revisions are preserved rather than overwritten. A VARIANT is intended to COEXIST with siblings — parallel alternatives (OS, window system, storage/runtime tradeoffs). A COOPERATION version supports parallel developers, each in a WORKSPACE governed by import/export policies. This revision/variant separation is the conceptual spine of the version space.

“A version intended to supersede its predecessor is called a revision. Revisions evolve along the time dimension”

“Versions intended to coexist are called variants.”

“Instead of performing modifications by overwriting, old revisions are preserved”

§ 3.4 Representations of the Version Space: Version Graphs and Grids. How extensional version sets are drawn. A VERSION GRAPH has nodes (groups of versions) and edges (relationships); it presupposes an explicit set, so it pairs with EXTENSIONAL versioning. One-level uses a single SUCCESSOR relation in three shapes: sequence, tree, acyclic graph (multiple predecessors = merge). Two-level (RCS) adds BRANCHES with successor + offspring; ClearCase RECORDS MERGES, yielding a DAG where branches keep existing rather than joining. The key failure mode: version graphs handle variants only weakly — multidimensional variation causes

COMBINATORIAL BRANCH EXPLOSION bounded by the product of domain cardinalities (foo: OS×DB×window = up to 27 branches). Escapes: classification-hierarchy clusters, or an n-dimensional GRID (Sciore) with an added orthogonal time dimension (Reichenberger).

“Since a version graph assumes an explicitly given set of versions, it is applied in conjunction with extensional versioning.”

“In the case of multidimensional variation, this approach breaks down because the number of branches explodes combinatorially.”

“ClearCase goes beyond the RCS organization by recording merges in the version graph.”

§ 3.5 State-Based and Change-Based Versioning. The SECOND great axis, declared explicitly ORTHOGONAL to extensional/intensional. STATE-BASED models describe a version by the STATE it represents (revisions + variants); changes are anonymous and must be DEDUCED from version-graph topology. CHANGE-BASED models describe a version by the CHANGES applied to a baseline; changes get Change Identifiers (CIDs) and attributes, giving a clean link to change requests. Change-based comes in extensional (change packages — documentation only) and intensional (the change-set model, Aide-de-Camp; changes freely composed as partial functions $c: V \rightarrow V$; $v = c_1 \circ \dots \circ c_n(b)$). The change space is drawn as an option grid (COV), a matrix (Aide-de-Camp), or a version graph with explicit change-labeled edges — which 'provides more information' than anonymous state-based graphs by recording WHEN each change applied across versions.

“Version models that focus on the states of versioned items are called state-based.”

“In change-based models, a version is described in terms of changes applied to some baseline... changes are assigned change identifiers (CID)”

“State- versus change-based is orthogonal to extensional versus intensional.”

“state-based versioning does not name the changes; that is, the changes are anonymous. As a consequence, the changes applied to a version must be deduced from the topology of the version graph.”

§ 4.1 AND/OR Graphs + Selection Order + Version Proliferation. The unifying mechanism JOINING product space and version space (Tichy 1982). Two node types: AND nodes (an object's specific versions/structure) and OR nodes (the versioned object = choice among versions); an unversioned product graph is all-AND, versioning adds OR nodes. AND edges carry both composition and dependency; an edge is BOUND if it ends at an AND node (specific version) and GENERIC if it ends at an OR node (unresolved choice). A CONFIGURATION is the subgraph reachable from a root; BOUND if all AND edges are bound, GENERIC otherwise (partially/totally generic). Binding = eliminating OR nodes by selecting one successor each — the precise vocabulary for pinned-vs-floating member binding. Models are classified by SELECTION ORDER: PRODUCT FIRST (SCCS/RCS, cannot express structural versioning), VERSION FIRST (PCTE/POEM, product version determines structure), INTERTWINED (ClearCase/Adele, supports structural versioning); the latter two version RELATIONSHIPS too. Cardinal warning: version-first triggers VERSION PROLIFERATION — a leaf bug-fix forces new versions of every ancestor — cured only by distinguishing versions of modules from versions of configurations.

“AND/OR graphs provide a general model for integrating product space and version space.”

“A relationship is bound to a specific version if the corresponding AND edge ends at an AND node; otherwise it is called generic.”

“in addition to objects, relationships are versioned as well.”

“in (b) new versions of all modules above c have to be created... This effect is called version proliferation... we have to distinguish between versions of modules and versions of configurations.”

§ 4.2 Granularity of Versioning (Component / Total / Product). The four-level / multilevel versioning core. Distinguishes EXTERNAL granularity (what the user sees versioned) from INTERNAL (much finer). Three external schemes: COMPONENT VERSIONING (only atomic leaves versioned; each has its own version graph; composition model; component graphs often only WEAKLY related, creating a selection problem); TOTAL VERSIONING (ALL composition levels versioned; composite objects are explicit first-class versioned entities; atomic+composite versioned UNIFORMLY; supports structural versioning — PCTE 'version first', ClearCase 'intertwined'); PRODUCT VERSIONING (all object versions in ONE global version space, a transparent single-version view — VOODOO, all change-based systems). Product versioning's global-view strength is also its

weakness: NO MODULARITY — every change/revision/variant is global. Also separates VERSION granularity (size of a version) from DELTA granularity (unit deltas are recorded in) — RCS versions files but deltas at text-line level.

“Component versioning means that only atomic objects are put under version control. Each object has its own version space”

“Total versioning applies to all levels of the composition hierarchy.”

“Product versioning differs from total versioning by arranging versions of all objects in a uniform, global version space.”

“A potential problem consists of lacking modularity of the version space. All changes, revisions, and variants are global.”

§ 4.3 Deltas (Physical/Logical, Directed/Symmetric, Interleaved). Implementation of deltas, kept ORTHOGONAL to the external version model. Separates PHYSICAL deltas (storage/runtime efficiency) from LOGICAL deltas (user-facing difference). DIRECTED deltas (Tichy/RCS) reconstruct by applying a change sequence to a base (RCS: backward on trunk, forward on branches). EMBEDDED deltas store all versions overlapping with shared fragments — either each version points to its fragments, or fragments carry CONTROL EXPRESSIONS marking which versions they appear in (INTERLEAVED deltas — SCCS, DSEE; lines tagged with their version set). Explicit claim: internal delta representation is orthogonal to external model — interleaved deltas can realize extensional OR intensional, state OR change-based. AND/OR graphs are themselves classifiable as embedded deltas; adding control expressions to constrain combinability turns them into an interleaved scheme.

“the internal delta representation is orthogonal to the external version model.”

“the fragments are decorated with control expressions for determining the versions in which they are visible (interleaved deltas)”

“Since versions are stored in an overlapping manner, AND/OR graphs can be classified as embedded deltas.”

§ 4.4 Relations Between Version Model and Data Model. Three architectural placements of versioning relative to the data model — directly relevant to whether Rama owns versioning or layers it. (1) VERSION MODEL ON TOP OF DATA MODEL: versioning is an ordinary DB application, data model unaware (PCTE, CoMa); simple but the DB can't store versions efficiently and the transaction manager ignores versioning, forcing awkward separate types for versioned-objects vs versions. (2) VERSION MODEL BUILT INTO DATA MODEL: data-definition language has versioned-object constructs, queries are version-aware (DAMOKLES, Adele); Adele uses ONE object type with common/modifiable/immutable attribute classes (updating an immutable attr mints a new revision). (3) DATA MODEL ON TOP OF VERSION MODEL: an instrumentable version ENGINE provides base delta storage + config rules, fully ORTHOGONAL to the data model so any data model plugs in (ICE, COV) — the most decoupled.

“the version model is completely orthogonal to the data model. This may be achieved through an instrumentable version engine that provides a basic delta storage and configuration rules by means of which specific version models may be expressed”

“Adele distinguishes between common attributes shared by all revisions, modifiable attributes whose values are revision-specific, and immutable attributes, where each update triggers the creation of a new revision.”

§ 5. Intensional Versioning (Combinability, Framework, Rules, Configurators, Merge). The combinatorial heart. Intensional versioning must reconcile COMBINABILITY (any version on demand) with CONSISTENCY CONTROL. The blowup is quantified: m modules in v versions $\rightarrow v^m$ configs; unconstrained change combination $\rightarrow 2^c$ configs. Change-based reduces the product-space problem but MOVES selection into the version space. Framework: a CONFIGURATION RULE has a PRODUCT PART (scope) + VERSION PART (selection), written $p:v$; a QUERY is a set of submitted rules; a CONFIGURATOR evaluates query against versioned-object-base + rule-base, satisfying version AND product constraints. Binding has a TIME axis: STATIC, DYNAMIC, or deferred to RUNTIME (Java applets). Rules are scoped (local/global/transitive-closure) and ordered by STRICTNESS: CONSTRAINT (mandatory) > PREFERENCE (optional if satisfiable) > DEFAULT (weakest). Configurators span paradigm $\times$ model into four cells, with state/rule-based using depth-first BACKTRACKING. MERGE tools combine versions/changes (raw / two-way / three-way-with-baseline), by semantic level (textual / syntactic / semantic / operation-based), distinguishing INCONSISTENCY (an op fails) from CONFLICT (contradictory non-commuting changes); conflicts can be pre-empted in version space via mutual-exclusion constraints $c1\vee c2$ (COV).

“a configuration rule consists of a product part, which determines its scope in the product space, and a version part, which performs a selection in the version space”

“A constraint is a mandatory rule that must be satisfied... A preference is an optional rule... a default is also an optional rule, but is weaker than a preference”

“the configurator has to backtrack from the wrong selection.”

“conflict detection can be performed in the version space... Conflicting merges can be excluded by constraints of the form $c1 \nabla c2$ (mutual exclusion).”

§ 6. Version Models in SCM Systems (Taxonomy + 24 systems). Surveys 20+ influential systems via an EVOLUTION GRAPH and two orthogonal classification tables, insisting the taxonomy's value is SEPARATING aspects systems conflate (change-based $\neq$ intensional $\neq$ product-level). Concrete anchors: conditional compilation (fine-grained intensional, interleaved deltas, functional), SCCS (tree, hidden interleaved deltas), RCS (directed deltas, trunk+branches, symbolic names = threads through version graphs), PIE (1980 change-based layers/contexts — change-based is OLD), Gandalf/Adele (separate immutable interface from realization variants/revisions; constraints/preferences/defaults), DSEE → ClearCase (config thread, derived-object cache, virtual file system, dynamic binding, merge recording), COV/EPOS (change-oriented, option grid, ambition/choice = write/read filters), ICE (feature logic; configuration = unification of feature terms, inconsistency = failed unification), VOODOO (orthogonal revision/variant/object dimensions), DAMOKLES (recursive versioning built into the data model). Repeatedly shows the SAME duality realized both ways: change-based on top of version graphs (Asgard) AND version graphs on top of change-based rules (COV/ICE).

“an important contribution of this article is to distinguish among different aspects that used to be mingled together.”

“symbolic names define threads through the version graphs and make reconstruction of configurations easier (RCS)”

“A configuration is inconsistent if unification fails (empty intersection of value sets...) (ICE)”

§ 7. Related Work & Disciplines + 8. Conclusion. Reviews prior surveys (Tichy 1988; Feiler 1991's four models — checkout/checkin, composition, long-transaction, change-set — critiqued as non-orthogonal; Katz 1990, limited to version-graphs + extensional). Maps SCM to neighbors load-bearing for Softland: TEMPORAL DATABASES record evolution but cover only time (no variants/changes) and distinguish VALID TIME vs TRANSACTION TIME — a distinction the authors call irrelevant for SCM because software objects aren't real-world referents (Softland disagrees: arrival-vs-commit IS that distinction). SCHEMA VERSIONING (versioning the rule-base/metadata, rarely taken seriously). DEDUCTIVE DATABASES 'urgently needed' for intensional versioning. LONG TRANSACTIONS (ACID doesn't survive them; versions crucial to cooperation). Current systems handle only coarse dependencies and lack FINE-GRAINED traceability. The Conclusion argues real systems are CONVERGING (change-based built on version graphs and vice versa), forecasts a UNIFORM CUSTOMIZABLE model integrating both axes plus revisions+variants+source+derived, and predicts versioning will be PULLED OUT of SCM systems and MOVED INTO database systems, powered by a deductive DB.

“Temporal databases focus solely on the time dimension and cover neither variants nor change-based versioning.

Furthermore, they often distinguish between valid time... and transaction time”

“Deductive capabilities are urgently needed for intensional versioning.”

“their underlying version models are converging... change-based versioning has been realized on top of version graphs and vice versa.”

“versioning can be pulled out of SCM systems and moved into database systems... This framework... must be customizable to suit the needs of a specific application.”

HOW IT MAPS TO OUR PARTS

- **Object-container kernel (stable CONTAINER = identity; REVISIONS = immutable values; current-revision = state)** — Gives our kernel its exact academic name and formalism. Section 3.1: 'A versioned item is a CONTAINER for a set V of versions' — our container is their versioned item. Their three-part identity machinery validates ours: identity needs a SAMENESS CRITERION (our container thread) provided by an OID (Section 2.1), each value needs a VID, and immutability is enforced at check-in or via explicit freeze (Section 3.2 = our immutable revisions). Adele's Figure-13 pattern is the sharpest mechanism: ONE object type with

common/modifiable/IMMUTABLE attribute classes where 'each update triggers the creation of a new revision' — precisely current-revision-as-state with copy-on-write minting. SUPPORTS our kernel and supplies tested vocabulary.

- **The code-identity question (file vs path vs FQN vs AST vs git-hash; atomic unit = file or function?)** — Directly addresses it and refuses a single answer — itself the lesson. Section 3.1: 'Versioning can be applied at ANY level of granularity, ranging from a software product down to text lines,' and the ITEM is 'anything that may be put under version control' (file, directory, entity, relationship, attribute). Section 2.1 separates external OID (name/path) from internal system OID, warning the name is not the identity. Section 4.2 splits EXTERNAL granularity (file/function) from INTERNAL (much finer). And representation changes identity comparison: text-file diff = differing lines, syntax-tree diff = differing syntactic units (Section 2). So 'file or function' resolves as: choose external granularity per workflow, keep internal finer, and make the OID/sameness-criterion explicit rather than inferring it from the path. WARNS against conflating name with identity — reinforcing our 'inferred structure is never accepted as truth at ingest' rule.
- **Versioning as its own layer; two distinct times (arrival-time history vs committed/minted versions)** — The paper's spine and our strongest borrow. Extensional vs intensional (3.2) maps onto our two times: Rama's arrival-time log = EXTENSIONAL (enumerated, immutable, reconstructable — partial persistence / 'when it arrived'), git-style minted branchable versions = the COMMITTED layer on top. The emphatic ruling — 'extensional and intensional are by no means mutually exclusive, but can (and SHOULD) be combined' — green-lights both times in one system. Section 4.4 gives the architecture decision: version-model-on-top vs built-into vs DATA-MODEL-ON-TOP-OF-version-engine (ICE/COV's orthogonal instrumentable engine = the decoupled layer we want). CAUTION (7.2): they dismiss valid-time/transaction-time as 'not relevant for SCM' — but Softland's arrival-vs-commit IS exactly that temporal-DB distinction, so we deliberately revive a separation they set aside. SUPPORTS the layer + NAMES both times.
- **Groupings as versioned objects (own history; member binding pinned/floating/effectivity-scoped)** — Section 4.1 gives the exact mechanism. A configuration (= our grouping/local world) is a versioned object; member references are BOUND (pinned to a revision = AND edge to AND node) or GENERIC (floating = AND edge to OR node), with partially/totally-generic covering mixed binding. Effectivity/condition-scoped binding = configuration rules with product part + version part (5.3), scoped local/global/transitive-closure, ordered constraint>preference>default. The VERSION PROLIFERATION warning is a critical failure mode: naive version-first forces a new version of every enclosing grouping per member edit — fixed only by their rule to 'distinguish between versions of modules and versions of CONFIGURATIONS,' i.e. give the grouping its OWN history partly independent of members, exactly our requirement. GIVES MECHANISM + warns of a concrete failure mode.
- **Four-level graph versioning (graph / node / relation-TYPE / node-DATA, must compose)** — The AND/OR graph (4.1) is the composition engine: AND edges carry BOTH composition and dependency, and 'in addition to objects, RELATIONSHIPS are versioned as well' — our relation-type level versioned, not just nodes. Section 4.2's component/total/product versioning is the level-composition spectrum: TOTAL VERSIONING ('all levels of the composition hierarchy,' composite objects as first-class versioned entities, atomic+composite versioned UNIFORMLY) is 'version at every level and compose them.' Node-DATA versioning maps to their fine-grained internal granularity (deltas to text lines / long attributes). DAMOKLES' RECURSIVE versioning ('any version may be versioned in turn') formally guarantees the levels nest. SUPPORTS the four-level design with total/recursive versioning as realization — while warning (4.2) that a single GLOBAL version space (product versioning) sacrifices modularity, so our levels should stay distinct, not collapse.
- **Trail-to-code join (resolve path+timestamp to code-at-that-moment; DECLARED vs INFERRED change)** — Section 3.5's state-based vs change-based axis IS the declared-vs-inferred distinction, formalized. CHANGE-BASED = DECLARED: changes carry CIDs and 'a version may be described in terms of the change REQUESTS it implements' — an agent saying what it did is a first-class change package. STATE-BASED = INFERRED: 'changes are anonymous... must be DEDUCED from the topology of the version graph' (reconstructed by diffing). The verdict favors us: the explicit-change graph 'provides MORE information' than

the anonymous state graph because it records WHEN each change applied. For resolving path+timestamp-to-code, RCS symbolic names that 'define threads through the version graphs' are the join primitive. CAUTION (7.2): current systems lack FINE-GRAINED dependency traceability — our trail join must operate below file granularity to deliver what they couldn't. SUPPORTS declared-change as superior and names both sides.

- **Preserved disagreement / plurality (conflicting views representable, late-bound synthesis)** — Section 3.3 VARIANTS are exactly preserved, coexisting disagreement: 'versions intended to COEXIST,' and ClearCase records merges so 'the branches are not joined; rather, each of them continues to exist' (3.4) — divergence kept, not overwritten. The merge taxonomy (5.5) is our late-bound synthesis vocabulary: INCONSISTENCY (an op fails) vs CONFLICT (contradictory non-commuting changes), with three-way merge consulting a baseline and synthesis deferred to a human when automatic resolution isn't safe. COV's mutual-exclusion constraints c17c2 mark known-conflicting views WITHOUT silently picking a winner — modeling plurality structurally. SUPPORTS preserved disagreement; offers conflict/inconsistency as synthesis primitives.
- **Substrate choices (Rama event-sourced; whether the substrate owns versioning)** — Section 4.4 + Conclusion speak straight to Rama. Their 3.1 directed-delta = change explicitly 'corresponds to transaction logs in databases' — Rama's append-only log IS a change/directed-delta substrate, giving us extensional change-based versioning for free. But 4.4 warns that putting the version model purely ON TOP of an unaware data model means 'no support for storing versions efficiently' and 'the transaction manager does not take versioning into account' — Rama-as-substrate must be deliberately made version-aware, not assumed. The Conclusion's prophecy — 'versioning can be pulled out of SCM systems and MOVED INTO database systems... customizable to suit the needs of a specific application' — directly endorses building versioning as a reusable substrate layer (our intent), ideally with deductive/rule evaluation for the intensional (committed/branch) side. SUPPORTS substrate-owned versioning, with an efficiency/awareness caveat.

Takeaway. The single most important contribution is the pair of ORTHOGONAL axes — extensional-vs-intensional and state-vs-change — sitting atop a clean product-space/version-space split, all unified by AND/OR graphs and configuration rules. For Softland this directly licenses our two-times design: Rama's arrival-time append log is extensional change-based versioning (their directed-delta = transaction log), and our git-style minted/branchable layer is the intensional side, and the paper's emphatic ruling is that these 'are by no means mutually exclusive, but can (and SHOULD) be combined.' It also hands us the exact kernel vocabulary (versioned item = container; OID = sameness criterion; VID; immutable revisions; bound vs generic member binding) and one concrete failure mode to engineer around — VERSION PROLIFERATION, where naive version-first grouping forces a new version of every ancestor, fixed only by giving groupings their own configuration-level version history distinct from their members.

6. Making Data Structures Persistent

James R. Driscoll, Neil Sarnak, Daniel D. Sleator, Robert E. Tarjan · 1989 · Journal of Computer and System Sciences (JCSS), Vol. 38, No. 1 (condensed preliminary version appeared at the 18th ACM STOC, 1986)

In one line. Defines the partial/full persistence taxonomy and gives three general techniques — fat nodes, node copying, node splitting — that make any bounded-in-degree linked structure persistent at $O(1)$ amortized space and time overhead per update, then applies them to balanced search trees.

Source access: full. [ACM \(dl.acm.org/citation.cfm?id=64317\)](https://dl.acm.org/citation.cfm?id=64317) returned HTTP 403. Read the full open-access author copy hosted on Dan Sleator's CMU page: <https://www.cs.cmu.edu/~sleator/papers/another-persistence.pdf> — the complete 39-page JCSS reprint (pp. 86-124, all sections, all figures, references). Every section summary and quote below is from that PDF read directly, not from a secondary summary.

THE PAPER, SECTION BY SECTION

§ **Abstract.** States the dichotomy that organizes the whole paper: ordinary structures are 'ephemeral' — a change destroys the old version — whereas a persistent structure preserves access to old versions. The paper promises 'simple, systematic, and efficient techniques for making linked data structures persistent' and a headline

application: persistent binary search trees with logarithmic access/insert/delete time and $O(1)$ space per insertion or deletion.

“Ordinary data structures are ephemeral in the sense that making a change to the structure destroys the old version, leaving only the new one.”

“We use our techniques to devise persistent forms of binary search trees with logarithmic access, insertion, and deletion times and $O(1)$ space bounds for insertion and deletion.”

§ 1. Introduction — definitions and model. Defines the two persistence levels precisely: partially persistent = all versions readable but only the newest is modifiable; fully persistent = every version both readable and modifiable. It formalizes a 'linked data structure' as nodes with a fixed set of named fields (information fields or pointer fields) and constant out-degree, accessed via named 'access pointers' to entry nodes — modeled as a labeled directed graph. It distinguishes access operations (read-only, building an 'accessed set' of nodes) from update operations (intermixed access + update steps that create nodes or change one field/pointer). Versions are indexed by the update that produced them (version 0 = empty). The paper notes that, prior to Overmars, persistence had been handled only by ad hoc constructions, and that a systematic study (growing out of Sarnak's PhD thesis) is the goal. Bounds are stated as amortized (worst-case averaged over a sequence).

“The structure is partially persistent if all versions can be accessed but only the newest version can be modified, and fully persistent if every version can be both accessed and modified.”

“We define a linked data structure to be a finite collection of nodes, each containing a fixed number of named fields.”

“By amortized cost we mean the cost of an operation averaged over a worst-case sequence of operations.”

§ 2.1 Known Methods (Overmars). Reviews three prior partial-persistence schemes and why each is too costly, motivating the paper's contribution. (a) Store every version explicitly by copying the whole structure each update: $\Theta(n)$ time and space per update. (b) Store no versions, only the update log, and rebuild on access: $O(m)$ space but $\Omega(i)$ rebuild time for version i . (c) A hybrid storing every k -th version plus the log, which gives a time-space tradeoff but, for any k , blows up either space or access time by a $\sqrt{m}$ factor. Bentley-Saxe dynamization adds a logarithmic blowup. The diagnosis is sharp: these are poor precisely because they assume almost nothing about the underlying structure.

“One reason that the results of Overmars are so poor is that he assumes very little about the underlying ephemeral structure.”

“By restricting our attention to linked structures and focusing on the details of the update steps, we are able to obtain much better results.”

§ 2.2 The Fat Node Method (partial). The first technique: never erase a field value — instead let each node grow 'fat' by holding an arbitrary number of stamped values per field. Each node and each extra value carries a version stamp; at most one value per field per version is kept (a same-version rewrite overwrites). Reads navigate by selecting, for field f in version i , the value with the maximum stamp $\leq i$. An auxiliary access array stores the access-pointer values per version so any version's entry point is reached in $O(1)$. Cost: $O(1)$ worst-case space per update step, but $O(\log m)$ per access/update step because choosing among many stamped values in a node requires a search tree on stamps — a logarithmic-factor slowdown. Two drawbacks: fat nodes need linked fixed-size representations, and the per-step search costs the log factor.

“Our first idea is to record all changes made to node fields in the nodes themselves, without erasing old values of the fields.”

“Each extra field value has an associated field name and a version stamp. The version stamp indicates the version in which the named field was changed to have the specified value.”

“If the values of a field within a fat node are ordered by version stamp and stored in a binary search tree, simulating an access or update step takes $O(\log m)$ time.”

§ 2.3 The Node-Copying Method (partial). The technique that removes the fat-node log factor: persistent nodes hold only a FIXED number of field values (d original pointers + e extra slots + 1 copy pointer). Extra slots absorb a bounded number of stamped changes; when a node overflows, a fresh copy holding only the newest values is

made, and the copy must be threaded into all predecessors in the newest version (using maintained inverse pointers), which can cascade. Each ephemeral node maps to a 'family' — a copy-pointer-linked list of persistent nodes, the last being 'live.' A potential-function (amortized) argument proves $O(1)$ amortized space AND time per update step, with $O(1)$ WORST-CASE access — the best of both. This is the paper's main partial-persistence result for bounded in-degree.

"We eliminate the drawbacks of fat nodes with our second idea, node copying. We allow nodes in the persistent structure to hold only a fixed number of field values."

"Each ephemeral node corresponds to a set of persistent nodes, called a family."

"Hence the amortized space cost of the entire update operation is $O(t) = O(u)$, i.e., $O(1)$ per update step. The same analysis shows that the amortized time per update step is also $O(1)$."

§ 3.1 Version Tree and Version List (full persistence). Full persistence is harder because, when you can update ANY version, versions form a partially-ordered 'version tree' (version i is the parent of j if j is obtained by updating i), not a line — so simple numeric stamps no longer order navigation. The fix: impose a total order consistent with the tree, recorded as a 'version list' (a preorder of the tree) with the crucial property that all descendants of i appear consecutively starting at i . Navigation then needs an order query — 'does version i precede j in the list?' — solved by the Dietz-Sleator list-order structure in $O(1)$ query and $O(1)$ amortized insert (or $O(1)$ worst-case with a more complex variant).

"The versions of a fully persistent structure are only partially ordered. This partial ordering is defined by a rooted version tree."

"We impose a total ordering on the versions consistent with the partial ordering defined by the version tree. We represent this total ordering by a list... the version list."

"For any version i , the descendants of i in the version tree occur consecutively in the version list, starting with i ."

§ 3.2 Fat Node Method, full (valid intervals). Fat nodes extend to full persistence by comparing version stamps by position in the version LIST rather than numerically. Inserting a new version mid-list means an update may have to store TWO new field values, not one, because it splits an existing value's range. The key new concept is the 'valid interval': each stored value is valid over the span of version-list positions until the next stamp for that field — and these intervals partition the node's version range. Gives full persistence at $O(1)$ worst-case space per update and $O(\log m)$ (more precisely $O(\log h)$, $h = \max$ changes to a node) per access/update — same asymptotics as the partial fat-node case.

"Updating differs slightly from what happens in the partially persistent case, because the insertion of new versions in the middle of the version list makes it... necessary to store two updated field values per update step, rather than one."

"The valid intervals of the values of a field f in a fat node x partition the interval of versions from the version stamp of x to the last version."

§ 3.3 The Node-Splitting Method (full, $O(1)$). The paper's hardest construction: a B-tree-style variant of node copying achieving full persistence at $O(1)$ amortized space/time per update and $O(1)$ worst-case access. To support modifying old versions, the ephemeral structure is first 'augmented' with explicit inverse (predecessor) pointers, giving symmetry invariants ($x \rightarrow y$ iff $y \rightarrow x$). When a persistent node overflows, instead of full copy it SPLITS, moving roughly half the extra pointers to a new node so both retain room — like B-tree splits. Updates run in two phases: phase one records changes (possibly making pointers 'improper' or 'overlapping'); phase two is a cascading repair of alternating pointer-copying and node-splitting until a 'pointer invariant' (all pointers proper and non-overlapping) is restored. A potential argument over vacant extra slots proves $O(1)$ amortized. The authors note kinship to fractional cascading.

"The major difference between node splitting and node copying is that in the former, when a node overflows, a new copy is created and roughly half the extra pointers are moved from the old copy to the new one, thereby leaving space in both... for later updates."

"The second phase repairs this damage by judiciously copying pointers... a cascading process of alternating pointer copying and node splitting, which ends when the pointer invariant is entirely restored."

“We note in conclusion that our node-splitting process is very similar to the fractional cascading method of Chazelle and Guibas.”

§ 4. Persistent Balanced Search Trees (red-black, partial). Applies the techniques to a concrete structure: red-black trees, whose color constraints bound depth at $2 \log_2 n$ so access is $O(\log n)$, with rebalancing by $O(1)$ -time rotations and recolorings. A bottom-up strategy gives $O(1)$ AMORTIZED recolorings per update. Making them partially persistent via node copying with $e=1$ extra pointer per node yields $O(m)$ space for m insertions/deletions, $O(\log n)$ access to any version, $O(\log n)$ worst-case insert/delete. The method generalizes to weight-balanced trees, weak B-trees, and finger search trees, always at only a constant-factor blowup over the ephemeral structure.

“The color constraints imply that the depth of an n -node red-black tree is at most $2 \log_2 n$.”

“A partially persistent red-black tree representing a history of m insertions and deletions occupies $O(m)$ space and allows $O(\log n)$ -time access to any version.”

§ 4.2 Full Persistence via Lazy Recoloring. To make red-black trees FULLY persistent with node splitting, two obstacles arise: amortized (not worst-case) recoloring, and the inability to overwrite old colors. The solution is 'lazy recoloring' (refining Tsakalidis): postpone all recoloring except at path ends, encoding deferred changes as vertex-disjoint 'recoloring intervals' (insertion/deletion intervals) that store header info — a flag and the bottom item — in their top node. Maintaining these intervals disjoint, and shrinking/splitting them with $O(1)$ recolorings, stops color-change propagation and converts the $O(1)$ -amortized recoloring bound into $O(1)$ WORST-CASE update time. This beats the only competitive prior method (full path copying, due to Myers, Krijnen-Meertens, Reps-Teitelbaum-Demers, Swart) by a logarithmic factor in space.

“We exploit this observation by postponing all the recoloring except that at the path ends.”

“The crucial property of recoloring intervals is that they stop the propagation of color changes during an insertion or deletion.”

“Their method... requires $O(\log n)$ space and time in the worst case per insertion or deletion. Thus our method saves a logarithmic factor in space.”

§ 5. Fully Persistent Balanced Trees Revisited (displaced storage). An alternative giving $O(1)$ WORST-CASE space and $O(\log n)$ worst-case time (not amortized). The new idea is 'displaced storage of changes': instead of recording a change in the node it applies to, store it in a possibly different node on the access path in the new version, keeping 'displacement paths' sufficiently disjoint. Navigation switches from a version list to a 'traversal list' (each version twice — preorder + postorder) for $O(1)$ ancestor queries. The structure maintains three displacement-path invariants and supports shrinking/splitting of displacement paths. The authors leave the most elegant variant of displaced storage as future research.

“Instead of indicating a change to an ephemeral node x by storing the change in the corresponding persistent node $\bar{x}$, we store information about the change in some possibly different node that lies on the access path to $\bar{x}$ in the new version.”

“We obtain a fully persistent form of red-black trees with the same resource bounds as in Section 4, but the $O(1)$ space bound and the $O(\log n)$ time bound per insertion or deletion are worst-case rather than amortized.”

“We leave as a topic for future research the discovery of the most elegant and practical variant of the method.”

§ 6. Applications, Extensions, Open Problems. Catalogs what becomes persistent: $O(1)$ -per-op persistent stacks/queues (linked lists) and dequeues (doubly-linked lists); partially persistent red-black trees with planar point-location applications (companion Sarnak-Tarjan paper, $O(\log n)$ query, $O(n)$ space); fully persistent trees for text editing (Myers), program editing (Reps et al.), and high-level languages (Krijnen-Meertens). A theoretical highlight: node copying can be made write-once except for access pointers, implying any bounded-in-degree augmented-LISP structure (with replaca/replacd) is simulable in linear time in pure LISP (cons/car/cdr only) — illuminating the power of purely applicative languages. Four open problems are flagged: (i) make node copying/splitting $O(1)$ WORST-CASE per step; (ii) CONFLUENT updates combining two old versions (the version tree becomes a DAG — i.e. merge); (iii) updates that change many versions at once; (iv) a better-than-fat-node method for UNBOUNDED in-degree, where copying/splitting break down.

“Any data structure built using augmented LISP (in which replaca and replacd are allowed) can be simulated in linear time in pure LISP (in which only cons, car, and cdr are allowed), provided that each node in the structure has constant bounded in-degree.”

“Find a way to allow update operations that combine two or more old versions of the structure. This would allow concatenation of lists... the version tree of Section 3 becomes a directed acyclic graph.”

“The node-copying and node-splitting methods completely break down in this [unbounded in-degree] case; the fat node method needs only $O(1)$ space per update step but increases access times by a logarithmic factor.”

HOW IT MAPS TO OUR PARTS

- **Versioning as its own layer — Rama arrival-time history (partial persistence) vs explicit git-style committed/minted versions (full persistence) — GIVES IT A NAME** and a precise formal distinction. The paper's central taxonomy IS our two-times split: partially persistent = 'all versions can be accessed but only the newest version can be modified' (a linear log you read but only append to the head — Rama arrival-time history); fully persistent = 'every version can be both accessed and modified' (branch from any committed point — git). The paper proves both are achievable at $O(1)$ amortized overhead, and crucially shows full persistence is strictly harder because its versions form a TREE not a line ('the versions of a fully persistent structure are only partially ordered... defined by a rooted version tree'). This validates that our two times are genuinely different layers, not the same mechanism viewed twice — and warns that the committed/branchable layer needs extra machinery (a version tree + a total-order 'version list') that the arrival-log layer does not.
- **Object-container kernel — stable CONTAINER = identity thread, REVISIONS = immutable values inside, current-revision = state — OFFERS A MECHANISM.** The 'fat node' is the canonical implementation of a container: one stable node identity that 'hold[s] an arbitrary number of values of each field,' every value tagged with a version stamp, navigation = 'choosing among several such values the one with maximum version stamp no greater than i.' This is exactly container-with-revisions: the node is the identity thread, each stamped field-value is an immutable revision, current = newest stamp. The paper hands us the read rule (rightmost-stamp-not-after-i) and the write rule (never erase, only append a stamped value) for free.
- **Hard rule: 'inferred structure is never accepted as truth at ingest' / DECLARED vs INFERRED change — SUPPORTS IT** with a sharp engineering reason. The whole paper assumes changes are DECLARED — update operations explicitly name the node and field they mutate, and persistence records exactly that declared delta with a version stamp. It never reconstructs history by diffing two states; it stores the change as it happens. The Section 6 LISP result is the payoff: an augmented-LISP program using declared mutations (replaca/replacd) 'can be simulated in linear time in pure LISP.' Declared mutation is cheap and exact; the paper's amortized $O(1)$ bounds depend on knowing the change rather than inferring it. Our INFERRED path (diff-reconstruct) is the lossy fallback the paper deliberately avoids.
- **Four-level graph versioning — graph / node / relation-TYPE / node-DATA must version AND compose — OFFERS A MECHANISM** and a partial WARNING. The fat-node method versions a node's identity, its DATA (information fields), and its RELATIONS (pointer fields) independently, each with its own stamp — that is node-level + node-data-level + relation-instance-level versioning composing under one stamp ordering. So three of our four levels have a worked, composing implementation. WARNING: the paper's clean $O(1)$ bounds require 'constant bounded in-degree'; for unbounded in-degree only the costlier fat-node (log-factor) survives — graph-level edges with high fan-in (relation TYPES touching many nodes) are exactly the open problem (vi) the authors flag as unsolved. Our graph-level and relation-TYPE-level versioning live in that harder regime.
- **Groupings as versioned objects — a curated grouping / local world has its OWN version history, partly independent of members; binding pinned/floating/effectivity-scoped — OFFERS A MECHANISM —** the 'valid interval' / effectivity primitive. Each stored value has a 'valid interval... the interval of versions in the version list from the version stamp of v up to but not including the next version stamp' — i.e. a field value is bound to a node only over an explicit version range. This is precisely effectivity/condition-scoped membership: a grouping's pointer to a member can be PINNED (single stamp) or FLOATING/scoped (a valid interval that

auto-extends until overwritten). The valid-interval partition gives groupings a self-owned timeline whose member bindings each carry their own validity window.

- **Preserved disagreement / plurality — conflicting views representable, synthesis late-bound** — PARTIAL SUPPORT + a clean WARNING about its limit. Full persistence (the version tree with sibling branches) is exactly preserved divergence: two updates off the same parent produce two co-existing versions, neither overwriting the other — branches ARE preserved disagreement. But the paper STOPS at branching: it explicitly lists CONFLUENT merge as unsolved — open problem (ii), 'allow update operations that combine two or more old versions... the version tree of Section 3 becomes a directed acyclic graph.' So the paper supports our plural-branches but warns that late-bound SYNTHESIS (merging branches back) is genuinely hard and was left open in 1989 (later named 'confluent persistence').
- **The trail-to-code join — resolve a trail's path+timestamp to code-at-that-moment** — OFFERS THE CORE MECHANISM. Resolving 'code at version i' is exactly the paper's access primitive: navigate to the node, then pick the field value whose stamp is 'rightmost in the version list but not to the right of i.' A trail's timestamp maps to a version stamp; the path maps to the access pointers / entry nodes (the paper keeps 'an access array, one per access pointer name, to store the various versions of the access pointers' so 'initiating access into any version takes $O(1)$ time'). That access-array-of-roots is the index our join needs: given (path, t) $\rightarrow$ root-at-t $\rightarrow$ walk $\rightarrow$ value-at-t.
- **Substrate choices — Rama event-sourced, multi-LLM, code-identity (file vs function)** — SUPPORTS Rama's arrival-log model and FRAMES the atomic-unit choice. 'The event log is primary, rendered state is projection' is the paper's own stance inverted into truth: it stores the sequence of stamped update steps and treats every version as a projection navigable from that log — partial persistence is literally 'rebuild the version from the change log.' On code-identity: the paper defines the atomic versioned unit as the NODE (fixed named fields, constant bounded in-degree for the good bounds). The lesson for file-vs-function: the cheaper your atomic unit's fan-in, the cheaper versioning — a function (low in-degree, stable field set) behaves like the paper's well-behaved node; a whole file (high churn, many referrers) drifts toward the unbounded-in-degree open problem. This is an argument that the function, not the file, is the natural container.

Takeaway. This paper is the formal backbone for Softland's 'versioning as its own layer': it names and separates exactly our two times — partial persistence (read-any-version, modify-only-newest = Rama's arrival-time log) versus full persistence (modify-any-version = git-style branchable committed versions) — and proves both are achievable at $O(1)$ amortized overhead, while showing full persistence is fundamentally harder because its versions form a version TREE (with a total-ordered version-list for navigation), not a line. Its fat-node mechanism is a ready-made object-container kernel (stable node identity + immutable stamped revisions + a 'rightmost-stamp- $\leq$ -i' read rule), and its 'valid interval' primitive directly models effectivity/floating member bindings for groupings. Two warnings are load-bearing for us: cheap $O(1)$ versioning requires constant bounded in-degree (favoring the function, not the high-fan-in file, as the atomic container), and confluent MERGE of branches — our late-bound synthesis of preserved disagreement — was explicitly left open here (the version tree becoming a DAG), confirming it is the genuinely hard part of our four-level graph-versioning ambition.

7. Pijul model — the patch-based model ('How Pijul works' essay + FAQ on pijul.org), grounded in 'A Categorical Theory of Patches' (Mimram & Di Giusto, 2013)

Pierre-Étienne Meunier & Florent Becker (Pijul); formal theory by Samuel Mimram & Cinzia Di Giusto (CEA, LIST) · 2013 (theory) / 2017+ (Pijul site) · pijul.org documentation/blog; theory in Electronic Notes in Theoretical Computer Science (MFPS), arXiv:1311.3903

In one line. A file is a graph of identity-bearing line-vertices; a patch is a morphism that adds vertices or remaps edge-status; merge is a categorical pushout, and conflicts are first-class objects in the pushout-completed category rather than errors.

Source access: full. pijul.org/model is a SvelteKit SPA serving no static prose; I extracted the actual page content (FAQ + 'How Pijul works' model essay + 2017 blog posts) from the JS chunk `/_app/immutable/chunks/ftENTFLE.js` by stripping KaTeX/SVG. The site states the model 'merges concepts from category theory' and links the formal paper. To honestly ground the pushout/free-monoid/conflict-completion claims (which the site only summarizes informally), I fetched and read the full text of the underlying paper, Mimram & Di Giusto 'A Categorical Theory of Patches' (arXiv:1311.3903, PDF via pdftotext, 1869 lines). Verbatim quotes below are sourced from whichever of the two I actually read.

THE PAPER, SECTION BY SECTION

§ FAQ: Why a new VCS? (pijul.org). Frames the whole project as resolving a two-camp split in version control: snapshot-based systems (git, mercurial, svn) are fast but hard to use beyond simple operations, while patch-based systems (darcs) are easy and have nice algebraic properties but historically slow. Pijul's pitch is that it COMBINES the two by 'representing data in a more general way that allows efficient patch operations,' with merges and conflicts as the place this generality matters most. The FAQ explicitly indicts three-way merge as algorithmically wrong, not merely buggy, citing real cases where git/mercurial/svn 'just do the wrong thing.'

"There are basically two approaches to version control: snapshot-based (git, mercurial, svn) or patch-based (darcs)."
"Pijul combines these approaches, representing data in a more general way that allows efficient patch operations."
"there are real-world examples where Git, Mercurial and SVN just do the wrong thing"

§ How repositories used to work — the graph-of-lines model (essay). Defines the core data structure: a single file is a directed graph $G=(V,E)$ of lines, each vertex a line, each edge labelled by a change number meaning 'according to change c , line u comes before line v .' The decisive design choice is that vertices are UNIQUELY IDENTIFIED by the hash of the change that introduced them plus a position within that change — so two textually identical lines from different changes are different objects, and a line keeps its identity even when its change is applied in a wholly different context. This is content-independent, provenance-based identity.

"a single file [is represented] by a directed graph $G=(V,E)$ of lines, where each vertex represented a line"
"vertices are uniquely identified, by the hash of the change that introduced them, along with a position in that change"
"a line keeps its identity, even if the change is applied in a totally different context"

§ Append-only deletion and the two action kinds (essay). Deletion never removes data: the system is append-only, and a deletion is a change that REMAPS the edge introduced by an earlier change from 'alive' to a 'dead' status, labelled by the deleting change's name. Live edges are 'alive', removed ones 'dead' (drawn dashed). From this the essay claims a minimal action vocabulary: there are exactly TWO kinds of action — one adds vertices with surrounding alive edges, the other maps an existing edge label onto a different one. Edge labels carry two parameters: a status (alive/deleted/others) and the introducing change.

"this system is append-only, in the sense that deletions are handled by a more sophisticated labelling of the edges"
"We have just described the two basic kinds of actions in Pijul. There are no other."
"the edge labels are given by two parameters: their status ... and the change that introduced them"

§ Dependencies (essay). Changes form a dependency relation derived from the graph: a change that ADDS a vertex depends on the changes that introduced its context (the lines immediately before and after the insertion point); a change that DELETES a vertex (remaps an edge introduced by change c) depends on c . The essay stresses this is the MINIMAL dependency set needed for the text edits to make sense, and that higher-level semantic dependencies can be layered on via hooks. This dependency closure is what makes patches partially (not totally) ordered and independently transferable.

"If a change adds a vertex, we must have its 'context', i.e. the lines before and after it"
"this is just the minimal set of dependencies needed to make sense of the text edits"
"Hooks and scripts may add extra language-dependent dependencies based on semantics"

§ Why keep change-labels on edges? (essay). A minimality probe: could edges store only status (alive/dead) and drop the introducing-change label? The essay shows two failure modes that force keeping the label. (1) Change inverses: if two authors delete the same line in parallel and one reverts, label-free statuses would merge the two

deletions into one and a single revert would wrongly resurrect the line; keeping per-change labels distinguishes the two deletions so revert only undoes its own. (2) Missing contexts: when Alice edits inside a paragraph that Bob deletes in parallel, the situation is asymmetric, and only the change-labels let one side detect that Alice's edit lands in context Bob marked deleted. So the change-id on the edge is not redundant; it is what makes revert and conflict-detection correct.

“why even keep the change number on the edges?”

“the two deletions get merged into one, and the inverse applies to both ... this is not what we expect”

“the context of Alice's edits is labelled as deleted in his repository”

§ Patches commute / associativity vs three-way merge (2017 blog). Argues the practical payoff of the patch algebra: patches COMMUTE, enabling patches that touch only one part of the repo and stay compatible with concurrent work elsewhere; and in the darcs lineage patches are ASSOCIATIVE, 'a really useful property to have in a VCS.' The post is emphatic that three-way merge's misbehavior 'is not a bug/implementation mistake ... It's a problem in the algorithm itself' — i.e. snapshot-diff merging is structurally, not incidentally, wrong, which is the motivation for an algebraic model.

“patches are associative, which is a really useful property to have in a VCS”

“This is not a bug/implementation mistake in 3-way merge/git ... It's a problem in the algorithm itself.”

“patch commutation [enables] patches touching just one part of the repository, compatible with the rest of the work”

§ Theory — Category L of files and patches (Mimram & Di Giusto, §2). The formal underpinning the site cites. A file is a function $A:[n] \rightarrow L$ assigning lines (from a fixed set L) to positions $0..n-1$. A patch (morphism) between files $A:[m] \rightarrow L$ and $B:[n] \rightarrow L$ is an INJECTIVE INCREASING PARTIAL function $f:[m] \rightarrow [n]$ that preserves line content where defined. Category L has files as objects and patches as morphisms; it is strictly monoidal (concatenation of files), and Proposition 2 shows every patch is generated by two primitives — insertion $\eta_a:l \rightarrow a$ and deletion $\epsilon_a:a \rightarrow l$ — i.e. the same two-action vocabulary the Pijul essay describes, here proven to be a free monoidal category. This is the exact formal version of 'there are only two kinds of action.'

“A morphism between two files ... is an injective increasing partial function ... Such a morphism is called a patch.”

“The category L has files as objects and patches as morphisms.”

“patches are generated by the operations of inserting and deleting a line”

§ Theory — Merge as pushout; conflicts as non-existent pushouts (§3). Merging the effects of two coinital patches (the diverging g,h from a shared file) is DEFINED as the pushout of the diagram — the universal way to combine two independent edits. The key theorem: not every pair of coinital patches has a pushout in L, and exactly those that don't are the CONFLICTING ones. The fix is to embed L into a larger category P obtained by the FREE COMPLETION under finite colimits (initial object + all pushouts), so that conflicting files become first-class objects 'in the most general way possible.' These generalized files are finite sets of lines with only a PARTIAL (transitive) order rather than a linear one — the conflict is literally an unresolved partial order, and the paper even draws it as git's <<<<<<< HEAD / ===== / >>>>>>> markers to contrast snapshot tools' textual conflict with Pijul's structural one.

“the operation of merging the effect of two coinital patches is defined by pushout”

“not every diagram in L has a pushout ... the two patches f_1 and f_2 are said to be conflicting”

“its objects are finite sets labeled by lines equipped with a transitive relation and morphisms are partial functions”

“patches are only partially ordered ... there is no global notion of 'time'”

HOW IT MAPS TO OUR PARTS

- **Object-container kernel: container = identity thread, revisions = immutable values, 'inferred structure is never accepted as truth at ingest.'** — STRONG SUPPORT + a mechanism. Pijul gives each line a stable identity = hash(introducing-change) + position, independent of content or context. That IS a container: the identity thread is the vertex, its successive alive/dead status are the revisions. Critically, identity is DECLARED by the change that introduced it, never INFERRED by diffing content — 'two lines with the same content,

introduced by different changes, will be different.' This is exactly our 'inferred structure is never truth at ingest' rule, validated at line granularity. The kernel should mint a container id at the declaring event, not reconstruct it.

- **The code-identity question: is identity the file, path, name/FQN, AST, or git hash? Is the atomic unit the file or the function?** — OFFERS A NAMED ANSWER at a finer grain than we'd considered. Pijul's atomic unit is the LINE, and its identity is neither content nor position but provenance: the change-hash that introduced it plus an offset. This warns against our snapshot instincts (path/hash) and AST-normalization: identity should be the introducing-event id, so a unit 'keeps its identity even if the change is applied in a totally different context.' Translated up: a function's identity could be the kernel event that first declared it, surviving moves/renames — closer to git-hash-of-introducing-commit than to FQN or AST.
- **Versioning as its own layer: arrival-time (partial persistence) vs committed/minted (full persistence); two times.** — WARNS + clarifies. Pijul's deepest claim is that patches are only PARTIALLY ordered — 'there is no global notion of time' — and that linear snapshot history (git) imposes a false total order that produces silently-wrong merges. This validates our two-times split but pushes harder: the committed/true-time layer is a DAG/partial order, not a line, and commutation (independent patches can be reordered) is the property that makes branching-from-any-point clean. Our 'minted version you can branch from' should be a partially-ordered patch set, not a linear commit chain.
- **Groupings as versioned objects; member binding pinned/floating/effectivity-scoped.** — OFFERS A MECHANISM via dependencies. In Pijul a change carries a minimal DEPENDENCY set: an insertion depends on the changes that introduced its surrounding context; a deletion depends on the change that introduced the deleted edge. A grouping-version is then a closed set of changes under dependency. This is a model for 'pinned vs floating' membership: pinning = include a specific change-id; floating = include whatever satisfies the dependency closure. 'Hooks and scripts may add extra language-dependent dependencies based on semantics' maps directly to effectivity/condition-scoped binding.
- **Four-level graph versioning (graph / node / relation-TYPE / node-DATA) that must COMPOSE.** — GIVES A COMPOSITIONAL SUBSTRATE. Pijul versions a file as a graph $G=(V,E)$ where vertices (nodes) and edges (relations) are BOTH versioned by the same primitive — a change either adds vertices+edges or remaps an edge's label (status). Node identity, edge existence (alive/dead), and edge type (status enum) are all expressed in one append-only edge-relabeling calculus, so the levels compose by construction. This is direct evidence that node-level and relation-type-level versioning can share one event vocabulary rather than four bolted-on schemes — and the categorical paper proves the whole thing composes (associativity of patch composition).
- **The trail-to-code join: DECLARED change (agent says what it did) vs INFERRED change (reconstructed by diffing).** — STRONG SUPPORT — this is Pijul's central thesis transposed. Pijul exists because three-way merge INFERS structure by diffing snapshots and 'just does the wrong thing'; Pijul instead records the change as a first-class declared object with explicit context/dependencies. Our agent trails are declared edit operations — so we are already on Pijul's side of the line. The mapping: store the trail's declared ops as the truth and treat diff-reconstruction only as fallback/verification, exactly as Pijul stores patches not snapshots.
- **Preserved disagreement / plurality: conflicting views representable, not overwritten; synthesis late-bound.** — GIVES IT A FORMAL NAME AND A FAILURE MODE TO AVOID. This is the paper's headline result: a conflict is the case where the pushout (merge) does NOT exist in the base category, and the fix is to WORK IN THE COMPLETED CATEGORY P where conflicting files exist as first-class objects — 'finite sets labeled by lines equipped with a transitive (partial, not linear) relation.' A conflict is a value, not an error; it persists until resolved by a later patch ('resolve a conflict once — pulling never brings it back'). This is precisely late-bound synthesis over preserved plurality, with a proof that it's the universal (most general) way to represent disagreement.
- **Substrate choices: Rama event-sourced, back-arrow rule, append-only log as primary.** — SUPPORT + design discipline. Pijul's model is append-only at its core — 'this system is append-only; deletions are handled by a more sophisticated labelling of the edges' (a dead edge, never a removed one). That is event-sourcing for code structure: state is a projection of an append-only set of change-events, identity is the event id. It validates Rama-as-truth: never mutate a node, append a status-remap event. The warning: keep the change-id ON the

relation (edge label), because dropping it breaks inverse/revert and missing-context detection — analogous to keeping provenance on every Rama fact.

Takeaway. The single most important contribution to our versioning/identity question is the principle that IDENTITY AND CHANGE ARE DECLARED, NEVER INFERRED FROM CONTENT: every line carries a provenance-based identity (hash of the introducing change + position) that survives moving, copying, and re-application, and every edit is a first-class patch object — never a snapshot to be diffed. From this one move Pijul gets the two things we want most: (1) conflicts/disagreement become first-class VALUES (objects of the pushout-completed category, a preserved partial order) instead of errors silently resolved by a lossy three-way merge, and (2) versions are partially-ordered, commuting patch-sets you can branch and synthesize late from — exactly our committed/minted layer over an append-only log. It is direct evidence that 'inferred structure is never accepted as truth at ingest' is not just a safety rule but the foundation a correct version model is built on.

8. A Categorical Theory of Patches

Samuel Mimram, Cinzia Di Giusto · 2013 · arXiv (cs.LO), CEA LIST; later in ENTCS (MFPS 2013)

In one line. Models files as objects and patches as identity-preserving partial functions in a category L ; defines merge as pushout, and since conflicting patches have no pushout, builds the free finite cocompletion P whose objects are finite label-sets with a transitive relation — giving conflicts a first-class mathematical representation.

Source access: full. Read the full PDF (including Appendix A) via pdftotext on the arXiv source arXiv:1311.3903v1 saved locally by WebFetch. WebFetch itself could not parse the FlateDecode binary; pdftotext extracted the complete 14-page text including all sections, examples, references, and the geometric-interpretation appendix.

THE PAPER, SECTION BY SECTION

§ Abstract. States the complementary-to-implementation strategy: rather than verify a vcs implementation, introduce a theoretical model 'defined abstractly by the universal property that it should satisfy' and work out a concrete description. Defines a category of files and patches where merging two coinital patches is a PUSHOUT. Observes the pushout need not exist (incompatible patches), raising the question of the correct category for files in conflicting state. Answers via the free completion of the file category under finite colimits, with an explicit description: objects are finite sets labeled by lines with a transitive relation; morphisms are partial functions respecting labeling and relations.

“the operation of merging the effect of two coinital patches is defined by pushout”

“objects are finite sets labeled by lines equipped with a transitive relation and morphisms are partial functions”

§ 1. Introduction. Motivates via git/Darcs: a vcs records changes as patches and imports others' patches; 'there is no global notion of time: patches are only partially ordered.' Walks the canonical diamond: file A, patch $f:A \rightarrow B$ imported by both users, then concurrent $g:B \rightarrow C$ and $h:B \rightarrow D$; the merge E must be the same regardless of import order, computed via residuals h/g and g/h . Formally residuals are 'the arrows of pushout cocones' so the merge square is a pushout. Notes not every coinital pair has a pushout in L — this is exactly conflict (e.g. two users editing the same line). To represent ALL conflicting files, build the free completion of L under all pushouts (plus initial object = finite colimits), called P , 'adding all conflicting files in the most general way.' P exists abstractly; the paper's main contribution is an explicit description via presheaf theory, paving the way to a provably correct vcs.

“there is no global notion of time: patches are only partially ordered”

“there should be a patch h/g , the residual of h after g , which transforms C into E and has the same effect as h once g has been applied”

“not every pair of coinital morphisms have a pushout in the category L : this reflects the fact that two patches can be conflicting”

“we investigate the free completion of the category L under all pushouts ... which corresponds to adding all conflicting files”

§ Related work (within Intro). Positions against prior patch formalizations. Darcs formalizes patches via commutation properties; operational transformations axiomatize residual patches (Ressel et al.). The paper's distinguishing claim: in both, 'the fact that residual should form a pushout cocone is never explicitly stated,' only said informally. Cites Jacobson's inverse-semigroup formalization of Darcs, and most importantly Houston's category-with-pushouts for conflicting files (discussed in Section 6) as the closest prior work — but Houston's morphisms are reversed, so it is not the free cocompletion (not the most general model).

“the fact that residual should form a pushout cocone is never explicitly stated, excepting in informal sentences”

“Houston has proposed a category with pushouts, similar to ours, in order to model conflicting files”

§ 2. Categories of files and patches. Defines the base category L . Fix a set L of lines. A file is a finite sequence $A: [n] \rightarrow L$ where $[n] = \{0, \dots, n-1\}$ indexes lines. A patch / morphism $f: A \rightarrow B$ between files $A: [m] \rightarrow L$, $B: [n] \rightarrow L$ is an INJECTIVE INCREASING PARTIAL function $f: [m] \rightarrow [n]$ such that $B(f(i)) = A(i)$ whenever $f(i)$ is defined — i.e. the patch 'keeps track of the unchanged lines.' Deleted lines = those undefined under f ; inserted lines = those not in the image. Prop. 2: L is the free monoidal category generated by line insertions $\eta_a: I \rightarrow a$ and deletions $\epsilon_a: a \rightarrow I$ with $\epsilon_a \circ \eta_a = \text{id}$ (deleting an inserted line = nothing). Prop. 4: in the unlabeled (singleton L) case, L is the free category on generators s^n_i (insert at position i) and d^n_i (delete) subject to standard simplicial-like relations. Defines subcategory L^+ (same objects, TOTAL injective increasing morphisms = insertion-only patches), the simpler case for Section 4.

“A morphism between two files ... is an injective increasing partial function $f: [m] \rightarrow [n]$ such that $B(f(i)) = A(i)$ whenever $f(i)$ is defined. Such a morphism is called a patch”

“ f keeps track of the unchanged lines”

“The category L is the free monoidal category containing ... morphisms $\eta_a: I \rightarrow a$ (insertion) and $\epsilon_a: a \rightarrow I$ (deletion) such that $\epsilon_a \circ \eta_a = \text{id}_I$ ”

§ 3. Towards a category of conflicting files. Shows by example that merge = pushout: two non-conflicting insertions have a pushout (the smallest file containing both edits, e.g. ab merging to $acbcd$), but diagram (3) — both users inserting different lines at the same position — has NO pushout in L : these patches are 'conflicting.' Defines P as the free completion of L under finite colimits (initial object + pushouts), intuitively adding files whose lines are only PARTIALLY ordered, explicitly drawn to mirror git's conflict markers (a , then c -or- d , then b). Def. 5: P is the free finite conservative cocompletion — the unique finitely cocomplete category with colimit-preserving embedding $y: L \rightarrow P$ satisfying the universal factorization property; 'conservative' = preserves colimits already in L . Theorem 6 (Kelly, folklore): this cocompletion is the full subcategory of presheaves $L^{\text{op}} \rightarrow \text{Set}$ preserving finite limits, restricted to finite colimits of representables. Example 7: FinSet is the cocompletion of the terminal category 1 . Notes the surprising consequence that the construction forces 'cycles' into objects of P .

“it should be a pushout of the diagram”

“the two patches f_1 and f_2 are said to be conflicting”

“ P is the free finite conservative cocompletion of L ”

“perhaps a bit surprisingly, it turns out that we have to allow cycles in the objects of the category P ”

§ 4. A cocompletion of files and insertions of lines. Works out P concretely for the simplified case (L^+ , insertions only, singleton labels). Uses the subcategory G of objects $\{1, 2\}$: presheaves on G ARE graphs ($P(1)$ =vertices, $P(2)$ =edges). The inclusion $I: G \rightarrow L$ is DENSE (Def. 9), so every file n is canonically a colimit of 1 's and 2 's; the nerve $N_I(n)$ is the graph on $[n]$ with an edge $i \rightarrow j$ iff $i \neq j$, and $P(n+1)$ = paths of length n . Lemma 11 — it suffices to check preservation of colimits of diagrams 'generated by a graph.' Lemmas 12-13 — such a generated diagram has a colimit iff the graph is 'tree-shaped': ACYCLIC and CONNECTED (these become a total order). Prop. 14 / Theorem 15: P is equivalent to the full subcategory of graphs where every non-empty path has exactly one edge — equivalently, FINITE SETS WITH A TRANSITIVE RELATION and relation-respecting functions. Prop. 16: pushout of $B \leftarrow A \rightarrow C$ is $(B \cup C) / \sim$ (gluing along shared images) with the TRANSITIVE CLOSURE of the inherited relation. Extends to labels via a slice category $L / !L$.

“a presheaf P in $\hat{G}$ is a graph”

“A finite graph G induces a diagram which admits a colimit if and only if it is tree-shaped, i.e. acyclic and connected”

“objects are finite sets equipped with a transitive relation $<$, and functions respecting relations”

“the pushout ... is $B \cup C / \sim$... equipped with the transitive closure of the relation inherited by $<B$ and $<C$ ”

§ 5. Examples. Concrete merges as pushouts. Example 17: starting from ab , one user inserts a' at the start and c in the middle, another inserts d in the middle; the pushout is the merged file with a partial order branching at the c/d position — a non-conflicting concurrent edit. Example 18: defines a SEQUENTIALIZATION morphism $\text{seq}: G_1 + G_1 \rightarrow G_2$ (turn two conflicting alternative lines into two consecutive lines, a user's resolution choice); the pushout of seq against seq' (opposite order) produces a CYCLIC graph, showing concretely why cycles appear in P . Example 19: by coproduct of two $\text{id}: G_1 \rightarrow G_1$ there is a universal MERGE morphism $G_1 + G_1 \rightarrow G_1$ collapsing two independent/conflicting lines into one — the formal act of conflict resolution.

“After merging the two patches, the resulting file is the pushout”

“the sequentialization morphism ... given two possibilities for a line, a user can decide to turn them into two consecutive lines”

“which illustrates how cyclic graphs appear in P ”

“two independent lines can be merged by a patch (in order to resolve conflicts)”

§ 6. Handling deletions of lines. Generalizes from $L+$ (insertions only) to the full L (insertions AND deletions).

Theorem 20: the free finite cocompletion P of L is the category whose objects are TRIPLES $(A, <, l)$: A a finite set of lines, $<$ a transitive relation, $l: A \rightarrow L$ a labeling; morphisms are PARTIAL functions $f: A \rightarrow B$ respecting labels ($l(f(a)) = l(a)$) and relations ($a < a'$ implies $f(a) < f(a')$). Pushouts computed as in Prop. 16, generalized to partial functions. Example 21: from abc , one user inserts d after a , another deletes b ; the pushout is adc , with the deletion realized as a PARTIAL morphism (b has no image). Compares to Houston's independently-proposed similar category: NOT equivalent because Houston's morphisms are reversed partial functions, so it is not the free cocompletion / most general model — illustrated by FinSet being the cocompletion of 1 while FinSet-op is not, and Houston's category lacking the merge morphism (it has a dual duplicating morphism instead).

“objects are triples $(A, <, l)$... and morphisms ... are partial functions $f: A \rightarrow B$ such that ... $l(f(a)) = l(a)$, and $a < a'$ implies $f(a) < B f(a')$ ”

“Notice that the morphism f_2 is partial: b has no image”

“it is thus not the most general model (in the sense of being the free finite cocompletion)”

§ 7. Concluding remarks and future works. Claims success: a finitely cocomplete category where merging any modifications is always well-defined, derived from universal constructions, giving correctness from universal categorical properties. Outlines extensions: the methodology 'adapts easily to other base categories' — multiple files (move/delete), structured file types (text, XML trees); repositories modeled as EVENT STRUCTURES labeled by morphisms in P ; modeling cherry-picking, branches, patch removal; adding patch inverses. Plans to study algorithmic representation/manipulation of conflicting files (objects of P) toward a 'theoretically sound and complete distributed version control system (no unhandled corner-cases as in most current implementations).'

“It is finitely cocomplete, thus the merging of any modifications of the file is well-defined”

“the structure of repositories (partially ordered sets of patches) is naturally modeled by event structures labeled by morphisms in P ”

“no unhandled corner-cases as in most current implementations of vcs”

§ Appendix A. Geometric interpretation of presheaves on $L+$. Supplementary intuition: recalls how presheaves over $L+$ (forgetting labels, empty file disallowed) can be read geometrically/simplicially, viewing files and their higher path-structure topologically. Provides the simplicial/geometric reading that underlies the graph-as-presheaf identification used in Section 4, helping the reader grasp why $P(n)$ corresponds to length- n paths and why the construction behaves like gluing simplices.

“Since presheaf categories are sometimes a bit difficult to grasp, we recall here the geometric interpretation”

HOW IT MAPS TO OUR PARTS

- **Object-container kernel: container = identity thread, revisions = immutable values, current-revision = state; 'inferred structure is never accepted as truth at ingest.'** — Gives a NAME and a clean mathematical separation for our split: in this paper objects (files) ARE the states/values and morphisms (patches) ARE the identity-preserving transitions between them. A file at a given moment is a finite sequence $A:[n] \rightarrow L$ (an immutable value, our revision); a patch is the structure-carrying thread between values (our container's history of transitions). Crucially, the patch is an INJECTIVE PARTIAL FUNCTION on line-indices that 'keeps track of the unchanged lines' — declared identity of which lines survive — NOT a diff inferred after the fact. This supports our hard rule: identity (which line is which) is carried explicitly by the morphism at edit time, not reconstructed by diffing two revisions. The warning: their model only has identity where a morphism is recorded; if you only store snapshots (objects) and try to recover the patch, you must INFER it, and inference is lossy (many morphisms can connect the same two files).
- **The code-identity question: is the atomic unit the file or the function? Is identity path/name/AST/hash?** — Offers a concrete stance and a generalization hook. Their base unit is the LINE (an element of $[n]$), not the file: a file is just a finite ordered set of lines, and identity is tracked PER LINE through the partial function. The file is merely the current container; the line carries the surviving-identity. This is a direct analogue to 'the function, not the file, is the atomic unit' — pick the granularity at which a morphism preserves identity. The conclusion explicitly says the method 'adapts easily to other more complicated base categories L ' — 'multiple files (which can be moved, deleted, etc.), different file types (containing text, or more structured data such as xml trees).' So an AST-node identity model is the same construction over a richer base category, warning us the choice of base category L (line / function / AST node) is THE design decision and everything else (merge, conflict) is derived.
- **Versioning as its own layer; two distinct times (arrival vs committed/true); branch-and-merge.** — Gives the mechanism for the COMMITTED/MINTED layer. The repository is 'partially ordered sets of patches' — 'there is no global notion of time: patches are only partially ordered.' This is exactly our committed-time-as-DAG (not a line). Merge of two concurrent commits is the PUSHOUT of the two coinitial patches; the residual h/g is defined as the pushout arrow, formalizing git's three-way merge as a universal construction. This validates separating arrival-time (Rama log order) from committed causality (the partial order of patches), and tells us merge should be a derived universal operation over the committed layer, not over arrival order.
- **Preserved disagreement / plurality: conflicts representable, not silently overwritten; synthesis late-bound.** — This is the PAPER'S CENTRAL CONTRIBUTION and our strongest mechanism match. Not every pair of patches has a pushout in L — 'two patches can be incompatible, such a pushout does not necessarily exist.' Rather than erroring or auto-resolving, they FREELY COMPLETE the category ($P =$ free finite cocompletion of L) so that EVERY conflict has a representable object: a file whose lines are only PARTIALLY ordered (a transitive relation, possibly with cycles), literally mirroring git's '<<<<<<< / ===== / >>>>>>>' markers as a first-class mathematical object. Conflict is a stored state, not a failure. Synthesis is late-bound: a 'merge' morphism (Example 19) or 'sequentialization' morphism (Example 18) resolves the conflict LATER, by an explicit user-chosen morphism — exactly our 'synthesis is a transformation of preserved plurality, not summary.'
- **Four-level graph versioning (graph / node / relation-type / node-data) that must COMPOSE.** — Offers a composition discipline and a partial caution. Their final object (Theorem 20) is a triple $(A, <, l)$: a finite SET of lines A (nodes), a transitive RELATION $<$ (relations/edges), and a LABELING $l:A \rightarrow L$ (node-data). Morphisms must SIMULTANEOUSLY respect set membership (partial function), relation ($a < a'$ implies $f(a) < f(a')$), AND labels ($l(f(a)) = l(a)$). This is a worked example of node-level + relation-level + data-level versioning composing through a SINGLE morphism condition, and merge (pushout) is computed by union-quotient + transitive closure across all three at once. Caution: they have no separate 'relation-TYPE' level (only one relation $<$) and only one node label set, so our 4th level (typed edges) is a genuine extension — the construction suggests it is another coordinate in the labeling/relation structure, but they do not prove it composes.

- **The trail-to-code join: DECLARED change (agent says what it did) vs INFERRED change (reconstructed by diffing).** — Sharpens the declared-vs-inferred distinction with a formal reason to prefer DECLARED. Their patch $f:[m] \rightarrow [n]$ is the DECLARED operation: it explicitly records which lines are kept, inserted (not in image), or deleted (undefined). The Related Work notes Darcs/operational-transforms axiomatize residuals but 'the fact that residual should form a pushout cocone is never explicitly stated.' The lesson for our trail join: if the agent DECLARES its edit as a morphism (kept/inserted/deleted line-identities), residuals and merges are computable by universal property; if we only have before/after snapshots and must INFER the morphism by diffing, we lose the identity information the pushout needs, and multiple incompatible patches could explain the same snapshot pair. Declared edits make the path+timestamp $\rightarrow$ code-at-that-moment resolution exact rather than heuristic.
- **Substrate choices: Rama event-sourced, multi-LLM, the 'correctness from universal properties' goal.** — Supports the event-sourced + derived-state stance philosophically. The paper's thesis is that the model is 'defined abstractly by the universal property that it should satisfy' and then given a concrete description — so behavior (merge, conflict representation) is DERIVED from a specification, giving 'a vcs whose correctness is deduced from universal categorical properties' with 'no unhandled corner-cases.' This is the formal-method analogue of our Rama-as-truth/projection-as-derived split: define the truth-layer's universal operations (merge=pushout) once, and all projections follow. Warning for substrate: they note the cocompletion FORCES cyclic / partially-ordered objects to even exist ('we have to allow cycles in the objects') — the truth representation for conflicts is necessarily richer (a relation with cycles) than the clean linear file, so our PState schema for a conflicted code-unit cannot be just an ordered list; it must store a transitive relation.

Takeaway. The single most important contribution for our versioning/identity question is the move from 'merge as procedure' to 'merge as universal property, conflict as a first-class object.' Mimram and Di Giusto show that if your edits are DECLARED as identity-preserving morphisms (which lines/units survive, not a diffed snapshot), then merge is exactly the pushout, residuals fall out for free, and — critically — when two edits conflict (no pushout exists), you do NOT fail or auto-resolve: you freely complete the category so the conflicted state is a genuine, storable object, namely a finite labeled set with a TRANSITIVE RELATION (a partial, possibly cyclic order) directly mirroring git's conflict markers. This is the precise mathematical license for our preserved-disagreement + late-bound-synthesis stance, and it tells us our truth-layer schema for a versioned/conflicted code-unit must store a relation (not a linear list), with synthesis modeled as an explicit later resolution morphism. The standing caution: the whole edifice depends on identity being declared at edit time — inferred diffs lose the morphism, and with it the guarantees."

9. Homotopical Patch Theory (Expanded Version)

Carlo Angiuli, Edward Morehouse, Daniel R. Licata, Robert Harper · 2014 · ICFP 2014 (expanded version, author homepage PDF)

In one line. Models version-control patch theory inside homotopy type theory: repository contexts are points, patches are paths, patch laws are paths-between-paths, and a model is a structure-preserving functor -- with identity defined as edit-history modulo commutation and an explicitly incomplete merge.

Source access: full. Read the full text from the author-homepage PDF (carloangiuli.com/papers/hpt-expanded.pdf). WebFetch returned only raw compressed PDF streams, so I downloaded the file (saved by the harness) and extracted text locally with pdftotext (1261 lines), reading all sections including the Appendix A addendum that was cut from the published ICFP version.

THE PAPER, SECTION BY SECTION

§ Abstract. States the thesis: homotopy type theory (HoTT) extends Martin-Lof type theory so that propositional equality is proof-relevant and corresponds to paths in a space, enabling higher inductive types (HITs) -- datatypes with constructors for points AND paths. The paper applies HITs to patch theory (the formalism behind Darcs), where a patch is a syntactic representation of an edit to a repository. The key architectural separation is between a *patch theory* (a formal language of patches, presented as a HIT) and its *models* (maps out of that HIT, which being

functors automatically preserve patch structure). Standard homotopy-theory tools (encode/decode, calculating fundamental groups) become practical programming techniques.

“Version control systems such as Darcs are based on the notion of patches—syntactic representations of edits to a repository.”

“A patch theory is presented as a higher inductive type. Models of a patch theory are given by maps out of that type, which, being functors, automatically preserve the structure of patches.”

§ 1. Introduction. Motivates the work: in HoTT, proofs of equality have computational content and can influence how a program runs, so the authors hunt for programming applications of computationally-relevant equality proofs. Patch theory is the chosen example. The central design decision: contexts act as TYPES for patches -- a patch 'delete file f' applies only where f exists (domain context) and yields a context where f is gone (codomain). Patches are closed under identity (no-op), composition (sequencing), and inverse (undo). They satisfy general laws (composition associative, inverses cancel) AND domain-specific laws ('the order of edits to independent lines of a file can be swapped'). The paper represents patches as paths and laws as paths-between-paths, and insists models depend crucially on proof-relevance: a model assigns each equality-proof a computational meaning as a function acting on repositories. Functoriality forces every model to respect identity/inverse/composition and validate the laws. Honest caveats: no implemented examples (would need HIT simulation in Agda/Coq), and no formal operational semantics yet, so 'how programs run' is given as sequences of propositional equalities.

“the contexts act as types for patches”

“a model must respect identity, inverses, and composition (e.g. sending composition of patches to composition of functions) and validate the patch laws”

“several of the examples in the paper are functions into contractible types ... up to paths/propositional equality, such a function can return any element of the type”

§ 2. Basics of Homotopy Type Theory. Reviews the machinery. 2.1 Paths: distinguishes definitional equality (proof-irrelevant, unmarked, includes beta-reduction) from propositional equality / the identity type (proof-relevant, marked: $\text{coe } p \text{ e coerces e along } p$). Path operations: refl , $\text{inverse } !p$, $\text{composition } q \circ p$, plus group(oid) laws ($\text{refl } o \text{ } p = p$, associativity, $!p \circ p = \text{refl}$) that hold as homotopies. $\text{ap } f$ lifts any function to an action on paths (functions are functors); apd is the dependent version producing a PathOver (heterogeneous equality across a type family). 2.2 n-types: a set/0-type has any two parallel paths equal; contractible = a mere proposition with an element. 2.3 Univalence: for sets, paths $A = B$ are given by bijections $A \leftrightarrow B$; ua turns a bijection into a path, and $\text{coe } (\text{ua } b) \text{ } x =$ applying the bijection -- so paths carry non-trivial computational content, and two different bijections give two differently-behaving paths. 2.4 Higher Inductive Types: generalize inductive types with path constructors; the circle = one point base + one path loop : $\text{base} = \text{base}$; circle recursion maps the generators (a point and a loop) into a target type, with computation rule $\text{ap } f \text{ loop} = !$ (taken as propositional, not definitional, equality in current models).

“In homotopy type theory, elements of the identity type are used to model a notion of paths in a space or morphisms in a groupoid.”

“two different bijections (f, g, p, q) and (f', g', p', q') determine two paths $\text{ua}(f, \dots)$ and $\text{ua}(f', \dots)$ that behave differently when coerced along.”

“Higher-dimensional inductive types ... generalize inductive types by allowing generators not only for points (terms), but also for paths.”

§ 3. General Patch Theory. The framework. A patch theory characterizes a VCS algebraically; a model is an implementation obeying the laws. Each patch has specified domain/codomain contexts (states it applies to / results in). 3.1 Patch Theories as HITs: contexts = points, patches = paths between context-points, patch laws = 2-dimensional paths (paths between paths). The big payoff: identity/inverse/composition AND the groupoid laws (associativity, unit, inverse-cancellation) 'come for free' from the HIT's path structure -- no boilerplate constructors needed. A subtlety flagged early: encoding patches as paths FORCES every patch to have an inverse ($p \circ !p = \text{refl}$ as well as $!p \circ p = \text{refl}$), which the authors note 'forces us to use some care' and sometimes 'gets in the way.' 3.2 Merging: merge takes a span $(f1, f2)$ (diverging patches from a common state) to a cospan $(g1, g2)$ reconciling them, required to satisfy $g1 \circ f1 = g2 \circ f2$ and to be SYMMETRIC ($\text{merge}(f1, f2) = (g1, g2) \Rightarrow \text{merge}(f2, f1) = (g2, g1)$) so

independently-computed merges agree -- essential for distributed collaboration. Darcs additionally requires merge to respect inverses, equivalent to 'pseudo-commutation,' Darcs' primitive operation. A total merge ALWAYS exists: $\text{merge}(f1, f2) = (!f1, !f2)$ undoes both changes, which is how a merge CONFLICT is signaled (human intervention needed). Critically, merge need NOT respect the groupoid structure -- a recursive 'tiling' merge can give different results depending on whether you compose first.

" $\text{merge}(f1, f2) = (g1, g2) \implies g1 \circ f1 = g2 \circ f2$ "

"It is always possible to define a total merge function, since for any span we may give $\text{merge}(f1, f2) = (!f1, !f2)$, the reconciliation that undoes both changes. This can be used to signal a merge conflict"

"a merge function that is a symmetric reconciliation need not respect the groupoid structure of a higher inductive type"

§ 4. An Elementary Patch Theory. Simplest example: repository = a single integer; patches = add/subtract n. One context num suffices (every patch applies anywhere), patches are paths $\text{num} = \text{num}$, generated by add1 . This HIT is 'just a renaming of the circle.' Remark 4.1 contrasts with the non-HIT alternative: a plain datatype with $\text{id}/\text{compose}/\text{inv}$ constructors would need a quotient type imposing 5 group laws (assoc, invr, invl, unitr, unitl) as boilerplate -- the HIT provides all this for free. 4.1 Interpreter: $I : R \rightarrow \text{Type}$ with $I \text{ num} = \text{Int}$, $\text{ap } I \text{ add1} = \text{ua}(\text{successor})$; $\text{interp } p = \text{coe-biject}(\text{ap } I \text{ p})$. Up to propositional equality $\text{interp add1} = \text{successor}$, and interp is functorial (takes composition to bijection-composition) because it is built from ap . Topologically I is the universal cover of the circle and interp computes the WINDING NUMBER -- a normal form counting net turns after detours like $\text{loop} \circ !\text{loop}$ cancel. Multiple sound models exist: I interprets in Bool with $\text{add1} = \text{not}(\text{add mod } 2)$, satisfying extra equations ($\text{add1} \circ \text{add1} = \text{refl}$) the Int model does not. The equational theory of R is complete for Int -- 'the fundamental group of the circle is $\mathbb{Z}$.' 4.2 Merge: since addition commutes, $\text{merge}(f1, f2) = (f2, f1)$; symmetric holds by definition; reconcile $(f2 \circ f1 = f1 \circ f2)$ requires a derived induction principle proving the loop group is abelian, via encode/repeat establishing $\text{num} = \text{num}$ is in bijection with Int -- 'our first example of a software correctness proof being a corollary of a theorem in homotopy theory.' merge' shows the technique of choosing canonical representatives (integers), computing on them with a function that need not respect group laws (merge! swaps), then mapping back -- the composite still respects the laws.

"This is, of course, just a renaming of the circle!"

"the integers give canonical representatives ("add x, for x in $\mathbb{Z}$ ") for equivalence classes of patches ... considered modulo the group laws"

"the correctness of merge follows from the fact that the fundamental group of the circle is $\mathbb{Z}$ —our first example of a software correctness proof being a corollary of a theorem in homotopy theory!"

§ 5. A Patch Theory with Laws. Adds domain-specific patch laws. Repository = one document of fixed n lines; one basic patch $s1 \leftrightarrow s2 @ i$ = 'permute strings $s1$ and $s2$ at line i ' (chosen because it induces a TOTAL bijection on $\text{Vec String } n$ -- self-inverse, fits the symmetric-paths requirement). The HIT R has point doc , path constructor $\langle - \rangle @ _$, and two 2-dimensional path constructors encoding laws: indep (edits at independent lines $i \neq j$ commute) and noop ($s \leftrightarrow s @ i = \text{refl}$). This is the paper's first type with both paths and paths-between-paths, so it details the recursion/induction principles: a function out of R must provide doc , swap , and proofs indep/noop , with computation rules beta1 (definitional) and $\text{beta21}/\text{beta22}$ (PathOvers , only propositional) -- the well-typedness of higher computation rules depends on lower ones. 5.2 Interpreter: $I \text{ doc} = \text{Vec String } n$, $\text{ap } I (s1 \leftrightarrow s2 @ i) = \text{ua}(\text{swapat} \dots)$; the law images become proof GOALS ($\text{GOAL0}/\text{GOAL1}$) that must validate the patch laws for the interpretation, solved via univalence rules ($\text{ua refl} = \text{refl}$; $\text{ua } b2 \circ \text{ua } b1 = \text{ua}(b2 \circ b1)$) plus the corresponding fact swapat -independent. 5.3 Optimizer: $\text{optimize} : (p : \text{doc} = \text{doc}) \rightarrow \Sigma(q : \text{doc} = \text{doc}). p = q$ -- returns an equivalent patch (optimizing $s \leftrightarrow s @ i$ to refl , saving two string comparisons). Two styles shown: 'program then prove' (write opt1 , prove opt1 -correct by induction, with the wrinkle that faces/2-paths are NOT automatically identified -- 'there is nothing that actually forces any two such faces to be identified' -- one can have proof-relevant patch laws, or use TRUNCATION to make them trivial); and 'program and prove' simultaneously by mapping into a $\Sigma(y : R). y = x$, which is CONTRACTIBLE (a singleton type $S(x)$), making the remaining coherence goals trivial. Key insight on singletons: even though optimize into a contractible type is homotopic to the identity function, it still has meaningful COMPUTATIONAL content -- 'computation does not respect extensional equality of functions.'

“indep ... (s <-> t @ i) o (u <-> v @ j) = (u <-> v @ j) o (s <-> t @ i)”

“there is nothing that actually forces any two such faces to be identified ... we could potentially consider proof-relevant patch laws—proof-relevant patch laws”

“even though the optimize function ... is equal (i.e., homotopic) to the function that simply returns p itself ... we expect ... it will in fact compute appropriately”

§ 6. A Patch Theory With Richer Contexts (the core contribution for versioning). The substantial example: patches ADD s@l (insert string s as line l) and RM l (remove line l) that are NOT total and NOT all composable -- removal from an empty file must be inadmissible. This needs MULTIPLE contexts. A naive indexing doc n by line-count fails because doc n = doc n+1 as a path, yet Vec n String and Vec n+1 String are not bijective, so univalence can't equate them. The decisive move (6.1): index contexts by PATCH HISTORIES -- 'contexts need only determine the contents of the repository, not literally contain them.' History m n is a quotient HIT of composable patches from m-line to n-line files, quotiented by commutation laws (e.g. ADD-ADD-< / ADD-ADD->= commute two additions with shifted line numbers; ADD-RM, RM-ADD, RM-RM omitted but analogous). Complete histories (History 0 n) applicable to the empty file UNIQUELY determine the file by 'replay' and serve as the patch contexts. R has point doc : History 0 n -> R and path constructors addP/rmP; because differing sequences are never parallel, the patch laws become PathOvers over the History quotient equations (addP-addP-< etc.). 6.2 Interpreter: I (doc h) = S(replay h) -- each context interpreted as the SINGLETON type of the exact file its history replays to; replay steps a complete history to its Vec String n; every function between singletons is automatically a bijection (single-biject). 6.3 Logs: an alternate model I'(doc h)=S(h) interprets contexts as singleton HISTORIES, giving interpH that updates the history; this shows histories can be RECOVERED from patches and ERASED at runtime ('it would be sensible to discard the histories at runtime'). 6.4 Merge: restricted to divergent paths with shared domain doc [] (so patches are forward sequences of ADD/RM with NO inverses -- 'a merge involving inverse paths is not sensible in this patch theory'). merge converts inputs to complete histories via interpH, calls mergeH (defined via Extension: h2 extends h1 if h1 is a prefix), and converts back via toPath/extToPath. mergeH itself is reduced to a merge on plain text files (e.g. via replay + existing diff algorithms), which 'would necessarily respect the patch laws because replay does.' This is the section the conclusion admits is INCOMPLETE.

“This is achieved by indexing contexts by patch histories, which are sequences of composable patches applicable to files of a given length. ... a history applicable to the empty file uniquely determines the resulting file's contents.”

“We can avoid these undesirable inverses by restricting merge to divergent paths with shared domain doc []. ... Users of a version control system will always encounter compositions of generating patches starting from the empty file”

“it would be sensible to discard the histories at runtime, through some erasure mechanism”

§ 7. Related Work. Positions against prior category-theoretic VCS analyses. Jacobson formalizes Darcs patches as inverse semigroups (partial bijections). Mimram & Di Giusto and Houston analyze merging as a PUSHOUT, giving a canonical merge for every pair including a primitive representation of merge conflicts; Houston also notes a duality with exceptions. The authors' distinguishing contribution: presenting patch theory in a categorical setting that is ALSO a programming formalism, so it leads directly to an implementation. They note these prior works allow non-invertible maps, whereas in HoTT all identity types are symmetric -- forcing the authors to either use naturally-total bijective patches (Sections 4-5) or restrict contexts via histories (Section 6) to fit the symmetric setting. Dagit uses advanced Haskell types to prove VCS invariants; the Camp (Commute And Merge Patches) project aims to prove Darcs patch theory correct in Coq. Swierstra & Loh use separation logic for patch behavior, which the authors suggest could describe histories with small-footprint specs.

“Mimram and Di Giusto analyze merging as a pushout, which provides a canonical merge for every pair of patches, including a primitive representation of merge conflicts.”

“In homotopy type theory all identity types are symmetric, and to fit patch theories into this symmetric setting, we either considered a language where all patches were naturally total bijections ... or used types to restrict patches to repositories where they are bijections”

§ 8. Conclusion. Summarizes: patch theories are HITs with generators for points (contexts), 1-paths (patches), 2-paths (patch laws); groupoid laws come free; a model is a functor into a univalent universe of sets and bijections

(sets = concrete repositories, bijections = patch actions), intrinsically functorial so it must respect the structure. Homotopy-theoretic 'encode/decode' identity-type characterizations become an implementation technique for merge: pass to a concrete representation, define merge there, pass back. Honest limitations stated plainly: the Section 6 merge is INCOMPLETE -- 'we have not proved the required properties of it,' because proving merge laws via mergeH needs a more precise inductive characterization of R's identity type (an inductive type equivalent to it, as Z is for the circle), left to future work. The computational interpretation of HoTT and the meaning of type erasure need further development. The most pointed reflection: requiring paths to be SYMMETRIC (have inverses) is 'both conceptually questionable and practically problematic'; they worked around it with histories (forward-only patches) but could NOT formulate pseudo-commutation, which seems to need a complete characterization of the space of spans of patches. Proposed future direction: a type theory with NON-symmetric paths grounded in directed homotopy theory, to express partial patches without refining contexts.

"the development of the merge operation in Section 6 is incomplete, because we have not proved the required properties of it."

"demanding inverses, rather than just retractions representing "undo" operations, is both conceptually questionable and practically problematic"

"a more precise characterization is needed, namely an inductive type which is equivalent to the identity type, as Z is to the identity type of the circle"

§ Appendix A. Addendum (interval, open interval, binary trees, quotiented trees, merge laws). Cut from the published version; clarifies the role of histories. A.1 Interval: the HIT I (zero, one, $\text{seg: zero}=\text{one}$) is CONTRACTIBLE (center zero), so $\text{zero}=\text{one}$ has no more elements than seg -- inverses add nothing because any path $\text{zero}=\text{one}$ must have one more seg than $!\text{seg}$, and groupoid laws collapse $\text{seg} \circ !\text{seg} \circ \text{seg}$ to seg . This is WHY restricting to forward paths eliminates inverses. A.2 Open Interval I ($\text{doc: Nat} \rightarrow I$, $\text{add1: doc } n = \text{doc } (n+1)$) is the half-line $[0, \infty)$; proving it contractible (center $\text{doc } 0$) shows there is a UNIQUE path $\text{doc } 0 = \text{doc } n$, so the codomain context determines exactly which patch sequence occurred -- 'patches applicable to $\text{doc } 0$ are characterized precisely by the index n of their codomain.' A.3 Binary Trees: two edits per context (add true/add false), contexts indexed by Bool List histories -- 'the patch theory looks like a tree, where the nodes are histories and the paths label the edges.' Contractibility again makes merge laws hold: reconcile follows because the type of merge forces a square and contractibility makes all squares commute; symmetric reduces to symmetry of mergeH. A.4 Quotiented Binary Trees: adds a commutation law (any two patches commute), so Bool Lists must be quotiented by permutation into MULTISSETS (MS, lists quotiented by Exchange of adjacent elements). The ex constructor equates $\text{add true} \circ \text{add false}$ with $\text{add false} \circ \text{add true}$ as PathOvers (since their endpoints differ as lists but are equal as multisets). Naively R seems to have non-trivial loops $\text{doc}[] = \text{doc}[]$, but the patch law ex 'trivializes these loops by equating the two sides' -- proving toPath respects the MS quotient 'verifies that ex fills in the loops.' A.5 Merge Laws for Richer Contexts: sketches that with a machine-checked proof (linked Agda: PatchWithHistories.agda / PatchWithHistories2.agda) R from Section 6 is contractible, so paths are uniquely determined by endpoints, the type of paths from $\text{doc}[]$ is equivalent to $\text{Sigma}(n). \text{History } 0 \ n$ (the path 'provides no additional information'), and merge laws on histories suffice for merge laws on paths.

"the codomain context $\text{doc } n$ determines exactly which sequence of patches has occurred."

"Now the patch theory looks like a tree, where the nodes are histories and the paths label the edges."

"The reconcile law follows from the contractibility of R : the type of merge specifies that p_1 , p_2 , q_1 , and q_2 form a square, and by contractibility, all squares in R commute."

HOW IT MAPS TO OUR PARTS

- **The code-identity question (identity = file vs path vs name vs AST vs git hash; declared vs inferred change)** — GIVES IT A NAME + MECHANISM. The paper's central move is that a repository context is NOT the file's contents but a *patch history* (a sequence of composable edits modulo commutation laws). Identity of a state = the canonical history that produced it, not the bytes. This is the 'identity = history modulo commutation' thesis: two edit sequences that commute denote the SAME context. Mapped to code identity, this argues identity is best modeled as a DECLARED operation-history (what the agent says it did), not an INFERRED

snapshot/AST-diff. The history 'uniquely determines the resulting file' but the file does not determine the history -- exactly our declared-over-inferred bias.

- **Versioning as its own layer (arrival-time log vs committed/minted versions; partial vs full persistence)** — SUPPORTS + WARNS. The paper separates a *patch theory* (formal language of edits, a higher inductive type) from a *model/interpreter* (how edits act on concrete repositories). This is precisely our split between the version layer and the materialized state it projects onto. Patches-as-paths give a free groupoid (identity/compose/inverse + associativity/unit laws) -- the algebra a commit/branch layer needs. WARNING: the paper found that requiring every patch to have an *inverse* (symmetric paths) was 'conceptually questionable and practically problematic'; an append-only arrival log (their History type contains 'only compositions of generators, and not their inverses') is the cleaner substrate. Validates Rama's append-only log as primary and committed versions as a derived layer.
- **Incomplete merge / preserved disagreement / plurality (synthesis is late-bound)** — SHOWS A FAILURE MODE + a name for our 'merge conflict as first-class'. The paper proves merge can ALWAYS be made total by 'undoing both changes' ($\text{merge}(f1, f2) = (!f1, !f2)$) -- a valid reconciliation that signals a conflict needing human intervention. Crucially it admits its own merge development is INCOMPLETE: 'we have not proved the required properties of it' because proving merge laws needs a complete inductive characterization of the *span of patches*, which they couldn't construct. Direct warning: a merge/synthesis operator that looks total can hide unproven correctness; conflicts (preserved disagreement) are the honest default, and forcing premature synthesis is where the theory breaks.
- **Object-container kernel: container = identity thread, revisions = immutable values, 'inferred structure is never accepted as truth at ingest'** — SUPPORTS strongly. Contexts (points) are the stable identity threads; patches (paths between points) are immutable transitions; the current file is a *projection* (replay of the history), never stored as truth-at-ingest. The paper's *I* interpreter even shows you can 'recover a history from a patch' and erase the concrete file at runtime -- the history (declared structure) is truth, the materialized vector-of-strings is a derived view. This is the container/revision split with the hard rule built in: inferred file contents are computed, never the source of identity.
- **Groupings as versioned objects with their own history, partly independent of members** — OFFERS A MECHANISM (functorial semantics / quotient HITs). A patch theory is a free structure; a *model* is a functor out of it that 'automatically preserves the structure of patches.' A grouping/local-world can be modeled the same way: define its own higher-inductive theory of admissible transitions, and bind members via a model (functor) that may differ (*I* vs *I'* -- one tracks file contents, another tracks logs/metadata). The quotient HIT (History modulo commutation, MS = multisets modulo Exchange) shows how to give a grouping a version history that is independent of member ordering -- pinned vs floating binding maps onto whether you quotient by commutation.
- **Four-level graph versioning that must COMPOSE (graph/node/relation-type/node-data)** — OFFERS A COMPOSITION DISCIPLINE. The paper's whole apparatus is layered paths-between-paths-between-paths: points (0-dim), patches (1-dim paths), patch-laws (2-dim paths/faces), and truncation/3-dim coherence. Each level's operations are forced to respect the level below by functoriality ('these come for free, from functoriality'). This is a worked example of multi-level versioning where higher levels COMPOSE coherently with lower ones by construction -- a model for making our 4 graph-versioning levels compose rather than drift, at the cost of coherence obligations (their GOAL0/GOAL1 face proofs) at every level.
- **The trail-to-code join (resolve a reasoning trail's path+timestamp to code-at-that-moment)** — OFFERS THE replay MECHANISM. Their 'replay : History -> Vec String' steps a complete history from the empty file to reconstruct the exact file at that point in time; 'histories also reify sequences of patches in a way which facilitates ... moving forward or backward in time.' This is exactly our path+timestamp -> code-at-that-moment resolution: an agent trail IS a declared history; replaying it to the relevant prefix yields the code state the trail was reasoning about, with no diffing/inference required.
- **Substrate choices (Rama event-sourced log) and the arrival-vs-commit two-times distinction** — SUPPORTS the event-sourced bet. The paper's most practical conclusion is that a VCS can be implemented by 'storing sequences of patches as paths, and repositories as vectors of strings,' updating by running *interp* on

each new patch -- an event-sourced design where the patch log is primary and state is a fold/projection. It also separates the *forward* History (arrival/append order) from reconstructed paths involving inverses (which it deliberately excludes), echoing our two distinct times: append-order is the safe substrate, committed/branchable structure is derived on top.

Takeaway. The single most important contribution to our versioning/identity question is the thesis that a state's IDENTITY is its edit-history modulo commutation, not its contents -- and that the clean substrate for this is a forward-only, append-style history (no inverses), with the concrete file/state computed by REPLAY as a derived projection that can even be erased. This directly validates our object-container kernel (history = truth, materialized state = projection, inferred contents never the source of identity) and our two-times model (the forward arrival-log is the safe primary; branchable structure is derived on top). The paper's most actionable warning is about merge: a total merge always exists by 'undo both' (the honest signal for an unresolvable conflict / preserved disagreement), but the authors could NOT prove their richer merge correct because that requires a complete characterization of the space of spans -- so we should treat merge/synthesis as late-bound and conflict-as-first-class rather than assume a clean total reconciliation.

10. Understanding Darcs/Patch Theory

Wikibooks contributors (community manual; draws on David Roundy's Darcs patch theory and FOSDEM 2006 conventions) · n.d. (living wiki page; content reflects mid-2000s Darcs patch theory) · Wikibooks (open community textbook)

In one line. An intuition-first, illustrated introduction to Darcs's patch algebra: patches are context-dependent transformations with inverses, commutation reorders them, and merge is defined purely from inverse+commutation — with conflicts explicitly deferred.

Source access: full. Read the full live Wikibooks page (en.wikibooks.org/wiki/Understanding_Darcs/Patch_theory) via two WebFetch passes — one for full section text, one to verify heading order and the (absent) conflict content. The page is open-access, no gating. Note: the deeper conflictor/conflict-resolution material is on a separate linked page ('More patch theory') NOT covered here; this breakdown covers only the Patch_theory page as requested.

THE PAPER, SECTION BY SECTION

§ 1. Math and Computer Science Nerds Only. A framing warning that the chapter is for 'math and computer science nerds (and the occasional physicist),' not casual users. It establishes that day-to-day Darcs use does not require this material, but that understanding how Darcs 'really works' demands engaging with the patch formalism. Sets expectation that the content is intuition-driven but mathematically grounded.

“Casual users be warned, the stuff you're about to read is not for the faint of heart!”

§ 2. What is the Theory of Patches?. Defines the patch formalism as the mathematical foundation for how Darcs exchanges patches between repositories, implemented concretely as Haskell data structures and functions over patch sequences. States the chapter's aim: convey the intuitions behind the math so readers can follow patch theory and the later 'conflictors' research. Frames a repository's behavior as operations over sequences of patches rather than over file snapshots.

“The patch formalism provides mathematical foundations for understanding how Darcs exchanges patches between repositories.”

§ 3. Why All This Math?. Argues that distributed VCS perform merging constantly, so correctness is critical, and that mathematical formalization serves two purposes: abstracting away implementation detail and guaranteeing the techniques stay 'fundamentally sound' as complexity grows. Crucially separates clean merging from conflict cleverness — the chapter begins with successful non-conflicting merges and defers conflicts. Also fixes notation: composition is left-to-right ($AB = B$ after A), per FOSDEM 2006 convention rather than the Darcs appendix.

“getting merging right does not necessarily consist of having clever behaviour with respect to conflicts”

“We will begin by focusing on successful, non-conflicting merges and move on to the darcs approach to handling conflicts”

§ 4. Context, Patches and Changes. Introduces the central concept: a CONTEXT names a repository state. Using a shopping-list file (apples/bananas/cookies/rice) labeled context o , it shows that when a user adds 'beer' at line 3, that act is a CHANGE, and Darcs records it as a PATCH capturing BOTH content ('beer') AND location (line 3). The notation oA^a denotes a patch A that takes the repository FROM context o TO context a . This is the load-bearing idea: a patch is inseparable from the contexts it connects — it is a typed arrow between states, not a free-floating diff.

“Darcs produces a patch recording both the content ('beer') and location (line 3)”

“The notation oA^a denotes moving from context o to context a via patch A .”

§ 5. Sequential Patches. Defines SEQUENTIAL patches as patches that apply successively, with shorthand collapsing the context annotations ($oA^a B^b \rightarrow AB$; $\dots \rightarrow ABC$). Establishes that a Darcs repository simply IS a sequence of patches. Motivates the rest of the theory: complex operations (like undo) require 'rearranging patches and putting them in different orders' while preserving repository coherence — which the following sections give precise definitions for.

“Darcs repositories are patch sequences”

“rearranging patches and putting them in different orders”

§ 6. Inverses. Defines the INVERSE: the inverse of patch P is P^{-1} such that composition PP^{-1} makes no net change to the context, and $(P^{-1})^{-1} = P$. Illustrated with the beer example: A adds beer (oA^a), A^{-1} removes it and returns to context o ($oA^a A^{-1}o$). States a hard requirement of the formalism: EVERY patch must have an inverse. This invertibility is what makes undo and (later) merge mechanically definable.

“The Inverse of patch P is P^{-1} , which is the patch for which the composition PP^{-1} makes no changes to the context and for which the inverse of the inverse is the original patch.”

“Every patch must have an inverse”

§ 7. Commutation. Defines COMMUTATION as reordering two adjacent patches while preserving correctness: $XY \leftrightarrow Y_1X_1$, where X_1 and Y_1 perform 'the same change' as X and Y but adjusted for the swapped context. Motivated by selective undo: if A (beer) then B (pasta), you cannot just delete A because B sits on top of it; you commute AB into B_1A_1 to surface A . Three sub-points: (a) 'Why Not Keep Our Old Patches?' — naive reordering breaks because line numbers/contexts shift (pasta at line 5 vs beer at line 3 must be recomputed), so commutation must produce contextually corrected variants; (b) 'The Complex Undo Revisited' — to undo a buried patch, commute it to the surface then apply its inverse (commute $AB \rightarrow B_1A_1$, then apply A_1^{-1}); (c) 'Commutation and Patches' — each patch type defines its own commutation rules (hunk patches adjust line offsets), and some pairs CANNOT commute (e.g., a file-add and a content-add into that same file).

“The commutation of patches X and Y is represented by $XY \leftrightarrow Y_1X_1$, where X_1 and Y_1 are intended to perform the same change as X and Y ”

“Some patches cannot commute (e.g., file addition with content addition to that file).”

§ 8. Merging. Builds MERGE entirely out of inverse + commutation. PARALLEL patches $A \vee B$ diverge from a common context o (Arjan's A reaches a , Ganesh's B reaches b). To let Ganesh pull Arjan's change, the algorithm: (1) start at oB^b ; (2) prepend B^{-1} to return to o ; (3) append A ; (4) commute $B^{-1}A$ into $A_1B_1^{-1}$ so A_1 now lives in Ganesh's context; (5) discard B_1^{-1} , leaving $oB^b A_1^c$. MERGE IS SYMMETRIC: the merge of A and B yields A_1, B_1 with $A \vee B \Rightarrow BA_1 \leftrightarrow AB_1$, so if Ganesh can pull Arjan, Arjan can equally pull Ganesh. A supporting 'commutation-with-inverse' property ($BA_1 \leftrightarrow AB_1$ iff $B_1A_1^{-1} \leftrightarrow A^{-1}B$, when both succeed) underwrites the algorithm's correctness. The whole construction handles only clean, non-conflicting merges — conflict is out of scope here.

“The result of a merge of two patches A and B is one of two patches A_1 and B_1 , which satisfy the relationship $A \vee B \Rightarrow BA_1 \leftrightarrow AB_1$ ”

“If you can commute the inverse of a patch A^{-1} with some other patch B , then you can also commute the patch itself against B_1 .”

§ 9. Definitions and Properties. A summary reference table consolidating the algebra: inverse (AA^{-1} has no effect), inverse-of-inverse ($(A^{-1})^{-1} = A$), inverse-of-composition ($(AB)^{-1} = B^{-1}A^{-1}$), commutation ($AB \leftrightarrow B_1A_1$), merge ($AVB \Rightarrow BA_1 \leftrightarrow AB_1$), and commutation-with-inverse. There is no reflective conclusion; the page ends by pointing to the next page, 'More patch theory,' where conflicts/conflictors are taken up.

“ $(AB)^{-1} = B^{-1}A^{-1}$ ”

“ $A \vee B \Rightarrow BA_1 \leftrightarrow AB_1$ ”

HOW IT MAPS TO OUR PARTS

- **Versioning as its own layer (when-it-arrived vs when-it-was-committed/true; partial vs full persistence)** — Darcs patch theory gives a name and an algebra to the distinction we care about: a patch is not a snapshot but a context-dependent transformation ($^oA^a$ means 'A taken FROM context o TO context a'). Identity of a change is the change-plus-its-context, not a fixed line-number diff. This warns us that our 'committed/minted version' layer must carry context, not just a value: the same logical edit re-expressed against a different base (A vs A_1) is a different patch object even though it 'performs the same change.' Commutation is exactly the machinery that lets you re-base a true/committed change onto a different arrival history without losing identity.
- **The code-identity question (is identity the file/path/AST/hash, atomic unit file vs function)** — Darcs answers a sibling question and the answer is instructive: the atomic unit of identity is the PATCH (the change), not the file state. A repository IS a sequence of patches, and any file-state is a derived projection. This supports treating our object-container's REVISIONS as the values and the EDIT OPERATIONS (in trails) as first-class identity-bearing things — but it also warns: Darcs identity is purely operational/syntactic (hunks at line offsets), it has NO semantic/AST notion of identity, so commutation can fail or mangle when changes touch the same region. If we want function-level identity that survives moves, we cannot get it for free from a patch algebra.
- **The trail-to-code join (DECLARED change vs INFERRED change)** — This is the strongest mapping. A Darcs patch is a DECLARED change carrying both content and location ('records both the content (beer) and location (line 3)') — it is the agent saying what it did, with its context, not a reconstructed diff. That is exactly our DECLARED-change pole. The paper shows the payoff: because the change is declared WITH its context, you can mechanically commute/re-base it to resolve a trail's edit onto a different code state, rather than re-diffing. It validates storing trail edits as context-tagged operations so the path+timestamp resolution is an algebraic re-contextualization, not a guess.
- **Object-container kernel hard rule: 'inferred structure is never accepted as truth at ingest'** — Darcs is built on the same instinct, restated: it never trusts a positional diff as ground truth across divergence. The 'Why Not Keep Our Old Patches?' subsection is a concrete failure mode — applying a patch made in context o directly onto context b inserts at the WRONG line because the inferred line number is stale. The discipline is: only the declared change-in-its-context is trusted; any cross-context application MUST go through commutation (an explicit, lawful transformation), never naive reapplication. This is a precise mechanism for our 'don't accept inferred structure at ingest' rule.
- **Preserved disagreement / plurality; conflicts late-bound synthesis** — Darcs's design philosophy directly supports late-bound synthesis: 'getting merging right does not necessarily consist of having clever behaviour with respect to conflicts' — it separates clean parallel composition ($AVB \Rightarrow BA_1 \leftrightarrow AB_1$, fully symmetric) from conflict handling, which it explicitly DEFERS to a later treatment (conflictors). The lesson for us: a merge of two divergent local worlds should first be modeled as symmetric re-contextualization where it commutes cleanly, and conflict should be a separate, named, deferred construct — not silently resolved at merge time. WARNING: this page does NOT actually deliver the conflictor model, so we cannot cite a concrete preserved-disagreement mechanism from it, only the architectural stance.
- **Groupings as versioned objects; Rama event-sourced substrate** — A Darcs repo = an ordered, append-style sequence of context-linked patches, which is structurally our event log: the patch sequence is primary and

any state is a fold over it. This supports our Rama 'event log is primary, rendered state is projection' stance. But the merge algorithm reveals a cost relevant to groupings: to undo a buried patch you must (1) commute it to the surface, (2) apply its inverse — i.e., reordering is a real algebraic operation with offset bookkeeping, not free. A versioned grouping whose members are patch-like changes inherits this: pinned vs floating member binding is essentially whether you re-commute (re-base) a member onto the grouping's new context or freeze its original context.

Takeaway. The single most important contribution is the reframing of a version as a CONTEXT-DEPENDENT, INVERTIBLE TRANSFORMATION rather than a snapshot or a context-free diff: a patch is a typed arrow α^a between named states, and reordering/re-basing it onto a different history is a lawful algebraic operation (commutation), not a re-diff or a guess. For Softland this directly underwrites the DECLARED-change pole of the trail-to-code join — store each agent edit as content+location tagged with its context, and resolve it onto code-at-a-moment by commutation rather than inference, satisfying our 'inferred structure is never truth at ingest' rule. The cautionary footnote is that Darcs's identity is purely syntactic (line/hunk offsets) and the page DEFERS conflicts entirely — so it gives us the algebra of clean re-contextualization and a model for late-bound conflict, but not a semantic/AST identity nor an actual preserved-disagreement mechanism.

What we can gather — Angle 2

This angle is the formal floor, and it returns three results that change how confidently we can build.

First, our design is a known framework, not an invention. Conradi & Westfechtel's 1998 survey is our architecture under older names: container = *versioned item* / OID, revision = VID, our two times = *extensional vs intensional* versioning — and their emphatic ruling is that these "are by no means mutually exclusive, but can (and SHOULD) be combined." They name our grouping hazard precisely — **version proliferation**, where naive version-first grouping forces a new version of every ancestor — and its fix is exactly our instinct: give configurations their own version history, distinct from their members. The cheapest high-value move available is to adopt this vocabulary wholesale; it makes every future contract and Codex gate cheaper to argue.

Second, the two times are formally distinct objects, and one of them is genuinely hard. DSST (1989) proves partial persistence (read any version, extend only the tip — our Rama arrival log) and full persistence (modify/branch from any version — our minted committed layer) are different mechanisms: full persistence needs a version *tree*, not a line, plus a total-order version list to navigate it. Both cost $O(1)$ amortized via structural sharing — but **confluent merge (synthesizing branches back together) was explicitly left open in 1989**, and it is still the hard part. This is the formal warning against promising clean automatic merge.

Third, the patch theories give us our deepest principles. Across Pijul, the categorical theory (Mimram-Di Giusto), the homotopical theory (Angiuli et al.), and Darcs, two ideas recur with proof behind them: (a) **identity and change are declared, never inferred from content** — a line's identity is the change that introduced it, and if you only store snapshots you must infer the morphism and inference is lossy; and (b) **a conflict is a first-class value, not an error** — formally the free cocompletion, concretely "a finite set of lines with a *transitive relation*, not a linear order." That last result is a literal schema instruction for us: a conflicted/plural code-unit in a PState must store a *relation*, not a list. And the homotopical result — identity = history *modulo commutation of independent edits* — is the formal statement of two things we already believe: the event log is primary, and semantic time is a DAG (independent edits commute, so they are unordered).

What we gather: build the two times explicitly and combined; encode "conflict is a stored relation" into the plurality layer; and treat synthesis/merge as the late-bound, still-open frontier rather than a feature we can promise total.

Angle 3 — Versioned Databases & Content-Addressed Stores

11. Mergeable Persistent Data Structures

Benjamin Farinier, Thomas Gazagnaire, Anil Madhavapeddy · 2015 · JFLA 2015 (26es Journées Francophones des Langages Applicatifs); system = Irmin / MirageOS

In one line. Irmin gives any OCaml data structure git-style persistence — an immutable content-addressed block store plus a tiny mutable tag store — and extends queues and ropes with a recursive 3-way merge whose only constant cost is a ~5x overhead.

Source access: full. Read the original full PDF (266KB, extracted via pdftotext) from the author's site anil.recoil.org/papers/2015-jfla-irmin.pdf — abstract, all 5 sections, type signatures (Figs 1,2,4,5,8), the queue worked example (push=6/pop=2), the rope merge_branch/merge_node functions (Figs 9,10), complexity tables (Figs 3,7), and benchmarks (Figs 12,14). Also cross-read Adrian Colyer's *The Morning Paper* summary at the requested blog URL for framing. The blog post is the requested URL; the primary technical source quoted here is the JFLA PDF itself.

THE PAPER, SECTION BY SECTION

§ Abstract. States Irmin is an OCaml library for purely functional data structures that 'can be persisted on disk and be merged and synchronized efficiently.' The paper's focus is the merge aspect, demonstrated on two structures — queues and ropes — that 'extend the corresponding purely functional data structures with a 3-way merge operation.' Promises early theoretical and practical complexity results, and notes Irmin is open-source within MirageOS.

“Irmin is an OCaml library to design purely functional data structures that can be persisted on disk and be merged and synchronized efficiently.”

“queues and (ii) ropes that extend the corresponding purely functional data structures with a 3-way merge operation.”

§ 1. Introduction. Frames the thesis: take DVCS workflows (used to manage source code, track provenance, relate bugs to the change that caused them) and apply them to application DATA. The motivating question is 'What if you could version-control a (mutable) persistent data structure, inspect its history, clone a remote state and revert it?' Rather than a conventional database, the functionality is delivered as a LIBRARY linked into application logic. Irmin reuses git's concepts (clone/push/pull/branch/rebase) and maintains a bidirectional mapping with the git on-disk format so standard tools (tig, gitk) work. Positions the work as a generalization of the Concurrent Revisions framework to arbitrary history-graph shapes and an extension of CRDTs that uses 3-way merge instead of encoding operation sequences into state. Crucially observes that while any data structure can be made persistent, the authors 'do not believe such a technique exists for mergeable data structures' — hence Irmin provides an immutable key-value store over which users build their own mergeable structures. Lists the two conflict situations (concurrent local edits; divergent remote edits) and three resolution techniques: CRDTs, a custom merge operator, or explicit first-class conflict handling.

“What if you could version-control a (mutable) persistent data structure, inspect its history, clone a remote state and revert it to a previous state?”

“Our work is a generalization of the Concurrent Revisions framework to handle arbitrary shape of history graphs.”

“we use 3-way merge instead of encoding the sequences of operations in the state itself.”

“While it has been shown that is possible to make any data structure persistent, we do not believe such a technique exists for mergeable data structures.”

§ 2. Irmin Architecture. Describes a mutable high-level interface built on two backend stores. (1) The BLOCK STORE: a low-level, append-only key/value store where keys are deterministically computed from values (e.g. SHA). Two consequences are spelled out: modifying a value creates a new key/value pair (the store is immutable),

and two stores sharing the same values get the same keys (the store is consistent; structure depends only on data). It holds serialized application content AND structured data (prefix-tree nodes, history metadata); being append-only it has no remove, grows forever, and relies on GC/compression. (2) The TAG STORE: 'the only mutable part of the system,' a key/value store whose keys are user-created names and whose values are block-store keys — 'a set of named pointers,' expected to be local and very small, and central to synchronization and GC. The high-level interface lifts immutable block operations into a mutable prefix-tree (Fig 1: read/update/remove over a path = list of strings, with mergeable contents at nodes). Figure 2 gives the merge contract: result is ['Ok of t | 'Conflict of string] and merge: old:t -> t -> result. The section closes with the GOOD-MERGE definition that serves as the paper's correctness spec.

"the block store: this is a low-level key/value append-only store, where values are a sequence of bytes and keys are deterministically computed from the values (for instance using SHA algorithms)."

"The tag store: this is the only mutable part of the system... a set of named pointers to keys in the block store... expected to be local to each replica and very small."

"Given two data structures to be merged, the result of every operation applied on these structures since their common ancestor has to be present in the result of the merge operation."

"type result = ['Ok of t | 'Conflict of string] val merge: old:t -> t -> result"

§ 3. Mergeable Queues (incl. 3.1 Implementation, 3.2 Merging). Queue chosen for ubiquity and for the simplicity of stating merge semantics: every element popped in either queue must be ABSENT from the result, and all elements pushed in either queue must be PRESENT in the same order. Implemented as a functional queue = two linked lists (a push list and a pop list); when the pop list empties, the push list is flushed into it ('normalization'), giving amortized $O(1)$ push/pop (Fig 3: push = 2 writes; pop $\approx$ 2 reads/1 write; merge = $O(n)$ reads, 1 write). The Irmin store holds three element types (Fig 5): Index (fields push, pop, top, bottom — the push/pop COUNTS are what merge uses), Node (next/previous/elt/branch pointers, where branch holds a concatenated queue's index), and Elt (the values). Merge takes q_1 , q_2 , and common ancestor old, and proceeds in steps: using only the push/pop counts (no element access), remove from q_1 the old-elements that q_2 popped, and likewise drop old-elements still shared, then concatenate q_2 onto q_1 by adding a node whose branch field is q_2 's index. Worked example: old={push 6,pop 2}, q_1 ={push 8,pop 2}, q_2 ={push 7,pop 4} normalizes to q_1 '={push 8,pop 4}, q_2 '={push 7,pop 6}, and the final concatenation is $O(1)$. Worst-case merge reads are linear in |old| but reducible to logarithmic via binary random-access lists.

"A functional queue is composed of two simple linked lists... When the pop list is empty, the push list is flushed into the pop one. This operation is called normalization."

"The two first fields, push and pop, are the number of pushes and pops applied to the queue since its creation. They are useful for the merge operation."

"elements of old which have been removed from q_2 are removed from q_1 . This is done without accessing these elements by using the push and pop values."

"using binary random-access lists instead of classical chained lists will reduce the overall number of reads to the logarithm of the size of the old."

§ 4. Mergeable Ropes (incl. 4.1 Implementation, 4.2 Merging). A rope is a self-balancing binary tree whose leaves hold short strings; each node indexes by the total length in its left subtree, giving $O(\log n)$ set/get/split/concat/insert/delete vs $O(1)/O(n)$ for raw strings (Fig 7). The design is generic: 'give me a container with a set of operations, and we will return you a rope on this container... achieving a better complexity' — so the user must supply a merge on the leaf container because a general merge over arbitrary containers is impossible. A key application: in MirageOS the rope is instantiated as a FILE SYSTEM with pages as atomic leaves, yielding 'a Git-like persistent file system that can be distributed over several machines.' The store again holds Index/Node/Leaf. Merge (4.2) uses the block KEYS of the two trees and their common ancestor to locate the smallest changed subtrees, via two recursive functions: merge_branch (Fig 9) does key comparison across three cases (both branches equal $\rightarrow$ return either; one side unchanged vs old $\rightarrow$ take the other; modifications in different branches $\rightarrow$ take the modified one; else None), and merge_node (Fig 10) pattern-matches on (left_option, right_option) into a GOOD case (deduce immediately), a RECURSIVE case (recurse into the None subtree), and a

BAD case (no automatic deduction → linearise both subtrees into containers and call the user merge). When changes are in 'differentiable places' the result is deduced conflict-free. Rebalancing is the efficiency enemy — rotations deeply reshuffle structure and defeat key comparison — so rotation is applied as little as possible and only to the smallest unbalanced subtree, making merge cost proportional to the VOLUME of modification.

“the rope can be instantiated as a file system, using pages as rope atomic elements. In combination with Irmin, we obtain a Git-like persistent file system that can be distributed over several machines.”

“their keys in the Irmin store are used to determine the smallest subtrees where modifications occurred.”

“if these subtrees are separated... we can therefore include all of them being sure there is no conflicts. If it is not the case, these subtrees are linearised and flushed into containers on which the user-defined merge operation is applied.”

“In order to minimize the scope of rebalancing operations, the rotation function is applied as less as possible, and only on the smallest unbalanced subtree.”

§ 5. Analysis (5.1 Automatic checking, 5.2 Benchmarking). Correctness (5.1) uses an oracle: a sequence of classical operations is run on the implementation and on a witness from another library, requiring exact match at every step. This cannot validate MERGE because no witness implementation exists, so they instead pick results with an easily-checkable invariant preserved by merge (e.g. a queue of increasing numbers, a rope of nondecreasing characters), decompose them, and re-merge to recover the original — confirming merge correctness at each use. Tests passed but cannot confirm asymptotic complexity, motivating benchmarks (5.2). Benchmarks confirm the theoretical complexities are reached and reveal a 'stair' pattern (each new tree level) and spikes (balancing vs leaf-length interactions). Backends compared: Memory (RAM), GitMem and GitDsk (git format in memory / on disk), Obj (an Obj.magic heap backend using memory addresses as block addresses, for raw comparison only), and Core (the Core library's functional queue, used as the optimal baseline). Headline result: 'there is a factor five between Core and the implementation of our queue on the Obj backend' — deemed 'an acceptable price to pay for maintaining a mergeable data structure.'

“it does not exist at our knowledge another data structure to be used as a witness.”

“we chose a result with a easily verifiable property which is preserved after each merge operation... a queue containing an increasing sequence of numbers, or a rope composed by a nondecreasing sequence of characters.”

“there is a factor five between Core and the implementation of our queue on the Obj backend. It is an acceptable price to pay for maintaining a mergeable data structure.”

§ Backends & Git format (within Analysis / 5.2). The bidirectional git-format mapping is concrete, not aspirational: GitMem and GitDsk backends store Irmin's content-addressed blocks in actual git object format so that application commits show up in the git repository and manual git changes flow back through Irmin. The append-only, content-addressed block model maps cleanly onto git's object store (blobs/trees keyed by hash), and the small mutable tag store maps onto git refs/branches. The Obj backend (Fig 13) deliberately bypasses hashing — 'maps Irmin blocks to the OCaml heap and use the memory address as block address (instead of its hash)' via Obj.magic — purely to measure raw overhead against Core, and is explicitly not advised for real use.

“any changes committed by an application are reflected in the associated Git repository, and any manual change in the Git repository is reflected back in the application via Irmin.”

“the OCaml heap, which maps Irmin blocks to the OCaml heap and use the memory address as block address (instead of its hash)... We do not advise to use it in other contexts.”

§ Conclusion / positioning. Irmin demonstrates that DVCS workflow — branchable history, clone/merge/revert, provenance — can be delivered as a library over purely functional, hash-consed, content-addressed data, at a modest (~5x) constant cost. The two showcase structures (queue, rope) prove the 3-way merge approach generalizes beyond text, with the rope-as-filesystem result directly enabling MirageOS's git-like distributed file system. The authors flag future work: formalizing the merge algorithms in a proof assistant such as Coq to strengthen the informal correctness argument, and reducing merge read-cost via binary random-access lists.

“We demonstrate this increased flexibility power comes with a very reasonable cost, thanks to the use of purely-functional data structures and (implicit) hash-consing.”

“We are planning to translate these algorithms into a proof assistant such as Coq in order to assert them more strongly.”

HOW IT MAPS TO OUR PARTS

- **Object-container kernel (stable container = identity thread; revisions = immutable values; 'inferred structure is never accepted as truth at ingest')** — Irmin's two-store split is a direct architectural template. The content-addressed block store is the immutable-revisions layer: every value is keyed by its own hash, so a modification creates a new key/value pair and 'the resulting data-store is immutable.' The mutable tag store ('named pointers to keys in the block store') is the container/current-revision layer: a stable user-chosen name whose value (a block-hash) moves over time. This validates our split of stable identity thread (tag) over immutable values (blocks). It also supports the hard rule: blocks are truth-by-content (hash = what it is), and only the tiny mutable tag layer asserts 'this is current' — structure is content-addressed, identity assignment is explicit and minimal.
- **The code-identity question (file vs path vs name/FQN vs AST vs git hash; atomic unit file or function)** — Irmin answers the analogous question for data: the atomic unit is the smallest content-addressed block, and identity-over-time is NOT the hash (which changes on every edit) but the externally-minted tag name. Mapped to code: the git-hash IS the revision identity (immutable value), while the container name (path/FQN) is the identity thread. The rope-as-filesystem instantiation ('pages as rope atomic elements') shows the same content can be sub-divided into finer content-addressed atoms (pages/functions) under one identity (file) — supporting 'atomic unit = function inside a file-container' rather than forcing file-granularity.
- **Versioning as its own layer: arrival-time history (partial persistence) vs explicit committed/minted git-style versions (full persistence) — two distinct times** — Irmin is the canonical full-persistence system: 'clone, push, pull, branch, rebase' over an arbitrary history DAG ('a generalization of the Concurrent Revisions framework to handle arbitrary shape of history graphs'). This is exactly our committed/minted-version layer — you can branch from any committed node. It warns, though, that full persistence of MERGEABLE structures is not free: 'while it has been shown that is possible to make any data structure persistent, we do not believe such a technique exists for mergeable data structures.' So our two-time design (Rama arrival-time log + explicit git-style commits) cannot assume merge comes for free at the committed layer; merge must be defined per data-structure.
- **Groupings as versioned objects (a curated local world has its own version history, partly independent of members; pinned/floating bindings)** — Irmin's prefix-tree (the high-level mutable interface 'generated over the block store, the tag store') is exactly a grouping that has its own commit identity while its leaf contents have theirs. Because every node — including interior structure/'history meta-data' — is itself a content-addressed block, a grouping's version is a hash over its members' hashes: change a member, the grouping gets a new version automatically (floating binding). To PIN a member you reference its specific block-hash; to FLOAT you reference through the tag/tree. This gives a concrete mechanism for member-binding modes.
- **Four-level graph versioning (graph / node / relation-type / node-data must compose)** — Irmin's hash-consed Merkle tree is the composition mechanism we need: 'if two data-stores share the same values, they will have the same keys; the overall structure only depends on the stored data.' A change at node-data level bubbles a new hash up through node, relation, and graph levels deterministically, so each level is independently versioned yet provably composed (a graph-hash uniquely determines all sub-level hashes). This is direct evidence that content-addressing makes multi-level versioning compose for free — the same property git relies on.
- **Preserved disagreement / plurality; synthesis is late-bound** — This is the paper's core contribution and its main caution. Irmin makes divergence first-class (two replicas, arbitrary history DAG) and offers THREE conflict strategies: CRDTs, a custom 3-way merge operator, or 'explicit handling of conflicts as first-class citizen in the API.' The recursive merge only auto-resolves when changes are in 'differentiable places'; otherwise it returns 'Conflict of string' or flushes subtrees to a user-defined merge — it never silently overwrites. This validates our late-bound synthesis stance and supplies the exact signature: merge: old:t -> t -> t -> result. The merge

correctness law ('the result of every operation applied since their common ancestor has to be present in the result') is a usable specification for our synthesis layer.

- **The trail-to-code join (resolve path+timestamp to code-at-that-moment; declared vs inferred change)** — Irmin's 3-way merge depends critically on a COMMON ANCESTOR ('old') being known, not inferred: the queue merge uses old's push/pop counts to compute the diff without reading elements, and the rope merge compares block-keys against old.left/old.right to locate changed subtrees. Mapped to trails: a stored commit DAG lets us resolve a trail's path+timestamp to an exact ancestor revision (declared change against a known base), rather than reconstructing change by diffing two states blind. The Acolyer summary flags the gap — the paper 'assumes knowledge of a common ancestor state but the discovery method remains unaddressed' — warning us that finding the LCA is our responsibility, not a freebie.
- **Substrate choices (Rama event-sourced; cost of the versioning machinery)** — Irmin gives an honest cost number for git-format versioned data: 'there is a factor five between Core and the implementation of our queue on the Obj backend... an acceptable price to pay for maintaining a mergeable data structure,' with backends ranging from in-RAM to GitMem/GitDsk. This calibrates expectations for layering explicit git-style commit/merge on top of Rama: a ~5x overhead on the optimal in-memory case is the order of magnitude to budget. The append-only block store ('there is no remove function; the store is expected to grow forever') matches Rama's append-only log substrate exactly — same growth assumption, same reliance on GC/compaction.

Takeaway. Irmin gives our versioning/identity layer its strongest existence proof and its exact shape: separate immutable, content-addressed REVISIONS (the hash-keyed append-only block store) from a tiny mutable IDENTITY layer of named pointers (the tag store), and you get git-style full persistence — branchable history, provenance, clone/merge/revert — over arbitrary data, with multi-level versioning composing for free because every level is a hash over its children. The critical, citable warning for us is that merge is NOT free at the committed layer: persistence is automatic but mergeability is per-structure, requires a KNOWN common ancestor (LCA discovery is the caller's problem), and resolves conflicts only by an explicit user merge (old:t->t->t->result) that must never silently overwrite — exactly our preserved-disagreement/late-bound-synthesis stance — all at a calibrated ~5x overhead.

12. Prolly Trees (Probabilistic B-trees) — Dolt Storage Engine Architecture

DoltHub (Dolt documentation team) · 2023 · DoltHub official documentation (docs.dolthub.com / www.dolthub.com)

In one line. A B-tree variant with content-defined, rolling-hash chunk boundaries and content-addressed storage that gives B-tree read/write performance plus O(change) diffs and automatic structural sharing across versions — the storage engine under the version-controlled SQL database Dolt.

Source access: full. Read the full official Dolt documentation page. Original URL <https://docs.dolthub.com/architecture/storage-engine/prolly-tree> 301-redirects to <https://www.dolthub.com/docs/architecture/storage-engine/prolly-tree>, which I fetched in full (three passes: full text, then two targeted verbatim-quote passes). This is a docs page, not a peer-reviewed paper; section names below are its real structural headings. The data structure was originally invented by the Noms team; this page is Dolt's description of its own implementation.

THE PAPER, SECTION BY SECTION

§ Introduction & Motivation. Defines a Prolly Tree as a 'Probabilistic B-tree' that is 'particularly effective as the basis of the storage engine for version controlled databases.' It frames three simultaneous goals that conventional structures cannot all meet: B-tree-class random read/write performance, fast diff computation, and structural sharing across versions. A motivation table claims prolly trees match B-trees on random ops at $\log k(n)$ but compute diffs in time proportional to the difference size d rather than total data n . The defining version-control advantage is content addressing: portions of the tree are shared between versions because intermediary storage is content-addressed.

""Prolly Trees can also structurally share portions of the tree between versions due to the content-addressed nature of their intermediary storage.""

“Prolly trees are “particularly effective as the basis of the storage engine for version controlled databases.””

§ How Prolly Trees Work — B-tree Review. Reviews ordinary B-trees as sorted key->value maps with pairs in leaf nodes, then names their two disqualifying limitations for versioning. First, finding differences between two B-trees requires scanning the entire structures ($O(n)$). Second, and central to the whole design, B-tree shape is order-dependent: the insert/update/delete sequence changes internal structure, so two trees with identical contents but different write histories can differ structurally. This non-determinism is exactly what defeats cheap diffing and structural sharing, motivating the history-independent construction that follows.

““writes to B-trees are not history independent, the order of the writes internally changes the structure of the tree.””

§ How Prolly Trees Work — Construction Algorithm. Gives the six-step build: (1) sort the key-value map; (2) determine chunk boundaries with a rolling/strong hash over keys, using a seed, target chunk size, and the key value — a boundary is declared when the hash falls below a target threshold; (3) hash each resulting chunk and store it content-addressed; (4) if a single content address remains, done; (5) otherwise build the next layer up as a map of each chunk's highest key to its content address; (6) recurse until one root remains. Because boundaries are content-defined (a function of the data itself, not of write order), the resulting tree is deterministic for a given dataset — the property the B-tree review identified as missing.

““You store the contents in a content addressed block store.””

““Prolly trees are stored in a content addressed block store where the content address forms the lookup key for the block.””

§ Modification Operations — Updates, Insertions, Deletions. Describes copy-on-write modification. A fixed-size value update rewrites the value in a copied chunk, recomputes that chunk's content address, then walks upward recomputing each parent's address up to a new root; only the path from the changed leaf to the root is rewritten, everything else is shared. Variable-length changes may force local re-chunking. Insertions behave by position: at the beginning they perturb the left edge, at the end the right edge, in the middle a 'diagonal spline' of affected chunks; deletions mirror insertions. Crucially the page quantifies boundary stability: with average 4KB chunks the chance that a single-byte change shifts a chunk boundary is about $1/4096$ ($\sim 0.02\%$), so almost all edits are local.

““In Dolt's Prolly Tree implementation, chunks are set to be on average 4 kilobytes. This means that you can imagine a probability of $1/4096$ or 0.02% of triggering a chunk boundary shift when changing a single byte.””

§ Key Properties — History Independence. States the foundational invariant: the tree is a pure function of its contents, independent of operation order. Any sequence of inserts/updates/deletes that yields the same key-value set yields the same tree, the same chunk boundaries, and the same root hash. This determinism is what makes the two downstream features (fast diff, structural sharing) possible — equal contents are guaranteed to produce equal, shareable subtrees, and two databases that converged to the same state are byte-for-byte identical in storage.

““No matter which order you insert, update, or delete values, the Prolly tree is the same.””

§ Key Properties — Fast Diff Algorithm. Explains the diff as a parallel descent over two trees comparing content addresses level by level: if two nodes share a hash, their entire subtrees are identical and are skipped wholesale; the algorithm recurses only into branches whose hashes differ, bottoming out at the actually-changed key-value pairs. Cost is therefore set by the number of differing chunks, not tree size. This is the structural payoff of history independence plus content addressing, and it is the property that makes Dolt's commit-to-commit diffs fast at any database scale.

““If the hash of the root chunk is the same the entire subtree is the same.””

““This algorithm scales with the size of the differences, not the size of the tree.””

§ Key Properties — Structural Sharing. Because each block's lookup key IS its content address, any two versions that contain a byte-identical chunk physically store it only once; unchanged subtrees across commits/branches are automatically deduplicated. A new version after an edit costs only the rewritten path from changed leaf to new root ($\log_k(n)$ chunks), with the rest of the tree referenced by shared hashes. This is the storage

basis for keeping many committed versions and branches cheaply — the cost of a version is proportional to its change, not the database size.

“any blocks that share the same content address are only stored in the block store once.”

§ Implementation Details — Chunk Size Control. Documents Dolt's improvement over the original Noms design, which produced geometrically/exponentially distributed chunk sizes (many tiny chunks, a few huge ones). Dolt instead targets a normal distribution around 4KB by adjusting the boundary probability with a cumulative-distribution-function formula $(\text{CDF}(\text{end}) - \text{CDF}(\text{start})) / (1 - \text{CDF}(\text{start}))$, which raises the boundary probability as a chunk grows. The result is more uniform chunk sizes, improving cache behavior and predictability without sacrificing content-defined boundaries.

“we use the formula $(\text{CDF}(\text{end}) - \text{CDF}(\text{start})) / (1 - \text{CDF}(\text{start}))$ to calculate the target probability.”

§ Implementation Details — Key-Only Hashing. A pivotal design decision: where Noms hashed both keys and values to place boundaries, Dolt hashes ONLY keys. Two consequences. First, value edits cannot move chunk boundaries, so an update is guaranteed local (rewrite one chunk, no re-chunking) — maximizing structural sharing and diff locality. Second, it sidesteps a low-entropy failure mode where ordered/sequential data hashed by value would chunk poorly. This separates the identity/ordering axis (keys) from the content axis (values), a clean engineering lesson for any identity-keyed versioned store.

“any updates to values is guaranteed not to shift the chunk boundary”

§ Implementation Details — Type-Less Block Storage. Notes that Dolt's block store, unlike Noms, does not encode type information inline within blocks; type metadata is kept out of band. This 'type-less' on-disk layout reduces per-block overhead and improves read and write performance. It is a comparatively minor optimization relative to the chunking and key-hashing decisions, but reinforces the theme that Dolt re-engineered Noms's proly-tree storage for performance and uniformity.

“type information is stored 'out of band'; the 'type-less layout improves Dolt read and write performance.”

§ Performance Analysis. Tabulates proly trees against B-trees: random read $\log k(n)$ for both; random write $\log k(n)$ for B-trees versus $(1 + k/w) \cdot \log k(n)$ for proly trees — a small penalty from the occasional chunk split; diff calculation n for B-trees versus d for proly trees. The narrative concedes a slight write cost from the low-probability boundary shift but emphasizes the asymptotic diff win. This is the quantitative case that proly trees buy version-control superpowers ($O(\text{change})$ diff, sharing) at near-B-tree operational cost.

“Proly Trees can produce differences in time proportional to the size of the differences.”

“writes incur 'a slight performance penalty on writes due to the small probability of a chunk split.’”

§ Real-World Usage in Dolt. Grounds the structure in the shipping product: every piece of Dolt's data is a proly tree — a table is a map of primary key to data columns; a secondary index is a map of index values to primary key; schemas and keyless tables also use proly maps, all over one shared content-addressed block store. Benchmarks put Dolt roughly comparable to MySQL, with gaps attributed to implementation language (Go vs C), SQL-analyzer maturity, and data-transformation overhead — NOT to the proly tree itself. The closing claim restates the payoff: Dolt computes diffs proportional to change size and structurally shares data across versions.

“a map of primary key to data columns is stored in a Proly Tree. Similarly, secondary indexes are maps of index values to the primary key.”

“Dolt 'can compute diffs in time proportional to the size of the differences' and 'structurally shares data across versions.’”

HOW IT MAPS TO OUR PARTS

- **Versioning as its own layer: arrival-time history (partial persistence) vs explicit committed/minted versions (full persistence)** — Proly trees are the mechanism Dolt uses to make EVERY committed version cheap to keep: structural sharing means a new commit stores only changed chunks (~the d in the diff), not a

full copy. This is a concrete proof-of-feasibility for our 'full persistence' / git-style minted-version layer: branching and retaining many committed snapshots is affordable when unchanged subtrees are shared by content address. It does NOT address arrival-time history (that is Rama's log); it addresses the OTHER time axis — when-it-was-committed — and shows the storage cost of keeping all those committed states is proportional to total change, not number-of-versions x size.

- **Object-container kernel: container = identity thread, revisions = immutable values, current-revision = state** — The proly tree gives a name to the substrate beneath our revision model: revisions are immutable content-addressed values; the container's history is a chain of root hashes over a shared block store. The hard rule 'inferred structure is never accepted as truth at ingest' is REINFORCED here — proly tree identity is the content hash of the actual bytes/keys, fully determined by data (history-independent), never by inferred or order-dependent structure. A revision's identity = its root content address; equality of two revisions is an $O(1)$ hash compare, exactly the 'if the root hash is the same the subtree is the same' property.
- **The trail-to-code join (first consumer): resolve a trail's path+timestamp to code-at-that-moment; declared vs inferred change** — Proly-tree diff (proportional to d , descend only into differing branches) is the mechanism for cheaply reconstructing INFERRED change between two committed code states. But the doc's strength is structural sharing + content addressing, which lets us pin a trail to an exact root hash (code-at-that-moment) and resolve a path against an immutable snapshot. Declared change (agent says what it edited) maps to a known key-range; the proly diff between adjacent roots is the inferred-change cross-check. Caveat: proly diff is key-level, not semantic — it tells you WHICH keys changed, not whether the change is meaningful, so it informs but does not replace declared change.
- **The code-identity question: file vs path vs FQN vs AST vs git hash; atomic unit = file or function** — Dolt's design choice — hash KEYS ONLY, not values, so 'any updates to values is guaranteed not to shift the chunk boundary' — is a direct lesson for our identity question. It separates the IDENTITY/ordering dimension (keys = stable address: path or FQN) from the CONTENT dimension (value = the code bytes/AST). If we make our atomic unit's key its stable identity (e.g. FQN or container-id) and its value the code text, edits to a function body change one chunk without re-chunking neighbors. This warns against keying by content hash (git-hash-as-identity), which would re-shuffle boundaries on every edit and destroy structural sharing.
- **Four-level graph versioning (graph / node / relation-type / node-data), must compose** — Proly trees compose hierarchically by construction (leaf chunks $\rightarrow$ parent map of highest-key+chunk-address $\rightarrow$ recurse to single root), and Dolt stores tables, secondary indexes, schemas all as nested proly maps sharing one block store. This is a working example of MULTI-LEVEL versioning that composes through a single content-addressed root: a change at node-data level bubbles new hashes up to the graph root, while untouched levels keep their old hashes and are shared. It supports our requirement that the four levels compose into one version identity, and gives the mechanism (one root hash over nested maps) for how lower-level changes roll up without recopying unchanged levels.
- **Groupings as versioned objects with their own history, partly independent of members** — A grouping = a proly map whose keys are member-ids and whose root hash is the grouping's version. Structural sharing means the grouping's identity changes ONLY when membership/binding changes, independent of churn inside members that are referenced by stable key. This supports pinned bindings (key $\rightarrow$ specific member root hash) vs floating bindings (key $\rightarrow$ member container, latest). Warning: proly trees are history-independent and have no native notion of effectivity/condition-scoped binding — that temporal/conditional logic must live in OUR layer above the tree, not be expected from it.
- **Preserved disagreement / plurality; late-bound synthesis** — Proly trees give cheap, persistent BRANCHES (any root hash is a fork point, unchanged subtrees shared across branches), which is the storage substrate for holding conflicting versions side-by-side without overwrite. But the doc offers NO merge/synthesis semantics — content-addressed diff tells you keys differ, not how to reconcile. So this maps as: proly trees make plurality cheap to STORE and cheap to DIFF, but late-bound synthesis (our merge policy) is entirely our responsibility above it.

- **Substrate choices: Rama (event-sourced log) as our chosen substrate** — This is a fork-comparison warning, not an endorsement. Dolt's two strengths (diff proportional to change, structural sharing across versions) are delivered by content-addressed proly trees, a DIFFERENT substrate than Rama's append-only log + PStates. Rama gives arrival-time time-travel for free but does not natively give git-style branchable committed versions with $O(d)$ diff and structural sharing. The takeaway for us: if we want Dolt-like committed-version semantics on top of Rama, we must BUILD a content-addressed, history-independent revision store (the proly-tree role) as a layer; Rama alone provides the log, not the full-persistence committed-version structure.

Takeaway. The single most important contribution is the proof that you can keep an unbounded number of committed, branchable versions cheaply by combining content-addressed storage with a history-independent (order-independent) tree: storage cost and diff cost both scale with the SIZE OF CHANGE (d), not the number or size of versions (n). For our versioning/identity question this gives a concrete mechanism for the 'full-persistence / git-style minted version' layer and a sharp design rule — key by stable identity, hash keys not values, so edits touch one chunk and unchanged subtrees are shared by content address. It also marks the boundary: proly trees make plurality, branching, and diff cheap to STORE and DETECT, but carry no semantic merge, no effectivity-scoped binding, and no arrival-time history — those remain our layer's job on top of Rama.

13. Version Controlled Databases: Defining a Category

Oscar Batori (DoltHub) · 2020 · DoltHub Engineering Blog

In one line. Coins 'version controlled database' as a product category — an append-only DB with native branch/merge/diff over both data and schema — and positions Dolt against Noms, Irmin, and TerminusDB by query interface.

Source access: full. Read the full blog post (a ~6-minute engineering post, not a formal paper) directly via WebFetch at the canonical DoltHub URL; no paywall. Confirmed it is a short category-definition piece with no deep technical sections on the storage structure.

THE PAPER, SECTION BY SECTION

§ Opening: the two-concepts distinction. The post's central move is to separate two phrases that are routinely conflated. 'Database version control' means storing your schema (and schema migrations) in a Git-like system external to the DB — 'the database itself knows nothing of its version history.' A 'version controlled database' is the opposite: a class of append-only databases that natively offer branch, diff, and merge across BOTH schema and data. The author's stated goal is reclaiming the term from schema-migration tooling for products that are actually databases with version control built in.

""Database version control' and 'version controlled database' are not the same thing.""

""a 'version controlled database' represents a class of append-only databases that offer traditional version control features like branch, diff, and merge across both schema and data""

""reclaim the term 'version control database' for products that actually are databases with version control features""

§ Version Controlled DB vs Database Version Control (the definition). This is where the category is formally defined. A version controlled database 'stores a full history of its own state, both data and schema.' The defining feature set is: the ability to store a history of state, PLUS some form of branching, merging, and diffing across that stored history. Crucially the history lives inside the database, not in an external VCS that is blind to the DB's contents. This is the criteria list the rest of the post measures candidate systems against.

""Is a database that stores a full history of its own state, both data and schema.""

""the ability to store a history of its state, along with some version of branching, merging, and diffing across the stored history""

""The database itself knows nothing of its version history.""

§ Why Now? (the enabling structure). Argues the category is newly feasible due to three converging factors: the rising value of data in production systems (invoking Karpathy's 'Software 2.0' — behavior determined by data, not code), a long-run decline in storage costs, and 'the advent of Merkle tree.' This is the ONLY place the enabling data structure is named: Merkle trees make it 'tractable to expose a full data history to a query interface.' The post gives no further technical depth — no proly tree, no content-addressing internals, no structural-sharing explanation — so the Merkle reference is asserted as the unlock, not elaborated.

""computer behavior is increasingly determined by data rather than code""

""the advent of Merkle tree make it tractable to expose a full data history to a query interface, and therefore to build true version controlled databases""

§ Why Adopt — Branching/Merging. Frames branch and merge as the core justification: they are 'fundamental primitives for collaboration,' letting many users work concurrently against the same base dataset and then 'resolve their differences in a principled and robust way.' The contrast is with conventional databases that assume a single concept of state, forcing any versioning/reconciliation logic to be hand-built in the application layer — which the author treats as a fragility this category removes.

""Branching and merging are fundamental primitives for collaboration.""

""work concurrently against the same base dataset, and then resolve their differences in a principled and robust way""

""any tools that permit versioning and reconciliation [must be] built into the application layer""

§ Why Adopt — Diffing & distribution. Elevates diff from a review convenience to an infrastructural primitive: robust diffing is 'what enables version control systems to effectively implement operations like clone and pull.' This makes a version controlled DB an 'excellent data distribution format' — history and semantics are native, so consumers do not reverse-engineer the producer's conventions (e.g. which string means NULL). Summarized benefits: collaboration as first-class (hundreds of concurrent editors), full reproducibility by tagging runs with the input/result commit hash, and distribution reduced to clone/pull instead of parse-and-load.

""not just useful for review, it is what enables version control systems to effectively implement operations like clone and pull""

""It shouldn't be necessary to figure out what strings a data vendor uses to signify NULL, it should be native to the distribution format.""

""associating runs with commit hashes""

§ Examples — TerminusDB, Noms, Irmin, Dolt. Surveys four members of the category along the implicit dimension of QUERY INTERFACE and host language. TerminusDB: a graph database with its own query language, Git-inspired branch/merge/diff. Noms: a Git-like distributed database storing structured objects defined by Go structs, versioning both the object DEFINITIONS and the VALUES they take — and the substrate Dolt is built on top of. Irmin: a storage-layer-agnostic, OCaml database 'quite similar in spirit to Noms' but objects are described/updated in OCaml. Dolt: the author's entry, a SQL database implementing the MySQL standard with Git primitives. The differentiator across all four is what language/interface the user must adopt.

""Dolt is built on top of Noms.""

""provides versioning and synchronization primitives for both the object definitions themselves, and values those objects take on""

""Irmin is a storage layer agnostic database inspired by Git, and written in O'Caml.""

""TerminusDB is a graph database... inspired by Git, and supports branching, merging and diffing in a Git-like interface.""

§ Why Dolt. Argues Dolt's strategic bet is the query interface: by implementing the MySQL standard it lets users keep SQL and the existing connector ecosystem ('nine programming languages with tested MySQL connectors') rather than adopting a bespoke language as Noms/Irmin/TerminusDB require. The honest tradeoff is stated plainly: the version-control storage (a commit graph) is a poor natural fit for a SQL engine, so building a MySQL superset over it is 'a huge technical commitment.' This is the post's only acknowledgment that marrying version-control storage to a familiar query model is hard.

""We believe that SQL is the language of choice for the vast majority of our potential users.""

""The underlying commit graph storage is not a natural fit for a SQL query engine, and implementing a super set of MySQL on top of that storage layer is a huge technical commitment.""

§ Conclusion. Restates the category claim: the industry has historically treated version control as something done TO databases, not a feature OF them, and Dolt's category inverts that. It stresses the two ideas are not competitors — version controlled databases do not replace the practice of version-controlling your schema; they serve users who specifically need collaboration, versioning, reproducibility, and distribution at the data layer, without writing application code.

""The software industry has long considered version control something that is done to databases, not a feature of databases.""

""not competitive with the practice of version controlling your database schema""

""elevate the quality of their data infrastructure without the need to write and maintain application code""

HOW IT MAPS TO OUR PARTS

- **Versioning as its own layer (arrival-time vs committed/minted time; full vs partial persistence)** — Names and validates our core distinction. Dolt's whole thesis is that a 'version controlled database' STORES its own history with explicit branch/merge/diff as first-class DB features — exactly our 'explicit, git-style COMMITTED/MINTED versions you can branch from' (full persistence) layered ABOVE whatever the substrate does. The post warns of the cost: 'The underlying commit graph storage is not a natural fit for a SQL query engine... a huge technical commitment.' That is the warning for our two-times design: bolting committed-version semantics onto an arrival-time log is real engineering, not free.
- **Object-container kernel (stable container = identity thread; immutable revisions inside; current-revision = state)** — Supports it via the Noms model the post cites: Noms 'provides versioning and synchronization primitives for both the object definitions themselves, AND values those objects take on' — i.e. a stable object identity (container) distinct from the immutable values (revisions) it takes over time. Merkle/content-addressing is the mechanism that lets revisions be immutable and shared while the container persists as an identity thread.
- **Substrate choices: Rama (event-sourced) as the versioning enabler** — Offers a mechanism + a name. The post pins the enabling condition: 'the advent of Merkle tree make it tractable to expose a full data history to a query interface.' Append-only history + content-addressed structural sharing is what makes time-travel queries affordable. Rama gives us the append-only log; this paper says the missing ingredient for cheap branch/diff is content-addressed (Merkle) structure over that history — a hint for how we'd build the committed-version layer.
- **The trail-to-code join: DECLARED change vs INFERRED change** — Reinforces the 'inferred structure is never accepted as truth' rule. The post elevates DIFF to a first-class, native, robust operation rather than a reconstruction: 'Robust diffing... is what enables version control systems to effectively implement operations like clone and pull,' and history should be 'native to the distribution format,' not parsed/guessed. This favors DECLARED change captured at commit time over INFERRED change reconstructed by post-hoc diffing — the same instinct behind our hard rule.
- **Four-level graph versioning (graph / node / relation-type / node-data, composing)** — Partial support + a gap. The post names a graph-shaped version-controlled DB (TerminusDB, 'a graph database... branching, merging and diffing in a Git-like interface') and stresses versioning spans 'both schema and data.' Schema-vs-data ~ our relation-TYPE-level vs node-DATA-level distinction. But the post offers NO mechanism for composing those levels — it's a category-definition piece, not an implementation, so it warns by omission that multi-level composition is unsolved here.
- **Groupings as versioned objects (a curated local world with its own history)** — Supports via the collaboration framing: 'Branching and merging are fundamental primitives for collaboration' letting users 'work concurrently against the same base dataset, and then resolve their differences.' A branch is exactly a grouping/local-world with its own independent version history that later merges back — our pinned/floating/merge semantics for grouped members.

- **Preserved disagreement / plurality (late-bound synthesis)** — Directly supports. Dolt's model treats divergence as normal: concurrent branches hold conflicting states co-present and 'resolve their differences in a principled and robust way' at merge time. That is preserved disagreement with late-bound synthesis, implemented at the data layer rather than 'built into the application layer' — the post's explicit critique of single-state databases.

Takeaway. The post's durable contribution is the clean category definition we should adopt verbatim for our versioning layer: a true version-controlled store keeps a FULL history of its own state (data AND schema) inside the substrate and exposes branch/merge/diff as native operations — never bolted on at the application layer and never reconstructed by guessing. For Softland that validates making committed/minted versions a first-class kernel feature over Rama's arrival-time log, and the Noms lineage it cites (versioning object DEFINITIONS separately from the VALUES they take) is precisely our object-container vs immutable-revision split. Its one honest warning: the version-control storage (a content-addressed commit graph) is 'not a natural fit' for a familiar query interface — wiring our committed-version graph to ergonomic queries is the real cost, not the history-keeping itself.

14. So you want Database Version Control?

Tim Sehn (DoltHub) · 2021 · DoltHub Engineering Blog

In one line. A vendor argument that 'true' database version control means full git-style branch/merge/diff of BOTH schema and data, achievable via proly-tree storage, with Dolt as the only SQL database delivering cell-wise diff and merge on rows.

Source access: full. Full original blog post read directly at dolthub.com (not gated). This is a company engineering blog, not a peer-reviewed paper; it has named structural headings rather than formal academic sections, and it argues a market thesis rather than proving results. Two WebFetch passes: one full-text, one targeted on proly trees / cell-wise diff / schema-vs-data / SQL system-tables / AS OF.

THE PAPER, SECTION BY SECTION

§ Introduction — the search-result framing. Opens with a rhetorical complaint: googling 'database version control' returns schema-migration tools (Liquibase, Redgate), which the author calls 'both too specific and antiquated.' The framing move is to claim the term has been wrongly narrowed to schema migration, and that new storage technology now makes true versioning of schema AND data possible. This is a positioning argument, not a technical contribution — it sets up the schema-vs-data axis the rest of the post organizes around.

“In 2021, this query response to 'database version control' is both too specific and antiquated.”

§ What do you mean by database version control?. Decomposes the term into a set of dimensions before evaluating solutions: database type (relational vs other), versioning target (schema, data, or both), desired capabilities (rollback, time travel, lineage, diff, branch/merge), and conflict detection/resolution. This taxonomy is the post's most useful analytic move — it makes explicit that 'versioning' is a bundle of distinct capabilities, and that most tools deliver only a subset. It maps cleanly onto our own need to separate arrival-time rollback from committed branch/merge.

§ Schema — migrations as the term of art. Defines the conventional approach: 'The term of art for version controlling database schema is database migrations.' You baseline a schema in version control and track changes as patches. The author notes DoltHub itself runs Postgres with schema+patches checked into Git as 'common practice,' and names Liquibase, Redgate, and PlanetScale as the representative tools. The implicit critique: migrations version the schema deltas but say nothing about the DATA, leaving half the problem unsolved.

“The term of art for version controlling database schema is database migrations.”

§ Data — four approaches and the proly-tree breakthrough. The technical heart. Surveys four ways people version data and rejects the first three. (1) Slowly-changing-dimensions / soft-deletes via active-inactive or temporal columns — dismissed because 'the application or an operator can still make a mistake and do really bad things.' (2)

Backups + transaction logs — rollback exists but on hours-to-days timescales, too coarse. (3) SQL AS OF temporal tables (SQL:2011, Oracle/SQL Server/MariaDB) — give 'instant rollbacks' but 'You don't get diffs and merges.' (4) The breakthrough: Noms' proly tree, 'a content addressed binary tree,' which 'allows for fast diff and merge on binary trees, the core data structure in most databases, without compromising too much read or write performance.' TerminusDB (graph/doc) and Dolt (relational) are built on this. Crystallizes the thesis: 'database version control means full Git-style versioning, branch, merge, and diff, of schema and data.'

"You don't get diffs and merges but you do get instant rollbacks."

"Noms laid the groundwork with a core storage format called a proly tree, a content addressed binary tree."

"This data structure allows for fast diff and merge on binary trees ... without compromising too much read or write performance."

"We think database version control means full Git-style versioning, branch, merge, and diff, of schema and data."

§ The options — database migration tools. Evaluates three schema-only tools against the data axis. Liquibase (2012): config-language descriptors (XML/SQL/JSON/YAML) that migrate cross-platform (Postgres to MySQL), supports rollback and preview; mature but schema-only. Redgate Deploy (2014): SQL-Server-only, Visual Studio integration, has limited 'static table data' versioning but explicitly 'intended only where the table has fewer than a thousand rows' — a concrete admission that row-level data versioning was an afterthought bolted onto a schema tool. PlanetScale (2016, from Vitess creators): adds schema branch/merge plus a modern deployment environment, but is MySQL-only and database-as-a-service. The common verdict: all version schema, none truly version data.

"intended only where the table has fewer than a thousand rows"

§ The options — version controlled databases (Dolt). Presents the two proly-tree databases. TerminusDB (2019, graph, WOQL query language, git-like branch/push/pull) gets a favorable nod. Dolt (2019, 'It's Git for Data') is the recommended answer. Key mechanisms: Dolt 'implements the Git command line and associated operations on table rows instead of files' — i.e. it re-bases git's unit from the file/blob to the row. Workflow: edit data+schema in a working set via SQL, snapshot via commits. The SQL surface is split by operation kind: 'dolt implements Git read operations (ie. diff, log) as system tables and write operations (ie. commit, merge) as functions.' Granularity: 'Dolt produces cell-wise diffs and merges, making data debugging between versions tractable.' Runs offline (data-as-source-code) or online (like Postgres/MySQL). Closing claim: 'the only SQL database on the market that has branches and merges.'

"Dolt implements the Git command line and associated operations on table rows instead of files."

"In SQL, dolt implements Git read operations (ie. diff, log) as system tables and write operations (ie. commit, merge) as functions."

"Dolt produces cell-wise diffs and merges, making data debugging between versions tractable."

"That makes Dolt the only SQL database on the market that has branches and merges."

§ Conclusion. Restates the thesis as a recommendation: for a SQL database that needs genuine version control over both schema and data, the author claims exactly one product qualifies. This is an unhedged vendor conclusion — useful as a clear statement of the position (committed branch/merge over rows beats temporal/AS OF rollback) but unsupported by independent evaluation or benchmarks.

"We think if you want Database Version Control for a SQL database, there is only one product that fits that label and that's Dolt."

HOW IT MAPS TO OUR PARTS

- **Versioning as its own layer: arrival-time history vs committed/minted versions (partial vs full persistence)** — Gives our two-times split a concrete, named foil. Dolt's **AS OF** temporal tables = arrival-time/partial-persistence rollback ('You don't get diffs and merges but you do get instant rollbacks'). Dolt's *commits* = explicit minted full-persistence versions you can branch/merge from. The post's central thesis is that backups/transaction-logs/AS OF are NOT 'true version control' precisely because they lack the committed-

branch layer — exactly the argument for why Rama's arrival-time log is insufficient and we must add a git-style committed layer on top.

- **Object-container kernel: stable container = identity thread, revisions = immutable values, 'inferred structure is never accepted as truth at ingest'** — Dolt's proly tree (content-addressed binary tree from Noms) is a mechanism for exactly our container/revision split: each commit is an immutable content-addressed value; identity/history is the chain of hashes. Content-addressing means a revision's hash IS derived from its bytes, not declared — which both supports our immutable-revision model and warns about our hard rule: a content hash is computed structure, so equating identity with hash conflates container with value. The post's 'cell-wise' granularity argues identity should be tracked below the file/blob boundary.
- **The code-identity question: is the atomic unit the file or the function; is identity path/name/AST/hash** — Dolt's core move — 'implements the Git command line and associated operations on table rows instead of files' — is a direct precedent for rejecting the file as the atomic unit. Git versions files (blobs); Dolt re-bases identity on rows+cells. The analogue for us: don't version the .clj file, version the function/AST-node, with cell-wise (field-wise) diff. This is the strongest single support in the source for 'atomic unit = function, not file.'
- **Four-level graph versioning (graph / node / relation-type / node-data), must compose** — Dolt separates row/cell-level data versioning from schema versioning, and the post repeatedly insists 'true' version control covers BOTH 'schema and data' together. That is a partial precedent for level-composition: schema-version maps to our relation-TYPE level, cell-data maps to our node-DATA level. But it's a WARNING too — the post admits Dolt doesn't cleanly separate them ('doesn't explicitly separate these as distinct features'); schema and data co-version in one commit. Our four-level requirement is more ambitious than anything shipped here.
- **The trail-to-code join: DECLARED change vs INFERRED change** — Dolt produces 'cell-wise diffs and merges' — i.e. INFERRED change by diffing two committed values. The post never offers a declared-change channel; all change knowledge is reconstructed post-hoc by diff. This both gives us a mechanism (cell-wise structural diff to resolve trail path+timestamp to code-at-that-moment) and marks the gap our design fills: the agent's DECLARED edit operation is information Dolt-style systems throw away and recompute.
- **Preserved disagreement / plurality, late-bound synthesis** — Branch + cell-wise merge with conflict detection is the mechanism for preserved disagreement: two branches hold conflicting values simultaneously, conflicts are surfaced at merge (late-bound synthesis) rather than silently overwritten. 'The only SQL database that has branches and merges.' Supports our plurality model; the proly-tree three-way merge is a concrete substrate pattern.
- **Substrate choices: Rama event-sourced log as our version substrate** — Cautionary mapping. Dolt's whole value rests on the proly tree giving 'fast diff and merge ... without compromising too much read or write performance' — a purpose-built versioned storage engine. Rama gives us the event log (arrival time) but NOT cell-wise structural diff/merge for free. The post implies that committed-version diff/branch/merge is a storage-engine-level concern, warning that we must build the diff/merge layer atop Rama PStates rather than assume the log provides it.
- **Groupings as versioned objects with independent history** — Weak/absent in the source — a noted gap. Dolt versions tables and rows but a Dolt commit versions the whole database snapshot atomically; there is no notion of a curated sub-grouping with its own independent version line and pinned/floating member binding. The post offers no mechanism here, signalling this is novel surface our design must invent rather than borrow.

Takeaway. The single most important contribution is the sharp distinction between rollback-versioning and branch/merge-versioning, and the claim that only the latter is 'true' version control. Mapped to us: Rama's arrival-time log ($\approx$ AS OF temporal tables — instant rollback, no diff/merge) is necessary but NOT sufficient; we must add an explicit committed/minted layer that supports branch and cell-wise merge, exactly our two-times split. Dolt's concrete mechanism — re-basing git from files to rows/cells via content-addressed proly trees — is direct precedent for making the function/AST-node (not the file) the atomic versioned unit with field-wise diff. The cautionary note: Dolt versions whole-database snapshots atomically and reconstructs all change by diffing

(inferred, never declared), so it offers no model for independently-versioned sub-groupings or for a declared-change channel — both of which our design must invent rather than borrow.

15. XTDB v1 Documentation — Bitemporality

JUXT (XTDB project documentation) · ~2021 (v1 docs, undated page) · XTDB v1 official docs (v1-docs.xtdb.com)

In one line. A documentation page defining XTDB's bitemporal model — independent valid-time and transaction-time axes, append-only transaction time, retroactive/proactive corrections, and a 'partially persistent, fully retroactive' index — illustrated by a border-crossing crime-investigation example.

Source access: *full. Read the live page at <https://v1-docs.xtdb.com/concepts/bitemporality/> in full via two WebFetch passes (concepts pass + worked-example pass). This is a docs page, not a peer-reviewed paper; the underlying theory citation it leans on is Snodgrass's bitemporal work and the Driscoll et al. 'persistent data structures' terminology, which the page references by name ('partially persistent and fully retroactive data structure') but does not reproduce.*

THE PAPER, SECTION BY SECTION

§ Overview. Establishes that XTDB is bitemporal: every fact is positioned on two independent timestamp axes, transaction-time and valid-time. These two axes together let the database answer point-in-time queries across both an audit history (what did we know, and when) and domain-specific temporal data (what was actually true in the world, and when). The framing is that these are orthogonal coordinates, not a single timeline — a database state is a function of a (valid-time, tx-time) pair.

“XTDB implements a bitemporal model using two independent timestamps: transaction-time and valid-time.”
“These axes enable point-in-time queries across both audit history and domain-specific temporal data.”

§ Transaction Time vs. Valid Time. Defines the two axes precisely and asymmetrically. Transaction-time is 'the point at which data is transacted into the database' — it is an immutable audit trail, is automatically set to 'now' when unspecified, and crucially CANNOT be written to the past (append-only). Valid-time is arbitrary: it can come from an upstream system or default to the transaction-time, and it is the axis users normally query against to ask 'what is the value of this entity at valid-time?' The asymmetry is the heart of the model: transaction-time is monotonic/append-only (you can never rewrite when you learned something), while valid-time is freely placeable in past or future (you can assert anything about when a fact was/will be true). A table assigns transaction-time to audit/event-sourcing purposes and valid-time to historical analysis and user queries.

“Transaction Time represents 'the point at which data is transacted into the database.' It provides an immutable audit trail and cannot be written to the past.”
“Valid Time is 'an arbitrary time that can originate from an upstream system, or by default is set to transaction-time.'”
“what is the value of this entity at valid-time?”

§ Retroactive and Proactive Operations. Describes the two directions writes can take along the valid-time axis. Retroactive operations write into the PAST of valid-time — they correct historical records when errors surface, asserting that something was true earlier than (or differently from) what was previously recorded. Proactive operations write into the FUTURE of valid-time — scheduling states that will become true later. The key consequence stated is that this decoupling lets data arrive out-of-order without compromising query consistency: because transaction-time independently records arrival, a retroactive correction does not destroy the prior belief, it just adds a later-known fact about an earlier valid-time.

“Writes can be made in the past of valid-time as retroactive operations.”
“Writes can also be made in the future of valid-time as proactive operations.”
“This decoupling allows data to arrive out-of-order without compromising query consistency.”

§ Bitemporal Querying (crime-investigation example). Demonstrates querying by pinning a coordinate on EACH axis simultaneously. The worked example tracks persons p1–p7 crossing US borders; each document carries xt/id,

entry-pt (SFO/LA/NY), arrival-time, and departure-time (:na if still present). Facts are ingested over several days: Day 0 records persons 2 and 3 arriving (valid-time Dec 31 2018); Day 2 records person 4 arriving NY; Day 3 person 4 departs; Day 4 (transaction-time) RETROACTIVELY records person 1's arrival back at valid-time Dec 31 2018 plus their departure, and records person 3 departing. The investigative query 'find all persons known present in the U.S. on day 2 (valid-time) as of day 3 (transaction-time)' is expressed by passing both ::xt/valid-time #inst 2019-01-02 ('as at') and ::xt/tx #inst 2019-01-03 ('as of'). Result: persons 2, 3, 4 — i.e., the world as it was BELIEVED on tx-day-3 about valid-day-2, deliberately excluding facts learned later. This shows the database can reconstruct not just past truth but past BELIEF about past truth.

```
"(xt/q (xt/db node {::xt/valid-time #inst "2019-01-02" ::xt/tx #inst "2019-01-03"})"
"Find all persons known present in the U.S. on day 2 (valid time), as of day 3 (transaction time)"
"corrections to data can flow in from various sources and at various times"
```

§ Correction Example (Person 3). Shows how a correction is represented WITHOUT destructive overwrite. On Day 7 it emerges that Person 3's recorded departure (valid-time Jan 4) was wrong. The system ingests two documents both at valid-time Jan 4 2019 but at a later transaction-time: first a fact asserting Person 3 did NOT depart on Day 4 (departure-time :na), then a fact that Person 3 actually departed on Day 7. Critically, the original (mistaken) belief remains queryable as-of the earlier transaction-time, while the corrected belief is visible as-of the later transaction-time. The two contradictory states coexist, separated along the audit axis — the correction is additive, not a mutation. This is the concrete demonstration that retroactive writes preserve, rather than erase, prior beliefs.

```
"Person 3 never departed on Day 4 (departure-time :na)"
"corrections to data can flow in from various sources and at various times"
```

§ Deletes and Evictions. Distinguishes two erasure semantics. A default DELETE only removes visibility of data from the current perspective — the historical record (both axes) is retained and remains queryable as-of past transaction-times, so the audit trail is intact. EVICTION is the stronger, irreversible operation that erases data completely, provided specifically for cases where the legal status of information changes (e.g., GDPR-style right-to-be-forgotten / compliance erasure). This separation lets the system keep its immutable-audit-trail guarantee by default while still supporting hard erasure as an explicit, exceptional act.

```
"By default, deleting data only erases visibility of that data from the current perspective."
"You may of course still evict data completely as the legal status of information changes."
```

§ Index Persistence Model. Names the data-structure theory underneath the model: XTDB's bitemporal indexes form 'a partially persistent and fully retroactive data structure.' Partial persistence (from the persistent-data-structures literature) means you can READ any previous version but can only UPDATE the latest version — this maps onto transaction-time's append-only immutability (history is read-only; only the present can be extended). Full retroactivity means you can efficiently make modifications to PAST operations while keeping global consistency — this maps onto valid-time, where retroactive writes edit the past and the whole structure stays coherent. The two properties together are what make bitemporal correction efficient rather than requiring full recomputation.

```
"a 'partially persistent and fully retroactive data structure'"
"Partially persistent: Transaction-time immutability allows querying any previous version but updating only the latest"
"Fully retroactive: Valid-time representation enables efficient modifications to past operations while maintaining global consistency"
```

§ Compliance and Auditing. Closes by motivating why bitemporality matters operationally. It enables auditing all data changes and performing data forensics when necessary, and lets organizations respond to new compliance regulations and audit requirements by maintaining complete historical visibility across BOTH temporal dimensions. The argument is that single-timeline databases lose the distinction between 'what was true' and 'what we believed' — bitemporality is what lets you reconstruct prior beliefs for audit and forensic purposes, which a mutate-in-place store cannot.

“Auditing all data changes and performing data forensics when necessary”

“Responding to new compliance regulations and audit requirements”

HOW IT MAPS TO OUR PARTS

- **Versioning as its own layer (arrival-time vs committed/true time)** — Gives our two-times distinction its canonical names and a working implementation: transaction-time = when-it-arrived (append-only, immutable audit log = our Rama default arrival history / partial persistence), valid-time = when-it-was-true (arbitrary, correctable = our git-style committed/minted truth). XTDB proves these are genuinely independent axes you query as a pair, not one derived from the other. Direct support for treating versioning as its own layer with two clocks.
- **Object-container kernel (container=identity thread, revisions=immutable values, current=state)** — `xt/id` is exactly our stable CONTAINER; each transacted document at a (valid-time, tx-time) coordinate is an immutable REVISION; 'what is the value of this entity at valid-time' is our current-revision/state query. XTDB shows the container's state is a FUNCTION of two coordinates, not one — a sharper model than a single current pointer. Validates the kernel shape and warns that 'current revision' is under-specified until you say current-as-of-which-time.
- **Declared vs inferred change (trail-to-code join, the first consumer)** — Retroactive writes = an agent DECLARING 'this was actually true back at valid-time T' even though it arrives now; tx-time records when we learned it. This is precisely the DECLARED-change channel: the trail says what changed and when it was effective, separate from when the kernel observed it. Supports resolving a trail's path+timestamp by querying valid-time=edit-moment while keeping tx-time=ingest-moment for provenance. Reinforces the hard rule: inferred structure is never accepted as truth — only declared (transacted) facts enter, and even those are corrigible without destroying prior beliefs.
- **Preserved disagreement / plurality (late-bound synthesis)** — The Person-3 correction shows the WRONG belief is NOT overwritten: the original (departed Day 4) remains queryable as-of the old tx-time, while the corrected fact (departed Day 7) is queryable as-of the new tx-time. This is structurally preserved disagreement across the audit axis — both views coexist, distinguished by transaction-time. A concrete mechanism for never silently overwriting a conflicting view.
- **Substrate choice: Rama as event-sourced log with time-travel** — XTDB labels its index 'partially persistent and fully retroactive,' mapping exactly onto our intuition that Rama's arrival-time log gives partial persistence (read any past, write only latest). XTDB warns this alone is insufficient: you ALSO need full retroactivity (efficient edits to the past of valid-time). Tells us Rama's default log gives us only one of the two axes; the valid-time/committed-version axis must be built explicitly on top.
- **Four-level graph versioning (graph/node/relation-type/node-data) and groupings as versioned objects** — XTDB versions only entity-documents on two time axes; it does NOT natively version graph structure, relation types, or groupings independently. This is a partial-coverage WARNING: bitemporality gives us per-object two-axis history for free but says nothing about composing version histories across the four graph levels or giving a grouping/local-world its own timeline. We must layer that ourselves; bitemporality is the per-node primitive, not the whole answer.
- **Deletes/evictions and the code-identity question** — XTDB distinguishes logical delete (hide from current perspective, history retained) from eviction (true erasure for legal/compliance). Bears on our container lifecycle: a code unit going away should default to logical delete (the container's thread continues, just no current revision) with eviction reserved for hard removal. Also: XTDB pins identity to an explicit `xt/id` rather than inferring it from content — a vote for an assigned-container-id over deriving code identity from AST/path/git-hash.

Takeaway. The single most important contribution is the rigorous, implemented separation of two clocks — transaction-time (append-only, immutable, 'when we learned it') and valid-time (arbitrary, correctable, 'when it was true') — as INDEPENDENT query coordinates, formalized as a 'partially persistent and fully retroactive' index. This

is exactly our versioning-as-its-own-layer distinction (Rama arrival-time = partial persistence; git-style committed/true time = full retroactivity), and XTDB shows that retroactive corrections can be additive rather than destructive, so prior (even wrong) beliefs stay queryable — a ready-made mechanism for both the declared-vs-inferred change channel and preserved disagreement. The caution: bitemporality is a per-OBJECT two-axis primitive; it does not natively compose across our four graph levels or give groupings their own timelines — we must build that layer on top.

What we can gather — Angle 3

The production-database angle does two things the theory cannot: it proves the two-layer pattern *ships*, and it puts numbers on the cost.

The pattern is convergent and identical everywhere. Irmin states it most cleanly — an immutable, content-addressed **block store** (key = hash of value) plus a tiny mutable **tag store** (named pointers into it) — which is our revision/container split exactly, offered as a library over arbitrary data. Dolt, Noms, TerminusDB, git all land on the same shape. The lesson is that we are not betting on an unproven architecture; we are implementing the convergent one.

Structural sharing is what makes keeping all of history affordable. Prolly trees (Dolt's engine) make storage *and* diff scale with the size of the change, not the number or size of versions — and hand us a sharp design rule: **hash the keys, not the values**, so an edit touches one chunk and unchanged subtrees are shared by content address. This directly vindicates "function-id is the key, body is the value" over "git-hash is the identity," because content-keyed identity would re-shuffle on every edit and destroy the sharing.

The two times are named and implemented by XTDB. Transaction-time (append-only, immutable, "when we learned it") versus valid-time (arbitrary, correctable, "when it was true") are independent query coordinates — and crucially, **retroactive correction is additive**: the earlier, wrong belief stays queryable as-of its old transaction-time. That is a ready-made substrate for our exactness flag *and* for preserved disagreement: a corrected trail resolution does not overwrite the approximate one.

The honest cost and the honest gap: Irmin measures git-format versioning at **~5x overhead** and flags that merge needs a *known* common ancestor (LCA discovery "remains unaddressed" — our problem to solve). The Dolt essays draw the line that **rollback (AS OF / temporal tables) is not version control** — you need the committed branch/merge layer — and warn that commit-graph storage "is not a natural fit" for a query engine. So: Rama gives us the append-only log (one axis, for free) but **not** the content-addressed, branchable, O(change)-diff committed-version structure — that is a layer we build on top (the proolly-tree role), at a budgetable ~5x, with LCA discovery as a known cost.

Angle 4 — Graph & RDF Versioning: The Multi-Level Case

16. TerminusDB Explanation (What is TerminusDB? / Graphs Explanation + supporting docs)

TerminusDB / TerminusCC (docs; Big Babushka post by Luke Feeney; succinct-structures material by the TerminusDB team) · 2020-2025 · TerminusDB official documentation (terminusdb.org/docs) + TerminusDB engineering blog/Medium

In one line. TerminusDB is a document-graph database that is 'git for data': it never mutates, appending every change as an immutable, content-addressed delta layer of RDF triples on positive/negative planes, with a separate commit graph providing branch, merge, diff/patch, clone and time-travel.

Source access: *partial. The primary URL (terminusdb.org/docs/terminusdb-explanation) is an introductory page that NAMES the mechanisms but does not detail them. I read it in full, then — because the deep technical substance was absent — pulled the open companion docs and posts to recover mechanism: the Graphs Explanation doc (graph hierarchy + commit/branch/layer control graphs), a community 'Storing data like git' write-up and search-surfaced excerpts (the +ve/-ve plane, HEAD-layer query, base vs delta layers), the Wikipedia/Grokopedia summaries (terminus-store Rust backend, WOQL, schema), and the terminus-store README/docs.rs (IdTriple, dictionaries, layers). The Medium 'Big Babushka' post (HTTP 522) and the succinct-data-structures blog (HTTP 522) and tdb-succinct README would not load, so wavelet-tree/rank-select internals are reported only at the level the loadable sources confirm. Numbers (~13 bytes/triple) come from the official intro page.*

THE PAPER, SECTION BY SECTION

§ What is TerminusDB? (intro page top). Frames TerminusDB as 'an open-source document graph database with built-in version control.' The single architectural commitment stated up front: it 'stores JSON documents in a schema-enforced graph, tracks every change as an immutable delta layer, and supports branch, diff, merge, clone, and time-travel operations.' This is the thesis the rest of the doc justifies — documents on the surface, a versioned triple graph underneath, version control as a built-in property rather than an add-on. The framing is deliberately 'like Git, but for structured data rather than files.'

"open-source document graph database with built-in version control"

"stores JSON documents in a schema-enforced graph, tracks every change as an immutable delta layer"

"like Git, but for structured data rather than files"

§ What Makes TerminusDB Different (the immutability principle). States the core invariant: 'every commit appends a new delta layer — an immutable record of what was added and what was removed.' Four consequences are derived directly from this one rule. (1) Full history: 'every commit is preserved. You can time-travel to any previous state instantly.' (2) Branch and merge: isolated branches edited independently then merged back. (3) Diff and patch: 'compare any two states... to see exactly what changed, at the document or triple level' — note the explicit TWO granularities. (4) Lock-free concurrency: 'because committed layers are immutable, readers never block writers.' The argument is that immutability is not a feature but the generator of all the version-control features.

"every commit appends a new delta layer — an immutable record of what was added and what was removed"

"every commit is preserved. You can time-travel to any previous state instantly"

"compare any two states of a database to see exactly what changed, at the document or triple level"

"because committed layers are immutable, readers never block writers"

§ How It Works (layer stack + content addressing). Describes the read/write mechanics. Documents 'decompose into RDF triples stored in succinct data structures called delta layers. Each layer is content-addressed and immutable.' Reads work by traversal: 'when you query the database, TerminusDB traverses the layer stack to resolve the current state' — i.e. the current value is computed by walking the chain of deltas, not stored as a flat snapshot. Because long chains slow reads, 'periodic delta rollup operations compress the stack to maintain read

performance.' Content-addressing means a layer's identity IS its content hash, so identical layers dedupe and a commit can name its parent layer by hash.

"RDF triples stored in succinct data structures called delta layers. Each layer is content-addressed and immutable"

"TerminusDB traverses the layer stack to resolve the current state"

"Periodic delta rollout operations compress the stack to maintain read performance"

§ Storage efficiency (succinct encoding). Quantifies the storage claim that makes keeping all history affordable: succinct encoding yields 'approximately 13 bytes per triple' measured 'on billion-triple datasets.' The deeper sources (and the Wikipedia/Grokipedia summaries) attribute this to the Rust 'terminus-store' backend introduced in v1.1 (Jan 2020), which encodes triples as ID-triples over node/predicate/value dictionaries in compressed (succinct) structures, operating in-memory with ACID persistence to disk. The point being argued: succinctness is what makes 'store every version forever' economically viable rather than a liability.

"approximately 13 bytes per triple"

"benchmarks on billion-triple datasets"

§ Positive / negative planes (the delta mechanism — from companion docs). The intro page hides HOW a delta encodes a change; the companion material reveals it. Each layer (except the base/initial layer, where 'everything was created') stores triples on one of two planes: '+ve means new data is added and -ve means that data is deleted.' A triple is 'the edge and the nodes that it links.' Query resolution walks layers from HEAD backward: 'if it is found in the +ve plane, it exists in the database; if it is found in the -ve plane, it existed at some point in time but was deleted... it does not exist anymore.' Committing builds the +ve and -ve planes of a new layer 'and then commit it to the series of layers, moving the HEAD forward.' This is the actual representation of declared additions and removals — deletion is recorded as a positive fact on the negative plane, never as erasure, which is why prior states remain recoverable.

"data... is stored either on the +ve plane or -ve plane (except the initial layer as everything was created)"

"+ve means new data is added and -ve means that data is deleted"

"if it is found in the +ve plane, it exists in the database; if it is found in the -ve plane, it... does not exist anymore"

"moving the HEAD forward"

§ Graph hierarchy: data graphs vs control graphs (Graphs Explanation doc). TerminusDB is not one graph but a hierarchy: 'a hierarchical collection of data graphs and control graphs.' DATA graphs split into instance, schema, and inference graphs — separating asserted data from the schema constraints that govern it (relevant to our 'inference is not truth at ingest' rule: structure lives in a distinct schema graph, enforced, not folded into instance data). CONTROL graphs 'govern data graphs and database functionality' and come in three kinds: repository, commit, and branch graphs. The lowest level of every graph 'is a single graph composed of a sequence of layers,' and transactions are themselves hierarchical to guarantee ACID. This layered hierarchy is what lets versioning operate uniformly at multiple granularities.

"A TerminusDB database is a hierarchical collection of data graphs and control graphs"

"The lowest level of the hierarchy is a single graph composed of a sequence of layers"

"Layers specify each change to data such as additions and deletions"

§ Commit graph, branch graph, repository graph (the version-control control plane). The version-control machinery lives in dedicated control graphs, separate from the raw layer stack. The COMMIT graph 'provides revision control... contains the information required for time-travel, branch, squash, reset and rebase operations' — each commit object carries a parent reference and is 'associated with layers representing the branch of interest,' linking the commit timeline to the underlying data deltas. BRANCH objects 'point to a commit object,' so a branch is a movable ref into the commit DAG. The REPOSITORY graph tracks other local/remote TerminusDB instances, enabling clone/push/pull across machines. This is the explicit, minted, branchable commit history layered ON TOP of the append-only delta storage — the two-time architecture made concrete.

“Commit graphs provide revision control. They contain the information required for time-travel, branch, squash, reset and rebase operations”

“Branch objects in a commit-graph point to a commit object”

“Repository graphs provide collaboration and communication with other TerminusDB instances”

§ Query interfaces and schema (WOQL / GraphQL / JSON schema — from summaries). Two query surfaces over the same versioned graph: WOQL, a datalog-based language that lets the store 'function as a document store or a graph interchangeably,' and GraphQL for typed link traversal. Schema moved from OWL (early versions) to a JSON-based schema in v10.0, giving 'constraints on the allowable shapes in the graph' without heavyweight ontology languages. The schema is itself a versioned graph, so type-level structure evolves through the same commit/layer mechanism as data. This matters for our relation-TYPE versioning concern: TerminusDB DOES version types, but through the same single mechanism rather than as an independent versioning level.

“as a document store or a graph interchangeably”

“constraints on the allowable shapes in the graph”

§ Git-like operation suite (synthesis across docs). The payoff repeatedly stated: the immutable-layer + succinct-structure combination yields 'the whole suite of revision control features: branch, merge, squash, rollback, blame, and time-travel' and enables 'CI/CD approaches on data.' Because all versions are retained, 'TerminusDB can query any past or present commit.' The explicit claim is that these are not separately engineered features but emergent consequences of two design choices — append-only immutable deltas (giving history and lock-free reads) and content-addressed succinct layers (giving cheap storage and dedup). This is the strongest external evidence that our append-only-substrate + explicit-commit-layer architecture can deliver the full git-for-knowledge feature set.

“the whole suite of revision control features: branch, merge, squash, rollback, blame, and time-travel”

“CI/CD approaches on data”

“TerminusDB can query any past or present commit”

HOW IT MAPS TO OUR PARTS

- **Object-container kernel: stable CONTAINER = identity thread; REVISIONS = immutable values; current-revision = state. 'inferred structure is never accepted as truth at ingest.'** — STRONG SUPPORT + a working mechanism. TerminusDB's layer stack IS exactly the container/revision split: the named graph (e.g. a branch's instance graph) is the stable container; each immutable, content-addressed delta layer is a revision; the HEAD layer is current-revision/state. Their hard invariant — 'never mutates existing data; every commit appends a new immutable delta layer' — is the production-proven version of our container-kernel discipline. It also validates our 'inference is not truth' rule from a different angle: TerminusDB separates the SCHEMA graph from the INSTANCE graph and enforces schema as a constraint, so structure is asserted/committed, never silently inferred into the data plane.
- **Versioning as its own layer: Rama gives arrival-time history (partial persistence / time-travel log); we ALSO want explicit git-style COMMITTED/MINTED versions to branch from (full persistence). Two times: when-it-arrived vs when-it-was-committed.** — DIRECT BLUEPRINT for the two-time distinction. TerminusDB physically separates these into two graph families: the LAYER stack (append-only deltas, the 'log' = arrival/storage time) and the COMMIT GRAPH (explicit minted commits with parent pointers, branch refs = committed/true time). 'The commit graph keeps track of the time at which facts are known... connects it to the parent commit.' This is precisely our Rama-log-vs-minted-version split. Their commit graph is a separate queryable control graph layered over the raw delta storage — exactly the architecture we want: derive cheap arrival-history for free, then mint explicit branchable commits on top.
- **Four-level graph versioning: version at graph level, node level, relation-TYPE level, AND node-DATA level, and these levels must COMPOSE.** — PARTIAL SUPPORT + a warning. TerminusDB versions at the GRAPH level (branch/commit granularity) and at the TRIPLE level (additions/removals on the +ve/-ve plane = node-DATA and relation granularity), and 'diff and patch... at the document or triple level' gives us node-level

(document) AND edge-level diffs that compose into a graph-level commit. But relation-TYPE versioning lives in the SCHEMA graph, which is versioned the SAME way as instance data (it is also a layered graph). WARNING: TerminusDB unifies all four levels into ONE mechanism (layered triples) rather than giving each level independent version history — so a schema-type change and a data change both become commits in the same graph. Our requirement that the four levels be SEPARATELY versioned yet COMPOSABLE is stronger than what TerminusDB offers out of the box.

- **The trail-to-code join: resolve a trail's path+timestamp to code-at-that-moment. DECLARED change (agent says what it did) vs INFERRED change (reconstructed by diffing).** — MECHANISM + sharp naming. TerminusDB's time-travel ('query any past or present commit') is exactly path+timestamp -> state-at-that-moment resolution. More importantly, the +ve/-ve plane gives us BOTH sides of our declared-vs-inferred axis natively: the layer stores the DECLARED delta (what the committer asserted as added/removed), while diff/patch RECONSTRUCTS change by comparing two states. TerminusDB shows these can coexist: the committed layer is the declared change; diffing two layers is the inferred change. Our trail join should store the agent's declared edit-op as the +ve/-ve delta and reserve diffing for verification.
- **Groupings as versioned objects: a curated 'local world' has its own version history, partly independent of its members; member binding may be pinned, floating, or effectivity-scoped.** — PARTIAL SUPPORT + a gap. A TerminusDB BRANCH is a versioned grouping with its own commit history independent of the underlying data layers — and clone/push/pull let groupings be shared and synced. This supports our 'grouping has its own version history' claim. GAP: TerminusDB branches PIN the entire dataset to a commit (one binding mode); it has no first-class notion of FLOATING or effectivity-scoped membership where a grouping tracks the latest revision of some members while pinning others. Our pinned/floating/scoped binding requirement is a generalization beyond TerminusDB's whole-branch pinning.
- **Preserved disagreement / plurality: conflicting views representable, not silently overwritten; synthesis is late-bound.** — SUPPORT via branch/merge model. TerminusDB's git-like 'branch, merge, squash, rollback, blame' lets divergent states coexist as branches without overwrite, with merge as an explicit late-bound reconciliation — structurally identical to our preserved-disagreement-then-synthesis stance. The +ve/-ve plane even preserves DELETED facts (found on the -ve plane) rather than erasing them, so prior assertions remain recoverable. WARNING: git-style merge forces eventual single-resolution; our plurality model may need to keep branches UNMERGED indefinitely as first-class plural truth, which TerminusDB treats as an unfinished merge rather than a stable state.
- **Substrate choices: Rama (event-sourced), content-addressing, succinct/compact storage at scale.** — VALIDATION of feasibility + a number. TerminusDB proves the append-only-immutable-delta + content-addressed-layer substrate is production-viable at scale: 'approximately 13 bytes per triple' on billion-triple datasets via succinct data structures, with 'lock-free concurrency... because committed layers are immutable, readers never block writers.' This de-risks our Rama-as-append-only-substrate bet and supplies a concrete cost target. It also flags an operational need we'd otherwise miss: 'periodic delta rollup operations compress the stack to maintain read performance' — long layer chains degrade reads, so we need a compaction story for our own version layer.

Takeaway. TerminusDB is the production existence-proof for our exact two-time architecture: an append-only stack of immutable, content-addressed delta layers (the arrival/storage log, where every change is a +ve/-ve plane delta and deletion is a recorded fact, never erasure) with a SEPARATE commit graph of parent-linked, branchable commits layered on top (the explicit minted-version time). It validates the object-container/revision split, the declared-delta-vs-inferred-diff distinction (diff/patch works at both document and triple level), and the cost story (~13 bytes/triple keeps all history affordable), while warning us where it stops short of our ambitions: it unifies all four versioning levels into one mechanism rather than versioning them independently, and its branch binding is whole-dataset pinning with no floating/effectivity-scoped membership — the two places our design must go beyond git-for-data.

17. Triple Storage for Random-Access Versioned Querying of RDF Archives (OSTRICH)

Ruben Taelman, Miel Vander Sande, Joachim Van Herwegen, Erik Mannens, Ruben Verborgh · 2018 · *Journal of Web Semantics (JWS)*

In one line. A hybrid IC/CB/TB RDF-archive index using snapshots followed by aggregated delta chains, preprocessing metadata at ingest to serve fast offset-capable VM/DM/VQ versioned queries — at the cost of monotonically growing ingestion time in a single linear history.

Source access: full. Read the full open-access HTML version at the authors' homepage (rdfostrich.github.io/article-jws2018-ostrich/), the canonical author-hosted copy of the JWS 2018 paper. Two WebFetch passes: one for full section content, one for verbatim quotes. Not blocked or paywalled.

THE PAPER, SECTION BY SECTION

§ Abstract. Introduces an RDF-archive indexing technique balancing storage efficiency against versioned-query performance. History is kept as compressed consecutive versions with added metadata to cut lookup time. Presents algorithms for queries AT a version (VM), BETWEEN versions (DM), and FOR versions (VQ). The implementation OSTRICH establishes a new trade-off point: by doing more work at ingestion, it lowers average versioned-query lookup time, supports result offsets for random access, and lowers the barrier to publishing historical datasets.

“By processing and storing more metadata during ingestion time, it significantly lowers the average lookup time for versioning queries.”

§ Introduction. Frames dataset history as essential as Linked Open Datasets evolve at dataset, schema, and instance levels. Identifies the core tension: query-optimized archives need large storage, while space-efficient ones query slowly. Argues for an elementary-but-efficient versioned triple-pattern index as the foundation for richer querying, because combining full SPARQL with archiving does not scale. Lists three needs (scalable storage, compression, expressive versioned indexing) and stream-based results, offsetting, and cardinality estimation as requirements. Contributions: a versioned compressed index with offset support, VM/DM/VQ algorithms, OSTRICH, and BEAR evaluation.

“supporting both RDF archiving and SPARQL at once is difficult to scale due to their combined complexity”

§ Related Work. Surveys non-versioned RDF stores (RDF-3X's lexicographic indexes, Hexastore's six sorted lists, TripleBit, K2-Triples, RDFCSA, and HDT's bitmap-encoded dictionary representation). Then categorizes RDF archiving into three non-orthogonal strategies — Independent Copies (IC, full snapshot per version), Change-Based (CB, store only deltas), Timestamp-Based (TB, store temporal validity) — reviewing SemVersion, R&WBase, R43ples (CB with SPARQL extensions), x-RDF-3X (TB with create/delete timestamps), RDF-TX (B+Trees), v-RDFCSA, Dydra, and TailR (hybrid IC/CB delta chains). Introduces five query atoms (VM, DM, VQ, plus cross-version CV and change CM) and the BEAR benchmark. Notes the known trade-off: IC fast for VM but storage-heavy, CB good for DM but slow for VM/VQ, TB constant for VM but weak for DM/VQ.

“Systems for archiving Linked Open Data are categorized into three non-orthogonal storage strategies: Independent Copies (IC), Change-Based (CB), Timestamp-Based (TB).”

§ Problem Statement and Hypotheses. Poses the central question: how to store RDF archives for efficient VM, DM, and VQ triple-pattern queries WITH offsets. States six hypotheses that frame OSTRICH's expected position between IC and CB: selected version should not affect VM/DM efficiency; storage smaller than IC but larger than CB; VM slower than IC but DM/VQ equal-or-faster; querying equal-or-better than CB; and lower average query time bought at the price of higher ingestion time.

“How can we store RDF archives to enable efficient VM, DM and VQ triple pattern queries with offsets?”

§ Fundamentals / Storage Approach. Core design: combine IC, CB, and TB via intermittent fully-materialized snapshots followed by AGGREGATED delta chains, where each delta is relative to the closest preceding snapshot

(not to the previous delta), bounding version reconstruction to snapshot + one delta. Uses three B+Trees (SPO, POS, OSP orders) rather than six, giving more balanced pattern-to-index distribution than HDT-FoQ. Additions and deletions are stored separately, dictionary-encoded and timestamped, each carrying a LOCAL CHANGE flag marking triples that revert an earlier change within the chain. Stores separate addition/deletion counts plus relative deletion positions to enable offsets and cardinality estimation. Snapshot stored via HDT; two dictionaries (snapshot + delta) distinguished by reserved bits.

“Each delta is independent of a preceding delta and relative to the closest preceding snapshot in the chain.”

“Triples that revert a previous addition or deletion within the same delta chain, are called local changes.”

§ Ingestion Algorithms. Transforms incoming CB-style deltas into the aggregated alternative-delta-chain structure. Batch ingestion loads a changeset in memory at $O(P + N \log N)$ but is memory-hungry for large versions. Streaming ingestion does a sort-merge join over three SPO-ordered streams (input changeset, existing deletions, existing additions), handling seven merge cases and keeping only stream heads in memory while maintaining relative-position counters. Crucially, ingestion time grows with every added version — a direct consequence of aggregating each delta against the snapshot. Measured: BEAR-A 2,256 s total; BEAR-B-daily 12.36 s; BEAR-B-hourly 4,497 s across 1,299 versions, hitting a swapping wall near version 1,200.

“Within the scope of this article, we only discuss ingestion of deltas in a single delta chain following a snapshot.”

“For each additional version in a dataset, the ingestion time increases. This is a direct consequence of our alternative delta chain method.”

§ Query Algorithms (VM / DM / VQ). VM (state at version i) uses a three-stream sort-merge join over snapshot, deletions, and additions, with an offset-convergence loop: the requested offset is iteratively increased by matched deletion positions to find the right snapshot offset, proven to terminate because offset is a non-decreasing function bounded by the finite deletion list — yielding near-constant lookup independent of version. DM (change set between i and j) either filters the delta directly (when $\text{start} = \text{snapshot}$) or iterates addition/deletion trees comparing flags across the two versions. VQ (versions in which a result is valid) joins snapshot and addition trees, querying deletion trees to annotate each triple with the version range it holds in (all versions except those where it was deleted).

“Version materialization (VM) retrieves data using a query Q targeted at a single version V_i .”

“Delta materialization (DM) retrieves query Q ’s result change sets between two versions V_i and V_j .”

“Version query (VQ) annotates query Q ’s results with the versions in which they are valid.”

§ Evaluation (BEAR benchmark). Implemented in C/C++ with HDT snapshots, Kyoto Cabinet compressed B+Trees, and gzip delta dictionaries; JS bindings expose a Triple Pattern Fragments API serving 30M triples over 10 versions. Storage: BEAR-A 3,117.64 MB (OSTRICH) vs 2,682.88 MB HDT-CB and 5,335.04 MB HDT-IC — between CB and IC as predicted. VM was ~2x faster than HDT-CB on BEAR-A with constant time as versions grew while CB degraded; DM matched HDT-IC and beat CB from version 7; VQ on BEAR-B-hourly beat all HDT approaches by nearly an order of magnitude. Offsets stayed below 1 ms for VM (offsets 2–4096) and were roughly offset-independent for VQ. Gzip savings reached 99.58% on BEAR-B-hourly. The clear bottleneck was ingestion swapping in long chains.

“outperforming all HDT-based approaches by almost an order of magnitude”

§ Conclusions and Future Work. OSTRICH establishes a new storage/ingestion/query trade-off for RDF archives by front-loading metadata computation at ingest. It confirms most hypotheses: less storage than IC, VM/DM/VQ equal-or-faster than CB, lower average query time at the cost of higher ingestion time. It excels for datasets with many small versions but struggles with few large versions due to materialization overhead, and does not scale past roughly 1,300 versions without dynamic snapshot renewal. Stated limitations: support for only a single snapshot and single delta chain (no branching), and monotonically increasing ingestion cost. Future work points to multiple snapshots and branch/variant support.

“For each additional version in a dataset, the ingestion time increases.”

HOW IT MAPS TO OUR PARTS

- **Versioning as its own layer (arrival-time vs committed-time; partial vs full persistence)** — OSTRICH is a pure single-axis versioning store: one linear history of versions $V_0..V_n$ over arrival-ordered deltas. It operationalizes 'when-it-arrived' time well (the snapshot+delta-chain IS arrival-time persistence) but has NO branch/commit axis at all ('we only discuss ingestion of deltas in a single delta chain following a snapshot'). This is a sharp warning: a single-time model is cheap and queryable but cannot express our second axis (minted/committed/branchable versions). It validates separating the two times — OSTRICH proves how far the arrival-only design goes and exactly where it dead-ends.
- **Object-container kernel: container = identity thread, revisions = immutable values, current = state** — OSTRICH's aggregated-delta model gives a concrete mechanism for the container/revision split. A triple-pattern identity is a thread; each version is an immutable value reconstructable as snapshot+one delta. The 'local change flag' (a triple that reverts a previous add/delete within the chain) is a direct mechanism for the kernel problem of a revision that cancels an earlier revision — they pre-compute and store this at ingest rather than re-deriving it, supporting our 'preprocess truth at ingest, not query time' instinct. But note: OSTRICH infers nothing — every delta is DECLARED input, never reconstructed by diffing, matching our hard rule 'inferred structure is never accepted as truth at ingest.'
- **Four-level graph versioning (graph / node / relation-type / node-data, composing)** — OSTRICH is explicitly a TRIPLE-pattern index — it versions at exactly one granularity (the triple = a node-relation-node statement) and composes nothing. It changes at 'dataset, schema, and instance levels' but stores all of it as undifferentiated triple adds/deletes. This is a failure-mode warning for our four-level model: collapsing all change into one flat statement-delta stream gives great storage/query numbers but loses the level structure entirely. If we want composition across graph/node/relation-type/node-data levels, we cannot adopt OSTRICH's flat single-grain delta as-is — we'd need per-level delta chains that compose.
- **VM/DM/VQ query atoms as the versioned-query vocabulary** — OSTRICH gives us a named, proven query vocabulary we can lift directly. VM = 'state at version i' (our current-revision / time-travel-to read), DM = 'what changed between i and j' (our trail-to-code diff / DECLARED-vs-INFERRED change), VQ = 'in which versions is this fact valid' (our effectivity/condition-scoped binding and provenance query). These three plus the offset/cardinality requirements are a ready checklist for what our versioning layer's read API must answer.
- **The trail-to-code join (resolve path+timestamp to code-at-that-moment)** — VM is exactly the trail-to-code resolution primitive: given a version (timestamp), materialize the state. DM is the DECLARED-vs-INFERRED distinction made concrete — OSTRICH ingests DECLARED changesets (the agent says what changed) and never reconstructs by diffing, which is precisely our preference. OSTRICH's offset-convergence VM algorithm (constant-time lookup regardless of which version, via stored deletion positions) is a mechanism for making 'code-at-that-moment' lookups not degrade as history grows.
- **Substrate choices: Rama event-sourced log + explicit committed versions** — OSTRICH's central limitation maps onto Rama's strength. Its ingestion time grows monotonically with chain length ('for each additional version the ingestion time increases') because it eagerly materializes aggregated deltas — it hit a swapping wall around v1200/1300 of BEAR-B-hourly. Rama gives the cheap append-only arrival log for free (no aggregation cost at write). The lesson: keep the raw event log primary and cheap (Rama), and treat OSTRICH-style aggregated-delta materialization as a DERIVED projection (a PState) we build for fast VM/DM/VQ — not as the write path. Aggregated deltas belong on the read side.
- **Groupings as versioned objects with independent history** — OSTRICH offers NO support here and thereby names the gap: it versions one flat triple set with a single global version counter. There is no notion of a sub-collection with its own version line, nor pinned/floating member binding. Its single-snapshot/single-chain constraint is the exact opposite of independently-versioned groupings. It tells us groupings-as-versioned-objects is a requirement the RDF-archive literature does not address — we are extending past it, not reusing it.

- **Preserved disagreement / plurality, late-bound synthesis** — OSTRICH assumes a single authoritative linear truth — every version is one consistent state, deltas are total and authoritative, conflicts are impossible by construction. This is the anti-pattern for plurality: it cannot represent two contradictory views of the same fact at the same version. It warns that the IC/CB/TB archive model is fundamentally single-truth; preserved disagreement needs a branch/variant axis OSTRICH deliberately excludes.

Takeaway. OSTRICH's core lesson for our versioning/identity layer is the explicit IC/CB/TB trade-off triangle plus a clean, named, proven read vocabulary — VM (state-at-version), DM (change-between-versions), VQ (versions-where-valid) — that our versioning layer's API should answer directly. Its central failure mode is the load-bearing warning: aggregating deltas eagerly at INGEST makes versioned reads fast but makes ingestion time grow monotonically with history length and confines you to a SINGLE linear chain with no branching — which is exactly why our design should keep the cheap append-only arrival log primary (Rama) and treat OSTRICH-style aggregated-delta materialization as a derived read-side projection, while adding the branch/commit axis (and per-level, per-grouping version lines) that OSTRICH deliberately excludes.

18. Evaluating Query and Storage Strategies for RDF Archives

Javier D. Fernández, Jürgen Umbrich, Axel Polleres, Magnus Knuth · 2019 · *Semantic Web Journal* (IOS Press); also SAGE DOI 10.3233/SW-180309

In one line. Foundations + the BEAR benchmark for RDF archiving: formalizes change/dynamicity metrics and a five-atom query taxonomy, then empirically shows that IC, CB, TB, and hybrid storage strategies each win on different change profiles and that version-count scaling breaks TB outright.

Source access: full. *SAGE/journals.sagepub.com* returned HTTP 403 (gated). Read the FULL open-access *Semantic Web Journal* preprint PDF instead: <https://www.semantic-web-journal.net/system/files/swj1608.pdf> (swj1608, 2808 lines extracted via pdftotext). All definitions, tables (Tables 2-8), and figures-as-text were accessible; only the rendered bitmap plots (Figs 2-5) are unreadable, but their findings are described verbatim in the running text I quote below.

THE PAPER, SECTION BY SECTION

§ Abstract. States the core problem: there is an emerging demand to efficiently archive and temporally query different versions of evolving Semantic Web data, and as archiving systems appear, there is no foundation or standard for benchmarking them on storage-space efficiency and retrieval performance. The paper's three deliverables: (1) theoretical foundations for designing benchmark data and queries for RDF archives, (2) a concrete instantiation of those queries over a real-world evolving dataset, (3) an empirical evaluation of current archiving techniques and querying strategies meant to be a reusable baseline.

“There is an emerging demand on efficiently archiving and (temporal) querying different versions of evolving semantic Web data.”

“we provide theoretical foundations on the design of data and queries to evaluate emerging RDF archiving systems”

§ 1. Introduction. Motivates archiving as a first-class problem: published Linked Data continuously changes at both data and schema level, and evolving vocabularies introduce inconsistencies between data relying on prior ontology versions. Web-of-Data archives must additionally offer time-traversing structured queries that neither SPARQL nor existing temporal SPARQL extensions natively support (e.g. knowing whether a dataset or a particular entity changed). Surveys prior benchmark gaps: EvoGen extends LUBM but is synthetic and single-dataset; existing RDF/SPARQL benchmarks target static, federated, or streaming data, none covering archiving where querying entity changes across time is crucial. Lists four contributions: theoretical foundations, an AnQL-based query instantiation, the BEAR test suite (BEAR-A/B/C from real datasets), and implemented archiving systems evaluated to set an extensible baseline.

“this published data is continuously undergoing changes (on a data and schema level)”

“archives for the Web of Data should additionally offer capabilities for time-traversing structured queries”

“querying entity changes across time is a crucial aspect”

§ 2.1 Retrieval Functionality (query taxonomy). Compiles a 2x3 classification of retrieval needs along two axes: query type (materialisation vs structured query) and focus (version vs delta). The six cells are: Version materialisation (retrieve a full snapshot at time t_i , the most common revision-control feature); Single-version structured queries (a query against one specific version); Cross-version / time-traversal structured queries (satisfied across multiple versions, the novel hard case); Delta materialisation (the added/deleted triples between two versions, tied to authoring/merge/conflict-resolution); and Single-delta / Cross-delta structured queries (the delta-focused counterparts). This taxonomy is the conceptual scaffold the rest of the paper builds queries and metrics around.

“This classification distinguishes six different types of retrieval needs, mainly regarding the query type (materialisation or structured queries) and the main focus (version/delta) of the query.”

“Cross-version structured queries, also called time-traversal queries, add a novel complexity since these queries must be satisfied across different versions.”

§ 2.2 Archiving Policies (the three storage strategies + hybrids). Defines the central IC/CB/TB trichotomy. Independent Copies (IC) stores each version as an isolated dataset; simple but duplicates static information across versions, so it is expected to face scalability problems. Change-based (CB) stores only the deltas (added/deleted triples) between versions, addressing IC's duplication but requiring a query mediator to propagate deltas to reconstruct a version, which costs version-focused retrieval. Timestamp-based (TB) annotates each triple with its temporal/version validity locally, a special case of temporal RDF, using compression/self-indexes to avoid repetition. Hybrid (HB) approaches combine these: HB_IC/CB inserts a fresh materialized IC version after some number of CB deltas (governed by a cost model deciding when to re-materialize); HB_TB/CB time-annotates triples only when added or deleted, often managed under named/virtual graphs so the mediator can reuse existing engines. A static core (triples present in all versions) is implicit in CB but explicit in IC.

“Main efforts addressing the challenge of RDF archiving fall in one of the following three storage strategies: independent copies (IC), change-based (CB) and timestamp-based (TB) approaches.”

“It is, however, expected that IC faces scalability problems as static information is duplicated across the versions.”

“CB requires additional computational costs for delta propagation which affects version-focused retrieving operations.”

“each triple locally holds the timestamp of the version. In order to save space avoiding repetitions, compression techniques can be used”

§ 3.1 Dataset Configuration (formal model + dynamicity metrics). Provides the formal data model adapting Gutierrez et al.'s temporal RDF graphs, but with version labels instead of time labels. Def 1: a version-annotated triple $(s,p,o):[i]$ holds in version i ; an RDF archive A is a set of these. Def 2: an RDF version $A(i)=V_i$ is the graph of triples labeled i . Then defines the dynamicity/change metrics that are the paper's measurement vocabulary: Def 3 version change ratio $\delta_{ij} = |\text{added} \cup \text{deleted}| / |V_i \cup V_j|$, split into insertion and deletion ratios; Def 4 version data growth $= |V_j|/|V_i|$; Def 5 static core = triples in ALL versions; Def 6 version-oblivious triples = the set of distinct triples ignoring timestamps (the size of the annotation-merged store); Defs 7-9 RDF vocabulary per version, per delta, and vocabulary set dynamicity v_{dyn} (symmetric-difference over union of subject/predicate/object sets). Notes IC is sensitive to insertion/deletion counts while CB and TB are sensitive to overall dynamicity delta, and vocabulary evolution matters because stores use term-to-integer dictionaries.

“A version-annotated triple is an RDF triple (s, p, o) with a label i in N representing the version in which this triple holds, denoted by the notation $(s, p, o) : [i]$. An RDF archive graph A is a set of version-annotated triples.”

“static core $CA = \{(s,p,o) \mid \text{for all } i \text{ in } N, (s,p,o):[i] \text{ in } A\}$ ”

“This computes the total number of different triples in an RDF archive independently of the timestamp.”

“the static core can be easily computed taking the first version and applying all the subsequent deletions.”

§ 3.2 Design of Benchmark Queries (five query atoms). Argues there is no standard language for querying RDF archives, then adapts cardinality/selectivity to the archive setting: archive-driven (Def 10) vs version-driven (Def 11) result cardinality and selectivity, plus version-driven result dynamicity (Def 12, how results change across versions).

Introduces five foundational query atoms that combine to express richer needs, explicitly aligned with the Memento Framework's datetime-negotiation needs: Version materialisation $\text{Mat}(Q, V_i)$; Delta materialisation $\text{Diff}(Q, V_i, V_j)$ (the result tuples present in one version but not the other); Version query $\text{Ver}(Q)$ (results annotated with the version label where they hold); Cross-version join $\text{Join}(Q_1, V_i, Q_2, V_j)$; and Change materialisation $\text{Change}(Q)$ (the consecutive version pairs where Q 's results differ, i.e. timemap construction). Emphasizes controlling cardinality/selectivity so that retrieval-time differences can be attributed to the archiving policy rather than query size.

"we introduce five foundational query atoms to cover the broad spectrum of emerging retrieval demands in RDF archiving"
"Change materialisation, $\text{Change}(Q)$: it provides those consecutive versions in which the given query Q produces different results."

"selecting only queries with the same cardinality and selectivity across all version should guarantee that the index performance is always the same... potential retrieval time differences can be attributed to the archiving policy"

§ 3.3 Instantiation in AnQL (and why off-the-shelf SPARQL is hard). Grounds the five atoms in AnQL, a SPARQL extension where a triple pattern is annotated with a version label $t:l$. Shows naive SPARQL translations of each policy: IC stores each version in a named graph (version name = graph name); TB stores each triple once as a reified triple with valid [start,end] interval metadata (more compact than IC's per-graph repetition); CB stores adds/deletes in separate per-version named graphs and must FILTER-test whether a triple was added in a prior version and not since deleted. The key methodological finding: implementing archives over off-the-shelf RDF stores with pure SPARQL is non-trivial and does not scale; CB's translation works only for single triple patterns, not full BGPs, and Ver/Change break under the TB recipe because the version becomes an unbound FILTER variable. This justifies using tailored optimized stores (HDT, Jena-TDB) in the evaluation rather than generic SPARQL.

"a change-based storage of RDF triples has no trivial implementation in an off-the shelf RDF store"
"an implementation of RDF archives and queries in off-the-shelf RDF stores and using SPARQL is a non-trivial exercise"
"this representation allows for a compact representation of several disjoint (maximal) validity intervals of the same triple, thus causing less overhead than the graph-based representation discussed for IC"

§ 4. BEAR datasets (A: Dynamic Linked Data, B: DBpedia Live, C: Open Data portals). Three real-world corpora with deliberately different change profiles. BEAR-A: 58 weekly snapshots from the Dynamic Linked Data Observatory (650+ domains); grows 30m->66m triples (growth 101% mean per version) yet a mean 31% of data changes between versions (27% deleted, 32% added); only 376m version-oblivious triples and a small 3.5m static core, confirming high churn over a relatively small distinct-triple universe. BEAR-B: DBpedia Live changesets for the 100 most volatile resources at three granularities -- instant (21,046 versions), hour (1,299), day (89) -- showing that aggregation granularity dramatically changes dynamicity (change ratio 0.011% instant to 1.78% daily) and that triples are deleted then re-inserted shortly after (Wikipedia reverts), so the static core stays large relative to version-oblivious triples. BEAR-C: 32 weekly snapshots of the European Open Data portal (~500k triples/snapshot), very high metadata dynamicity (delta 67.6%) but near-zero growth, with complex UNION/OPTIONAL queries deliberately beyond Basic Graph Patterns. Queries are sampled to be epsilon-stable (cardinality near-constant across versions) and entirely dynamic (results always differ between consecutive versions).

"a mean of 31% of the data change between two versions and that each new version deletes a mean of 27% of the previous triples, and adds 32%"
"the average version change ratio increases from very small portions of 0.011% for instant updates to 1.8% at the daily level"
"triples which have been deleted at some point are re-inserted after a short period of time (in the case of DBpedia Live this may happen when changes made to a Wikipedia article are reverted shortly after)"

§ 5.1 Storage Space Results. Implements IC/CB/TB and hybrids on Jena-TDB and HDT (a compressed store). Headline numbers (Table 6): on BEAR-A, raw gzip is 23GB; Jena-IC needs 230GB (~10x raw), HDT-IC 48GB (~2x gzip), Jena-CB 138GB, HDT-CB 28GB. CB space savings track data dynamicity -- diff data saves 40% on BEAR-A and 15% on BEAR-C but on low-change BEAR-B the diff is only 1-3% of original, so CB shrinks dramatically (BEAR-B-day Jena-CB is 10% of Jena-IC; instant is 5%). Critical failure mode: TB does not scale with version

count -- Jena-TB introduces large overhead on BEAR-B-hour (1,299 versions) and FAILED to load BEAR-B-instant (21,046 versions) entirely, because the named-graph-per-version notation is inefficient at high version counts; this is the stated motivation for hybrids. On highly dynamic BEAR-C, CB's add+delete indexes make it slightly LARGER than IC. Hybrids interpolate: small IC-gap behaves like IC in size, large gap like CB.

"Jena-TB introduces a large overhead in BEAR-B-hour, with 1,299 versions, and it even failed to load BEAR-B-instant, with 21046 versions. The main reason is that the notation for graphs and versions in TB is clearly inefficient for such large number of versions"

"in BEAR-B-day, Jena-CB only takes 10% the space of IC, and only 5% in BEAR-B-instant"

"The only exception is BEAR-C where data are so dynamic that the additional index overhead in CB (for adds and deletes) produces slightly bigger sizes than IC"

§ 5.2 Retrieval Performance. Maps each policy to each query atom. IC, TB, and CB/TB give constant retrieval time across versions; CB and IC/CB degrade as more deltas must be queried (querying a higher version means replaying more adds/deletes). HDT generally outperforms Jena under the same policy, but an IC policy in Jena can beat a CB policy in HDT (e.g. Jena-IC beats HDT-CB after version 5 in BEAR-A). For Mat and Diff, IC is best/most constant; TB in Jena performs worse than IC because it queries one potentially huge store (catastrophic on data-heavy BEAR-A) yet can beat CB when version count is large. Crucial reversal for Version queries (Ver(Q), Table 8): CB OUTPERFORMS IC because every version must be queried and CB can reuse the already-materialized previous version, while IC must query each full version independently -- Jena-CB 72ms vs Jena-IC 101ms on BEAR-A QSH, and 120ms vs 1189ms on BEAR-B-hour. Hybrid HB_TB/CB emerges as the fastest for version queries (single store query then split by version), whereas pure Jena-TB is catastrophic (56,693ms on BEAR-A, 6,473ms on BEAR-B-hour).

"the CB policy shows a clear trend that the query performance decreases if we query a higher version since more deltas have to be queried"

"an IC policy in Jena can be faster than a CB policy in HDT"

"As opposed to the previous Mat and Diff operations, Jena-CB outperforms Jena-IC in these version queries... the query of a version Vi in CB can leverage the already materialized version for the previous version Vi-1"

§ 6. Conclusions. Concludes RDF archiving is early-stage with no standard corpus or retrieval-functionality definition, which the paper's foundations (dynamicity metrics + five query atoms) and the BEAR suite (three real corpora + query testbeds + reference IC/CB/TB/hybrid implementations on Jena and HDT) are designed to fill as an extensible baseline. Results confirm both the scalability challenges and the strengths of each policy, guiding strategy selection by dataset change profile. Future work: a customizable synthetic generator preserving user-defined dynamicity characteristics while scaling to arbitrary size and version count, and extending to multiple versioned graphs in a federated scenario.

"RDF archiving is still in an early stage of research."

"there is not a standard way of defining neither a specific data corpus for RDF archiving nor relevant retrieval functionalities"

"Results clearly confirm challenges (in terms of scalability) and strengths of current archiving approaches"

HOW IT MAPS TO OUR PARTS

- **Versioning as its own layer (arrival-time vs committed-time; partial vs full persistence)** — Gives us the canonical names for our two storage axes. IC = full persistence (every committed version materialized, branchable, O(1) read, O(n) space). CB = partial-persistence-style delta log (cheap to store, O(deltas) to reconstruct a version). TB = per-fact validity intervals = exactly our arrival-time annotation model. The paper proves these are a genuine space/time trade-off, not interchangeable. Directly warns: do NOT pick one globally. Our Rama arrival-time log is TB-shaped; our git-style minted versions are IC/CB-shaped; the paper says keep both and choose per change-profile.
- **Object-container kernel (container = identity thread, revisions = immutable values, current = state)** — Def 1-2 is our model under another name: a version-annotated triple (s,p,o):[i] is a revision-labeled value; the

archive A is the container; $A(i)=V_i$ is current-revision-at-i. Supports the kernel design. The static-core / version-oblivious-triple distinction (Def 5/6) gives us a measurable identity invariant: the static core is what persists unchanged across all revisions of a container -- a natural anchor for stable identity vs churning state.

- **Four-level graph versioning (graph / node / relation-type / node-data, must compose)** — BEAR versions at the triple level only and the paper is candid that this is a limitation -- it explicitly focuses on low-level deltas and single-graph archiving, deferring multiple/federated versioned graphs to future work. This is a WARNING and a gap our four-level model must close: BEAR's vocabulary-dynamicity metrics (Def 7-9, separate subject/predicate/object evolution) are a partial answer at the relation-type/node-data levels, and we should adopt vdyn as a per-level composable change metric, but BEAR does not give us graph-level-vs-node-level composition -- that is our novel contribution.
- **Groupings as versioned objects with their own history (pinned / floating / effectivity-scoped binding)** — The TB validity interval [start,end] per triple is exactly effectivity/condition-scoped binding: a member is bound to a grouping over a version range, not absolutely. The paper's compact 'disjoint maximal validity intervals' representation is a concrete mechanism for our floating/effectivity-scoped membership. The hybrid HB_IC/CB cost-model (re-materialize a grouping snapshot after N member deltas) is a ready mechanism for deciding when to pin (snapshot) a curated local world vs keep it floating (delta-tracked).
- **The trail-to-code join (resolve path+timestamp to code-at-that-moment; declared vs inferred change)** — Version materialisation $Mat(Q,V_i)$ and the Memento datetime-negotiation framing IS our resolve-trail-timestamp-to-state operation. Change materialisation $Change(Q)$ -- the consecutive versions where results differ -- is precisely how we detect INFERRED change by diffing when the agent did not declare it. The paper's performance data warns us: if we store the code/trail history as pure CB (delta log, like Rama arrival order), point-in-time resolution degrades with distance from the materialized base -- so trail-to-code lookups need either IC anchors or a hybrid, not a raw delta replay.
- **Substrate choice: Rama event-sourced log as default arrival-time history** — Offers a direct failure-mode warning. Rama's arrival log is CB/TB-shaped. BEAR shows: (1) pure CB version-materialisation cost grows with version distance, and (2) pure TB with named-graph-per-version CATASTROPHICALLY fails at high version counts (Jena-TB couldn't even load 21,046 versions; 56,693ms queries). If our kernel mints a version per arrival event (DBpedia-Live-instant analogue: 21k versions), we will hit exactly this wall. Mitigation the paper hands us: aggregate updates into coarser granularities (instant/hour/day) and/or insert periodic IC materialized anchors (hybrid).
- **Hard rule: inferred structure is never accepted as truth at ingest** — BEAR operationalizes the declared/inferred split: deltas are low-level computed (inferred) facts, while version labels are asserted (declared) truth. The paper keeps both representable -- version-oblivious triples store every distinct fact with its annotation set rather than collapsing -- which supports our rule that an inferred delta is a derived view, not a new accepted truth. The static core being computable from V_0 + subsequent deletions shows derived state stays reconstructable, never overwriting the committed log.
- **Preserved disagreement / plurality (conflicting views representable, late-bound synthesis)** — Weakest mapping -- flag as inference, not something the paper addresses. BEAR assumes a single linear total order over version labels (N with a total order) and one authoritative sequence of snapshots; it has no notion of concurrent/branching versions or conflicting parallel histories. This is a GAP: our plurality requirement (and our git-style branch-from-any-version) needs a partial order / DAG of versions that BEAR explicitly does not model. Useful as a baseline to extend, but it does not support preserved disagreement out of the box.

Takeaway. The single most important contribution is the proof, on real data, that there is NO universal versioning storage strategy: IC (full copies), CB (delta log), and TB (per-fact validity timestamps) form a hard space/time trade-off whose winner flips by both query type and change profile -- CB beats IC for version-spanning queries (reuse the prior materialization) but loses for point reconstruction, and TB collapses entirely at high version counts (failed to load 21,046 versions). For Softland this validates keeping arrival-time history (TB/CB-shaped Rama log) and committed/minted versions (IC/CB-shaped) as two distinct layers, and it hands us a concrete escape hatch -- aggregation granularity plus periodic IC anchors (hybrids) -- for the version-count explosion our per-event kernel

will otherwise hit. The unmodeled gaps (multi-graph/four-level composition and branching/plural histories) are precisely where our design must go beyond BEAR.

19. Towards Fully-fledged Archiving for RDF Datasets

Olivier Pelgrin, Luis Galarraga, Katja Hose · 2021 · Semantic Web Journal (IOS Press), submission swj2538

In one line. A survey-plus-framework paper that defines a 5-tuple RDF archiving model with separate LOCAL (per-graph) and GLOBAL (per-dataset) revision tracks, proposes RDFev metrics for high-level (entity/object) change, empirically studies DBpedia/YAGO/Wikidata evolution, and sketches a modular fully-fledged archiving engine.

Source access: full. Read the full author/publisher PDF from semantic-web-journal.net (swj2538.pdf, 626KB). WebFetch could not parse the PDF binary, so I downloaded it via WebFetch's saved file and extracted text locally with pdftotext (5234 lines). Two-column layout produced interleaved line-numbers but all prose, formulas, definitions, and tables were recoverable. Figures (3-7) are bitmap plots; I read their captions and the surrounding analysis text, not pixel values.

THE PAPER, SECTION BY SECTION

§ Abstract. States that the dynamicity of RDF data motivates archiving (storing and querying previous versions). Argues the state of the art is under-developed on three counts: (i) systems are neither scalable nor easy to use, (ii) there is no standard way to query RDF archives, and (iii) solutions ignore the actual evolution patterns of real RDF data. The paper's contributions are a survey, RDFev (a framework to study RDF dynamicity), an empirical study of YAGO/DBpedia/Wikidata evolution, and a sketch of a fully-fledged solution.

"RDF archiving, i.e., the task of storing and querying previous versions of an RDF dataset"
"solutions do not exploit the evolution patterns of real RDF data"

§ 1. Introduction. Frames RDF archiving as storing/querying the entire edition history of a dataset, useful both for producers (fine-grained version control, debugging, evolution study) and consumers (analytics, mining correction patterns, trend analysis). Distinguishes two user camps that drive different designs: maintainer-facing Git-like version control systems (R&Wbase, R43ples, Quit Store) versus consumer-facing time-aware querying (RDF-TX). Attributes the absence of a fully-fledged solution to (i) engine limitations handling metadata, (ii) no query standard, (iii) disregard for real evolution. Lists five contributions: RDFev framework, the DBpedia/YAGO/Wikidata study, the survey, an Ostrich evaluation, and the sketch.

"Storing and querying the entire edition history of an RDF dataset, a task we call RDF archiving"
"RDF archives can serve as a backend for fine-grained version control in collaborative projects"

§ 2. Preliminaries (data model: graphs, graph archives, dataset archives, the 5-tuple). Builds the formal model that is the load-bearing part for our purposes. A graph G is a set of triples; an archive $A = \{G_s, \dots, G_{s+n-1}\}$ is an ordered set of revisions where each non-initial revision G_i is obtained by applying a changeset $u_i = (\text{added/deleted triples, disjoint})$ to G_{i-1} , i.e. $G_i = (G_{i-1} \cup u_i) \setminus u_i$. A graph archive is equivalently modeled as 4-tuples where ρ is the RDF identifier of a revision, $rv(\rho)$ maps it to a natural number, and $ts(\rho)$ is its commit time. Crucially the paper then separates LOCAL and GLOBAL history: a dataset is a set of named graphs G^k ; a dataset update $U_j = \{u_{\hat{j}}, u^0_j, u^1_j, \dots\}$ bundles a per-graph changeset PLUS a special GRAPH changeset $u_{\hat{j}}$ (added/deleted graph LABELS). A master graph G_M holds metadata about graphs. The dataset archive is then modeled as 5-tuples: ρ identifies the LOCAL revision of the triple within its graph (label $g = l(\rho)$), and ζ identifies the GLOBAL dataset revision $rv(\zeta)$, with $ts(\zeta)$ its timestamp. Adding a non-empty graph yields TWO revisions (one to create, one to populate); deletion is symmetric. Section 2.5 defines the five canonical archive query types: Version Materialization (VM), Delta Materialization (DM), Version (V), Cross-Version (CV), Cross-Delta (CD).

"we can model a dataset archive as a set of 5-tuples"
" ρ in l is the RDF identifier of the local revision of the triple in an RDF graph with label $g = l(\rho)$... ζ in l identifies a (global) dataset revision $j = rv(\zeta)$ "

“the addition of a non-empty graph to a dataset archive generates two revisions: one for creating the graph, and one for populating it”

“we also define the function ts ... that associates a revision identifier ρ to its commit time”

§ 3. RDFev: Framework for the Evolution of RDF Data. Proposes RDFev, a set of metrics plus a C++/Python tool (RocksDB-backed) that ingests successive N-triples dumps in chronological order and computes change metrics per consecutive revision pair. Splits metrics into LOW-LEVEL (triple-level: change ratio δ_{ij} , insertion/deletion ratios, vocabulary dynamicity ν_{dyn} over the term set $Upsilon$, growth ratio) and HIGH-LEVEL (entity- and value-level). The key argument is that triple-counting (as in BEAR) is insufficient for maintainers who need the SEMANTICS of a change. High-level metrics: entity changes ec (an entity = a subject; counts added/deleted subjects), triple-to-entity-change score (avg triples per entity change), object updates (a delete paired with an add, $o \neq o'$), and orphan object additions/deletions (triples NOT consumed by any higher-level change). Once a triple is assigned to a high-level change it is 'consumed' and cannot be reused. All metrics are domain-agnostic.

“metrics for low-level changes, and metrics for high-level changes”

“A high-level change confers semantics to a changeset”

“An object update in a changeset u_{ij} is defined by the deletion of a triple and the addition of a triple with $o \neq o'$ ”

“Once a triple in a changeset has been assigned to a high-level change, the triple is consumed and cannot be assigned to any other high-level change”

§ 4. Evolution Analysis of RDF Datasets. Applies RDFev to YAGO, DBpedia, Wikidata, treating each release as a revision. Finds two distinct evolution regimes. YAGO and DBpedia have a MAJOR/MINOR release cycle: major releases (YAGO 3.0.0 doubled the KB) carry order-of-magnitude larger changes and may not increase size, while minor releases focus on quality/correction with $\sim 10\times$ fewer changes. DBpedia's 2015-04 release is an inflexion point where deletion rate exceeds insertion rate (a design redesign). Wikidata, by contrast, shows a stable release cycle with low, near-constant change. High-level findings: in YAGO/DBpedia object UPDATES dominate (4.8x and 1.8x more frequent than orphan add/delete) meaning edits mostly revise existing values; in Wikidata orphan object additions dominate (19x more common than orphan deletions). The explicit design lesson: an effective archiver must ADAPT to at least these two evolution patterns.

“we have identified two patterns of evolution”

“Major releases are characterized by a large number of updates in the knowledge base and may not necessarily increase its size”

“object updates are 4.8 and 1.8 times more frequent than orphan additions and deletions”

“an effective solution for large-scale RDF archiving should be able to adapt to at least these two patterns of evolution”

§ 5. Survey of RDF Archiving Solutions. Taxonomizes systems along storage paradigm, data model, BGP support, query types, branch/tag, multi-graph, concurrency, source availability (Table 4). Four storage paradigms: Independent Copies (IC, full snapshot per revision, fast VM/CV, disk-prohibitive), Change-Based (CB, snapshots + delta chains, storage-efficient, costly VM/CV on long chains), Timestamp-Based (TB, per-triple time annotations + bit/visibility maps), and Fragment-Based (FB, Quit Store). Discusses SemVersion (IC, 5-tuples with per-graph-local revisions, no global graph-addition tracking), R&WBase/R43ples/Quit Store (Git-like, PROV-O metadata; Quit Store uniquely tracks BOTH local and global history but is Git-bound and memory-heavy), RDF-TX (multiversion B-tree, SPARQL-T), v-RDFCSA (suffix-array, per-triple revision bit-sequence), and Ostrich (hybrid IC+CB+TB; HDT snapshots + redundant delta chains stored against the LATEST snapshot, quads, single snapshot + single delta chain only). Section 5.2 surveys query languages: AnQL (quad patterns), T-SPARQL (interval + timestamp temporal operators), SPARQ-LTL (temporal-logic PAST/FUTURE), RDF/SPARQL. Section 5.3 covers benchmarks BEAR and EvoGen.

“three main storage paradigms ... namely independent copies (IC), change-based (CB), and timestamp-based (TB)”

“Quit Store [4] is the only available solution that supports both archiving of the local and joint (global) history of multiple RDF graphs”

“each revision in the chain is stored as a changeset containing the changes w.r.t. the latest snapshot - and not the previous revision”

§ 6. Evaluation of Related Work (Ostrich). Of all surveyed systems, ONLY Ostrich could ingest the experimental datasets; Quit Store crashed/MemoryError, R43ples could not ingest one DBpedia revision in four days, R&WBase choked on millions of SPARQL update statements, x-RDF-3X would not compile. Ostrich achieves strong compression (87% YAGO, 84% DBpedia, 94% Wikidata via HDT snapshots + delta compression). But its ingestion time grows LINEARLY with history length because it stores revision i as changeset $u_{\{0,i\}}$ against revision 0, so changesets only grow: ~22h for DBpedia rev 9 vs ~14h for the larger rev 5. Spearman analysis shows runtime correlates most with delta-chain length, changeset-size variance, and avg deleted triples. Conclusion: massive deletions and long histories degrade Ostrich; the single-snapshot/single-delta-chain design does not scale to long histories.

“only Ostrich can store the release history of very large RDF datasets”

“Ostrich stores the i -th revision of an archive as a changeset of the form $u_{\{0,i\}}$... can only grow in size. This fact makes it unsuitable for very long histories”

“the most correlated features are the length of the delta chain, the standard deviation of the changeset size, and the average number of deleted triples”

§ 7. Towards Fully-fledged RDF Archiving (the design sketch). The prescriptive core. Derives design lessons. (a) Temporal domain-specific metadata vs revision metadata must be SEPARATED: revision metadata (commit time, revision numbers, branch/tag) is IMMUTABLE and is attached to the revision identifiers rho/zeta, whereas mutable domain-specific time (e.g. validity of a marriage) is attached to graph labels $g=l(\text{rho})$ and can be edited by a later changeset on the master graph. (b) Global AND local history are both vital. (c) Provenance (PROV-O) is workflow history, separate from revision metadata, linkable to either revision ids or graph labels, with a hybrid default-plus-override scheme. (d) Concurrency should be a MODULAR middleware layer separated from the storage backend, so collaborative version control becomes an APPLICATION of archiving rather than the architecture. (e) Storage should hybridize IC/CB/FB/TB and be ADAPTIVE: use RDFev metrics to trigger snapshot creation on large deletions, divide each graph's history into interval BLOCK FILES (snapshot + its interval's changesets), and resize dictionary encodings on vocabulary spikes. (f) Internal serialization of 5-tuples: reification (compatible but quintuples query patterns), singleton properties (encodes rho in predicate, still needs second reification for zeta), named graphs, HDTQ, or binary matrices (triples x revisions, good compression from contiguous runs). *RDF / SPARQL (Turtle) is endorsed as the publication/query format because it natively nests multiple metadata levels; the authors propose extending SPARQL with REVISION and DELTA syntactic sugar.*

“revision metadata is immutable and should therefore be logically placed at a different level”

“we propose a modular design that separates the concurrency layer from the storage backend”

“collaborative version control systems for RDF become an application of fully-fledged RDF archiving”

“we envision an adaptive data-oriented system that relies on the metrics proposed in Section 3 and adjusts its parameters according to the archive's evolution”

§ 8. Conclusions. Recaps: RDF archiving matters for maintainers and consumers; evolution patterns must drive design; RDFev characterizes three real KBs and exposes design shifts; only Ostrich scales to large datasets but not to long histories and ignores evolution patterns; the field lacks a query standard, for which SPARQL* plus syntactic sugar is the promising path. Reaffirms that collaborative version control is itself an application of archiving rather than the whole of it.

“only a few solutions are available for download and use, and ... only Ostrich can store the release history of very large RDF datasets”

“its design still does not scale to long histories and does not exploit the data evolution patterns”

HOW IT MAPS TO OUR PARTS

- **Versioning as its own layer: arrival-time history (Rama default, partial persistence) vs explicit committed/minted versions (full persistence) -- 'two distinct times'** — Gives our two-times distinction a

precise, externally-validated name and structure. The paper's rho = LOCAL revision identifier and zeta = GLOBAL dataset revision are exactly committed/minted versions, and ts(rho)/ts(zeta) are commit timestamps -- separate from arrival order. It even warns of the failure mode where the two diverge: x-RDF-3X 'stores temporal metadata in special indexes that optimize for concurrent updates at the expense of temporal consistency, i.e., revision numbers may not always be in concordance with the timestamps.' That is precisely the arrival-time-vs-commit-time hazard our layer must keep coherent. SUPPORTS + names the mechanism.

- **Groupings as versioned objects: a curated grouping / local world has its OWN version history, partly independent of its members (pinned/floating member binding)** — This is the paper's central member-vs-group structure. A graph G^k has its own LOCAL revision track (rho); the dataset/grouping has a GLOBAL revision track (zeta) advanced by a dataset update U_j that bundles per-member changesets PLUS a graph changeset $u_{\hat{}}$ over member LABELS (membership add/delete). So the group version advances both when members change AND when membership itself changes -- and crucially, ' u^1_1 leaves G_1 untouched' shows a global revision can advance while a member's local revision does not. That IS partly-independent group-vs-member history. The master graph G_M holding membership metadata maps onto our grouping's own metadata record. SUPPORTS + gives a concrete mechanism for pinned vs floating binding (the global revision pins a specific tuple of local revisions).
- **Object-container kernel: stable CONTAINER = identity thread; REVISIONS = immutable values; current-revision = state** — Direct structural analogue at the triple/graph level. The graph label g (a stable IRI in I) is the CONTAINER identity that persists across revisions; each G_i is an immutable revision value; current-revision is the latest in the ordered set. The paper's hard insistence that 'revision metadata is immutable and should therefore be logically placed at a different level' echoes our 'inferred structure is never accepted as truth at ingest' discipline -- immutable committed facts live on one track, mutable/derived annotations on another. SUPPORTS the container/revision split and reinforces the immutability invariant.
- **Four-level graph versioning (graph / node / relation-type / node-data) that must COMPOSE** — PARTIALLY SUPPORTS but also WARNS. The paper genuinely composes TWO levels (member-graph local revision composed under dataset global revision) and shows the composition rule ($U_j = \text{graph-changeset} + \text{per-graph changesets}$, applied in a defined order). But it operates at triple granularity, not a typed node/relation-type/node-data hierarchy -- 'an entity is a subject for the sake of this analysis,' a flat collapse. So it validates that multi-level versioning must compose and gives the composition operator, but it is a cautionary data point that without typed levels you lose the node-data vs relation-type distinction we want. The 'object update' high-level change (a node-data edit) vs 'orphan addition' (structural add) is the closest the paper gets to separating node-data from structure.
- **Trail-to-code join: DECLARED change (agent says what it did) vs INFERRED change (reconstructed by diffing)** — Names and motivates our declared-vs-inferred split, and supplies RDFev as a concrete inference engine. Low-level changes are RAW diffs (inferred). High-level changes (entity change, object update, orphan add/delete) are the SEMANTIC reconstruction of intent from a diff -- 'A high-level change confers semantics to a changeset.' That is exactly the inference side of our join: given before/after states, reconstruct what operation the author intended. The 'consumed triple' rule (once a delete+add pair is bound to an object-update it cannot be reused) is a reusable disambiguation algorithm for our diff-to-edit-op reconstruction. Critically, the paper shows inference is LOSSY/ambiguous (orphans are leftovers), which is exactly why we prefer the agent's DECLARED edit operation when available. SUPPORTS + offers a mechanism + shows the failure mode of inference-only.
- **The code-identity question: file vs path vs FQN vs normalized-AST vs git-hash; atomic unit = file or function** — Offers a transferable answer by analogy: identity is the STABLE LABEL (the IRI/graph-label that threads across revisions), NOT the content. SemVersion's failure is instructive -- it used per-graph-LOCAL revision numbers and thereby 'makes it difficult to track the addition or deletion of named graphs in the history of the dataset.' Translation for us: if a code unit's identity is tied only to a content-derived key (git hash) or a position (path), you cannot track rename/move/delete as identity-preserving events -- you need a stable label decoupled from both content and position, with membership (path) recorded as MUTABLE metadata on a

separate track. WARNS against content-hash-as-identity for tracking history; SUPPORTS a stable-label container with path/content as revisable attributes.

- **Substrate choices: Rama event-sourced log as truth, derived state as projection; adaptive snapshotting** — OFFERS A MECHANISM and a tuning policy. The CB paradigm (snapshots + delta chains) is precisely event-sourcing with periodic materialization. The paper's hardest empirical lesson maps onto our snapshot strategy: Ostrich grew ingestion time LINEARLY with history because it diffed every revision against revision 0 - a direct warning that an append-only log without periodic re-snapshotting degrades. Their adaptive remedy (use RDFev metrics, e.g. a large-deletion spike, to TRIGGER a fresh snapshot / new block file) is a ready-made policy for when Rama should materialize a checkpoint PState vs keep replaying the log. SUPPORTS event-sourcing + WARNS about unbounded delta chains + gives an adaptive trigger mechanism.
- **Preserved disagreement / plurality; concurrency and late-bound synthesis** — Modestly supports via the modular-concurrency lesson. The paper separates the concurrency/conflict layer as a MIDDLEWARE distinct from storage, and explicitly contrasts manual conflict resolution (R&WBase: users resolve) vs automatic. That maps to our late-bound-synthesis stance: conflicts are not resolved at the storage/ingest layer; they are surfaced to a separable layer where resolution can be deferred or human-arbitrated. Branching/tagging as first-class (which most surveyed systems LACK, a noted gap) is the substrate our preserved-disagreement branches need. SUPPORTS keeping conflict resolution out of the truth-storage layer; the absence of good branching support across systems is a WARNING about how immature this is.

Takeaway. The single most important contribution is the two-track revision identity model: a triple is a 5-tuple where rho is the member's LOCAL revision and zeta is the grouping's GLOBAL revision, advanced by an update that bundles per-member changesets with an explicit membership changeset -- and where IMMUTABLE revision metadata is deliberately placed on a different logical track from MUTABLE domain data. This is a directly portable blueprint for our 'groupings as versioned objects' and 'two times' layers: it shows precisely how a curated local world can carry its own version history partly independent of its members, how membership changes are themselves versioned events, and (via the Ostrich finding that diff-against-origin degrades linearly) why our Rama-backed archive needs adaptive, metric-triggered snapshots rather than an unbounded delta chain.

20. Querying the Edit History of Wikidata

Thomas Pellissier Tanon, Fabian Suchanek · 2019 · ESWC 2019 (Extended Semantic Web Conference) — demo/poster paper

In one line. A SPARQL endpoint over Wikidata's full 700M-revision edit history that stores each triple once with validity-range annotations, exposing per-revision diffs AND global-state-after-revision via plain SPARQL 1.1 named graphs over a RocksDB index.

Source access: full. Read the complete author-hosted PDF at thomas.pellissier-tanon.fr/papers/2019-ESWC-wikidatahistory.pdf. WebFetch returned raw PDF bytes, so I converted it locally with `pdftotext -layout` and read the full extracted text (all 5 sections + references). It is a short ESWC demo paper (~5 pages); nothing was gated.

THE PAPER, SECTION BY SECTION

§ Abstract. States the problem and the contribution in five sentences: Wikidata has accumulated millions of contributions over 7 years, and the paper proposes a system exposing this through a SPARQL endpoint. The key design claim is that they index not only the diffs of each revision but also the GLOBAL STATE of the Wikidata graph after any given revision — this dual capability is the paper's whole point. They motivate it with concrete analytic use-cases that the endpoint enables: tracing human vs. automated (bot) contributions, areas of vandalism, big schema changes, and the adoption of different values for the 'gender' property over time. The deliverable is framed as standard SPARQL 1.1 query access to history, not a new query language.

“We index not just the diffs done by a revision, but also the global state of Wikidata graph after any given revision.”

“This allows users to answer complex SPARQL 1.1 queries on the Wikidata history”

§ 1 Introduction. Sets scale and gap. Wikidata started 2012; as of July 2018 it had 5B statements about 50M entities, 700M+ revisions by 56k contributors. The official SPARQL endpoint (query.wikidata.org) serves ONLY the latest version and is blind to history, so even 'how many people were in Wikidata two years ago?' is unanswerable. The only existing route to full history is a 250GB compressed XML dump of per-revision JSON blobs — hard to manipulate and index. Two naive strategies are rejected up front: materializing one RDF graph per revision is 'prohibitively expensive' (700M revisions × billions of triples), while indexing only inter-revision differences 'loses the global state at each revision.' The contribution is a system that smartly indexes revisions to deliver three query capabilities: (1) retrieve the diff (added/removed triples) for any revision; (2) evaluate any triple pattern against the global state after a given revision; (3) retrieve the revisions that added/removed triples matching a pattern. These answer questions like cities-over-time, entities-modified-exclusively-by-bots, and gender-values-over-time — all in standard SPARQL with no language additions.

"this endpoint emits only the latest version of the data, and is blind to the edit history"

"It would be prohibitively expensive to create one RDF graph for each of the 700 million revisions"

"If one indexes only the differences between the revisions, one loses the global state at each revision."

"without any additions to the language"

§ 2 Related Work. Surveys versioned-RDF approaches and argues each fails at Wikidata's scale or update model. Quad stores and archives storing each triple annotated per-revision would require storing a triple once for every revision it appears in — 'quadrillions of triples' for Wikidata. tgrin timestamps triples but has no time-range support and no available implementation. R&Wbase ('git for triples') is a SPARQL 1.1 wrapper giving git-like RDF versioning but stores only diffs, so it cannot efficiently query full graph state at a point in time. x-RDF-3X annotates each triple with validity ranges (add/remove timestamps) permitting any-state queries and consistency features, but by design CANNOT load data with already-known timestamps/version IDs — disqualifying for ingesting Wikidata's known history. v-RDFCSA stores versioned triples efficiently but allows no subsequent revision additions and no revision ranges. OSTRICH stores version-annotated triples via an HDT base version plus a changeset, but assumes a non-empty base version; with Wikidata's empty base it would have to store full global state per revision. The recurring distinction is theory vs. a working implementation that survives Wikidata's size and append-with-known-IDs requirement.

"each triple would have to be stored once for every revision in which it appears ... this would amount to quadrillions of triples"

"R&Wbase ... stores only the diff between different revisions. Therefore, it does not allow efficient querying of the full graph state at a given point of time."

"x-RDF-3X ... annotates triples with validity ranges ... However, by design, it does not allow loading data with already known timestamps or version IDs."

"This system is not tailored to the case of Wikidata, where the base version is empty."

§ 3 System Overview (data model + indexes + evaluation). The technical core. The DATA MODEL uses four named-graph families, all expressible in vanilla SPARQL 1.1: a Global State Graph per revision (full state of Wikidata after that revision is saved), an Addition Graph per revision (triples added by it), a Deletion Graph per revision (triples removed), and a single Default Graph holding revision METADATA — author, timestamp, modified entity id, the PREVIOUS REVISION OF THE SAME ENTITY (per-entity chain), the PREVIOUS REVISION IN WIKIDATA (global chain), and the IRIs of the addition/deletion/global-state graphs. They reuse the official Wikidata dump schema. Because materializing those graphs would take exabytes, they do NOT load into a triple store; they build custom indexes on RocksDB ('a scalable key value store, optimized for a mixed workload between query and edits,' chosen to anticipate future live updates). Four index types: (a) Dictionary indexes mapping strings ↔ integer ids; (b) Content indexes — each triple (s,p,o) stored in three permutation indexes keyed spo/pos/osp, with VALUE = a set of revision ranges [start, end] where start is the revision that introduced the triple and end the revision that removed it (or +∞ if never removed) — this validity-range encoding stores each triple ONCE; (c) Revision indexes giving the (small) set of triples added/removed by a given revision; (d) Meta indexes for per-revision author, previous revision, etc. Querying the global state = prefix-search a content index then FILTER results by revision range. This is cheap because of near-monotone growth: of 4931M triples that ever existed (as of July 1 2018), only

475M were ever removed, so the largest revision holds ~90% of all triples that ever existed. The QUAD-PATTERN EVALUATOR dispatches: metadata patterns → meta indexes; content patterns on an addition/deletion graph → revision indexes; on a global-state graph or unbound graph → longest-prefix content-index search, then either range-filter (global state) or reconstruct quads by mapping each triple's range to its addition/deletion graph ids (notably NOT returning global-state-graph ids, since one triple pattern could match hundreds of millions of state graphs). This single-quad evaluator is plugged into Eclipse RDF4J, which executes arbitrary SPARQL 1.1 via repeated evaluator calls and applies join reordering with cardinality estimates exploiting structure (one author per revision; few triples per entity).

"The default graph contains triples that encode metadata about each revision: the revision author, the revision timestamp, the id of the modified entity, the previous revision of the same entity, the previous revision in Wikidata"

"Each range is of the form [start, end[, where start is the id of the revision that introduced the triple, and end is the id of the revision that removed it (or +∞ if the triple has not been removed)."

"out of the 4931M triples that have existed in the Wikidata history as of July 1st 2018, only 475M have been removed. Therefore, the largest revision contains 90% of the total number of Wikidata triples that ever existed."

"we use RocksDB ... a scalable key value store, optimized for a mixed workload between query and edits. This choice anticipates the possibility of live updates from Wikidata in the future."

§ 3 (deployment numbers). Reports the actual deployed demo instance. Because of storage constraints on the Wikimedia Foundation server, only the direct-claim relations (wdt:) are loaded, covering Wikidata's creation through July 1st 2018. The instance stores 700M+ revisions and 390M content triples about 49M items. The RocksDB indexes occupy 64GB on disk using RocksDB gzip compression after compaction. These numbers are the paper's main empirical evidence that validity-range + permutation-index storage on a KV store makes full edit history tractable — full 5B-statement history is implied to fit the same scheme but was trimmed only for the public demo's disk budget.

"only the direct claim relations are loaded in our demo endpoint"

"stores more than 700M revisions and 390M content triples about 49M items"

"The RocksDB indexes are using 64Go on disk with the RocksDB gzip compression system, after having compacted the database."

§ 4 Demonstration Setting. Shows the model in action via a worked SPARQL query (Listing 1): the 10 most frequent replacements of the 'gender' (wdt:P21) value. The query pattern is the join the model enables — GRAPH ?addGraph matches added P21 triples, ?revision hist:additions/hist:deletions links to the corresponding deletion graph, and GRAPH ?delGraph matches the same subject's deleted P21 value, then GROUP BY/ORDER BY/LIMIT (plain SPARQL 1.1) compute the ranking. Insights surfaced: users correct erroneous gender values (e.g. 'man' → 'male'); a variant quantifies the gender gap over time (absolute man/woman difference grows but the relative difference shrinks); another variant traces increasing presence of non-traditional sex/gender values like 'trans-male' or 'non-binary.' The point is that historical analytics that previously required custom dump-processing pipelines become one declarative query against added/deleted/global graphs.

"GRAPH ?addGraph { ?s wdt:P21 ?addedGender } ?revision hist:additions ?addGraph ; hist:deletions ?delGraph ."

"users correct erroneous gender values (such as "man") by their intended variants ("male")"

"while the absolute difference between the number of men and women in Wikidata keeps increasing over time, the relative difference actually decreases"

§ 5 Conclusion. Restates that the system efficiently indexes the entire Wikidata edit history and answers arbitrary SPARQL 1.1 queries over both revision diffs and global state after each revision, with the live endpoint at wdhqs.wmflabs.org. Future work is to update the store live from the Wikidata edit stream (the same motivation that drove the RocksDB choice). Acknowledges grant ANR-16-CE23-0007-01 ('DICOS').

"Our system allows queries on both the revision diffs and the global state of Wikidata after each revision."

"we plan to update the content of our system live based on the Wikidata edit stream."

HOW IT MAPS TO OUR PARTS

- **Versioning as its own layer (arrival-time history vs committed/minted versions; partial vs full persistence)** — OFFERS A MECHANISM. The content-index entry stores triples once, each annotated with a set of revision ranges [start, end[— start = revision that introduced it, end = revision that removed it (or +infinity). This is exactly partial persistence done right: one immutable store, time-travel by range-filtering, not by snapshotting. It directly validates our refusal to materialize one graph per version. But note their 'revision' is a single linear ID assigned per edit (arrival/commit fused), so it does NOT split when-it-arrived from when-it-was-committed the way we want — a warning that a single revision counter collapses our two distinct times.
- **Object-container kernel (stable container = identity thread; revisions = immutable values; current = latest)** — SUPPORTS + NAMES IT. Each Wikidata entity is a container; the paper threads it via 'the previous revision of the same entity' stored in the default/meta graph — a per-entity revision chain — AND 'the previous revision in Wikidata' — a global revision chain. This is precisely our braided identity thread (per-container chain) co-existing with a global arrival log. Their current-revision = 'the largest revision contains 90% of the total number of Wikidata triples that ever existed' confirms current-state is just the open-ended range subset, computed not stored.
- **Four-level graph versioning (graph / node / relation-type / node-data, must compose)** — PARTIAL SUPPORT + GAP. They version at the triple grain via validity ranges and expose three composable named-graph views (global-state, addition, deletion) plus a default metadata graph — graph-level and data-level versioning that compose in plain SPARQL 1.1 with no language extension. But they do NOT separately version node identity, relation-TYPE/schema, or grouping objects; schema changes appear only as ordinary triple churn ('the big schema changes' are observed, not modeled). Warns us that triple-grain ranges alone don't give the relation-type level — we must add it.
- **Substrate choices: Rama as event-sourced store with RocksDB-style indexes** — STRONG CONVERGENT EVIDENCE. They explicitly reject loading version-annotated quads into a triple store ('would occupy exabytes') and instead build on RocksDB — 'a scalable key value store, optimized for a mixed workload between query and edits' — with dictionary, content (spo/pos/osp permutation), revision, and meta indexes. Rama's PStates are exactly this: hand-rolled, append-friendly, permutation-style indexes over an event log. The whole 390M-triples-in-64GB result is an existence proof that our substrate shape scales to full edit history.
- **The trail-to-code join: DECLARED change vs INFERRED change; resolve path+timestamp to code-at-that-moment** — OFFERS A MECHANISM. Their query capability #2 ('Evaluate any triple pattern against the global state of Wikidata after a given revision') is the join we need: given a revision (timestamp/version), reconstruct the full state at that moment by range-filtering. Capability #1/#3 (retrieve diff for a revision; retrieve revisions that added/removed a pattern) gives both directions of the trail join. Crucially their diffs are DECLARED — the addition/deletion graphs are stored as recorded by each edit, not reconstructed by diffing snapshots — matching our 'declared change' preference over inferred diffing.
- **Near-monotone growth / cost model for keeping all history** — VALIDATES FEASIBILITY. Of 4931M triples that ever existed, only 475M were ever removed; the open (un-removed) set is ~90% of all triples. Growth is near-monotone, so the validity-range representation has almost no end-bounded rows and the 'current state' is cheap. This quantifies why keeping full history costs little more than keeping current state — directly relevant to our claim that Rama arrival-time history is affordable by default.
- **Preserved disagreement / plurality and 'inferred structure is never accepted as truth at ingest'** — WEAK / CAUTIONARY. The paper has NO conflict model — revisions are a single linear authoritative chain, last-write-wins, vandalism is queryable-after-the-fact but not held as preserved disagreement. It treats every recorded edit as truth (it ingests Wikidata's declared diffs verbatim, which is consistent with our 'declared not inferred' rule). But it offers nothing for late-bound synthesis of conflicting views — a gap we must fill ourselves; their model is the single-authority case of ours.

Takeaway. The single most important contribution: validity-range triple storage as the right shape for keeping full edit history. Store every triple ONCE, annotated with a set of [start, end[revision ranges (end = +∞ if still live); the

current state is just the open-ranged subset, and any past state is a range-filter — no per-version materialization. Combined with their dual per-entity + global revision chains (braided identity threads in a metadata graph) and the near-monotone-growth observation (only ~10% of triples ever get removed, so history costs barely more than current state), this is a direct, deployed existence proof for our object-container kernel + arrival-time-versioning-as-its-own-layer on a RocksDB-style KV substrate — exactly Rama's PState shape. The cautions: their single linear revision counter fuses arrival-time and commit-time (we want them split), and they model no conflict/plurality and no relation-TYPE/schema-level versioning — gaps our four-level + preserved-disagreement layers must add.

What we can gather — Angle 4

This is the angle that stress-tests the hardest thing we want — versioning at four composing levels — and it comes back with both the blueprint and the walls.

The two-track theorem is the central result. The RDF-archiving framework formalizes a triple as a 5-tuple $\langle s, p, o, \text{local-revision}, \text{global-revision} \rangle$ and *proves* that systems with only member-local revisions "cannot track the addition or deletion" of members in the group's history — **member-level versioning alone cannot reconstruct group-level history**. Wikidata's history system implements the working version: each revision records both "previous revision of the same entity" and "previous revision globally" — per-entity chains braided inside a global chain. The practical consequence for us is strong: the group/configuration track must exist *from the start*; retrofitting it is provably lossy — which is a direct argument for putting the configuration noun into the kernel **before** the graph ingestor forces it.

Validity-range storage is the right shape, and it's cheap. Wikidata stores each triple *once*, annotated with `[start, end)` revision ranges (end = ∞ if still live); current state is just the open-ended subset, any past state a range filter — no per-version materialization. And because knowledge is near-monotone (only ~10% of Wikidata triples were ever removed), **keeping full history costs barely more than keeping current state** — encouraging for trails and discourse graphs, which mostly accrete.

The version-count wall is real and bounds our minting policy. The BEAR benchmark shows no universal storage strategy (snapshots/IC, deltas/CB, timestamps/TB each win on different change profiles), and TB *failed to load* 21,046 versions while queries timed out. If we mint a version per arrival event we hit this wall — so **commit-grain, not save-grain**, with periodic snapshot anchors triggered adaptively by change metrics. OSTRICH adds the same warning (ingestion time grows with history because it aggregates deltas eagerly) and the gift to take: the **VM / DM / VQ** query vocabulary (state-at-version, change-between-versions, versions-where-valid) — adopt it as the kernel's read API; trail-to-code is VM plus an exactness flag.

TerminusDB is the production proof and the boundary. It runs our exact two-family architecture — append-only immutable delta layers plus a *separate* commit graph — at ~13 bytes/triple, with deletion recorded as a fact, not an erasure. But it **unifies all four levels into one mechanism** (a schema-type change and a data change are both just commits), and pins whole branches with no floating/effectivity binding. So it validates the substrate and marks the two places we must go further: keeping the four levels *separately* versioned yet composable, and the one level **nobody in this angle versions** — relation-TYPE / schema evolution — which is genuinely our own contribution.

Angle 5 — Composite & Assembly Versioning: The Mature, Solved Part

21. Towards a Unified Framework for Version Modeling

Randy H. Katz · 1988 · UC Berkeley Computer Science Division Technical Report UCB/CSD-88-484 (later published as 'Toward a Unified Framework for Version Modeling in Engineering Databases,' ACM Computing Surveys 22(4), 1990, pp. 375-408)

In one line. A survey that distills ~15 CAD/engineering version-management proposals down to a small set of underlying mechanisms — version histories, configurations, equivalences, dynamic binding, change propagation, workspaces — and proposes a unified terminology rather than yet another model.

Source access: full. ACM DOI ([dl.acm.org/doi/10.1145/98163.98172](https://doi.org/10.1145/98163.98172)) returned HTTP 403. Read the OPEN Berkeley tech-report copy CSD-88-484 instead: a 45-page scanned-image PDF at www2.eecs.berkeley.edu/Pubs/TechRpts/1988/Archive/CSD-88-484.pdf (linked from Katz's Berkeley tech-reports page). The PDF has no text layer (CCITT scan, no tesseract available), so I read all 45 pages as rendered page images via the Read tool. This is the pre-publication TR of the 1990 ACM Computing Surveys article; section structure and figures match.

THE PAPER, SECTION BY SECTION

§ Abstract & Key Words. Frames the paper as a SURVEY of one aspect of CAD database support: version modeling — 'how best to describe the structure of a complex design artifact as it evolves across its multiple representations and over time.' Explicitly states it is NOT proposing another model but seeking 'a common terminology and a common collection of mechanisms that should underlie all version models.' Notes an operational model is also needed to describe how artifact descriptions are created and modified. The stated remaining challenge is to build a single tailorable framework from these mechanisms.

"it is our purpose not merely to describe such proposals but to attempt to unify them"

"a common terminology and a common collection of mechanisms that should underlie all version models"

"how best to describe the structure of a complex design artifact as it evolves across its multiple representations and over time"

§ 1. Introduction. Motivates CAD as a non-traditional database domain that doesn't fit tables/serializable-transaction models — it needs complex connected data, navigation, and long-duration interactive access. Identifies five CAD characteristics, the key ones being that it is HISTORY-ORIENTED ('current data is not overwritten with new data, but rather maintained as historical data') and handles multi-faceted, multi-representational, hierarchical aggregates. Crucially separates a CAD data model into two components: a VERSION MODEL ('primitives for organizing data across time') and a DESIGN TRANSACTION MODEL ('how such data can undergo change in a consistent manner'); the paper concentrates on the former. Distinguishes version-oriented from time-oriented databases: 'Versions represent a significant, semantically meaningful change. Thus, it is implicit in a version database that not every update need result in the construction of a new version.' States the goal of producing a CATALOG OF FEATURES every version model must support plus the mechanisms to implement them, separating mechanisms from policies ('knobs') to enable a true tailorable framework. Previews the roadmap: Sec 2 terms/requirements, Sec 3 a detailed model (the Version Server), Sec 4 a survey, Sec 5 the extracted mechanisms/unification, Sec 6 conclusions.

"the CAD environment is history oriented: current data is not overwritten with new data, but rather maintained as historical data"

"A data model for computer-aided design must consist of two components: a version model... and a design transaction model"

"Versions represent a significant, semantically meaningful change... not every update need result in the construction of a new version"

“Only once the appropriate 'knobs' are made visible and available to the user, with mechanisms clearly distinguished from policies, will it be possible to achieve a true version framework”

§ 2. Definition of Terms and Basic Requirements. Lays out a vocabulary because the field 'has not yet evolved a consistent terminology.' Key definitions: a DESIGN OBJECT is a named aggregate of design primitives ('better to think of them as units of the design that can be given a name, such as Arithmetic Logic Unit') — explicitly NOT necessarily a single file. A VERSION is 'a semantically meaningful snapshot of a design object at a point in time,' a descendant of existing versions and a potential ancestor of others. A CONFIGURATION is 'a binding between a version of a composite object and the versions of its components.' WORKSPACES are named repositories that are units of sharing and access control. Figure 2.1 presents a leveled architecture splitting a Design Management System (History Manager, Workspace Manager, Object Manager, Physical Database) from an independent Process Management System (Workflow, Event Log) — anticipating event-sourced, log-backed design. Section 2.2 enumerates 8 general requirements (named aggregation, hierarchical composition, separation of contents from interfaces, configuration construct/manipulate, multi-version support, version SELECTION with a default mechanism, instance-vs-definition reuse, and a HISTORY MECHANISM to record design decisions), then splits them into MODELING requirements (hierarchy across representations; versions a.k.a. alternatives/revisions/ECOs; configurations buildable STATICALLY 'all component references bound at creation time' AND DYNAMICALLY 'references bound at the time of access,' the latter called PARAMETRIZED VERSIONS; identifying equivalent objects across representations; object-oriented ADT/inheritance concepts) and DESIGN TRANSACTION requirements (loose vs tight coupling via check-in/out and workspaces; CONVERSATIONAL transactions that 'hold locks through crashes' but provide no recovery; cooperative-work support via change notification/propagation and workflow).

“a design object is a useful module of the design”

“A version is a semantically meaningful snapshot of a design object at a point in time”

“we define a configuration as a binding between a version of a composite object and the versions of its components”

“Configurations should be able to be constructed both statically (all component references bound at creation time) and dynamically (references bound at the time of access). Dynamic configurations are also known as parametrized versions”

§ 3. Detailed Presentation of a Version Model (the Version Server). Presents the author's own Version Server data model as a representative exemplar (not claimed ultimate), guided by the pragmatic constraint that it be implementable 'on top of existing design files... to avoid rewriting any existing design applications.' PHILOSOPHY (3.2.1): the model is E-R-like but with distinguished relationships; it separates STRUCTURAL objects (added by the model to organize the flat space of files, like directories) from REPRESENTATIONAL objects (the actual tool files) — 'It is a key aspect of our organization that structural information be separated from representational information.' Three structural relationships organize everything: COMPONENT HIERARCHIES / is-a-part-of (3.2.2) — aggregation, generalizable from trees to DAGs so a component can be shared by multiple composites; VERSION HISTORIES / is-a-kind-of + is-derived-from (3.2.3) — objects denoted name[version#].type, with is-derived-from recording ancestry because the naming scheme alone cannot ('it is not possible to determine from this naming scheme which version is descended from which'); ALTERNATIVES are parallel versions not on the same root-path, DERIVATIVES are on the same path; the generic object (e.g. ALU.Layout) is a structural object with NO corresponding file. CONFIGURATIONS (3.2.4): combining version history + component hierarchy yields a configuration; mapping info is stored PER NODE ('a composite object version carries with it the version numbers of its components'), enabling the static-vs-dynamic distinction. EQUIVALENCES (3.2.5): a structural object tying heterogeneous representations (layout/transistor/functional) of the same real-world thing via is-equivalent-to; equivalences are 'constraints on the database that can be exploited at design verification time' and can be inherited by new versions. Operations (3.3): CURRENCY (3.3.1) decouples 'current' from 'last created,' constrains branchiness (new versions must be descendants of current), allows multiple parallel currencies (e.g. May Release vs June Release) — 'our first example of an operational mechanism' with associated policy. DYNAMIC CONFIGURATIONS (3.3.2) implemented via PIE-style LAYERS and CONTEXTS (search order over layers), with the noted weakness that layers can be combined in nonsensical ways with no interface constraints. WORKSPACES (3.3.3): archive/group/private repositories with a worked example of a version migrating private-

>group->archive via check-out/check-in, relationships established automatically on archive check-in. LOGICAL vs PHYSICAL (3.3.4): the web of relationships is implemented as on-disk linked lists of configuration/version/equivalence nodes pointing at UNIX files, keeping per-version storage overhead small. CHANGE/CONSTRAINT PROPAGATION (3.3.5): change propagation 'automatically incorporates new versions into configurations'; constraint propagation enforces directional equivalences by regenerating versions. Two hard problems: LIMITING SCOPE (rarely propagate to the root) and DISAMBIGUATING PATH (DAG propagation order changes the result). Solution = GROUP CHECK-IN/CHECK-OUT making intent unambiguous: 'A configuration node spawns a new version at most once no matter how many times a change path passes through it,' so the result is order-independent; group check-out limits inherited equivalence to the group. INHERITANCE (3.3.6): argues plain type-instance inheritance forces an undesirable proliferation of subtypes to carry instance-specific equivalence constraints, and proposes direct INSTANCE-TO-INSTANCE inheritance along structural relationships (a new version inherits most of its description from its version-history ancestor). Summary (3.4): three orthogonal organizing relationships (version histories, configurations, equivalences); admits the model is weak on separating interface from contents and on the instance/definition distinction.

"It is a key aspect of our organization that structural information be separated from representational information"

"it is not possible to determine from this naming scheme which version is descended from which. Thus, we introduce the concept of a version history and is-derived-from"

"It is frequently the case that the default or current version is not the most recently created version... it is useful to separate 'current' from 'last'"

"A configuration node spawns a new version at most once no matter how many times a change path passes through it... the resulting configuration is independent of the check-in order"

§ 4. Survey of Version Data Models. Traces ~8 years of evolution and reviews ~15 systems. GENERAL TRENDS (4.1): three approaches — extend the relational model (complex objects, conversational/nested transactions, long fields), extend the E-R model (embed is-derived-from/is-a-part-of/is-equivalent-to; Sec-3 model is here), and new object-oriented models (operational with inheritance vs structural with distinguished relationships; combined is best for CAD). Chronology: 1980-81 timestamped records; 1982 complex objects; 1983-84 molecular/composite objects; 1985-86 derivation histories, configurations, equivalences recognized; 1986-87 change notification/propagation; 1988 schema evolution (Banerjee/Kim) and awareness of the need for comprehensive frameworks. PROPOSED MODELS (4.2), each contributing a distinct mechanism: Haskin & Lorie [HASK82] — complex objects in System-R, foreign-key COMP_OF traversal, long transactions (but ignores versions). McLeod/Narayanaswamy/Bapa Rao [MCLE83] — early compositions+versions via AND-OR graphs (AND=composition, OR=alternatives), explicit instance/definition distinction, but no is-derived-from. Dittrich & Lorie [DITT88] — design object = a SET of versions with a distinguished current; GENERIC references + hierarchical ENVIRONMENTS for dynamic binding; logical VERSION CLUSTERS for arbitrary, possibly-overlapping groupings. Klahold/Schlageter/Wilkes [KLAH86] — VERSION GRAPHS (derivation is just one of many threadings), PARTITIONS by consistency level, SUBSET VIEWS. Batory & Kim [BATO85] — MOLECULAR OBJECTS (interface/implementation separation), TYPE-VERSION GENERALIZATION (versions as instances of a type, inheriting the interface), INSTANTIATION (use vs definition), PARAMETRIZED VERSIONS; change notification via two timestamps T_CN (last changed) and T_CA (changes last acknowledged) with implementation/reference/representation consistency conditions. Landis [LAND86] (Ontologic) — non-linear histories, current+default-branch, fixed (historical) vs floating references, change propagation with DELTA SETS and PENDING VERSION CREATION (no object evolves more than one new version per propagation). Chou & Kim [CHOU86] — transient/working/released versions; message-based (immediate) vs flag-based (deferred) notification; single-level (non-recursive) propagation. Ketabchi & Berzins [KETA87] — REFINEMENT (template/explosion/instance); a Refinement Graph (DAG) plus an Incremental Refinement Graph recording DELTAS; dependent/independent/alternative refinements; Parts/Projects/Private databases. Eckland et al. [ECKL87] (DVSS) — FEDERATIONS (site-distributed workspaces with associate/participating access); version DAGs with a PRINCIPAL PATH; principal-successor/derivation/consolidation relationships; fixed vs floating references. Beech & Mahbod [BEEC88] (IRIS) — version is an object with its own OID, organized in version SETS around a generic instance; version graph via functions; new versions by MUTATION (copy-on-write) or PROPAGATION; specific vs generic references resolved

by a rule-based trigger CONTEXT. Rumbaugh [RUMB88] — PROPAGATION ATTRIBUTES on relationships (none/propagate/shallow/inhibit) to control operation/change spread. Vines/Vines/King [VINE88] (GAIA) — timestamp versions; version-sensitive and change-sensitive relationships; explicit CHANGE REQUEST and CHANGE NOTIFICATION objects providing an audit trail; configuration objects grouping changes. SUN NSE [SUN88] — File/Target/Component/Linkdb objects, interleaved-delta storage, acquire/modify/reconcile, hierarchical environments (release/integration/development), variants for target architectures, email notification. Apollo DSEE [LEBL84] — History Manager (SCCS-like reserve/replace), Configuration Manager with system models and CONFIGURATION THREADS (component+version tabulations, dynamically bindable via VERSION MAPS; a bound thread + keyword = a RELEASE), Task/Monitor/Advice Managers.

“The model distinguishes among versions representing alternative implementations, revisions, and different representations”

“all versions of an object are constrained to implement the same interface... the type system's inheritance mechanisms can be leveraged”

“An object is implementation consistent if its T_CA exceeds T_CN... reference consistent if its T_CA exceeds the T_CN of all of its components”

“References among objects can be fixed to a specific version or left floating... floating references are resolved to the current version”

§ 5. Version Modeling Mechanisms and Policies (the unification). Re-cuts all the surveyed models by MECHANISM rather than by system, arguing 'much of the variation across models is due not so much to different concepts, but to different terminology for the same concept.' Reduces everything to SEVEN classes of mechanism (5.1): (1) ORGANIZING THE VERSION SET — version set/history/graph/refinement graph are all the same thing arranging versions by ancestor/descendant, with branches=alternatives, a distinguished current, and a generic object; identifiers are version numbers, timestamps, or OIDs. (2) DYNAMIC CONFIGURATION DEREFERENCING — static/specific/fixed vs dynamic/generic/floating/parametrized references; resolved either by layers+contexts/environments/version-maps OR by 'current on a distinguished path,' with Beech-Mahbod's rule-based approach the most general. (3) HIERARCHICAL COMPOSITIONS — flat configurations (a table mapping objects to versions, e.g. VINE88) vs hierarchical configurations (composite/molecular objects, explosion refinements, configuration threads). (4) VERSION GROUPING — Dittrich's logical version clusters (general, overlapping) vs Klahold's single-level partitions. (5) INSTANCES vs DEFINITIONS — only MCLE83 and BATO85 cleanly separate a use (instance, a copy) from a definition (version). (6) CHANGE NOTIFICATION & PROPAGATION — notification via timestamps/messages/flags; propagation = incorporating new versions up the configuration hierarchy, needing disambiguation (group check-in, pending version creation) and scope-limiting (propagation attributes, change-sensitive relationships). (7) OBJECT SHARING — workspaces moved through via check-in/out (a.k.a. acquire/modify/reconcile, reserve/replace), in repositories (workspaces/databases/federations/environments) that most agree need at least THREE levels (archive/group/private; parts/projects/private; release/integration/development). THE GRAND UNIFICATION (5.2) proposes consensus terminology, mostly adopting Section 3: version history with ancestor/descendant links, generic object, main derivation (= principal path = default branch), branches=alternatives, current decoupled from last-created, merged descent (= reconciliation = consolidation). It prefers DITT88's hierarchical ENVIRONMENTS for dynamic binding (subsumes layers/context and current-on-path), the composite-object terminology for configurations (plus a flattened-configuration form for archives), Dittrich's version clusters for grouping (orthogonal to ancestry), an explicit instance-hierarchy-referencing-definition-hierarchy for instance-specific attributes, and RUMB88's propagation attributes as the most fundamental scope-limiting mechanism (subsuming timestamp and change-sensitive methods).

“there are really only a small number of notions underlying the various proposals... seven classes of mechanisms”

“We feel it is important to decouple the notion of current from last created”

“[KATZ85] distinguishes between static and dynamic configuration binding... [ECKL87] calls these fixed and floating... [BEEC88] call them specific and generic”

“the propagation attribute approach of [RUMB88] can be made to cover the timestamp methods of [BATO85] and [CHOU86], as well as the change sensitive relationships of [VINE88], and is more general”

§ 6. Summary and Conclusions. Restates that despite many proposals 'with its own idiosyncratic terminology... most proposals were really minor variations on a small number of themes,' and that unification largely succeeded for version histories, dynamic configurations, and workspaces — apparent differences being terminological. Identifies mechanisms that GENERALIZE others (notably propagation attributes subsuming timestamp methods). Honestly concedes no ultimate model exists or is likely soon; the author's PREFERRED model = the Section-3 Version Server extended with Dittrich's logical version clusters + hierarchical environments, Eckland's federation concept, Rumbaugh's propagation attributes, and a merged-path version history. Closes by stating the framework-construction challenge — a single tailorable framework — 'is something that is still left to be accomplished. This paper represents a first step in that direction.'

“most proposals were really minor variations on a small number of themes”

“While we would have liked to have been able to present the ultimate version management model, we do not feel that such a model is likely to exist for some time”

“The final challenge, that of creating a single framework... that can be tailored for the needs of a given version environment, is something that is still left to be accomplished”

§ 7. Bibliography. ~30 references spanning 1982-1989, the foundational corpus of CAD/engineering version management: Haskin & Lorie 1982, Lorie & Plouffe 1983, McLeod et al. 1983, Batory & Buchmann 1984, Leblang & Chase 1984 (DSEE), Katz & Lehman 1984, Batory & Kim 1985 (ACM TODS), Katz 1985 (Springer survey monograph), Chou & Kim 1986, Klahold et al. 1986, Landis 1986, Ketabchi & Berzins 1987, Eckland et al. 1987 (DVSS), Banerjee/Kim/Korth 1987 (schema evolution), Dittrich & Lorie 1988, Beech & Mahbod 1988 (IRIS), Rumbaugh 1988 (OOPSLA, propagation attributes), Vines et al. 1988 (GAIA), SUN NSE 1988, plus the author's own Version Server papers (Katz/Chang/Bhateja 1986-88) and the Bhateja & Katz validation-subsystem paper.

“BANE 87... 'Semantics and Implementation of Schema Evolution in Object-Oriented Databases'”

“RUMB 88 Rumbaugh, J., 'Controlling Propagation of Operations Using Attributes on Relations,' OOPSLA '88”

HOW IT MAPS TO OUR PARTS

- **Object-container kernel: stable CONTAINER = identity thread, REVISIONS = immutable values, current-revision = state** — Gives our kernel its exact historical name and validates the shape. Katz's 'generic object' (e.g. ALU.Layout) is a STRUCTURAL object with NO corresponding file — pure identity thread — while each ALU[i].Layout is an immutable versioned instance hanging off it via is-a-kind-of/is-derived-from. That is our container/revision split almost line-for-line. His 'currency' mechanism is precisely our current-revision pointer, and his key insight directly supports our hard rule: he DECOUPLES current from last-created ('It is frequently the case that the default or current version is not the most recently created version'), proving the head pointer must be explicit state, not inferred from arrival order.
- **Versioning as its own layer: Rama arrival-time history (partial persistence) vs explicit git-style committed/minted versions (full persistence) — two distinct times** — Strong support, and a warning. Katz separates a 'version model' (organizing data across time) from a 'design transaction model' (how change happens), and insists 'not every update need result in the construction of a new version' — a version is 'a semantically meaningful snapshot.' That is exactly our two-times distinction: Rama's per-update arrival log is the substrate, but a minted/committed version is a deliberate, semantically-significant act layered on top. He also warns the cost of conflating them: uncontrolled version creation causes 'proliferation of versions, many of which are likely to be of little interest,' which his currency mechanism exists to constrain — i.e. minting must be gated, not automatic.
- **Groupings as versioned objects: a curated grouping/local world has its OWN version history, partly independent of members; member binding pinned/floating/effectivity-scoped** — Directly mechanized. Katz's configuration = 'a binding between a version of a composite object and the versions of its components,' and crucially a composite is 'a new version in its own right' even when 'its contents have not changed' but a

component changed — the grouping versions independently of and because of its members. His pinned/floating/effectivity vocabulary IS our binding spectrum: static/specific/fixed reference (pinned), dynamic/generic/floating reference (floating), and parametrized versions / layer-context resolution (effectivity-scoped). Dittrich's logical 'version clusters' (arbitrary, overlapping member groupings with their own identity) are our 'curated local world' precisely.

- **Four-level graph versioning: graph / node / relation-TYPE / node-DATA, and these must COMPOSE** — Partial support + a named gap. Katz delivers node-level (version history), node-DATA-level (revisions of contents), and graph/composition-level (configurations compose component versions hierarchically, and configuration mappings are stored PER NODE so versions compose without re-mapping the whole tree). The relation-TYPE level he does NOT version — his three relation kinds (is-a-part-of, is-derived-from, is-equivalent-to) are fixed schema, not versioned objects. He flags this frontier himself: by 1988 'the field has moved towards extending version concepts to model schema evolution... how the structure of the database changes over time, not just how its contents change' (citing Banerjee/Kim). That is our relation-TYPE-versioning requirement, named as the open edge of the field.
- **Preserved disagreement / plurality: conflicting views representable not overwritten; synthesis late-bound** — Mechanism + name. Katz's ALTERNATIVES are 'parallel versions... NOT on the same path to the root' — coexisting, non-overwriting branches; merging them is an explicit late act ('merged descent' / 'reconciliation' / 'consolidation'), never automatic. His EQUIVALENCES are an even stronger gift: a structural object that ties heterogeneous representations of the same real-world thing (layout/transistor/functional views) together as a CONSTRAINT 'exploited at design verification time,' not a forced merge. That is our model for holding plural representations in preserved tension with late-bound, checkable synthesis.
- **The trail-to-code join: DECLARED change (agent says what it did) vs INFERRED change (reconstructed by diffing)** — Strong support for declared-over-inferred. Katz's is-derived-from is an EXPLICITLY RECORDED ancestor relationship precisely because naming-scheme alone (name[version#].type) cannot recover lineage: 'it is not possible to determine from this naming scheme which version is descended from which.' This is our hard rule that inferred structure is never truth at ingest — derivation must be a declared, first-class edge. Ketabchi/Berzins' Incremental Refinement Graph (recording the actual DELTAS that differentiate a version from its ancestor) and Ontologic's 'delta sets' are the declared-edit-operation analog of our trail's edit operations resolving path+timestamp to code-at-that-moment.
- **The code-identity question: file vs path vs name/FQN vs AST vs git-hash; atomic unit = file or function** — Directly addressed, with a stance. Katz separates 'structural objects' (identity/organization, owned by design management) from 'representational objects' (the actual files, owned by tools), and insists 'structural information be separated from representational information... absolutely necessary if the same model is to be applicable to a wide range of design domains.' His unit is the named 'design object' ('better to think of them as units of the design that can be given a name, such as Arithmetic Logic Unit') — NOT the file. The generic object has no file at all. This argues our atomic unit should be a named structural identity (the container), decoupled from any single file/path/hash, which are merely its representational backing.

Takeaway. Katz's deepest contribution to our versioning/identity question is the separation of STRUCTURAL identity from REPRESENTATIONAL backing: a versioned object's identity lives in a 'generic object' that has NO file of its own (our container/identity-thread), while versions (our immutable revisions) and configurations hang off it via explicitly-recorded relationships — and 'current' must be decoupled from 'last-created' as deliberate state, never inferred from arrival order. He proves that derivation, equivalence, and grouping must be DECLARED first-class edges because naming/arrival data cannot reconstruct them — directly underwriting our hard rule that inferred structure is never accepted as truth at ingest. He also hands us the precise vocabulary for the whole problem (version history, configuration, alternatives vs derivatives, static/dynamic binding, equivalences, change propagation, version clusters) and names our one genuine frontier beyond his 1988 scope: versioning the relation-TYPE level (schema evolution), which the field had only just begun to address.

22. Versions of Schema for Object-Oriented Databases

Won Kim and Hong-Tai Chou (MCC, Austin TX) · 1988 · *Proceedings of the 14th VLDB Conference, Los Angeles, California*

In one line. Extends ORION's single-object version model to version the entire SCHEMA (the class lattice) as one versioned object, with a derivation hierarchy and an inherited 'access scope' that ties each schema version to a non-destructively branchable snapshot of the database.

Source access: full. Read the FULL paper. WebFetch downloaded the 1.3MB PDF from vldb.org but returned only binary; I extracted the complete text locally with pdftotext (805 lines, all 8 sections + references intact). OCR noise present (e.g. 'CHOUSS'=CHOU88, 'arz'=are) but technical content fully legible. Source read: the canonical VLDB-1988 PDF itself, not a secondary copy.

THE PAPER, SECTION BY SECTION

§ Abstract. States that prior version research versioned only single design objects, never the schema (the definition of those objects). In a multi-user design environment where the schema undergoes dynamic change, you must be able to version the schema too. The paper's contribution: a model of versions of schema obtained by EXTENDING ORION's existing model of versions of single objects — presenting its semantics, implementation issues, and alternatives.

“research in versions has been focused exclusively on versioning single design objects”

“we propose a model of versions of schema by extending our model of versions of single objects”

§ 1. Introduction. Sets the motivation. ORION is a prototype OODB at MCC supporting CAD/CAM/AI/OIS apps, already with object versions, composite objects, and dynamic schema evolution. The key argument: if the schema cannot be versioned, a schema change (e.g. dropping an attribute) IRREVERSIBLY harms pre-existing objects — dropped-attribute values become invisible. Versioning the schema instead lets old objects stay fully visible under the OLD schema version while the new version sees them without the dropped attribute. Second motivation: checkout/checkin in a federated private/public database — a modified schema must be copied to the public DB as a new schema version before a modified object can be checked in. Claims this is largely unexplored (only [SKAR86] touched versioning a single class) and promises two original contributions: a semantic model with a small command interface, and detailed implementation techniques.

“if an attribute of a class is dropped, the values of the attribute in existing objects ... become no longer visible to the application”

“If, however, a new version of the schema is derived in which the attribute ... is dropped, the values ... continue to be visible to the application under the old version of the schema”

“except for [SKAR86] ... the semantics and implementation of versions of schema have not been reported in the literature”

§ 2. Basic Object-Oriented Concepts. Review (extracted from [BANE87a]). Everything is an object with private memory (attribute/instance-variable values) and behavior encapsulated in methods; objects communicate only via messages, the public interface. 'Similar' objects are grouped into a CLASS sharing the same attributes and methods; objects of a class are its instances. The class hierarchy encodes IS-A: a subclass specializes a superclass and inherits its properties (attributes+methods collectively), inheriting transitively up the superclass chain. The domain (type) of an attribute is itself a class. ORION generalizes the hierarchy to a class LATTICE allowing multiple inheritance, which creates name conflicts: conflicts between a class and its superclass are resolved by giving the local definition precedence; conflicts AMONG superclasses are resolved by superclass ORDERING (the first superclass in the application-specified list wins). This lattice + flattening + ordering machinery is what later sections must version.

“A class describes the form (attributes) of its instances, and the operations (methods) applicable to its instances”

“a class can have more than one superclass, generalizing the class hierarchy to a lattice”

“the one chosen by default is that of the first superclass in the list of (immediate) superclasses of C”

§ 3.1 Versions (review of object versions). The foundational two-level model the whole paper builds on. Versions carry CAPABILITIES: a TRANSIENT version can be updated or deleted at will; a WORKING version can be deleted

but NOT updated. A transient version is promoted to working explicitly by the user, or IMPLICITLY when a new transient is derived from it. An object is versioned (instance of a class declared versionable) or non-versioned. A versioned object is a hierarchy of versions = a version-derivation hierarchy. Critical terminology: a VERSION INSTANCE is a specific version in that hierarchy; a GENERIC INSTANCE is the abstract versioned object that maintains the derivation history. Both the generic instance AND each version instance get their OWN unique object identifier. A reference to a version instance's OID = STATIC binding; a reference to the generic instance's OID = DYNAMIC binding, resolving to a default version instance (user-specified, else the most-recent timestamp). Dynamic binding is useful because transient/working versions may be deleted and new ones created.

"We use the term version instance to refer to a specific version ... and generic instance to refer to the abstract versioned object"

"both the generic instance and a version instance ... have object identifiers"

"If the reference is to a version instance, we say that the object is statically bound ... If the reference is to a generic instance, the object is said to be dynamically bound to a default version instance"

§ 3.2 Schema Evolution (review). Defines what a schema change IS, since the paper versions schema changes. The schema = the class lattice; two change types are meaningful: changes to a class DEFINITION (node contents: add/drop attributes & methods) and changes to lattice STRUCTURE (edges/nodes: create/drop a class, add/drop a superclass-subclass IS-A edge). It then gives the precise semantics of five operations: (1) Add attribute V to C — inherited by subclasses, overrides conflicting inherited attribute, existing instances get nil for V; (2) Drop attribute V from C — V removed from C and inheriting subclasses, those instances lose V's value, and an alternate same-named attribute from another superclass may be re-inherited; (3) Make S a superclass of C — must not create a cycle; C and subclasses inherit from S; (4) Add class C — inherits from all specified superclasses (or CLASS if none); (5) Drop class C — edges to subclasses dropped (they lose inherited properties), edges from superclasses removed, C deleted but its subclasses survive. These exact semantics are replayed in Section 5's implementation.

"the schema of an object-oriented database is the class lattice"

"Existing instances of the classes to which V is added receive the nil value"

"The addition of a new edge from S to C must not introduce a cycle in the class lattice"

§ 4. Versions of Schema (the model — 7 rules). The core contribution. Three candidate approaches are named — versions-of-SCHEMA (whole lattice = one versioned object), versions-of-CLASS (each class = a versioned object), and VIEWS-of-schema (dynamic views, not versions). They choose versions-of-schema and extend version capabilities, derivation hierarchies, and default versions to it, adding the view that 'a version of schema is associated with a SET OF OBJECTS created under it.' Key definitions: ancestor/descendant, parent/child, CURRENT schema version (where you operate now), CREATOR schema version (where an object was created), DIRECT access scope (objects created under SV), and ACCESS SCOPE (all objects visible to SV). They note only INSTANCE objects matter for versioning — class objects need not be versioned. The model is seven rules: (1) Capability Rule — transient vs working schema versions; working can't be updated but can be demoted if it has no child; (2) Derivation Rule — derive any number of new schema versions from any existing one, forming a derivation hierarchy; deriving from a transient auto-promotes it to working; (3) Deletion Rule — only a LEAF schema version can be deleted, and deleting it also deletes its direct access scope (so 'a schema version OWNS the objects created under it'); (4) Access-Scope Inheritance Rule — SV-j by default inherits SV-i's access scope, but the user can BLOCK inheritance, and once inherited, SV-i's scope by default becomes non-updatable under SV-i (optionally left updatable); (5) Access-Scope Rule — SV's access scope = objects created under SV plus those inherited from ancestor scopes; all are readable AND updatable, nothing else is visible; (6) Direct-Access-Scope Update Rule — SV's direct scope is frozen once any child is derived from it WITHOUT blocking/leaving-updatable; (7) Inherited-Access-Scope Update Rule — updates/deletes of inherited objects under SV are visible ONLY to SV and descendants that inherited from SV (branch-local). Three derivation options follow (inherit+freeze, inherit+keep-updatable, don't-inherit), changeable later. Figure 1 walks SV-0 → SV-1 (drop at3) → SV-2 (add at4) showing objects flowing into each scope minus/plus attributes.

“a schema version 'owns' the objects created under it, that is, its direct access scope”

“SV-j by default inherits the access scope of SV-i. However, the user may optionally block the inheritance”

“The access scope of a schema version SV is the set of objects created under SV and those objects in the inherited access scopes of the ancestor schema versions”

“updates and deletes under SV of the objects inherited into SV are only visible to SV and the descendant schema versions of SV which inherited the access scope from SV”

§ 5.1 Implementation — Auxiliary Data Structures for Objects. To make versions-of-schema cheap, every object gets one system-defined CREATOR attribute (its creator schema version) and, when needed, an ANCHOR INSTANCE side-structure that describes a SET OF COPIES of the object — analogous to how a generic instance describes the set of version instances. The trick: copy-on-write across schema versions. An object first exists with no anchor instance, just its creator id. When it is first deleted/replaced under a NON-creator schema version, the system makes a new COPY and an anchor instance. All copies AND the anchor share the SAME object identifier (so existing references stay valid); the CREATOR attribute distinguishes copies. The anchor instance holds two attributes: a list of TERMINATOR schema versions (where the object was deleted — always a descendant of the creator) and a list of copies. Figure 2 traces: copy-0 created under SV-0; updated under SV-4 → anchor instance + copy-1 (creator SV-4); copy-0 deleted under SV-1 → SV-1 recorded as a terminator.

“we need to include in every object one system-defined attribute, called the creator attribute, and to associate a data structure, called an anchor instance, with every object”

“An anchor instance, and each of the copies of the object it describes, are all identified by the same object identifier of the object”

“A terminator schema version is the schema version under which the object was deleted”

§ 5.2 Object Access Algorithms. Precise fetch/insert/delete/replace algorithms using creator/anchor/terminator. Two booleans: Ancestor-Of (true if arg1 is an ancestor whose access scope is inherited by arg2) and Frozen (true if a SV's direct scope is non-updatable). Helper find-closest-copy walks the copy-list returning the closest ancestor copy; copy-blocked checks whether a terminator hides a copy from sv. ALGORITHM-SV-FETCH: locate object; if it's an anchor instance, find the closest copy and error if none visible / blocked by a terminator; else error if the lone copy's creator isn't an ancestor of sv. INSERT errors if frozen, else stamps creator=sv and allocates an id (new objects join the current version's DIRECT scope). DELETE/REPLACE: if creator=sv, physically delete/update in place; else (inherited object) create/extend an anchor instance, adding sv as a terminator (delete) or pushing a new copy with creator=sv (replace) — making the change branch-local. Performance: the Ancestor-Of check dominates, so the ancestor relationship is encoded in per-schema-version BIT VECTORS (root = all zeros; deriving SV-j inheriting from SV-k sets $BV-j[k]=1$ and copies $BV-k$), reducing the ancestor test to an $O(1)$ vector access; provably $BV-j[i]=1$ iff SV-j inherits from SV-i. A second bit vector tracks class-object inheritance since class objects are always inherited at derivation time.

“the system will create a new copy of the object when the object is first deleted or replaced under a schema version which is not its creator schema version”

“the ancestor relationship can be encoded in bit vectors, one for each schema version”

“we have reduced the check for ancestor relationship to a vector access”

§ 5.3 Storage Representation for the Schema. How a single schema is stored, then how versions are stored cheaply. A single schema = a set of CLASS OBJECTS, each represented as instances of system-defined classes Class, Attribute, Method (like system catalogs). Class holds ClassName, Attributes, Superclasses, Subclasses, Methods — and crucially Attributes/Methods hold BOTH locally-defined AND inherited properties ('FLATTENING' of the lattice, [ZARA85], for fast access). Attribute holds Class, VariableName, Domain (the class the value binds to), InheritedFrom. For VERSIONS, the objective is NOT to copy the whole schema per version. Insight: every schema change is either a class-definition change or a class-relationship change, and relationships are encoded inside class objects — so they keep Class/Attribute/Method instances NON-versioned but apply the SAME anchor-instance copy mechanism to CLASS OBJECTS. A class-definition change copies that one class object; a relationship change copies the class object of EACH affected class (both endpoints of an edge). Figure 4 traces

five schema changes (define C-a/b/c; add subclass C-d under SV-1 forcing a copy of C-b; add C-e under SV-2 forcing another C-b copy; make C-c a superclass of C-e under SV-3 copying both; delete C-b under SV-4 copying C-a/C-d/C-e). The schema-version derivation hierarchy itself is stored via a system class SCHEMA with a single versionable instance — each schema version is a version instance of that one object; SCHEMA carries Update-Instances (boolean, false when a child froze the scope) and Ancestor-SVs (list of ancestors whose scope is inherited).

“Attributes and Methods attributes for a class hold values for not only the attributes and methods defined for the class, but also those inherited ... This technique is known as 'flattening'”

“we ... apply the anchor instance structure to class objects to support updates to a class object under different schema versions”

“Each schema version is a version instance of this versioned object, and the schema-version derivation hierarchy is maintained in the generic instance”

§ 6. User Interface. A deliberately small command set despite rich semantics. `derive-schema-version(parent-SV, highest-ancestor-SV, update-instances)` returns a new schema version derived from `parent-SV` (auto-promoting parent if transient); `highest-ancestor-SV` sets the highest ancestor whose access scope is inherited (default root = full inheritance; value 'self' = no inheritance; otherwise the union of direct scopes from parent up to that ancestor); `update-instances` defaults False (inherited scope becomes non-updatable under its creators) — True leaves them updatable. `change-inheritance(highest-ancestor-SV, update-instances)` lets you change those two parameters AFTER derivation. `delete-schema-version(SV)` deletes SV plus all version instances created under it, rejected if SV has a child. `promote-schema-version(SV)` upgrades transient → working. `set-current-schema-version(current-SV)` switches the active schema version (all ops run under it). `current-schema-version()` returns it.

“The set of messages is surprisingly small, despite the rich semantics that the model captures”

“If the value of the parameter is self, the access scope of the parent schema version is not inherited into the new schema version”

§ 7. Alternative Approaches (versions-of-class, views-of-schema). Defends the design against the two rejected approaches. 7.1 Versions of Classes ([SKAR86]): treat each class object as versionable and build a 'virtual' schema as a lattice picking one version per class. Problems: the user must MANUALLY track which class-versions form which virtual schema and maintain per-virtual-schema default-version lists for every generic class reference; dynamic binding of inter-class references means the virtual schema CANNOT be FLATTENED (losing fast access). Even with static binding it's worse: because each class version has a different OID and class objects carry Superclasses/Subclasses, deriving a new version of class C forces a new version of EVERY subclass AND superclass — cascading into 'a new version of the entire class lattice for each virtual schema version.' ORION's own approach uses COPIES (not versions) of class objects, allowing only one copy per class within a schema version — so users can't experiment with alternative defs of a class within one schema version, but avoid the cascade and the cross-version surprise-update hazard. 7.2 Views of Schema: provide dynamic views over a single underlying schema with a view-derivation hierarchy, but with NO inheritance of objects between views — every object belongs to the single schema, and a view's scope is just the objects whose attributes the view defines. Consequence: ALL updates under one view become visible to all views sharing those attribute definitions — so a view does NOT capture a point-in-time DB snapshot, sharply contrasting ORION where objects under SV can be preserved against derived versions.

“if we are to use versions of the class objects to support 'virtual' schema versions, we will end up generating a new version of the entire class lattice for each 'virtual' schema version”

“the use of dynamic binding of references to class objects implies that the 'virtual' schema version cannot be flattened for efficient access”

“the schema view approach does not associate with any view a snapshot of the state of the database at some point in time ... This contrasts sharply with our model”

§ 8. Summary. Recaps that versioning both design objects AND their schema matters in multi-user design, that object-version research was extensive but schema versioning had virtually no formal semantics before this paper,

and that they built schema versions atop ORION's object-version model. Reasserts the two admitted weaknesses (non-trivial overhead for inherited-object updates, and user confusion when an object deleted in a descendant reappears when operating under the ancestor — 'exactly the desired effect ... however ... somewhat confusing'). Frames the central thesis: each schema version captures the DB state at a point in time, inheritable into derived versions for read and update — and the two alternatives (class versioning, dynamic views) were shown to fall short of this.

"each version of the schema captures the state of the database at some point in time and that the state may be inherited into any derived schema version for read and update"

"there has been virtually no formal proposal to define the semantics of versioning the schema"

HOW IT MAPS TO OUR PARTS

- **Object-container kernel (container=identity thread, revisions=immutable values, current-revision=state)** — GIVES IT A NAME and a battle-tested mechanism. The paper's generic instance / version instance split is exactly our container/revision split: a 'generic instance' is the abstract versioned object that carries a stable object identifier and the derivation history; a 'version instance' is a specific concrete version with its OWN identifier. ORION even gives BOTH the container AND each revision distinct OIDs, then lets references target either one. This validates our two-level model and warns us we must decide which identity a reference resolves to.
- **Versioning as its own layer (arrival-time history vs committed/minted versions)** — SUPPORTS the two-times distinction and shows the cost of the explicit-commit (full-persistence) side. ORION's schema versions are git-style MINTED versions you branch from (derive-schema-version), with transient (mutable) vs working (frozen) capability — directly our 'committed/minted versions you can branch from.' The paper has NO arrival-time layer; everything is explicit derivation. It thus shows the full-persistence half of our design in isolation, and the bookkeeping (anchor instances, terminators, bit-vectors) it costs.
- **Groupings as versioned objects (a curated local world has its OWN version history, partly independent of members; member binding pinned/floating)** — OFFERS A DIRECT MECHANISM and a failure-mode warning. ORION treats the ENTIRE schema (the whole class lattice = a grouping of class objects) as a single versioned object with its own derivation hierarchy — independent of any individual class's evolution. Each schema version is itself a version instance of one system SCHEMA object. The rejected 'versions of classes' alternative is our 'floating member binding' failure mode: when members version independently and reference each other, deriving one member forces re-versioning of all neighbors ('a new version of the entire class lattice for each virtual schema version'). Lesson: version the GROUP, snapshot the members.
- **Four-level graph versioning (graph / node / relation-type / node-data, composing)** — PARTIALLY VALIDATES and shows the composition tax. ORION versions at graph level (the schema/lattice = SchemaVersion), node level (class objects, via copies under each SchemaVersion), relation-TYPE level (IS-A edges encoded inside class objects, so an edge change forces copies of both endpoint class objects), and node-DATA level (instance objects via anchor instances + creator/terminator). It proves these levels CAN compose, but the cost is the warning: an edge change cascades into multiple node copies, and the access-scope rule is the glue that makes node-data inheritance compose with graph versions.
- **The trail-to-code join (resolve a path+timestamp to code-at-that-moment; declared vs inferred change)** — OFFERS THE RESOLUTION ALGORITHM. ORION's ALGORITHM-SV-FETCH(object-id, sv) is precisely 'resolve identity X as-of version-context sv': find-closest-copy walks ancestor copies, copy-blocked checks terminators. This is the template for resolving a trail's file-identity at its commit/version context. Our 'declared vs inferred' maps to ORION's static vs dynamic binding: a reference to a specific version instance = DECLARED/pinned; a reference to the generic instance resolving to a default = INFERRED/floating.
- **Preserved disagreement / plurality (conflicting views representable, synthesis late-bound)** — SUPPORTS via the access-scope model. ORION lets sibling schema versions hold DIVERGENT updates to inherited objects that are 'visible only to SV-j; viewed from SV-i it is as if the updates never took place' —

concurrent, non-destructive, branch-local truth, exactly preserved disagreement. The paper contrasts this with the rejected 'views of schema' approach where 'all updates under one view become visible to all views' — i.e. silent overwrite, the anti-pattern we want to avoid. ORION chose snapshot-isolation-per-branch over shared mutable views.

- **Hard rule: 'inferred structure is never accepted as truth at ingest'** — SUPPORTS through static binding and creator-attribute provenance. ORION records a CREATOR schema version on every object (declared provenance, stamped at write) rather than inferring which version an object belongs to. Static binding pins a reference to an exact version instance so the system never guesses. The dynamic-binding/default-version path is explicitly the convenience/inference path, and the paper flags that 'virtual versions cannot be flattened' and 'may generate surprises' — i.e. inference is second-class, exactly our rule.
- **Substrate choices: Rama event-sourced full vs partial persistence** — WARNS about implementation overhead if we hand-roll. ORION must add a CREATOR attribute to every object, an ANCHOR INSTANCE side-structure (terminator list + copy list) for any object touched outside its creator version, and per-version BIT VECTORS to make ancestor-checks $O(1)$. The authors concede 'non-trivial system overhead' and user 'confusion' as the two admitted weaknesses. In Rama these become PState design choices — and the paper argues a log/copy-on-derive substrate is the right shape, which Rama already gives us for free.

Takeaway. The paper's enduring contribution to our versioning/identity question is the generic-instance / version-instance split with static vs dynamic binding — a stable identity-bearing container (its own OID) holding immutably-derived version instances (their own OIDs), where a reference EITHER pins an exact version (static = our 'declared/pinned') OR floats to a default (dynamic = our 'inferred/floating'). Equally important, it proves that versioning a GROUPING (the whole schema lattice) as a single versioned object — and snapshotting members via copy-on-derive plus an inherited 'access scope' — beats versioning each member independently, because independent member-versioning cascades into re-versioning the entire neighborhood. This is direct, 1988-tested guidance for our 'groupings as versioned objects' and 'four-level graph versioning' layers: version the group, snapshot the members, and make inheritance (not shared mutation) the default so branches preserve disagreement non-destructively.

23. Versioning and Configuration Management in an Object-Oriented Data Model

Edward Sciore · 1994 · The VLDB Journal, vol. 3, no. 1, pp. 77–106

In one line. Extends the EXTRA data model / EXCESS query language so that versions form a multidimensional version SET inside the object, and configurations are declarative (intensional) queries over those dimensions — yielding one application-independent versioning model that subsumes CAD, CASE, and temporal databases.

Source access: blocked-used-alternative. The primary article on Springer (link.springer.com/article/10.1007/BF01231359) is paywalled — it 303-redirects to an IdP login; no open PDF, tech-report, CiteSeerX, or author-homepage mirror exists (searched extensively; scispace 403s, Google Scholar blocks). I reconstructed the model from (1) the paper's published abstract as quoted across Springer/scispace search results, and (2) the open, full-text Conradi & Westfechtel survey 'Version Models for Software Configuration Management' (ACM Computing Surveys 30(2), 1998, pp.232–282), which describes Sciore 1994/EXTRA-V directly in several passages (its n-dimensional variant grid in Fig.5b is attributed to Sciore 1994; it cites Sciore 1994 on the awkwardness of separate types for versioned objects). The section-by-section walk below is therefore a faithful RECONSTRUCTION of Sciore's argument from its abstract + this authoritative secondary source, NOT a reading of the primary article's own section text. Page-level section titles of the primary paper could not be verified; section names below are reconstructed from the abstract's stated contributions and are flagged as such.

THE PAPER, SECTION BY SECTION

§ **Abstract (verified — quoted from search-surfaced abstract).** States the problem: many database applications (CAD, CASE, temporal/historical DBs) need to store and manipulate multiple versions of data objects, but contemporary DBMSs support versioning only at a low, procedural level. Sciore's thesis is that application-independent versioning can be supported at a SIGNIFICANTLY HIGHER level of abstraction. The mechanism:

extend the EXTRA data model and the EXCESS query language so configurations can be specified conceptually and non-procedurally; show that a version SET can be viewed MULTIDimensionally, letting configurations be expressed at a higher abstraction level. A version-specification language is given at the conceptual level and a multidimensional specification language at the logical level, encoding an application's logical versioning semantics into orthogonal dimensions — producing a unified, application-independent treatment of versioning.

“application-independent versioning can be supported at a significantly higher level”

“version sets can be viewed multidimensionally, thereby allowing configurations to be expressed at a higher level of abstraction”

“encoding the logical versioning semantics of an application into orthogonal dimensions”

§ Introduction / Motivation (reconstructed from abstract + survey framing). Frames versioning as cross-cutting CAD, CASE, and temporal databases, each of which historically built its own ad-hoc versioning. The argument (corroborated by the survey crediting Sciore with attacking the 'separate types' problem) is that prior models force a distinction between a versioned object and its versions, requiring two object types — which 'is awkward and complicates both schemata and queries.' Sciore positions the contribution as moving versioning UP into the data model and query language so that one mechanism serves all three application domains rather than each reinventing it.

“As argued in Estublier and Casallas [1994] and Sciore [1994], this distinction is awkward and complicates both schemata and queries.”

“DAMOKLES, EXTRA-V [Sciore 1994], and Adele follow this approach [version model built into the data model].”

§ Version sets and the multidimensional model (the core — reconstructed via survey Fig. 5b). The central mechanism. A versioned object owns a SET of versions, and that set is organized not as a linear version graph nor merely as a tree of branches, but as an n-dimensional GRID: each dimension corresponds to a variant attribute, and a specific version is a point in this coordinate space. The survey attributes exactly this grid organization to Sciore 1994 and explains WHY it matters: pure branch-based variant modeling explodes combinatorially — the number of branches is dominated by the product of dimension cardinalities ($b \approx |A_1| \cdot \dots \cdot |A_n|$; their worked example of $OS \times DB \times window = 3 \times 3 \times 3$ needs up to 27 branches). The multidimensional version set tames this by making variation orthogonal rather than enumerated. Time/revision can be added as an additional orthogonal dimension (orthogonal version management).

“versions may be arranged in a grid (Figure 5b), that is, an n-dimensional space whose dimensions correspond to variant attributes [Sciore 1994]”

“In the case of multidimensional variation, this approach [branches] breaks down because the number of branches explodes combinatorially.”

“ $b \approx |A_1| \cdot \dots \cdot |A_n|$ [the branch count is dominated by the product of domain cardinalities]”

§ Version-model-built-into-the-data-model: EXTRA-V (reconstructed via survey §4). Sciore's architectural choice. The survey's taxonomy distinguishes three options: version model ON TOP OF the data model (ordinary DB application; simple but inefficient — 'no support for storing versions efficiently, the transaction manager does not take versioning into account'), version model BUILT INTO the data model (the DDL gives customized constructs for defining versioned object types and the query language is modified for natural queries over versioned data), and data model on top of version model (orthogonal version engine). Sciore's EXTRA-V is explicitly placed in the BUILT-INTO category alongside DAMOKLES and Adele. The payoff is that versioned and unversioned objects need not be different types and queries stay convenient; the cost is data-model complexity and the lack of a universally accepted uniform version model.

“If the data model is extended with versioning, applications can be supported through a data-definition language that provides customized constructs for defining versioned object types.”

“the query language is modified so that queries against a versioned database can be written in a convenient and natural way”

§ Declarative configurations as intensional selection (reconstructed via survey §3.2 / §5). Sciore's 'conceptual, non-procedural' configuration specification corresponds to the survey's INTENSIONAL versioning: rather than enumerating members extensionally ($V = \{v_1, \dots, v_n\}$, each immutable and uniquely numbered — retrieval of previously built versions), an intensional configuration defines the set by a predicate ($V = \{v \mid c(v)\}$) and CONSTRUCTS versions on demand. A configuration query is a (partial) function $q: A_1 \times \dots \times A_n \rightarrow V$ over the coordinate dimensions — i.e., supplying a tuple of coordinate values selects/constructs the version. This is precisely Sciore's claim that configurations can be 'specified conceptually': you describe WHICH version by its properties/coordinates, and the system selects or builds it, rather than hand-assembling a parts list. The survey notes intensional versioning demands both combinability (any version constructed on demand) and consistency control (constructed versions must satisfy constraints).

"Intensional versioning... the version set is defined by a predicate: $V = \{v \mid c(v)\}$ "

"a query corresponds to a (partial or total) function q that creates versions in V based on attributes ranging over the domains: $q: A_1 \times \dots \times A_n \rightarrow V$ "

"an SCM system must provide for both combinability... and consistency control—a constructed version must meet certain constraints"

§ Composite/assembly objects (the requested research angle, reconstructed via survey §2.2). Directly relevant to our research angle 5 (composite/assembly). A composite object / configuration is an object o representing a subgraph of the product graph — all objects transitively reachable from o via composition relationships. It can carry structural constraints (tree vs DAG), existence dependencies (a component may depend on its superobject), and acts as a UNIT with respect to structural operations (copy/delete), abstraction (encapsulation), concurrency control (locking the composite locks its components), AND version control. The key composite-versioning insight Sciore operationalizes: the assembly is itself a versioned item with its own history, and its members are bound either extensionally (pinned, specific version IDs) or intensionally (floating, by coordinate predicate / effectivity). Dependency relationships (orthogonal to composition) connect a dependent to a master and require the dependent be kept consistent as the master changes.

"Objects that are decomposed are called composite objects or configurations."

"a composite object is defined as an object o that represents a subgraph of the product graph. All objects that are transitively reachable from o via composition relationships belong to this subgraph."

"a composite object may act as a unit with respect to structural operations... concurrency control... and version control"

§ Unification across CAD / CASE / temporal databases (reconstructed from abstract). Sciore's closing claim of generality: by reducing each application's versioning to orthogonal dimensions plus declarative selection, one mechanism subsumes CAD assembly configurations, CASE software configuration management, and temporal/historical databases (where TIME is simply one more orthogonal dimension of the version set). This 'application-independent' framing is the paper's headline contribution — versioning ceases to be a per-domain feature and becomes a data-model-level capability parameterized by which dimensions an application declares.

"a unified, application-independent treatment of versioning"

"Revisions can be represented in the grid by adding a time dimension (orthogonal version management)"

§ Conclusion (reconstructed). The paper concludes that versioning belongs at the data-model and query-language level, expressed as multidimensional version sets with declarative (intensional) configurations, rather than as procedural application code. The dual specification languages — conceptual version-specification and logical multidimensional-specification — let an application encode its versioning semantics as orthogonal dimensions, achieving abstraction, application-independence, and natural querying. The survey's longevity-test corroborates the influence: Sciore 1994 is cited as the canonical source of the n -dimensional grid version model and as an early argument for collapsing the versioned-object/version type distinction.

"a version specification language was developed at the conceptual level, and a multidimensional specification language at the logical level"

HOW IT MAPS TO OUR PARTS

- **Object-container kernel (container=identity, revisions=immutable values, current-revision=state)** — Strongly supports and names it. Conradi/Westfechtel's framing of Sciore: 'a versioned item is a container for a set V of versions' is exactly our container/revision split. Critically, Sciore (and Estublier/Casallas) ARGUE AGAINST the naive form of our model: requiring different TYPES for the container vs its versions is 'awkward and complicates both schemata and queries.' Sciore's fix (version model built INTO the data model, EXTRA-V) folds versioning into the object type itself rather than wrapping it. Warning for us: if our container and revision are modeled as two distinct kernel object types, every query and schema pays a join tax. Consider Adele's resolution (common attrs shared by all revisions, modifiable revision-specific attrs, immutable attrs whose update mints a new revision).
- **The code-identity question (file vs path vs FQN vs AST vs git-hash; atomic unit = file or function)** — Offers a mechanism, not a single answer. Sciore decouples identity from any one coordinate: a versioned item has identity, and a specific version is selected by a point in a multidimensional coordinate space. Mapped to us: a code unit's identity is the CONTAINER; file/path/FQN/AST-hash/git-hash become DIMENSIONS (coordinates) along which versions vary, not competing identities. This dissolves the either/or: you don't pick one identity criterion, you make several orthogonal selection axes. Caveat: this is inference — Sciore's dimensions are variant attributes (os/db/window-system), and I am analogizing them to our identity coordinates.
- **Versioning as its own layer (arrival-time history vs committed/minted versions; partial vs full persistence)** — Names the deeper distinction we need. The survey's extensional vs intensional split maps onto our two times. Extensional versioning ('V defined by enumerating members v1..vn, each immutable, identified by a unique number, checked in once') = our explicit COMMITTED/MINTED git-style versions. Intensional versioning ('V = {v | c(v)}', constructed on demand from a predicate/query) is a SECOND axis we did not yet name. Sciore's contribution is that a configuration is an intensional query against the version space, not a stored list. Warns us: Rama's arrival-time log alone gives only extensional history; branchable committed versions need an explicit immutability/mint operation, and declarative configuration needs an intensional selection layer on top.
- **Groupings as versioned objects (a grouping/local-world has its own version history; member binding pinned/floating/effectivity-scoped)** — This is the paper's center of gravity (CAD/CASE configuration management) and directly validates us. A composite object/configuration is itself a versioned item with its own history, AND its binding to members is a separate concern. Sciore's declarative configurations let the grouping specify members by predicate over coordinates (floating/effectivity-scoped) rather than by pinned IDs (extensional). The survey: configuration construction 'may be viewed as a selection against a versioned object base... directed by configuration rules.' So 'pinned' = extensional binding, 'floating/condition-scoped' = intensional binding — and Sciore's whole point is you want BOTH expressible declaratively in one query language (EXCESS).
- **Four-level graph versioning (graph / node / relation-type / node-data, must compose)** — Supports composition; warns about the type explosion. Sciore's grid model shows why per-branch variant modeling explodes: 'b = |A1|...|An|' — with 3 OS x 3 DB x 3 window = 27 branches. Our four levels are four such dimensions; if we model their combinations as explicit branches they multiply combinatorially. Sciore's answer — an n-dimensional coordinate space with intensional selection — is precisely how to make four levels COMPOSE without enumerating the cross-product. The relation-type level versioning especially benefits: visibility of a relation can be made coordinate-conditioned (cf. COV: 'a relationship is visible only when both ends belong to the currently selected version').
- **The trail-to-code join (resolve path+timestamp to code-at-that-moment; DECLARED vs INFERRED change)** — Maps to extensional retrieval + the source/derived distinction. Resolving a trail's (path,timestamp) to the immutable revision at that moment is exactly extensional versioning: 'supports retrieval of previously constructed versions... each identified by a unique number... made immutable when checked in.' Our DECLARED-vs-INFERRED split aligns with the survey's source-object (created by a human/agent, here the agent DECLARES the edit) vs derived-object (reconstructed automatically by a tool — INFERRED by diffing).

The kernel rule 'inferred structure is never accepted as truth at ingest' matches treating derived objects as recomputable projections, never as checked-in source revisions.

- **Preserved disagreement / plurality (conflicting views representable, synthesis late-bound)** — Partial support via variants-not-merged. The survey notes that with branch/merge 'the branches are not joined; rather, each of them continues to exist' — co-present alternatives, our plurality. Sciore's multidimensional variant space is the cleaner mechanism: divergent views become distinct coordinates in the same version set rather than overwrites, and a 'configuration' (synthesis) is a late-bound intensional query that picks/combines among them. This is inference-adjacent: Sciore targets engineering variants (os/db), not epistemic disagreement, but the structural move (preserve as orthogonal coordinates, resolve by query at read time) is exactly late-bound consensus.
- **Substrate choices (Rama event-sourced; version model relative to the data model)** — Gives us a design-axis taxonomy. The survey's three architectures — version-model-ON-TOP-of, BUILT-INTO, and data-model-on-top-of-version-model — is a direct decision frame for Rama. Rama-as-substrate with a versioning layer above is 'version model on top of the data model' (simple, but 'no support for storing versions efficiently, transaction manager unaware of versioning'). Sciore's EXTRA-V chose BUILT-INTO for query convenience. Warns us: bolting git-style versions purely on top of Rama PStates risks the inefficiencies the survey names; if versioning is first-class we may want it woven into the kernel object type (container with versioned attributes) rather than as an external wrapper.

Takeaway. Sciore's single most important contribution for us: model a versioned thing as ONE container holding a multidimensional version SET, where every axis of variation (revision/time, variant, platform — for us: path, FQN, AST, git-hash, graph/node/relation/data level) is an ORTHOGONAL COORDINATE rather than a branch, and make a 'configuration' a DECLARATIVE (intensional) query over those coordinates instead of a pinned parts list. This is the structural escape from the combinatorial branch explosion our four-level graph versioning would otherwise hit ($\approx$ product of dimension cardinalities), and it cleanly separates extensional retrieval (resolve trail path+timestamp to the immutable revision-at-that-moment) from intensional selection (a floating/effectivity-scoped grouping). The load-bearing warning: Sciore argues a versioned object and its versions should NOT be separate types — fold versioning into the kernel object so schemata and queries don't pay a perpetual join tax, which directly challenges any design that makes our container and revision two distinct Rama object types.

24. Configuration Management of Industrial Products in PDM/PLM

Olivier Rey · 2021 · Author homepage (orey.github.io), technical essay, GNU FDL V3

In one line. An industrial PLM/PDM essay arguing that dynamic filtering-based configuration management (serial-number effectivity + recomputed assemblies) is a structural source of unreliability, and that static 'wired' graph data — explicit, persistent, immutable version-to-version links — is the only configuration model that secures past configurations forever, supports branching, and enables the digital twin.

Source access: full. Read the full article text via WebFetch directly from the author's open homepage <https://orey.github.io/papers/articles/config-mgt/> (Olivier Rey, July 2021, GNU FDL V3). Not paywalled. The fetch reproduced all section headings and substantive content including the Figure 6/7 examples; I did not see the figures themselves, so Figure descriptions rely on the surrounding prose.

THE PAPER, SECTION BY SECTION

§ Introduction. Frames CM in industrial PLM/PDM as a contrast between dynamic filtering-based CM and static graph-based CM. The thesis: software engineering matured its configuration practices while traditional manufacturing still relies on outdated CM that impedes digital advancement and limits digital-twin capability. The core operational goal of CM is stated precisely — obtain the exact applicable parts/design-solutions for a given product instance while managing options and tracking change across the lifecycle. The complete tree is the '150% vision': it holds every option, but no single instance uses all of them.

“To be able to get the exact list of applicable parts (or applicable design solutions) for a certain instance of the product”
“the 150% vision of the product”

§ The Product Tree. Products are modeled as trees whose leaves are parts/design solutions, assembled from multiple viewpoints: functional/systems-engineering splits, regulatory structures (ATA codes in aerospace), organizational divisions, and the PDM tool's own technical constraints. The over-complete tree sets up the central problem: how to extract the real per-instance configuration from a tree that contains more than 100% of any actual product.

“The complete product tree contains the 150% vision of the product—all options, though not all apply simultaneously to any single instance.”

§ Manual Version Management. Legacy PDM systems have no native version management; new versions are created by copy-and-rename with version numbers — a manual discipline. The bicycle-brake example: when a supplier discontinues a brake, a new design solution V2 is created while V1 is preserved to support already-manufactured units; manufacturing switches from V1 to V2 only when old stock is exhausted. Configuration changes thus have immediate physical industrial consequences (stock, support), not just data consequences.

“When design solutions change, new versions must be manually created by copying and renaming with version numbers.”

§ Configuration Management Using Links (S/N effectivity). The traditional mechanism: applicability is attached to product-tree LINKS as serial-number ranges. Brakes V1 apply to S/N [1,N], V2 to [N+1, infinity). Filtering the complete tree with a specific S/N yields that instance's options; filtering with a future S/N can return ALL options (over 100% of the product). This is the canonical serial-number-effectivity model and the prototype of the 'dynamic filter' the paper attacks. The Product Catalog complements it: external compatibility tables encoding non-trivial interdependent option rules, binding S/Ns to option lists.

“Brakes V1 are applicable to bicycle which S/N is in the range [1, N], and brakes V2 are applicable to the products which S/N is in the range [N+1, infinity].”

“non trivial rules that depend on one another”

§ Managing the Changes / Change tracking. Components are versioned and change-tracking is fundamental because of industrial impact on stock, procurement, manufacturing. Change management justifies modifications and masters their consequences. In aerospace it carries certification weight: change tracking proves certification inheritance, and a significant change triggers re-certification. This establishes change as a first-class, regulated object, setting up the aerospace 'stack of changes' model.

“In aerospace, change tracking proves certification inheritance; significant changes require re-certification.”

§ Applicability Through Changes (stacking) and the no-fork failure mode. In aerospace the product is 'seen through changes' rather than as physical components: Change 1 creates Brakes V1, Change 2 retrofits V1 (applicable only when both Change 1 AND 2 apply), Change 3 replaces with V2. A product becomes 'a set of changes', requiring 'complex calculation' to replay the iterative sequence. Key warning: recomputing a past configuration gives NO guarantee of matching what was actually delivered if business rules changed. The model also establishes lineage that typically PREVENTS forks — a fork from V1 while Change 4 targets V2 'cannot be represented', forcing every later change to be manually undone ('a manual, costly and error prone process'). This is the paper's strongest demonstration of why a linearizing history substrate becomes unusable.

“calculating past configurations offers no guarantee of matching originally-delivered configurations if business rules change —leading to perceived PDM unreliability”

“The model establishes lineage between changes, typically preventing forks.”

“all changes must be manually undone—a manual, costly and error prone process”

§ From PDM to CLM. Stacking-based change management degrades PDM into Change Lifecycle Management (CLM), where the change is central and the product secondary — the inverse of true PLM where 'the product is at

the center and the change on a secondary level.' This inversion 'closes the door to static graph data usage' and cascades problems through manufacturing. The product-first vs change-first framing becomes the paper's organizing axis.

"the product is at the center and the change on a secondary level"

"closes the door to static graph data usage in PLM"

§ Limitations of Filtering-Based CM + workarounds. Filtering-based systems (S/N applicability or change-based) are 'structural sources of problems' rooted in 1990s PDM limits; modern graph-data PLM enables 'static configuration management', the only approach supporting digital twins. Central question: how can you work on the digital product if your view of it is a non-persistent dynamic-filtering calculation? Concurrent engineering breaks immediately — a component applicable at T0 but inapplicable at T1 forces finding a replacement and migrating all attached manufacturing/support data, binding downstream teams to engineering's rhythm. The common workaround — caching intermediate configuration snapshots with timestamps in separate databases — is exactly an ad-hoc persistence layer compensating for the missing wired graph.

"How can you work on the digital product if the vision you have of the product is the result of a dynamic filtering calculation that will not be persistent enough to enable the creation of the many links you need?"

"structural sources of problems"

§ The New Way — Reuse, Interfaces, Multi-level Tree. Modern manufacturing shifted from making all components to assembling supplier subsystems, requiring compatibility through well-defined interfaces (mechanical, electrical, fluid) that enable component reuse and specialization; like software, lifecycles carry many component versions in series production. CM must operate at MULTIPLE levels: component (variants/options), product (which components apply), and intermediate 'big components' aggregating smaller ones. Figure 6 distinguishes internally-mastered design solutions (blue), small assemblies SA (yellow, only SA2 internal), and big assemblies BA (purple, BA1/BA2 internal, BA3 straight from suppliers) — making the multi-level hierarchy explicit. Drawback noted: optimizing within an existing supply chain can suppress global innovation.

"mastering supplier component compatibility through defined interfaces—mechanical, electrical, fluid-based"

"big components aggregating smaller ones (cars, ships, aircraft)"

§ Adding Versions to the Product Tree. Component versions are folded into the tree, showing each component's full lifecycle with explicit change linkages. A propagation convention: any change to a lower component version implies a version change of the components above it. Figure 7: DS2 v1.0 → v1.1 is tagged by Change 1a2c3; SA2 uses DS2 v1.1 in its v1.18/v2.0 and DS2 v2.0 in its v12.0/v12.1; black arrows are structural dependencies. The product head P points to all component heads (latest), though not all apply to every instance — distinguishing 'head/floating' from 'wired/pinned'.

"any lower component version change implies upper component version changes"

"Product head P points to all component heads (latest versions), though not all apply to every product instance."

§ Materializing the S/N in the Product Tree. The crux of the static model. Product instance S/N 1212 version v234.3 points to specific BA1 v25.4 and BA2 v56.6, which point to contained small assemblies, which point to design solutions — a fully resolved, wired graph head. There is NO calculation mechanism: the S/N version points to existing, fully-defined component versions. Upgrades mint a NEW S/N version pointing to updated components, and once wired a version never changes — ten years later that subgraph is identical. This is the paper's definition of immutable, content-pinned, time-stable versions.

"There are no more calculation mechanism: The S/N 1212 in version v234.3 is pointing to existing versions of big assemblies that are fully defined."

"When a version is 'wired' with links, it will never change in the future, and ten years later, this past part of the graph will be exactly the same."

§ Applicability and Configuration (static). Applicability splits into 'real usage' (this S/N genuinely uses this component version — de facto applicable) and 'future applicability' (components available for future use, an open choice since all releases stay accessible). Configuration needs no algorithm: each S/N version is itself a component-tree head extending to the needed depth, so every configuration is already 'solved'. A study S/N (XXX01) can fork back to a past version (DS2 v1.2) for analysis/innovation. The decisive payoff: all S/N configurations are 'secured forever', enabling true concurrent engineering — manufacturing assemblies SA2 v12.0 while engineering develops DS2 v2.1 with no conflict.

“All versions of the S/N configuration are secured forever inside the system.”

“Different departments work on different component versions without conflict.”

§ Change Management (graph model). Changes tag the LINK between two specific component versions; the old aggregate multi-component before/after change 'does not exist anymore' — changes restrict to single-component transitions, while an assembly version may simply embed different sub-components. Computing a configuration delta vs a baseline becomes graph analysis: aggregate all changes that produced all versions of all sub-components (SA2 v11.8 → v12.0, change 3df59 = union of 12bb5, aze129, 1a44ho). Comparison tools allow direct version-to-version analysis (DS2 v2.0 vs v1.1). The two structural differences from filtering: configurations are solved/explicit, and change-sets are computed by graph diff rather than rule replay — 'the product comes first and the change comes second'.

“changes tag links between component versions”

“Change 3df59 corresponds to the union of changes 12bb5, aze129, and 1a44ho.”

“The product comes first and the change comes second. A system with this kind of management can be called a product lifecycle management system.”

§ Product Catalog, PLM Features, Digital Twin, Conclusion. A 'solved graph' holding all design solutions, assemblies and serial numbers in all versions trivially generates catalogs (applicable element heads at a chosen time/granularity plus compatibility tables). Baselines + option choices let an S/N be minted directly as a new graph head — 'the direct fruit of the product catalog' — giving a non-ambiguous link across marketing, engineering, manufacturing, support. Graph data unlock software practices: branching/forking (retrofits as parallel branches, merges with defined 3D/BOM/document strategies) and 'studies' (temporary products from future, fake, or old components). The whole PLM graph IS the product line's digital twin; each S/N is one product's twin. Conclusion: filtering brings problems that worsen downstream; static graph-based CM secures all data, stays simple, and enables concurrent engineering — 'the graph data are at the core of the PLM systems.'

“The S/N, in its first version, is the direct fruit of the product catalog.”

“The complete PLM graph represents the product line's digital twin, containing all possible products.”

“The graph data are at the core of the PLM systems.”

HOW IT MAPS TO OUR PARTS

- **Object-container kernel (stable container = identity thread; revisions = immutable values; current-revision = state; 'inferred structure is never accepted as truth at ingest')** — STRONGLY SUPPORTS and gives a name. The paper's 'wired static graph' IS the container-with-immutable-revisions pattern: an S/N version 'is pointing to existing versions of big assemblies that are fully defined' and 'when a version is wired with links, it will never change in the future.' The S/N (container) threads identity over time; each version points to specific, frozen component versions (immutable revisions). Critically, the paper's central warning — that dynamic filtering 'calculates' configuration and 'offers no guarantee of matching originally-delivered configurations if business rules change' — is the industrial proof of our hard rule: inferred/computed structure must never be treated as committed truth. Wire it explicitly or you cannot trust it ten years later.
- **Versioning as its own layer: Rama arrival-time history (partial persistence) vs explicit git-style COMMITTED/MINTED versions (full persistence); two times = when-it-arrived vs when-it-was-committed** — DIRECTLY VALIDATES the two-times distinction and warns against relying only on the derived one.

Filtering-based CM = reconstructing-from-a-log-and-rules (arrival-time/derived); graph-based CM = explicitly minted, branchable versions (committed/full persistence). The paper shows a purely recomputed view is unreliable because the recomputation rules drift ('perceived PDM unreliability'). Lesson: Rama's arrival-time log is necessary but NOT sufficient as truth; we need explicitly minted committed versions ('wired' heads) you can branch from. This is the strongest external argument we have for why the committed-version layer must exist on top of Rama's default history.

- **Groupings as versioned objects (a grouping/local-world has its own version history, partly independent of members; member binding pinned vs floating vs effectivity-scoped)** — GIVES A MECHANISM and the exact pinned-vs-floating vocabulary. Assemblies (SA/BA) are groupings with their OWN version lifecycle: 'any lower component version change implies upper component version changes' yet an assembly version can 'embed different sub-components than previous versions' and skip member versions (SA2 jumps DS2 versions). 'Pinned' = wired S/N pointing to fully-defined component versions. 'Floating' = product head P 'points to all component heads (latest versions).' 'Effectivity/condition-scoped' = serial-number-range applicability ('S/N in range [1,N]'). The paper proves a grouping's history is partly independent of members and shows how change-aggregation rolls member changes up into the grouping's delta.
- **The code-identity question: is the atomic unit the file or the function; is identity the path, name/FQN, AST, or git hash?** — OFFERS AN ANALOGY by structural inversion. PLM resolved 'is the unit the whole product or the component?' by making the COMPONENT the atomic identity-bearing unit with its own lifecycle, and the product an assembly of pinned component versions — 'the product comes first and the change comes second' only once components are first-class. Mapped to us: the FUNCTION (component) is the better atomic identity unit, with the FILE as an assembly/grouping that pins specific function versions. The paper warns the wrong choice (CLM: change-first instead of product-first) cascades problems — analogous to choosing git-hash/diff-first identity over a stable container, forcing 'complex calculation' to recover what a thing actually was.
- **The trail-to-code join: DECLARED change (agent says what it did) vs INFERRED change (reconstructed by diffing); resolve path+timestamp to code-at-that-moment** — WARNS AGAINST inferred-by-diff and ENDORSES declared. The paper's whole thesis is that reconstructing 'what was true at a past moment' by recomputation/filtering is unreliable, while explicit declared links are 'secured forever.' Its change model tags an explicit link between two specific component versions ('changes tag links between component versions') — that is DECLARED change as a first-class edge, exactly our preferred trail-to-code join. Recovering configuration by re-diffing baselines is possible but is the costly, drift-prone path. Direct evidence to privilege the agent's declared edit-operation over diff-reconstruction when resolving a trail to code-at-that-moment.
- **Preserved disagreement / plurality; synthesis late-bound; branching/forking from past committed versions** — SUPPORTS via forking and the named failure mode of NOT supporting it. Graph-CM enables 'branching/forking,' 'studies' (temporary products with fake/old/future components), and forking a study S/N back to a past version (DS2 v1.2) for innovation. The killer evidence is the failure mode of the filtering/stacking model: it 'typically prevents forks' — a fork from V1 while a later change targets V2 'cannot be represented,' forcing a 'manual, costly and error prone' undo of all changes. Concrete industrial proof that a substrate which silently linearizes history becomes unusable — exactly our case for representing plurality structurally rather than overwriting.
- **Four-level graph versioning (graph / node / relation-TYPE / node-DATA), and these levels must COMPOSE** — PARTIAL MECHANISM. The paper composes versioning across structural levels: design-solution (node-data), small/big assemblies (node groupings), product head (graph), and changes that 'tag links' (relation/edge versioning). The change-aggregation rule ('SA2 v11.8 to v12.0 ... Change 3df59 = union of 12bb5, aze129, 1a44ho') is exactly level-composition: an edge/grouping version's delta composes from its members' deltas. It is weaker on relation-TYPE versioning (edges are mostly structural containment), so it confirms node + node-data + grouping + edge-instance composition but leaves relation-TYPE versioning as our own contribution.
- **Substrate choice: Rama (event-sourced) as ground truth, with graph data as the persistent core** — SUPPORTS the substrate stance with a caveat. The paper's verdict — 'the graph data are at the core of the

PLM systems' and only persistent graph data 'enables the creation of the many links you need' and digital-twin operations — endorses graph-as-truth over recompute-on-demand. The caveat for our Rama choice: an append-only event log is the substrate, but truth must be the explicitly wired graph projected/minted from it, not a query-time filter over the log. Otherwise we rebuild the filtering anti-pattern on a fancier engine.

Takeaway. The single most important contribution: an independent, decades-deep industrial validation of our object-container kernel and committed-version layer. Rey shows that reconstructing 'what a thing was at a past moment' by RECOMPUTING it from a log plus filtering rules (serial-number effectivity / change-stacking) is structurally unreliable — it cannot guarantee a match to what was actually delivered once rules drift, it cannot represent forks, and it forces costly manual undo — whereas explicitly WIRED, immutable version-to-version links ('a version, once wired, will never change; ten years later this subgraph is exactly the same') secure every configuration forever and make branching and concurrent work trivial. This is the strongest external argument that Rama's arrival-time log alone is not truth: we must mint explicit committed versions wired to specific revisions ('the product comes first, the change comes second'), and never accept a filter-/diff-reconstructed structure as truth at ingest. The assembly model also hands us a ready vocabulary for groupings-as-versioned-objects (pinned vs floating member binding, grouping deltas composed by aggregating member changes)."

What we can gather — Angle 5

This is the mature, boring, solved part — forty years of CAD/SCM/PLM that answers "groupings as versioned objects" directly — and it is the angle with the least to invent and the most to simply *adopt*.

The configuration is a first-class versioned object. Katz names it exactly: a configuration is "a binding between a version of a composite object and the versions of its components," and the composite "is a new version in its own right" even when its own contents did not change but a member did. The grouping has its own identity and its own history, partly independent of its members. Two further gifts from Katz: **current ≠ last-created** (the head pointer is deliberate minted state, never inferred from arrival order — our hard rule, stated in 1988), and **equivalences** — a declared relationship tying heterogeneous representations of the same real thing (layout/transistor/functional views), checked at verification time rather than merged. That is precisely our "the same understanding as code, as trail, as discourse node," with a 1988 mechanism.

Version the group, snapshot the members — because the alternative cascades. ORION tried versioning members (classes) independently and found that, because members reference each other, deriving one forces re-versioning the whole neighborhood — the version-proliferation failure mode, demonstrated. Its fix was to version the *grouping* (the whole schema lattice) as one object and snapshot members via copy-on-derive, with an inherited **access scope** that makes branch-local divergence visible only to its branch — non-destructive preserved disagreement. Static vs dynamic binding = our pinned vs floating.

Declarative configuration is the escape from combinatorial explosion. Sciore models versions as a *multidimensional coordinate space* and a configuration as a **declarative (intensional) query** over coordinates, not a pinned parts list. Our four levels are four such dimensions; intensional selection composes them without enumerating the cross-product ($3 \text{ OS} \times 3 \text{ DB} \times 3 \text{ window} = 27$ branches, or one query). His load-bearing warning: do **not** make the container and the revision two separate object types — fold versioning into the kernel object, or every schema and query pays a perpetual join tax. That is a direct caution about how we model container-vs-revision in Rama.

And industry has already proven the deepest point. The PLM essay shows that reconstructing "what a product was at a past moment" by recomputing it from a log plus filtering rules (serial-number effectivity, change-stacking) is *structurally unreliable* once the rules drift, and **cannot represent forks at all** — whereas explicitly **wired, immutable version-to-version links** are "secured forever; ten years later this subgraph is exactly the same." This is the strongest external argument in the entire research that **the arrival-time log alone is not truth**: you must

mint explicit committed versions wired to specific revisions, and never treat a filter-/diff-reconstructed structure as truth. Build the wired graph from the log; do not rebuild the filtering anti-pattern on a fancier engine.

What we gather: three binding modes (pinned / floating / effectivity-scoped); the configuration as one first-class noun that unifies our manifest, a git commit, a grouping, and a synthesized local world; version-the-group-snapshot-the-members to dodge the cascade; declarative intensional selection as the sophisticated regrouping end-state; and the join-tax warning about how we type container-vs-revision in the kernel.

What we can gather — overall

Reading all twenty-four together, the same conclusions arrive from independent directions — code systems, formal theory, production databases, graph stores, and forty years of mechanical-engineering PLM — which is the strongest kind of evidence we could have. Nine things are now load-bearing.

1. The architecture is convergent, not chosen. Immutable content-addressed *values* plus a thin mutable *naming* layer is where git, Irmin, Unison, Dolt, Noms, TerminusDB, Monticello, Molhado, Katz, and Conradi-Westfechtel all independently land. Our container/revision split is that pattern. We are implementing the answer, not gambling on one — and the one thing we add that most predecessors lacked is a natively event-sourced value layer (Rama), so we keep history by default instead of bolting it on.

2. Versioning is two times, formally distinct, and they must be combined. DSST (partial vs full persistence), XTDB (transaction vs valid time), and Conradi-Westfechtel (extensional vs intensional) are three names for one split. Rama gives us the first axis for free — the append-only arrival log. We must *mint* the second — committed, branchable, semantically-meaningful versions — as a deliberate act, and PLM proves the arrival log alone cannot be trusted as truth. The mint is an event; the object it produces is a **configuration**.

3. The function is the atomic unit; the file is an assembly. Angle 1 and Angle 5 fuse here. Every system that cared about code rejected file-granularity, and every system that cared about groupings made the *member* the identity-bearing unit and the *grouping* a binding over member versions. So a function is a container; a file is a configuration that pins function versions — and identity is a stable opaque handle, with path, name, content, and git-hash demoted to *attributes or coordinates*, never the identity itself.

4. Groupings are first-class versioned objects — version the group, snapshot the members. This is the most thoroughly solved part. Configuration-as-object (Katz), the cascade/proliferation failure mode of doing otherwise (ORION, Conradi-Westfechtel), three binding modes — pinned / floating / effectivity-scoped (Katz, ORION, PLM, DSST's valid intervals) — and declarative intensional selection as the end-state (Sciore). One noun, *configuration*, unifies our version-manifest, a git commit, a curated grouping, and a synthesized local world.

5. Declared beats inferred — and Softland uniquely owns the declared channel. This is not merely our safety rule; the patch theories show it is the foundation a *correct* version model is built on (you cannot recover the morphism from snapshots without lossy inference), MolhadoRef dogfooded it (declared refactorings preserved history that diffing lost), and Katz and PLM both insist derivation must be a declared first-class edge because naming and arrival data cannot reconstruct it. The asymmetry that no prior system had: **agent reasoning trails are a declared-operation log** — edits with intent narrated in language. MolhadoRef needed to own an IDE to get this; we get it because agents already make and narrate most edits. This is the single biggest structural advantage the research surfaces.

6. Preserved disagreement can be a data structure. Pijul and the categorical theory prove a conflict is a first-class value — a transitive *relation*, not a linear list — and tell us the PState schema for a conflicted unit must store a relation. XTDB (additive correction), ORION (access-scope branch isolation), and every branch/merge system show divergence held without overwrite. Synthesis is late-bound — and total automatic merge is the explicitly *unsolved* frontier (DSST's confluent persistence, the homotopical theory's incomplete merge), so we design for held plurality and human-or-late synthesis, and never promise clean auto-merge.

7. Four-level graph versioning has a blueprint and three hard constraints. You provably need both a member track and a group track, linked (the 5-tuple theorem; Wikidata's braid). Store facts once with validity ranges. Mint at commit-grain with adaptive snapshot anchors, or hit the version-count wall (21k versions = system failure). And **relation-TYPE / schema versioning is the one level nobody in the literature solved** — it is our genuine contribution, and Sciore's coordinate-space + intensional-selection is the most promising tool for making the levels compose without combinatorial explosion.

8. The warnings to encode now, before code. Don't model container and revision as two separate Rama object types (Sciore's join tax) — prefer Adele's shape: one item with common attributes, revision-specific attributes, and immutable attributes whose update mints a revision. Budget ~5x overhead for the committed layer (Irmin). LCA discovery is your problem, not the merge function's (Irmin). Don't rebuild filtering-as-truth on Rama (PLM) — truth is the wired graph minted from the log, not a query over it. And don't let delta chains grow unbounded — re-snapshot on a metric trigger (OSTRICH, BEAR, the RDF framework).

9. The reordering this whole exercise earns. The versioning vocabulary — item/OID/VID, configuration, mint, the two times, bound/generic/effectivity, partial vs full persistence, VM/DM/VQ — is **kernel physics**, and it belongs in a kernel-physics document *above* the ingestor contracts. The code ingestor is merely the first place the question got loud; Roam, groupings, and the four-level graph all route through the same nouns, and the 5-tuple theorem proves the group track cannot be safely retrofitted. Settling the vocabulary is cheap and on paper; building the mechanism should still wait for a consumer to pull it.

What this changes for the code-ingestor slice 1

The research **validates more than it overturns**. Stable container + revisions = OID/VID, confirmed ~ten times. Commit-grain minting = Katz's "semantically meaningful snapshot, not every update," and the version-count wall actively vindicates it over save-grain. Refusing git's rename heuristic is right *because* the correct answer is declaration, not better inference (MolhadoRef, Pijul, Katz, PLM all agree). Whole-file artifacts now with forms later = MolhadoRef's own deliberate granularity floor. The earned amendments, each now with citations behind it:

1. **Identity = opaque stable handle, not content-hash** (Molhado over Unison) — keep a content hash as a *derived* attribute for dedup and the exactness flag, not as identity. Same bytes at a new path = same value, new occurrence: half of rename continuity as *fact*, not inference (Irmin/Proolly/Unison dedup).
2. **Promote the version-manifest to a first-class configuration object** (Katz) — it is simultaneously the group-version thread and the noun the future Softland-native commit needs; unify them now at zero cost. Version the group, snapshot the members (ORION) to avoid cascade.
3. **Make the two times explicit** (XTDB) — commit author-time (valid) vs ingest-time (transaction); the exactness flag becomes a projection of this, and corrected resolutions are additive, not overwriting.
4. **Adopt the VM/DM/VQ read vocabulary** (OSTRICH) and **validity-range storage** (Wikidata) for the code-at-time / trail-resolution projections.
5. **Snapshot the name/path → identity table as-of each trail moment** (Unison's namespace-version) so a trail's recorded path resolves later even after renames.
6. **For the regrouping vision: composition model only** — pinned / floating / effectivity-scoped selections (Katz/ORION/PLM/Sciore), never total versioning (proliferation), never a single global version space (Molhado's inexpressible cross-version reference, which would kill anchors and trails).
7. **Don't make container and revision separate kernel types** (Sciore) — fold versioning into the item; this is a Rama PState modeling decision to make before writing the adapter.

The open frontier the literature itself hands us, unsolved by anyone: **semantic merge** (declared refactorings × ordinary edits), **relation-TYPE versioning** (the fourth graph level), **confluent/late-bound synthesis of preserved plurality**, and **history-length-independent performance**. These are not gaps in our design; they are gaps in the field — and three of the four are exactly the places where having agents (declared intent) and LLMs (semantic judgment) in the substrate is the new leverage none of these systems had.